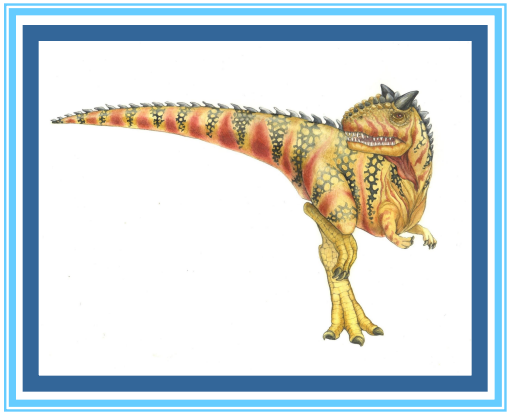
Chapter 11: File-System Implementation



Silberschatz, Galvin and Gagne ©2013

Chapter 11: File-System Implementation

11.1 File-System Structure

11.4 Allocation Methods

11.5 Free-Space Management

11.7 Recovery

11.10 Summary

Objectives

- To describe the details of implementing local file systems and directory structures.
- To describe the implementation of remote file systems.
- To discuss block allocation and free-block algorithms and trade-offs.

File-System Structure

Overview

- File structure:
  - Logical storage unit.
  - Collection of related information.

- **File system** resides on secondary storage (disks)
  - Provided user interface to storage, mapping logical to physical.
  - Provides efficient and convenient access to disk by allowing data to be stored, located retrieved easily.

File-System Structure

- Disk provides in-place rewrite and random access:
  - I/O transfers performed in **blocks** of **sectors** (usually 512 bytes).
- **File control block** – storage structure consisting of information about a file.
- **Device driver** controls the physical device.

- File system organized into layers (next slide).

File-System Structure

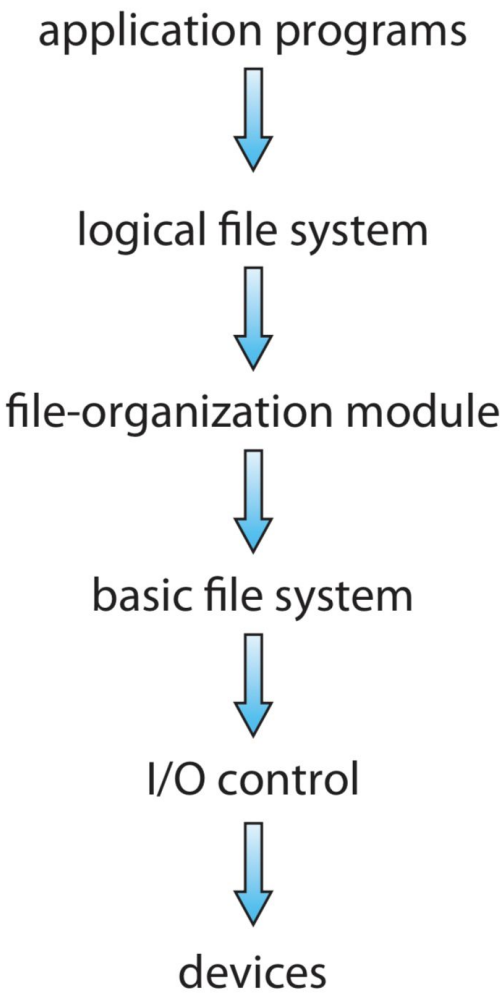
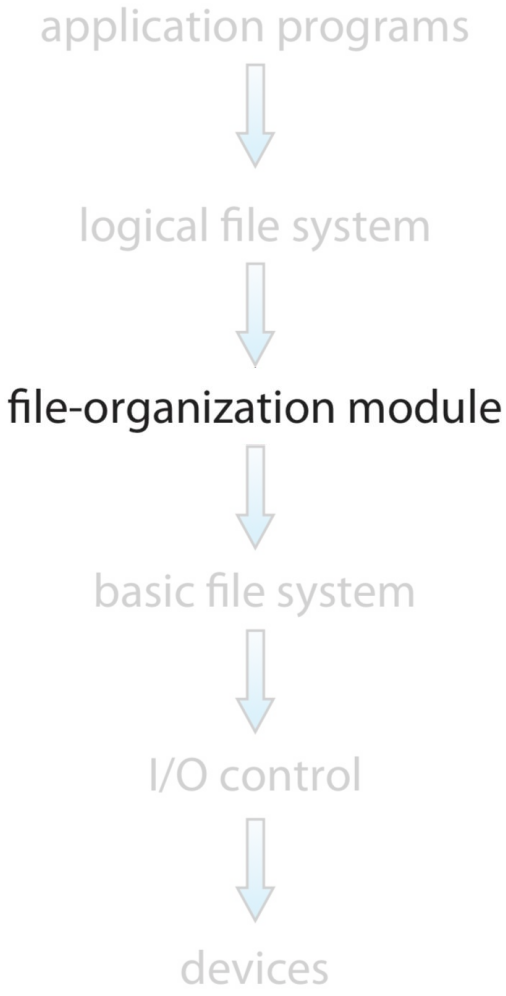


Figure 11.1 Layered file system.

- **Logical file system** manages metadata information includes all the file-system structure, except the actual data.
- **File-organization module** understands files, logical address, and physical blocks.
- **Basic file system** to issue generic commands to the appropriate device driver to read/write physical blocks on the disk.
- **I/O control** consists of device drivers and interrupt handlers to transfer information between main memory and the disk system.

11.1

File-System Structure



- **File-organization module** understands files, logical address, and physical blocks.
  - Translates logical block # to physical block #
  - Manages **disk allocation** and **free space**.

Figure 11.1 Layered file system.

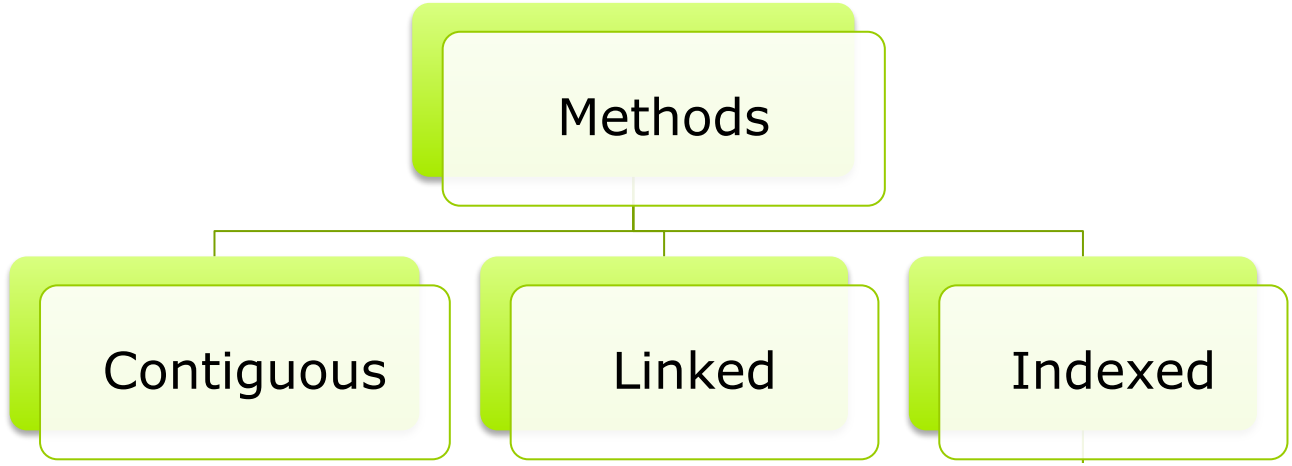
Allocation Methods

Overview

- A disk is a direct-access device and thus gives us flexibility in the implementation of files.
- The main issue is :
  - ✔ how to allocate space to these files so that disk space is utilized effectively, and
  - ✔ files can be accessed quickly.
- An **allocation method** refers to how disk blocks are allocated for files.

Allocation Methods

- Three major methods of allocating disk space are in wide:



Extent-based system

Index Block Structure

Allocation Methods

(a) Contiguous allocation

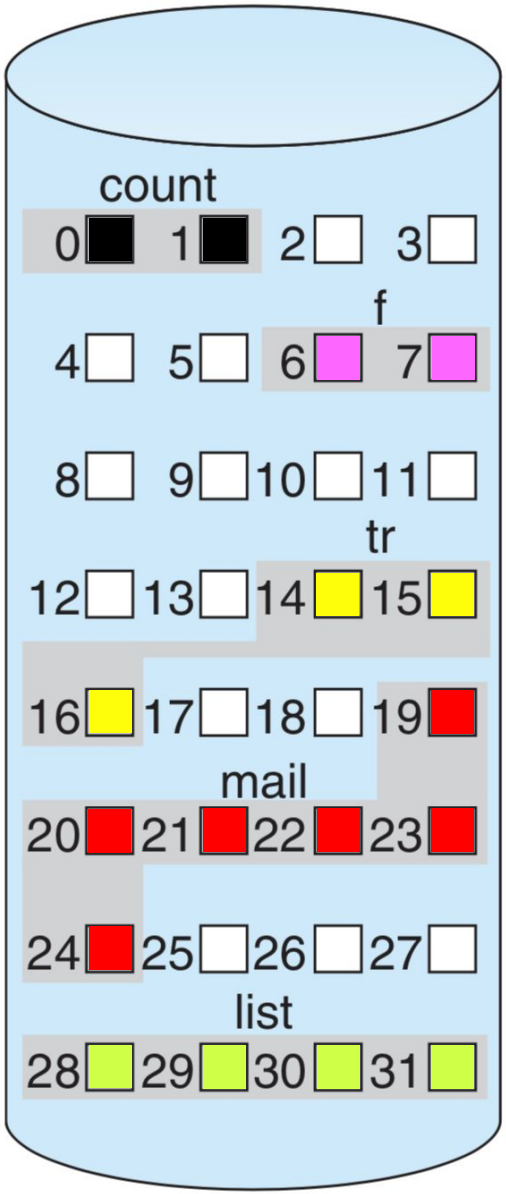
- Each file occupies a set of contiguous blocks.
- Best performance in most cases.
- Simple – only starting location (block #) and length (number of blocks) are required.
- Problems include:
  - Finding space for file,
  - Difficult file extension,
  - Knowing file size (max size),
  - External fragmentation,
  - Need for compaction.

Allocation Methods

Example 1 :

(Page: 506)

Mapping from logical to physical.



directory

file	start	length
count	0	2
tr	14	3
mail	19	6
list	28	4
f	6	2

Figure 11.5 Contiguous allocation of disk space.

Allocation Methods

Example 2 : Contiguous file storage for File A and B.

Free Space	File A Record 1	File A Record 2	File A Record 3	File A Record 4	File A Record 5	File B Record 1	File B Record 2	File B Record 3	File B Record 4	Free Space	File C Record 1
Slot: 0	1	2	3	4	5	6	7	8	9	10	11

File A:

- This file cannot be expanded for new record.
- **Solution**: rewrite into a larger storage area.

File B:

- This file can be expanded by only one record that replacing the free space at slot 10.

Allocation Methods

Extent-based systems

- A file will grow slowly over a long period (months or years) must be allocated enough space for its final size, even though much of the spaces will be unused for a long time
  - internal fragmentation.

- **Solution**: Modified Contiguous-Allocation scheme.
  - ✔ A contiguous block of space is allocated initially.
  - ✔ If not enough, another contiguous block, known as extent, is added.
  - ✔ A file may consists of one or more extent blocks.

Allocation Methods

(b) Linked allocation

- Each file is a linked list of blocks.
- No external fragmentation.
- Each block contains pointer to next block.
- Free space management system called when new block needed.

Allocation Methods

Example 3 :

(Page: 508)

- A file of five blocks start at blocks 9 and continue at block 16, then block 1, then block 10, and finally block 25.
- Each block contains a pointer to the next block.

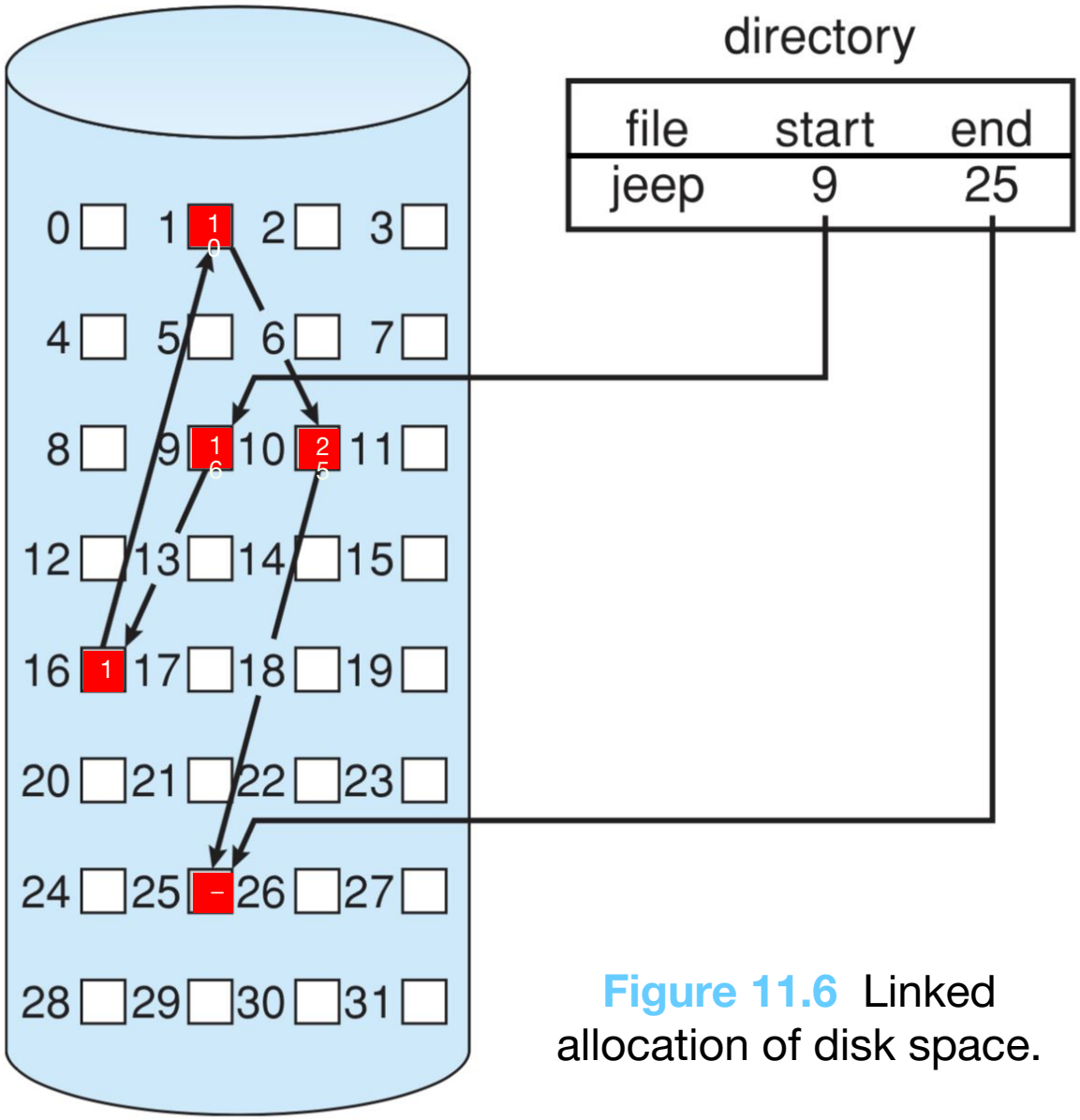
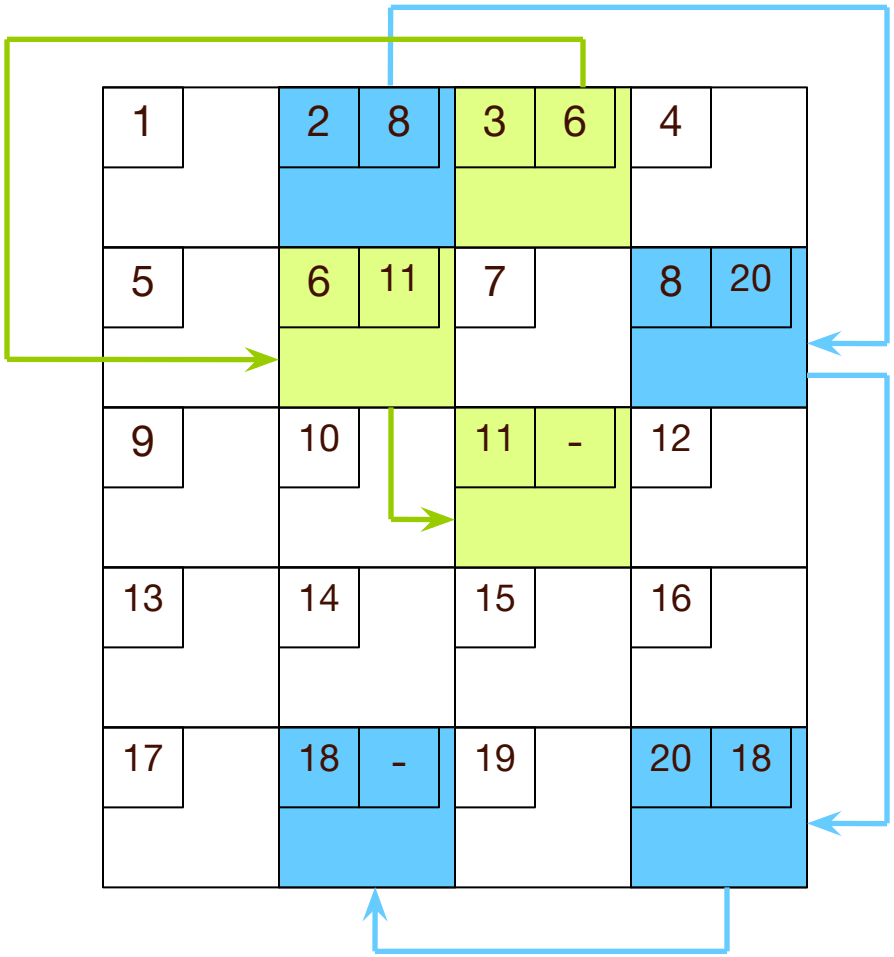


Figure 11.6 Linked allocation of disk space.

Allocation Methods

Example 4 :



- A linked allocation with File A starts at block 2 and continues at block 8, 20, and 18.
- Each block includes its address and a pointer to the next block of the file as well as the data itself.

Directory Entry

File No.	Start	End	No. Extents
A	2	18	3
B	3	11	2

Allocation Methods

- Example 3 and 4 are a sequential-access files.
- To find the i-th block of a file, it starts at the beginning of that file and follow the pointers until the block.

Disadvantages:

- Locating a block can take many I/Os and disk seeks.
- Reliability can be a problem (what if one of the pointers get corrupted?)

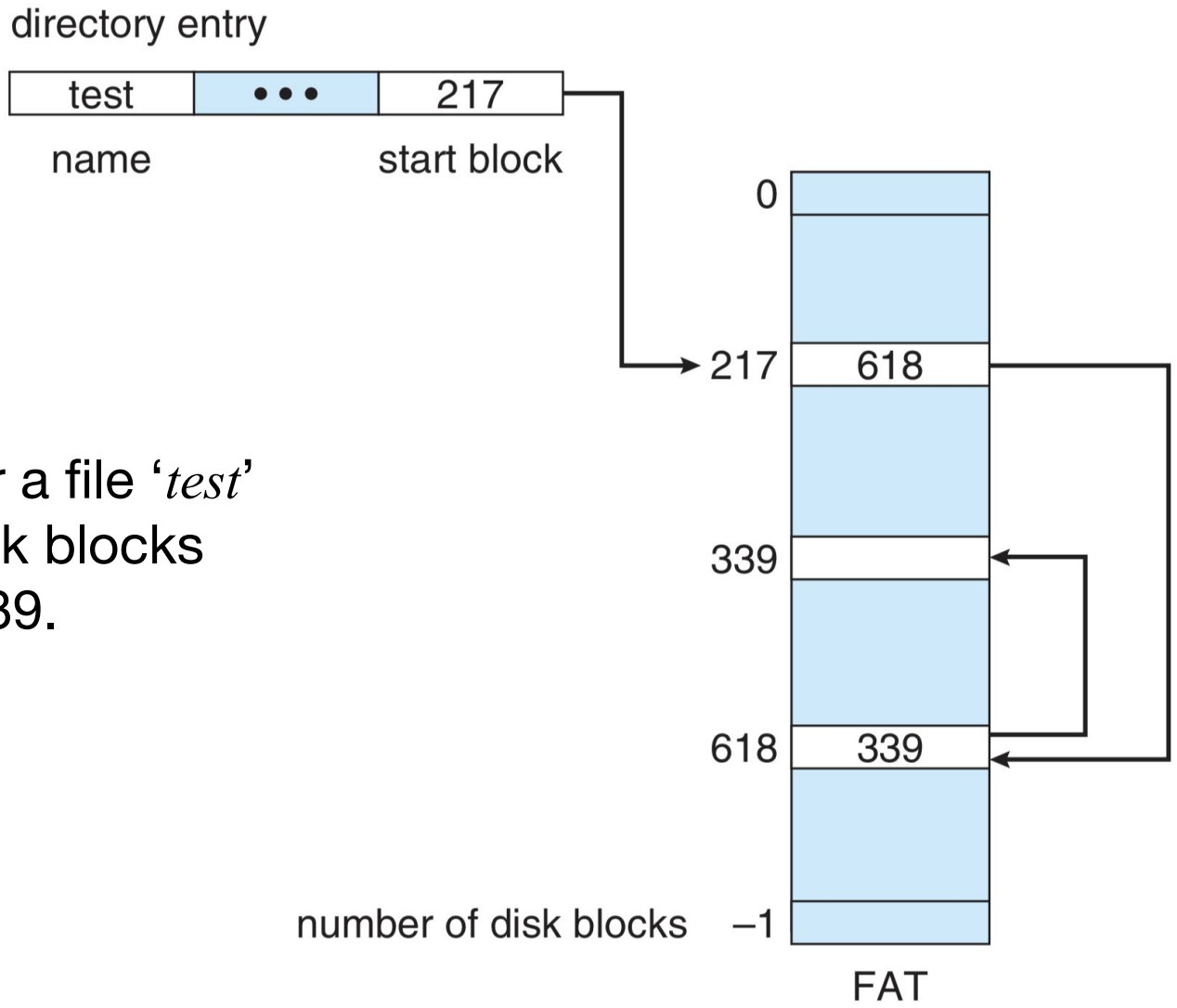
Solution: File-Allocation Table (FAT)

- Beginning of volume has table, indexed by block number.
- Much like a linked list, but faster on disk and cacheable.
- New block allocation simple.

Allocation Methods

Example 5 :

(Page: 510)



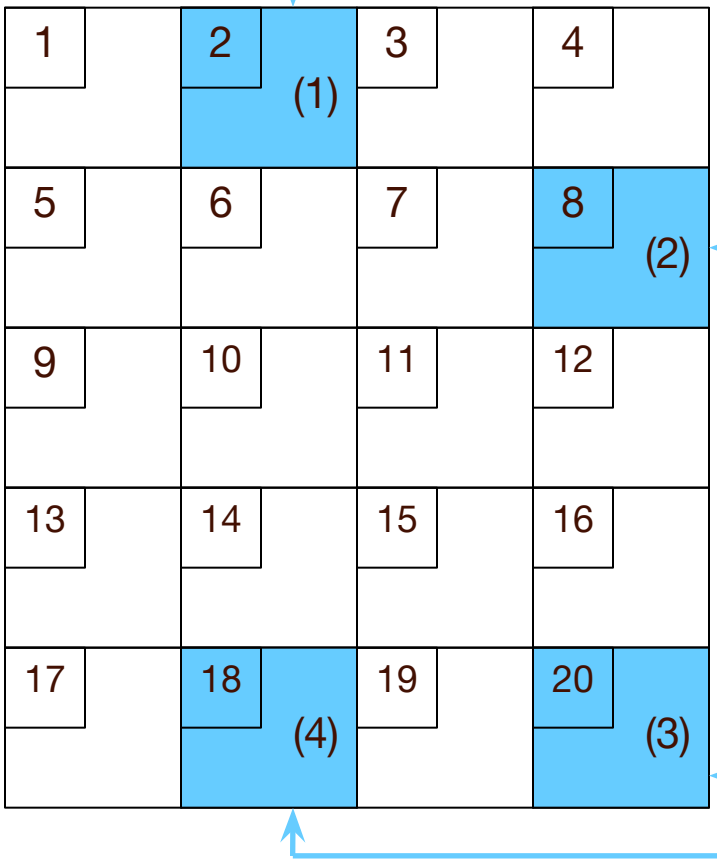
- FAT structure for a file '*test*' consisting of disk blocks 217, 618, and 339.

Figure 11.7 File-Allocation Table (FAT).

Allocation Methods

Example 6a :

MS-DOS which uses linked allocation - **FAT**.



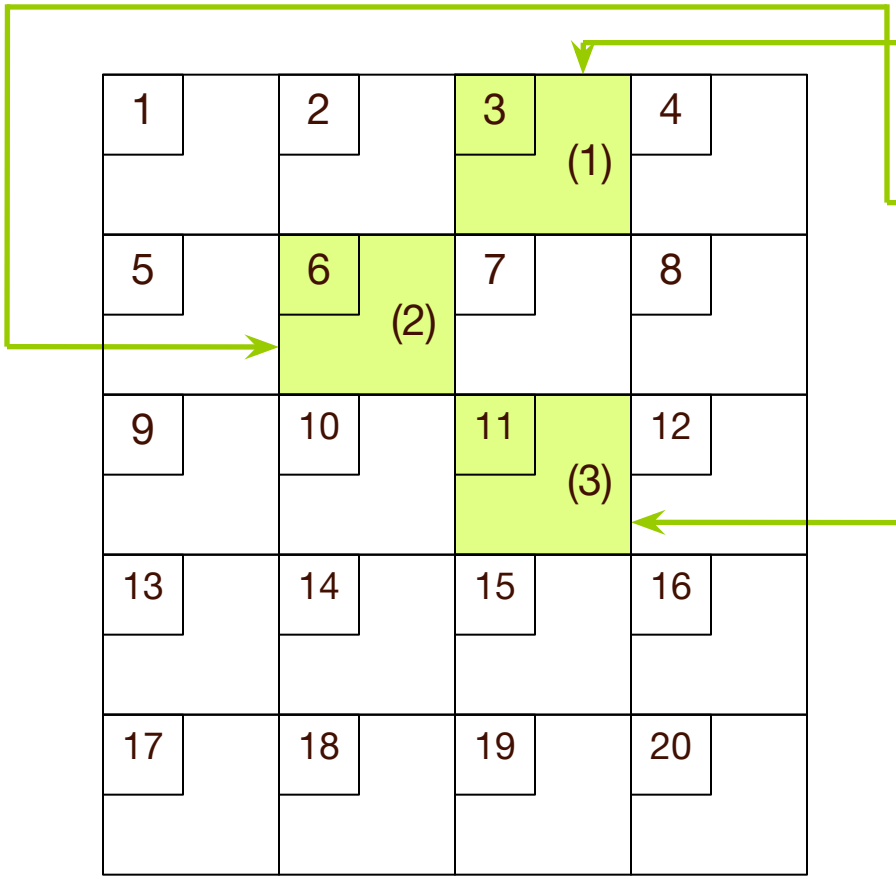
Directory Entry

File	Address	Size (KB)	Next
	1	512	
File A (1)	2	512	8
File B (1)	3	512	6
	. . .	. . .	
File B (2)	6	512	11
	7	512	
File A (2)	8	512	20
	. . .	. . .	
File B (3)	11	512	-
	. . .	. . .	
File A (4)	18	512	-
	19	512	
File A (3)	20	512	18

Allocation Methods

Example 6b :

MS-DOS which uses linked allocation - **FAT**.



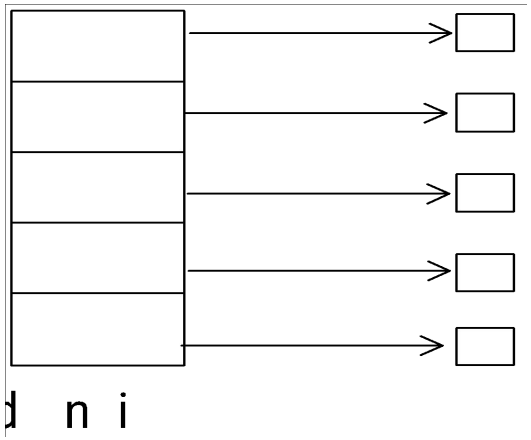
Directory Entry

File	Address	Size (KB)	Next
	1	512	
File A (1)	2	512	8
File B (1)	3	512	6
	. . .	. . .	
File B (2)	6	512	11
	7	512	
File A (2)	8	512	20
	. . .	. . .	
File B (3)	11	512	-
	. . .	. . .	
File A (4)	18	512	-
	19	512	
File A (3)	20	512	18

Allocation Methods

(c) Indexed allocation

- Each file has its own **index block**(s) of pointers to its data blocks.
- Logical view:



- Need index table.
- Random access.
- Dynamic access without external fragmentation, but have overhead of **index block**.

Allocation Methods

Example 7 :

(Page: 510)

- The directory contains the address of the index block.
- To find and read the *i*th block, the pointer in the *i*th index-block entry is used.

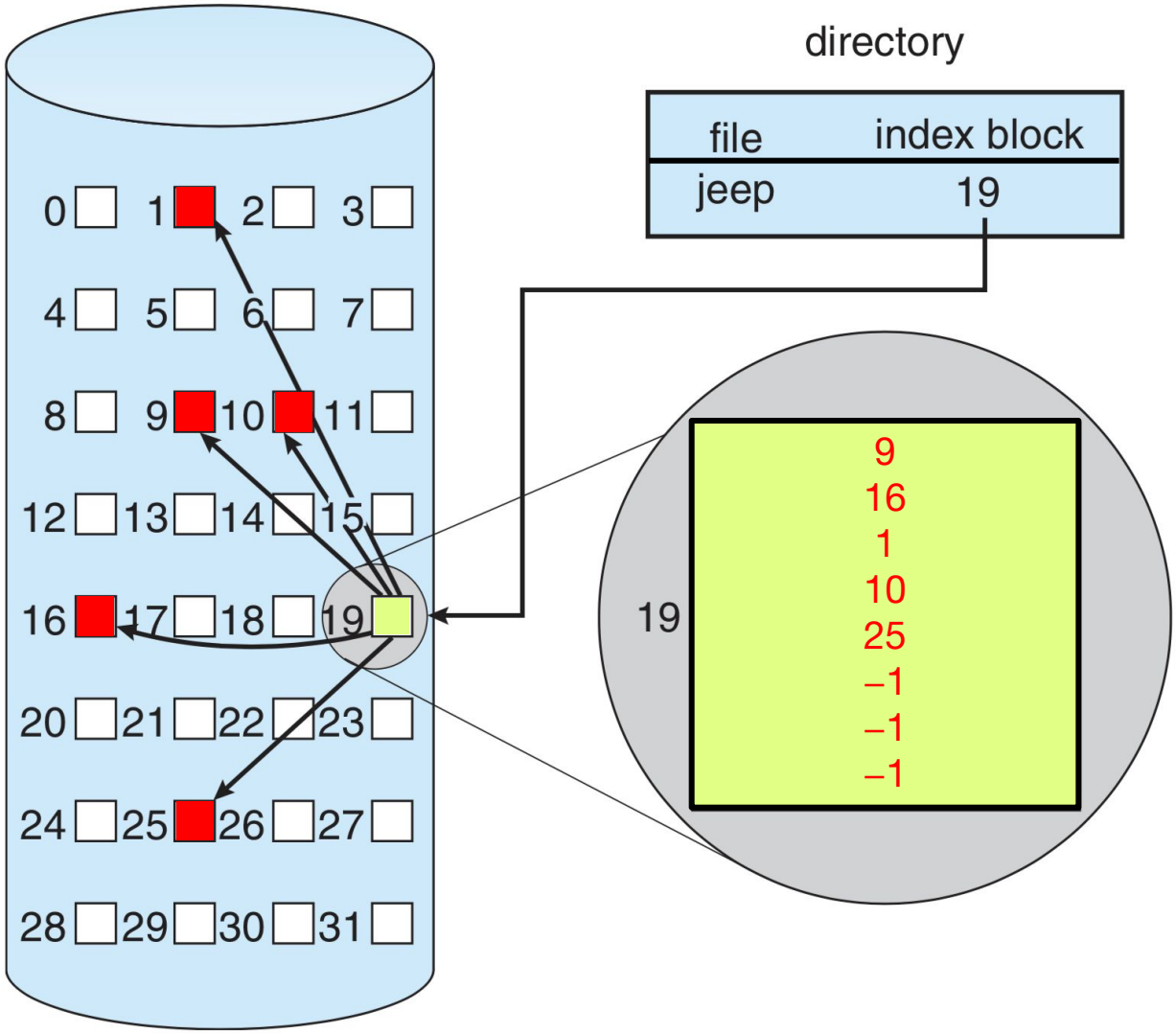
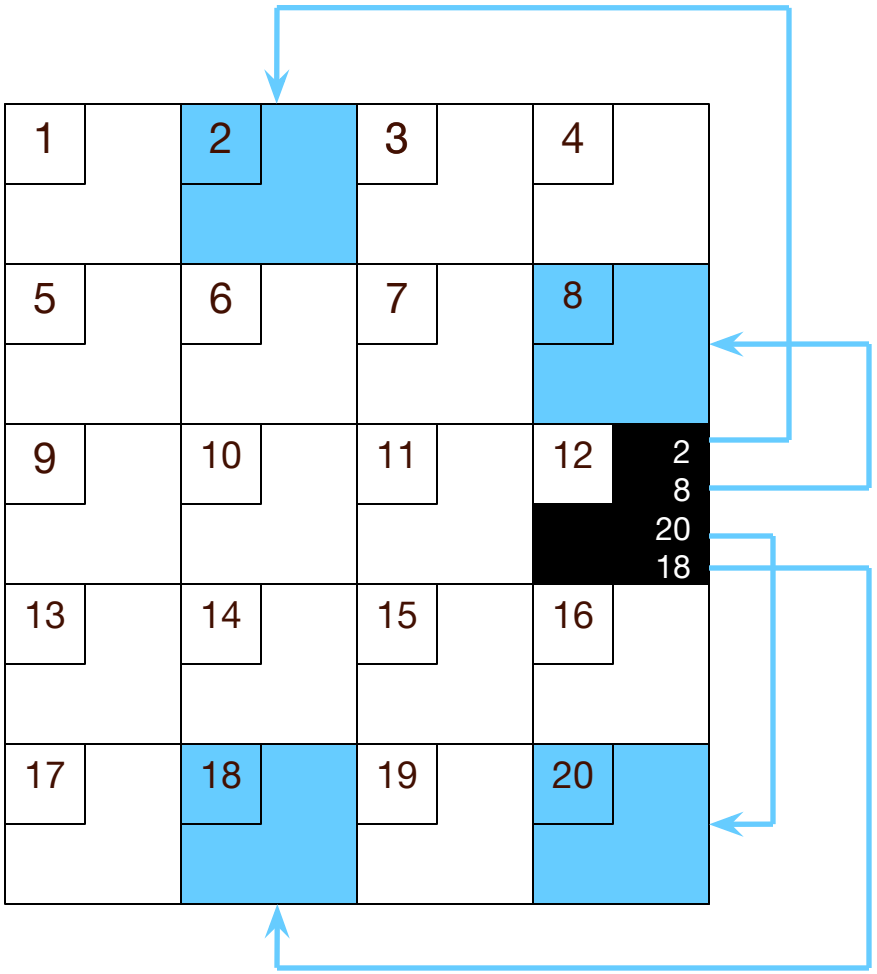


Figure 11.8 Example of Indexed Allocation. Assume each block can only stores 8 block addresses.

Allocation Methods

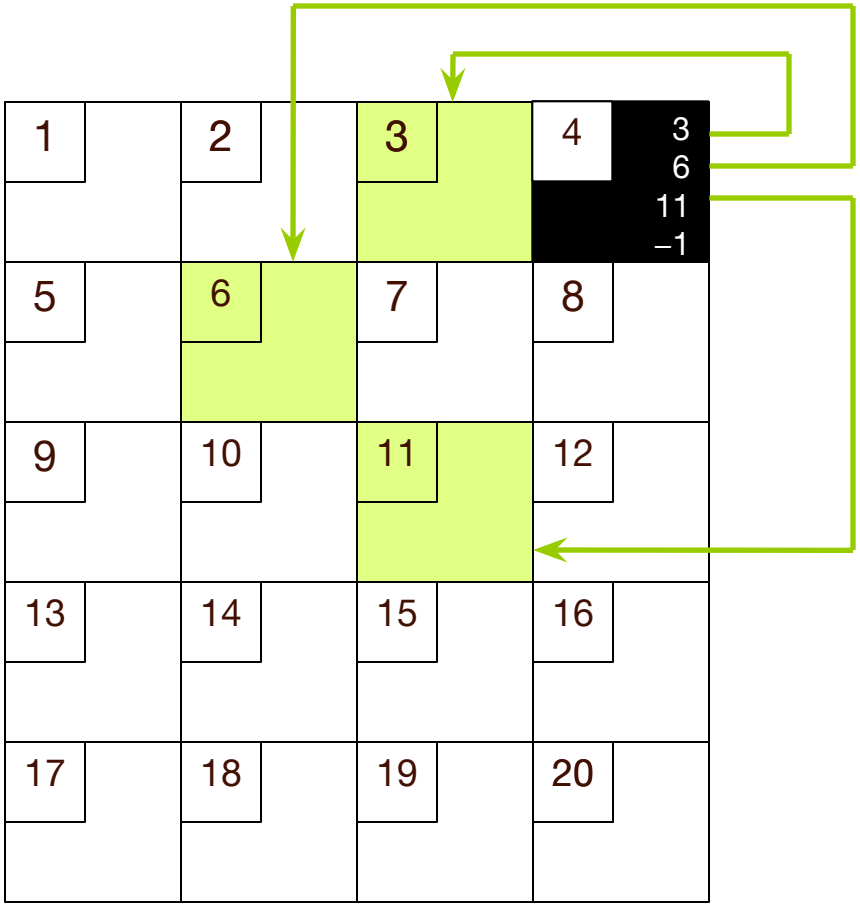
Example 8a : Indexed allocation for 4 bytes each block.



File No.	Index Block
1	12
2	4

Allocation Methods

Example 8b : Indexed allocation for 4 bytes each block.



File No.	Index Block
1	12
2	4

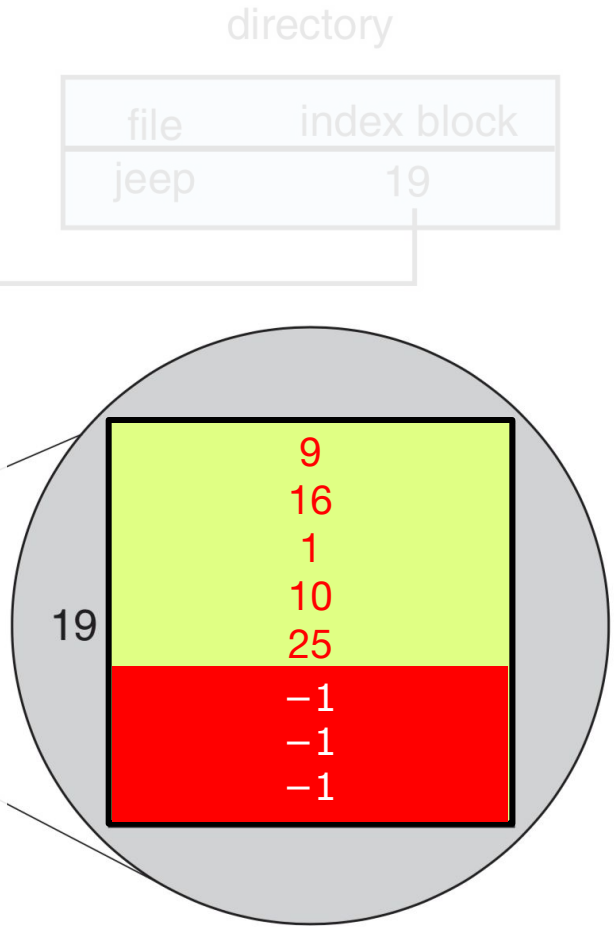
Allocation Methods

Index Block Size Dilemma

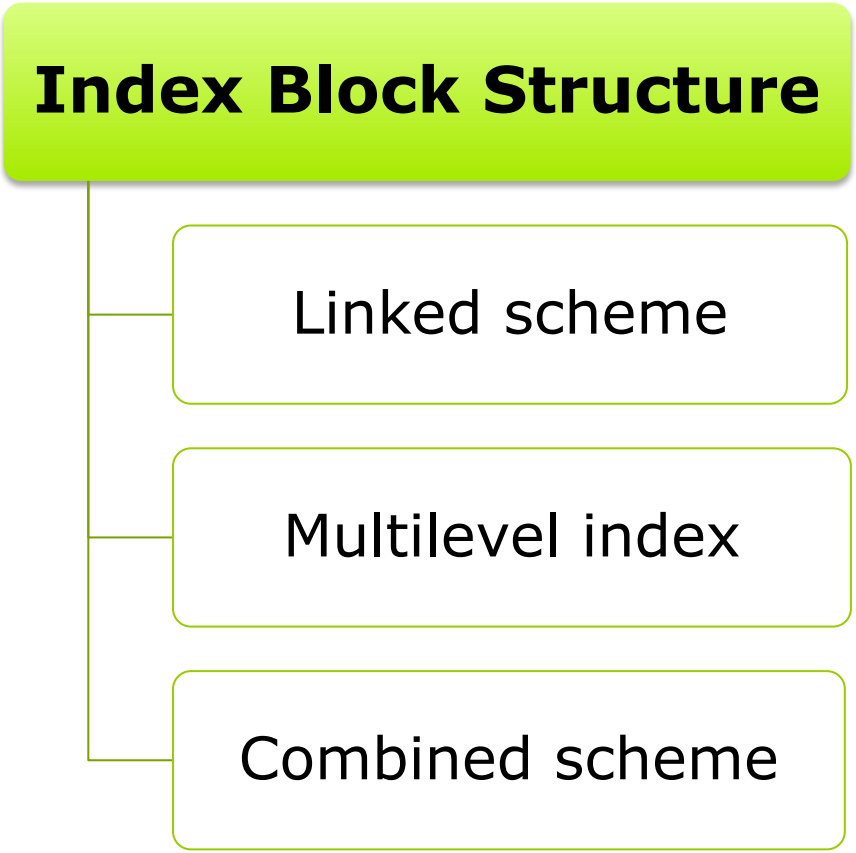
- If the index block is:
  - **small** ☐ it will not be able to hold enough pointers for large files.
  - **large** ☐ there may be lots of unused space if the file size is small.

- **Solution**:

  Use **index block structure**



Allocation Methods



Allocation Methods

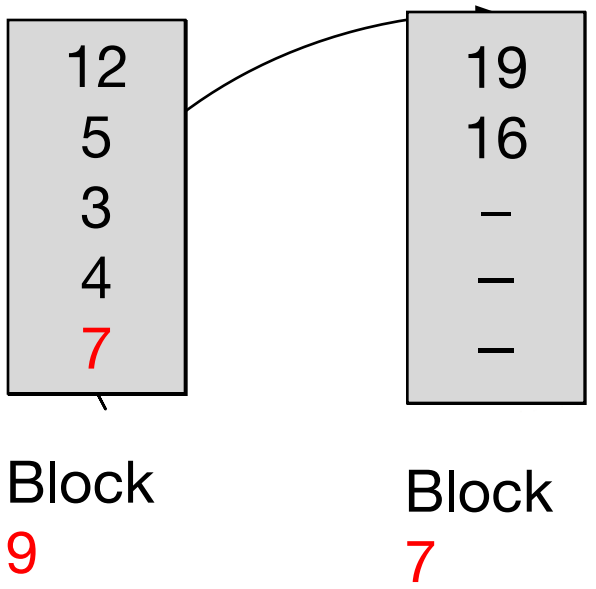
Indexed Allocation: (1) Linked scheme

Example 9 : File blocks □ **9**, 12, 5, 3, 4, **7**, 19, 16.

Assume each block can only stores 5 block addresses.

Directory:

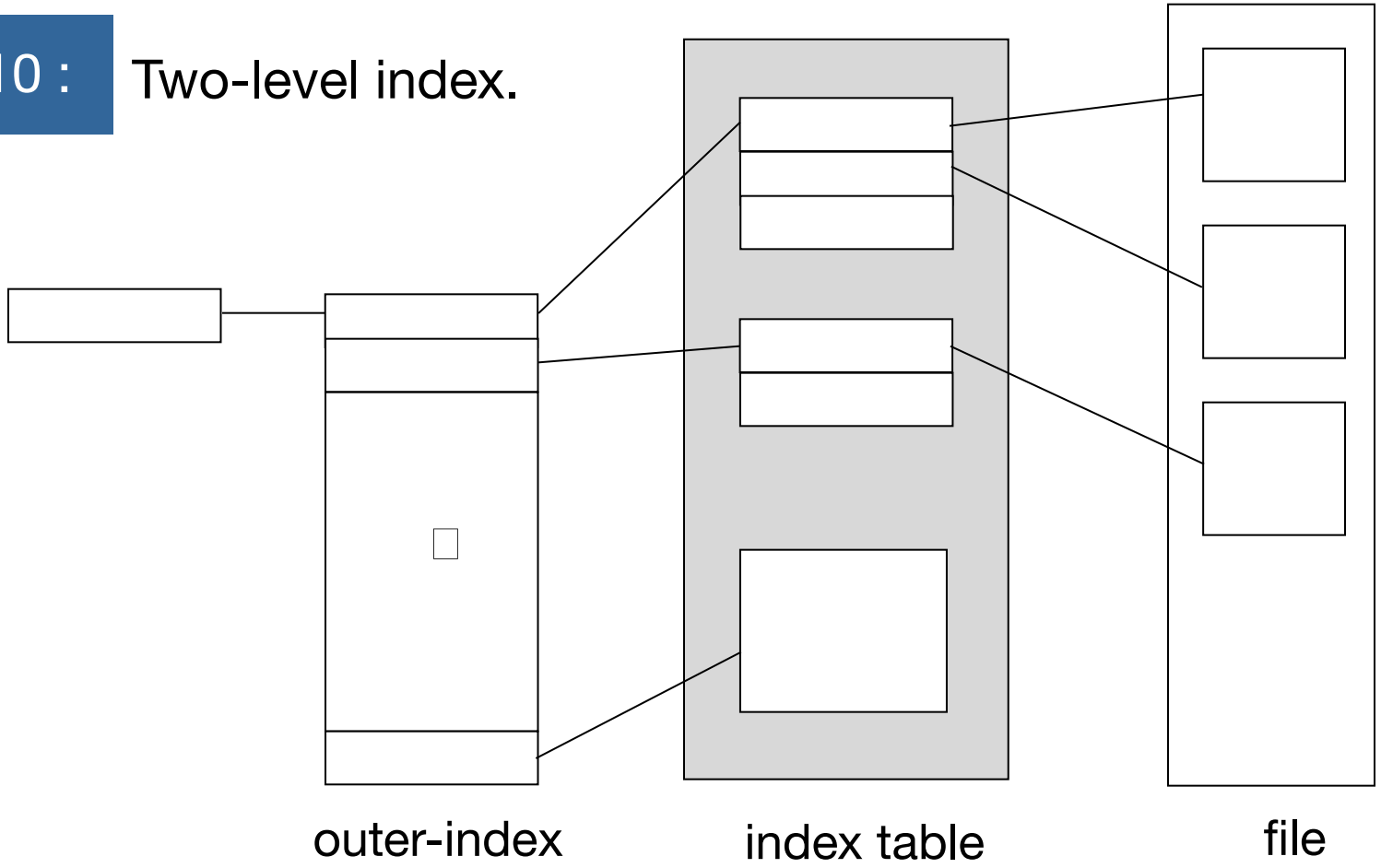
File	Index Block
DATA	9



Allocation Methods

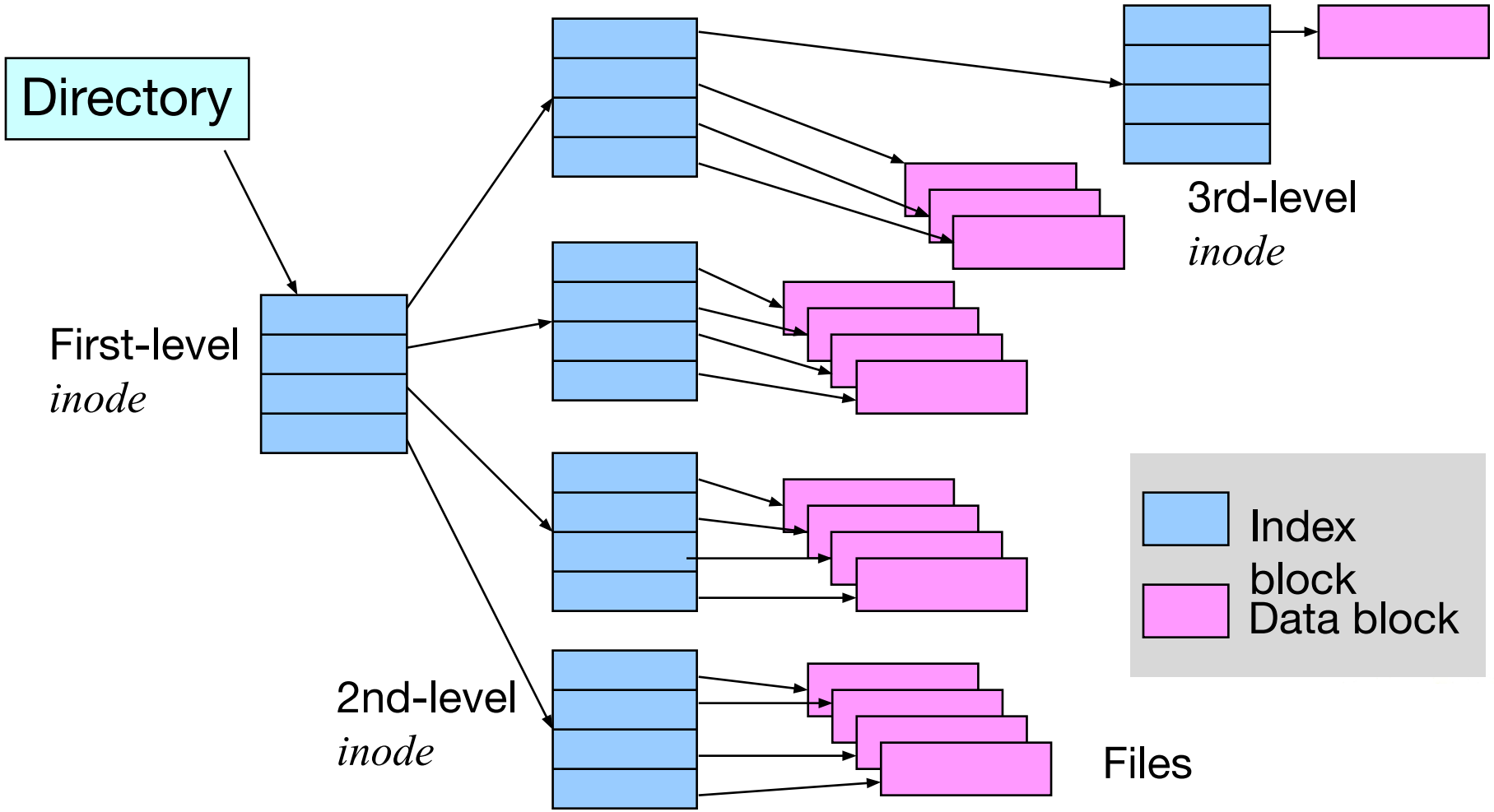
Indexed Allocation: (2) Multilevel index

Example 10 : Two-level index.



Allocation Methods

Example 11 : Multilevel index.



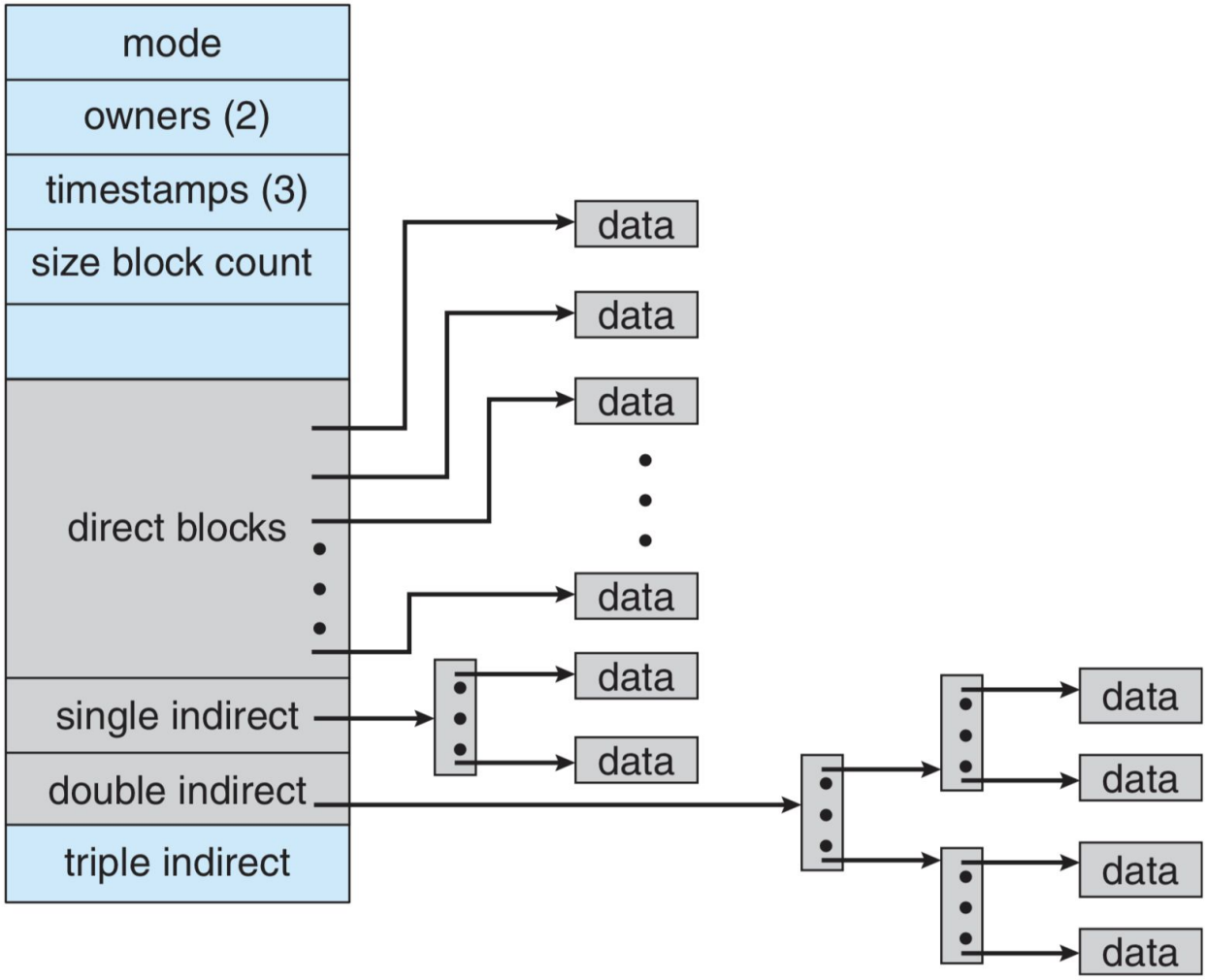
Allocation Methods

Indexed Allocation: (3) Combined scheme

Example 12 :

The UNIX File System (UFS).

- 4K bytes per block, 32-bit addresses.



Allocation Methods

Performance

- Best method depends on file access type.
  - Contiguous great for sequential and random.
- Linked good for sequential, not for random.
- Declare access type at creation select either contiguous or linked.
- Indexed more complex.
  - Single block access could require 2 index block reads then data block read.

11.5

Free-Space Management

Overview

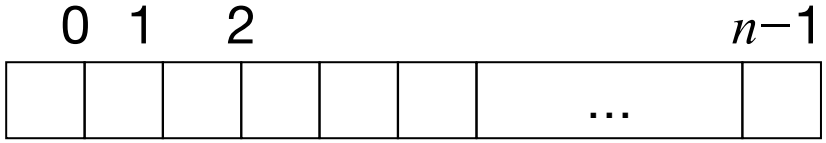
- Since disk space is limited, we need to reuse the space from deleted files for new files, if possible.
- File system maintains **free-space list** to track available **blocks** / clusters.
  - (Using term “block” for simplicity)

11.5

Free-Space Management

(a) Bit map

- **Bit map** or **bit vector** (n blocks)



$$\text{bit}[i] = \begin{cases} 1 & \Rightarrow \text{block } [i] \text{ is free} \\ 0 & \Rightarrow \text{block } [i] \text{ is used} \end{cases}$$

- The calculation of the first free block number.
  - The first non-0 word is scanned for the first 1 bit, which is the location of the first free block.

Free-Space Management

Example 13 : Assume there are 8 bits per word and the bit map is as follows:

Block No.	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	
Bit map :	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0	1	…
Offset :	0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7	0	…

word

First free block

To find the address of the first free block

= (number of bits per word) * (number of 0-value words) + offset of first 1 bit

= (8 * 1) + 4 = 12

Allocation Methods

Exercise 11.1 : Given a bit map value (0 □ used; 1 □ unused/free).

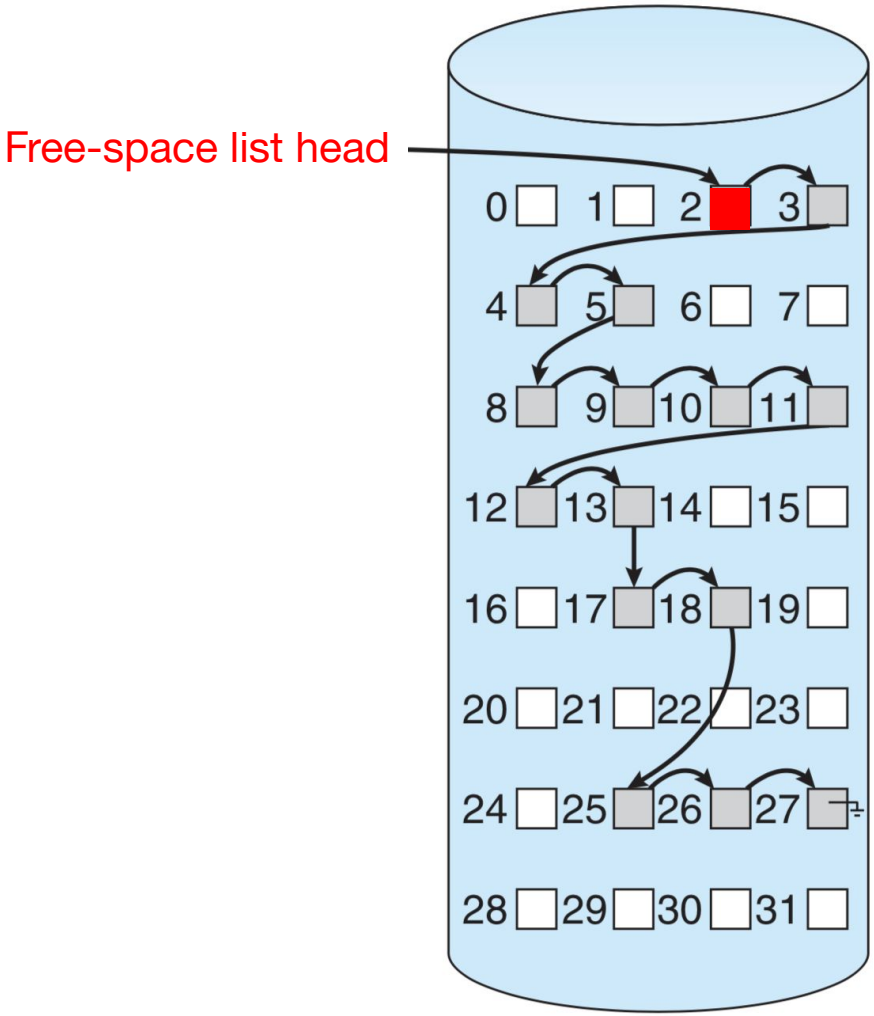
Block No.:	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	…
			1 KB	1 KB					1 KB		1 KB		1 KB				
Bit Map:	0	0	1	1	0	0	0	0	1	0	1	0	1	0	1	0	…

- List the block addresses needed by a file with size 5KB for the different block size of file system.

Case(s)	Block size	Address(es)
1	1 KB	2, 3, 8, 10, 12
2	2 KB	2, 3, 8
3	3 KB	2, 3
4	5 KB	2
5	0.5 KB	2, 3, 8, 10, 12, 14, Another 4 addresses

Free-Space Management

(b) Free list



- Linked list (free list):
  - ✔ Cannot get contiguous space easily.
  - ✔ No waste of space.
  - ✔ No need to traverse the entire list (if # free blocks recorded).

Figure 11.10 Linked free-space list on disk.

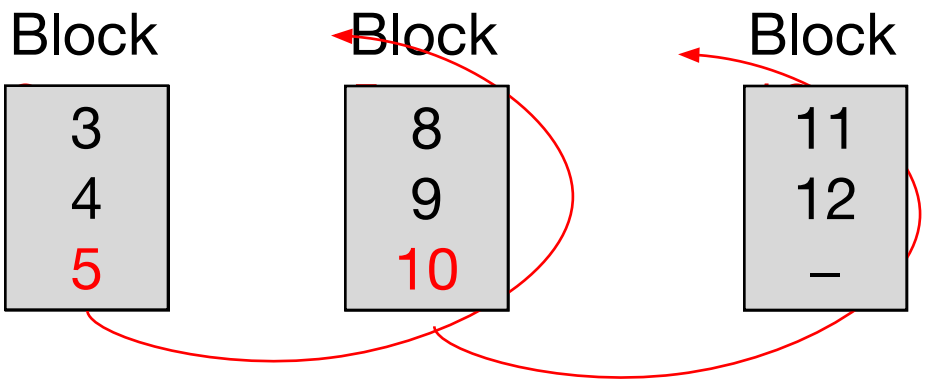
Free-Space Management

(c) Grouping

- Modify **linked list** to store address of next n-1 free blocks in first free block, plus a **pointer** to next block that contains free-block-pointers (like this one)

Example 14 : Free-blocks list □ **2**, 3, 4, **5**, 8, 9, **10**, 11, 12.

Assume each block can only stores 3 block addresses.



Note: block 2, 5 and 10 have become the index block; cannot be used for Data block

Free-Space Management

(d) Counting

- Because space is frequently contiguously used and freed, with contiguous-allocation algorithms, extents, or clustering:
  - Keep address of first free block and count of following free blocks.
  - Free space list then has entries containing addresses and counts.

Recovery

- **Consistency checking** – compares data in directory structure with data blocks on disk, and tries to fix inconsistencies.
  - ✔ Can be slow and sometimes fails.
- Use system programs to **backup** data from disk to another storage device (magnetic tape, other magnetic disk, optical).
- Recover lost file or disk by **restoring** data from backup.

Summary

- The **file system** resides permanently on secondary storage disk.
- Physical disks may be segmented into partitions to control media use and to allow multiple file systems.
- Any file-system type can have different structures and algorithms.
- The various files can be allocated space on the disk in three ways: **contiguous**, **linked**, or **indexed** allocation.

Summary

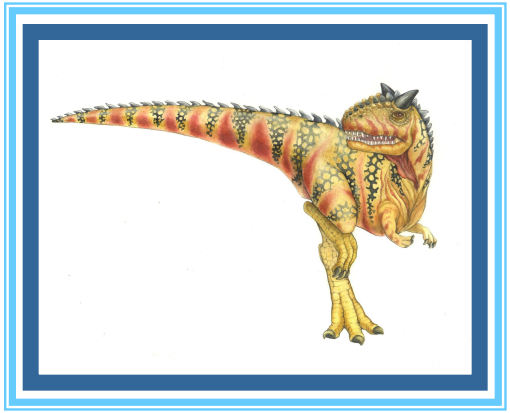
- Free-space allocation methods also influence the efficiency of disk-space use, the performance of the file system, and the reliability of secondary storage. **bit map**, **linked lists**, **grouping**, and **counting**.

Exercises

From Silberchatz, Operating System Concepts
Chapter 11 Exercises (page: 533 - 534)

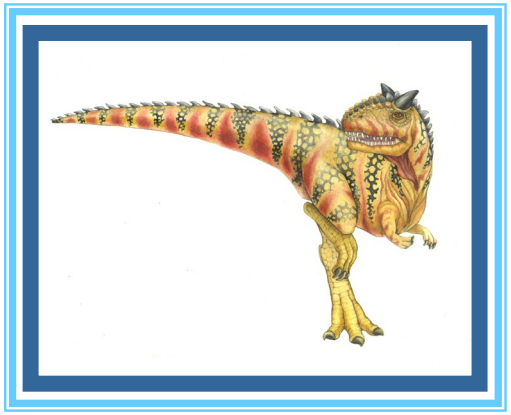
- 11.2
- 11.7
- 11.8
- 11.9
- 11.10

End of Chapter 11



Silberschatz, Galvin and Gagne ©2013

Chapter 7: Deadlocks



Silberschatz, Galvin and Gagne ©2013

Chapter 7: Deadlocks

- 7.1 System Model
- 7.2 Deadlock Characterization
- 7.3 Methods for Handling Deadlocks
- 7.4 Deadlock Prevention
- 7.5 Deadlock Avoidance
- 7.6 Deadlock Detection
- 7.7 Recovery from Deadlock
- 7.8 Summary

Chapter Objectives

- To develop a description of deadlocks, which prevent sets of concurrent processes from completing their tasks.
- To present a number of different methods for preventing or avoiding deadlocks in a computer system.

7.1

System Model

Overview

- System consists of resources
- Resource types R_1, R_2, . . ., R_m
  - *CPU cycles, memory space, I/O devices*
- Each resource type R_i has W_i instances.
  - Eg: *If a system has two CPUs, then the resource type CPU has two instances. Resource type printer may have five instances.*
- Each process utilizes a resource as follows:
  - **request**
  - **use**
  - **release**

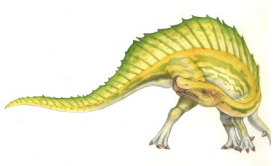
Resources

- Resources can be:
  - a **physical** (eg. *Printers, disk drives*) or
  - a **logical** (eg. *A record in a database, semaphores, monitors*)

- Resources are partitioned into several types,

  $R_1, R_2, \ldots, R_m$

  each consisting of some number of identical instances.



…Resources

- A process may utilize a resource in only the following order:
  - **Request**
    - If the request cannot be granted immediately, then the requesting process must **wait** until it can acquire the resource.
  - **Use**
    - The process can operate on the resource
  - **Release**
    - The process releases the resource

Deadlock

- Many programs competing for limited resources
- **Deadlock** : a state when every process in the set of processes is waiting for an event that can be caused only by another process in the set.

- Example 1:
  - System has 2 CD drives.
  - P_1 and P_2 each hold one CD drive and each needs another one.

System Model

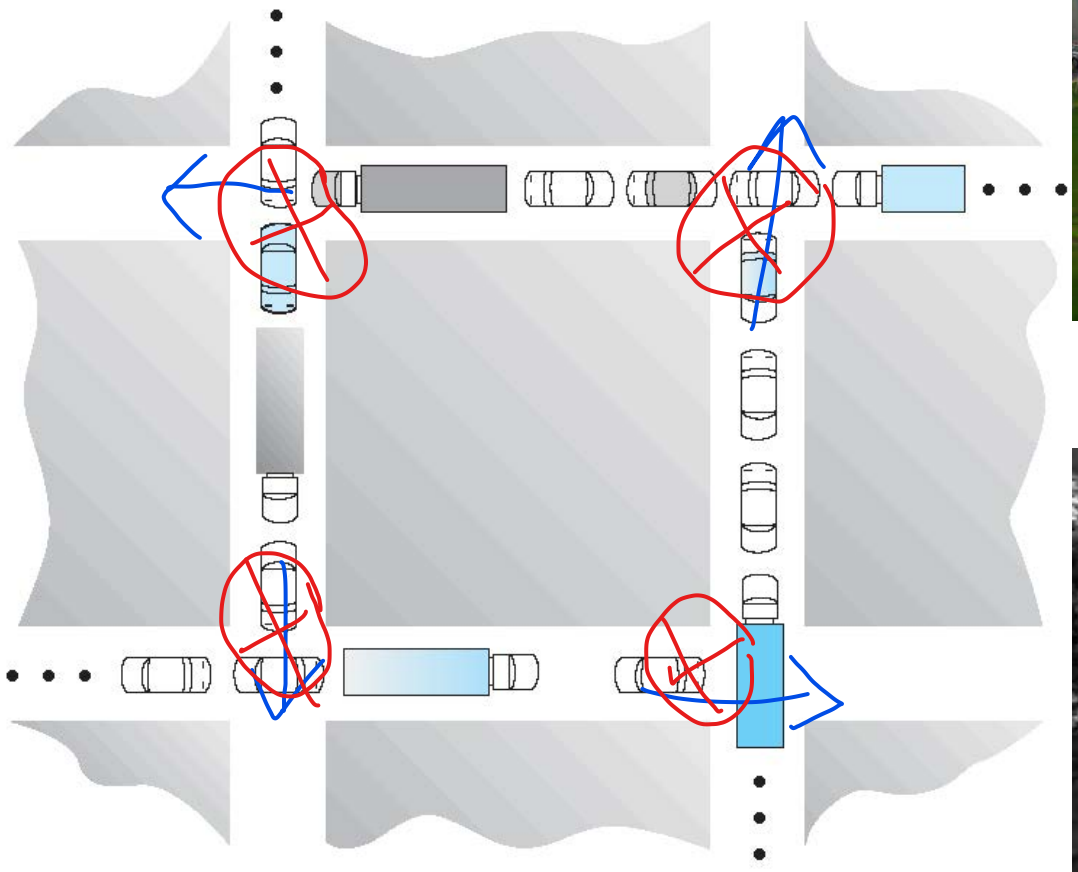


Figure Deadlock example

7.2 Deadlock Characterization

Necessary condition

Deadlock can arise if four conditions hold simultaneously.

- **Mutual exclusion**: only one process at a time can use a resource
- **Hold and wait**: a process holding at least one resource is waiting to acquire additional resources held by other processes
- **No preemption**: a resource can be released only voluntarily by the process holding it, after that process has completed its task
- **Circular wait**: there exists a set $\{P_0, P_1, \ldots, P_n\}$ of waiting processes such that P_0 is waiting for a resource that is held by P_1, P_1 is waiting for a resource that is held by P_2, …, P_{n-1} is waiting for a resource that is held by P_n, and P_n is waiting for a resource that is held by P_0.

7.2 Deadlock Characterization

Resource-Allocation Graph (RAG)

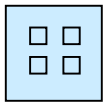
A set of vertices V and a set of edges E.

- V is partitioned into two types:
  - $P = \{P_1, P_2, \ldots, P_n\}$, the set consisting of all the processes in the system
  - $R = \{R_1, R_2, \ldots, R_m\}$, the set consisting of all resource types in the system
- **request edge** – directed edge $P_i \rightarrow R_j$
- **assignment edge** – directed edge $R_j \rightarrow P_i$

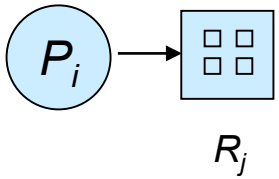
7.2 Deadlock Characterization

- Process

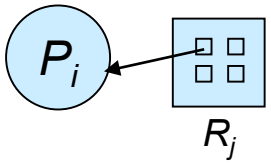
- Resource Type with 4 instances



- P_i requests instance of R_j



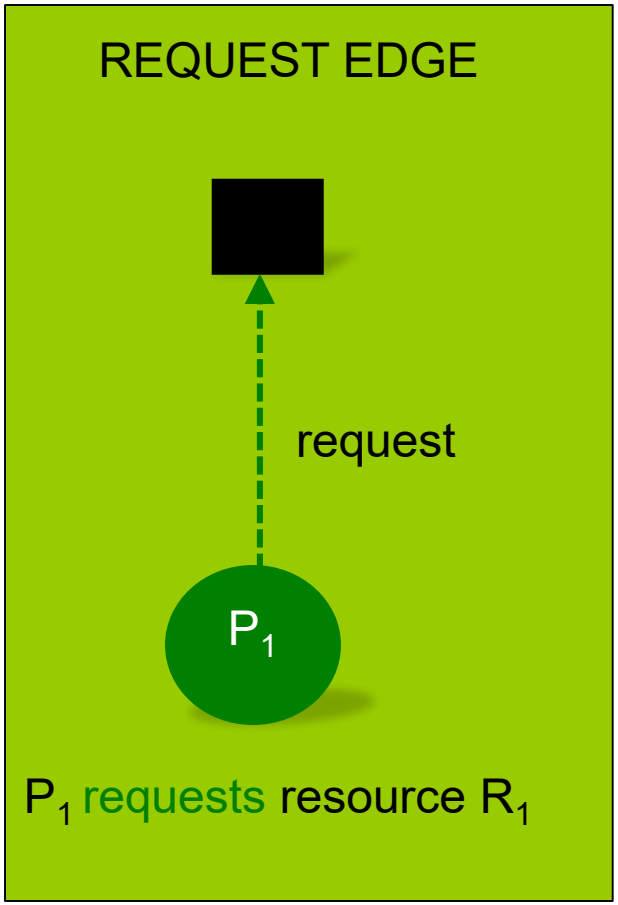
- P_i is holding an instance of R_j

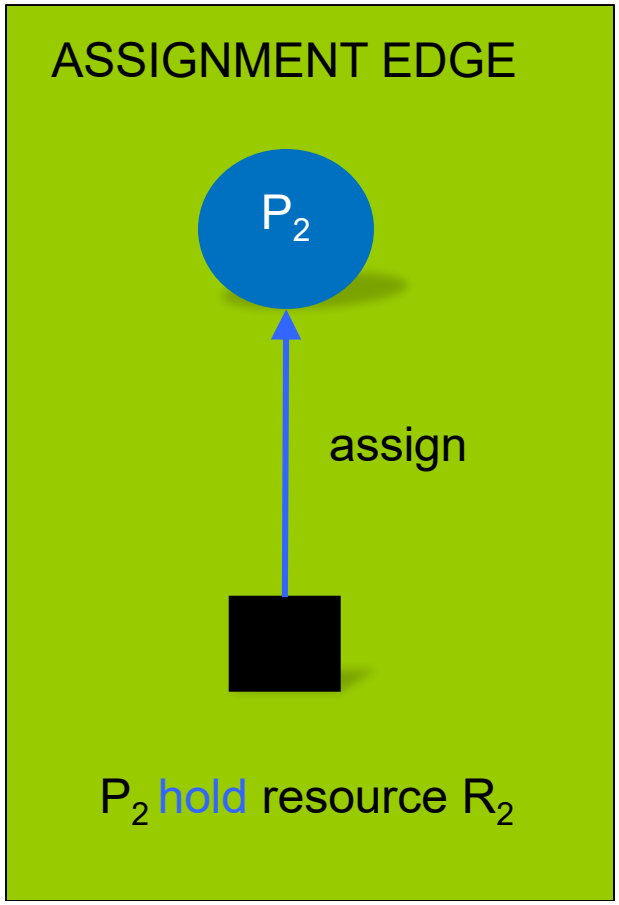


7.1

Deadlock Characterization

- Type of edges, E

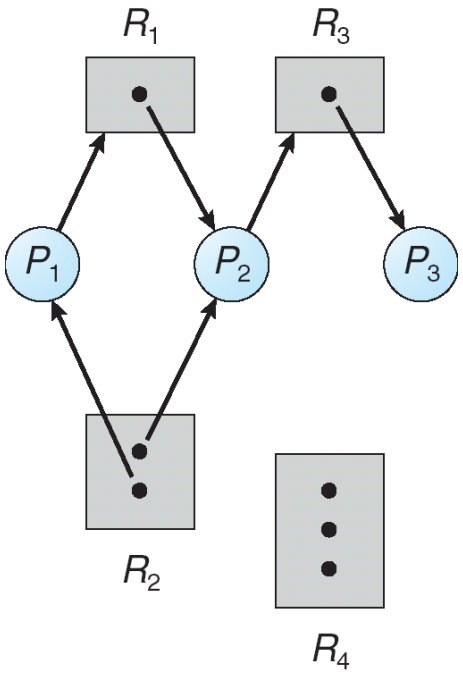




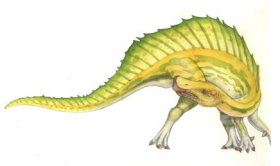
Request edge:
directed edge $P_i \rightarrow R_j$

Assignment edge:
directed edge $R_j \rightarrow P_i$

7.2 Deadlock Characterization

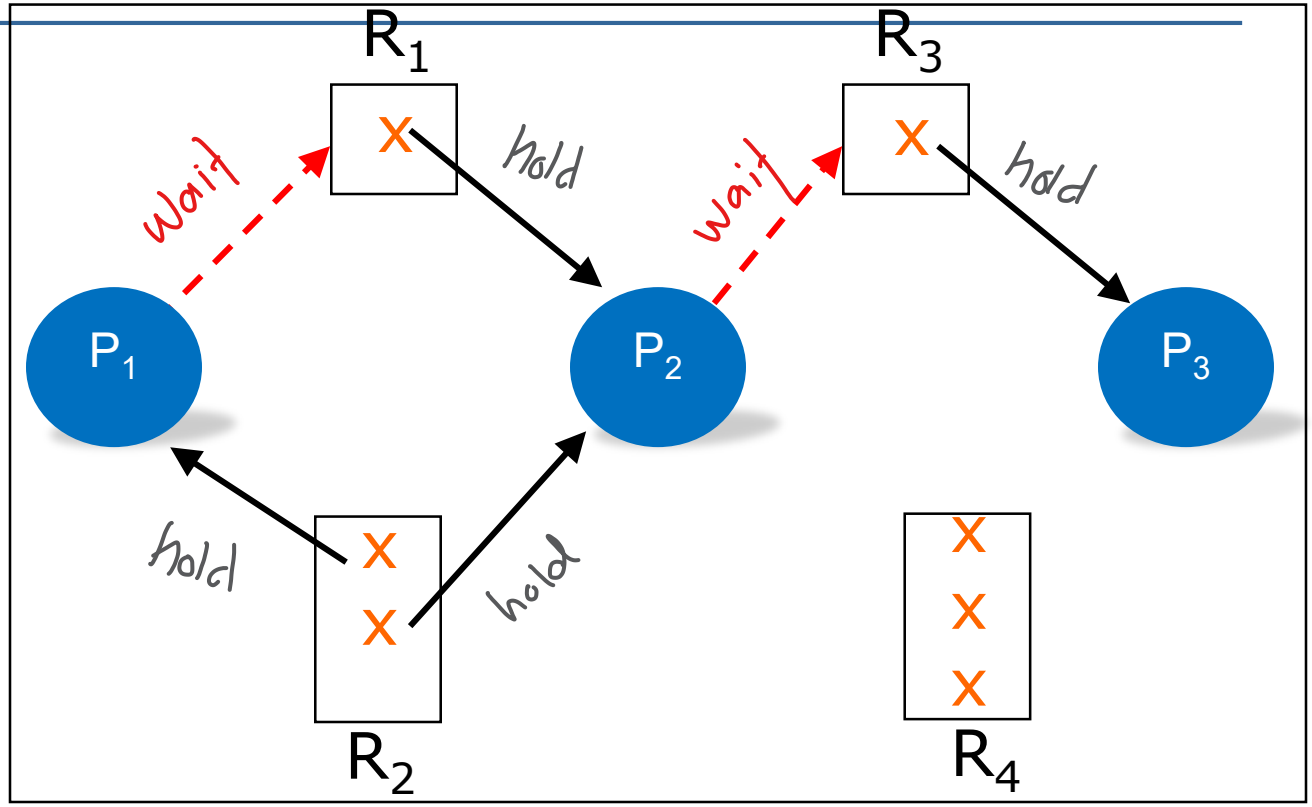


Example of a Resource Allocation Graph



RAG: Example 1

(Several Instances of Resources)



Set **P**, **R** and **E**:

- **P** = {P_1 , P_2 , P_3}
- **R** = {R_1 , R_2 , R_3}
- **E** = {$P_1 \rightarrow R_1$, $P_2 \rightarrow R_3$,
  $R_1 \rightarrow P_2$, $R_2 \rightarrow P_2$,
  $R_2 \rightarrow P_1$, $R_3 \rightarrow P_3$}

Process state:

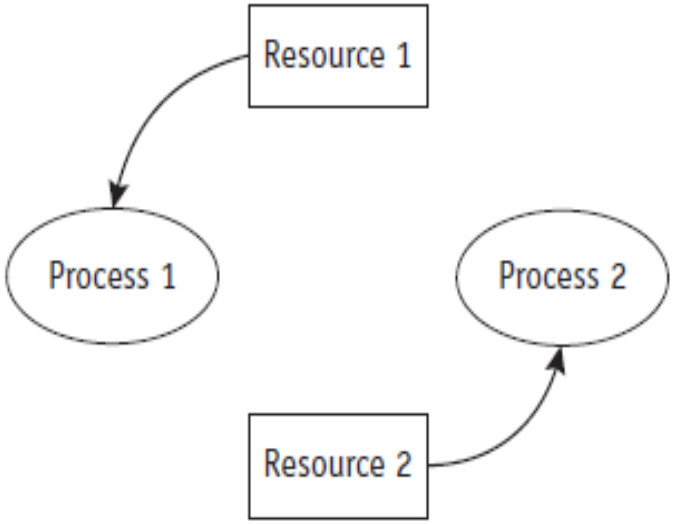
- P_1 holds R_2 and waits for R_1
- P_2 holds R_1 and R_2, and waits for R_3,
- P_3 holds R_3

Deadlock Characterization Basic Facts

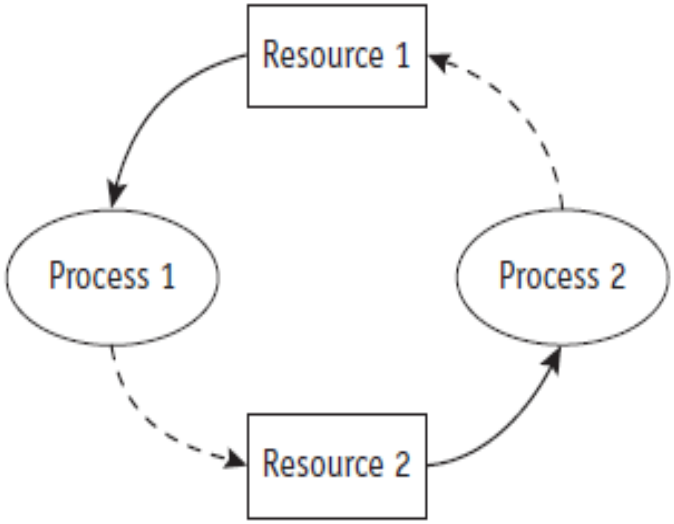
(a) If graph contains no cycles $\Rightarrow$ no deadlock

(b) If graph contains a cycle $\Rightarrow$

- If only one instance per resource type, then deadlock exist
- If several instances per resource type, then possibility of deadlock

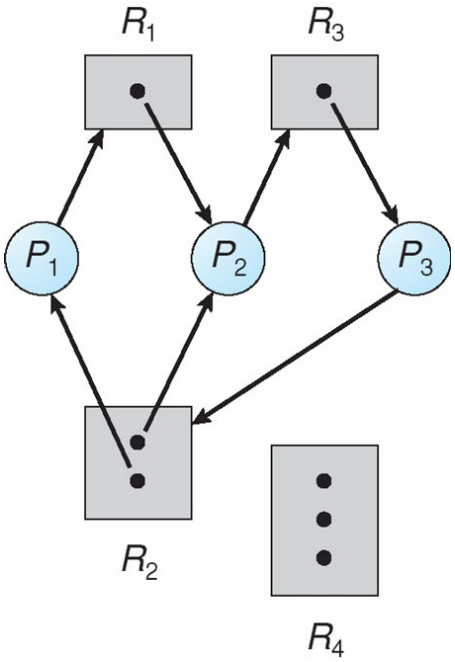


(a)



(b)

7.2 Deadlock Characterization



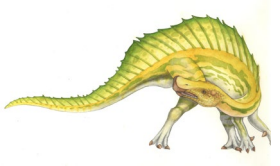
① $P_1 \rightarrow R_1 \rightarrow P_2 \rightarrow R_3 \rightarrow P_3 \rightarrow R_2 \rightarrow P_1$

② $P_2 \rightarrow R_3 \rightarrow P_3 \rightarrow R_2 \rightarrow P_2$.

Deadlock.

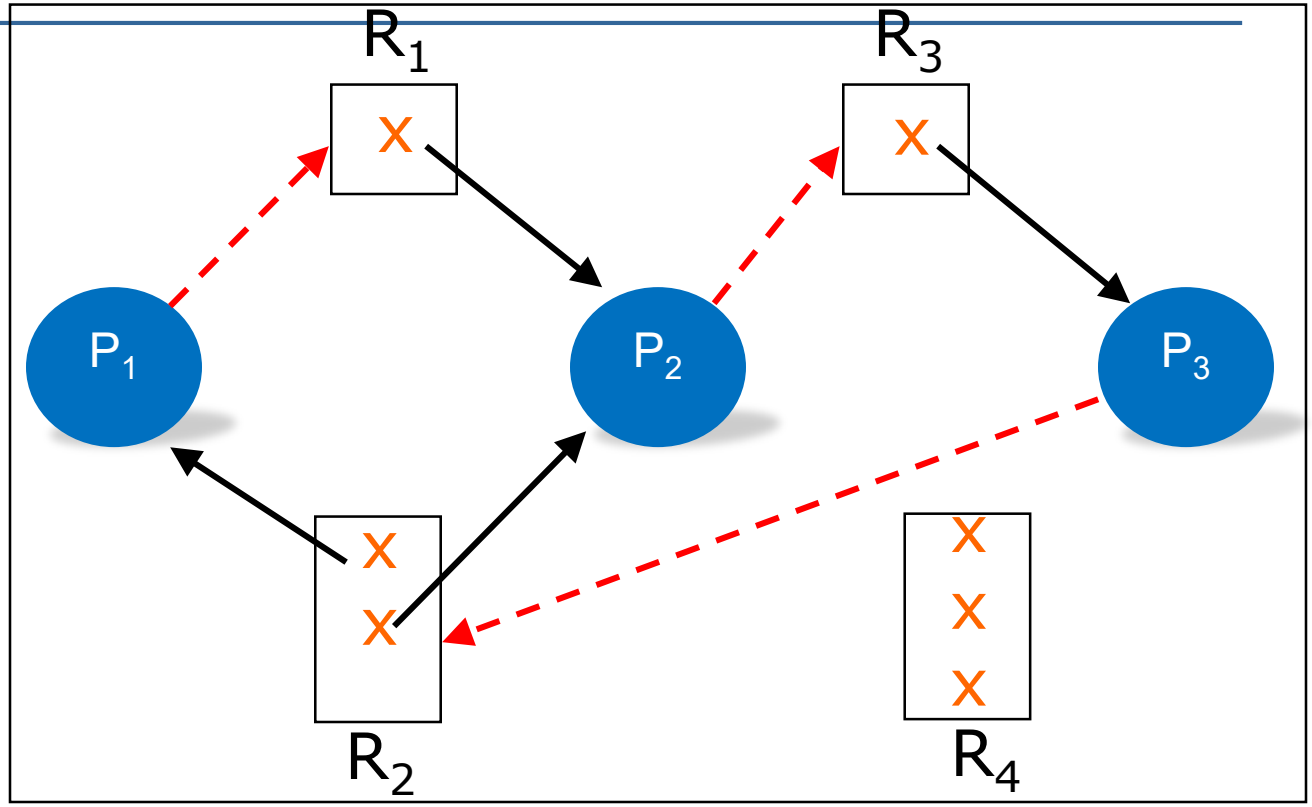
Is there a deadlock?

Resource Allocation Graph With a Cycle



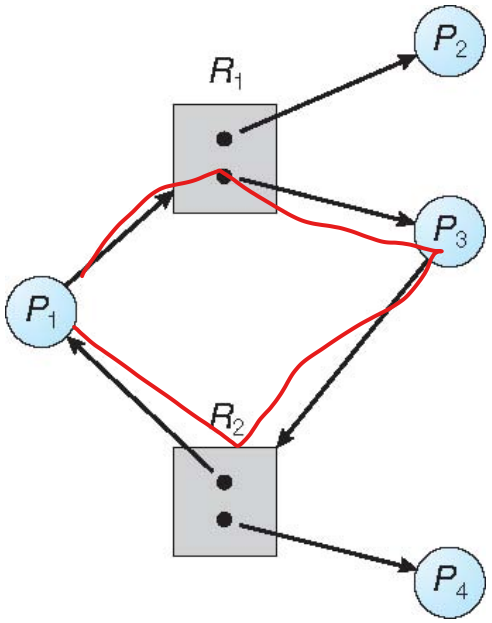
RAG: Example 2

(Several Instances of Resources)



- There exists 2 cycles:
  - $P_1 \rightarrow R_1 \rightarrow P_2 \rightarrow R_3 \rightarrow P_3 \rightarrow R_2 \rightarrow P_1$
  - $P_2 \rightarrow R_3 \rightarrow P_3 \rightarrow R_2 \rightarrow P_2$
- P_1 , P_2 and P_3 are in a deadlock state

7.2 Deadlock Characterization



$P_1 \rightarrow R_1 \rightarrow P_3 \rightarrow R_2 \rightarrow P_1$

Deadlock.

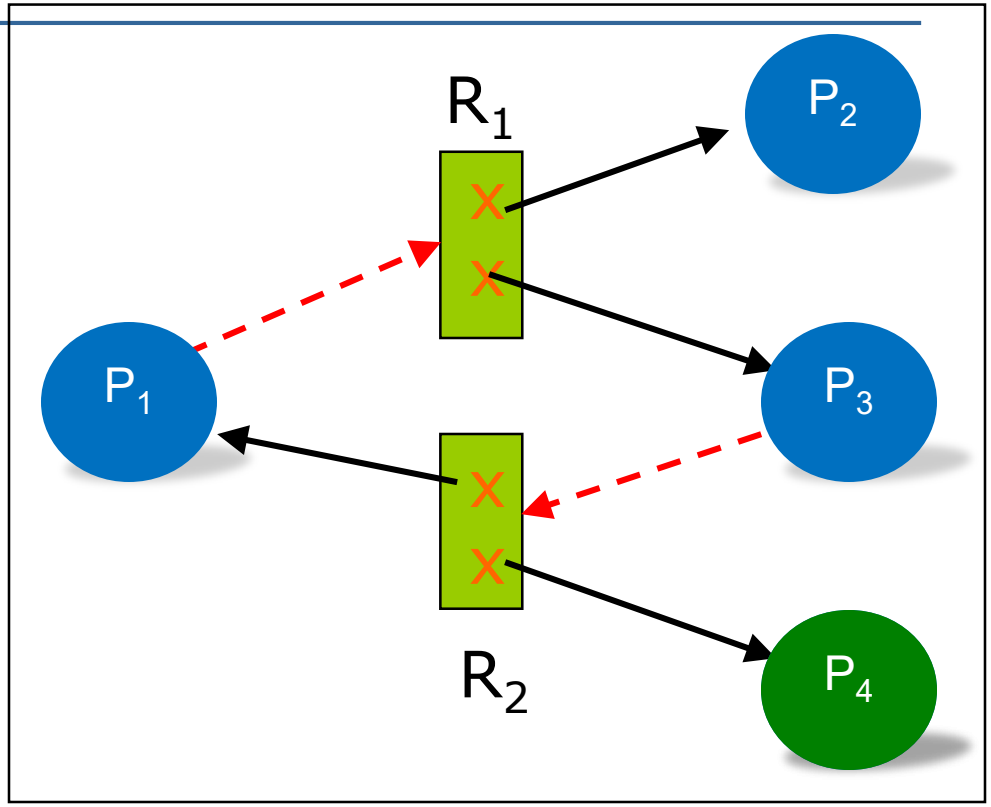
Is there a deadlock?

P_1 and P_3 are in a deadlock state.

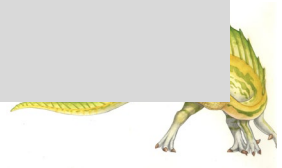
Resource Allocation Graph With a Cycle

RAG: Example 3

- However, the system is not deadlock:
  - when P_4 releases R_2, this resource can be assigned to P_3

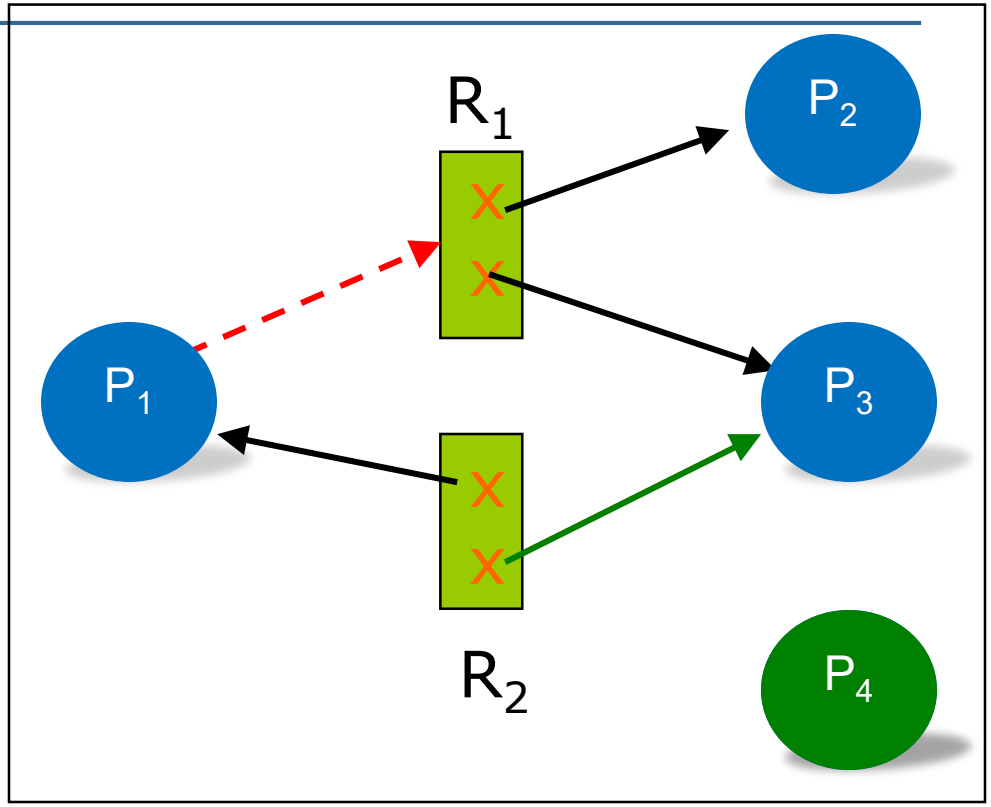


- The RAG has one cycle: $P_1 \rightarrow R_1 \rightarrow P_3 \rightarrow R_2 \rightarrow P_1$
- P_1 and P_3 are in a deadlock state



RAG: Example 3

- However, the system is not deadlock:
  - when P_4 releases R_2, this resource can be assigned to P_3



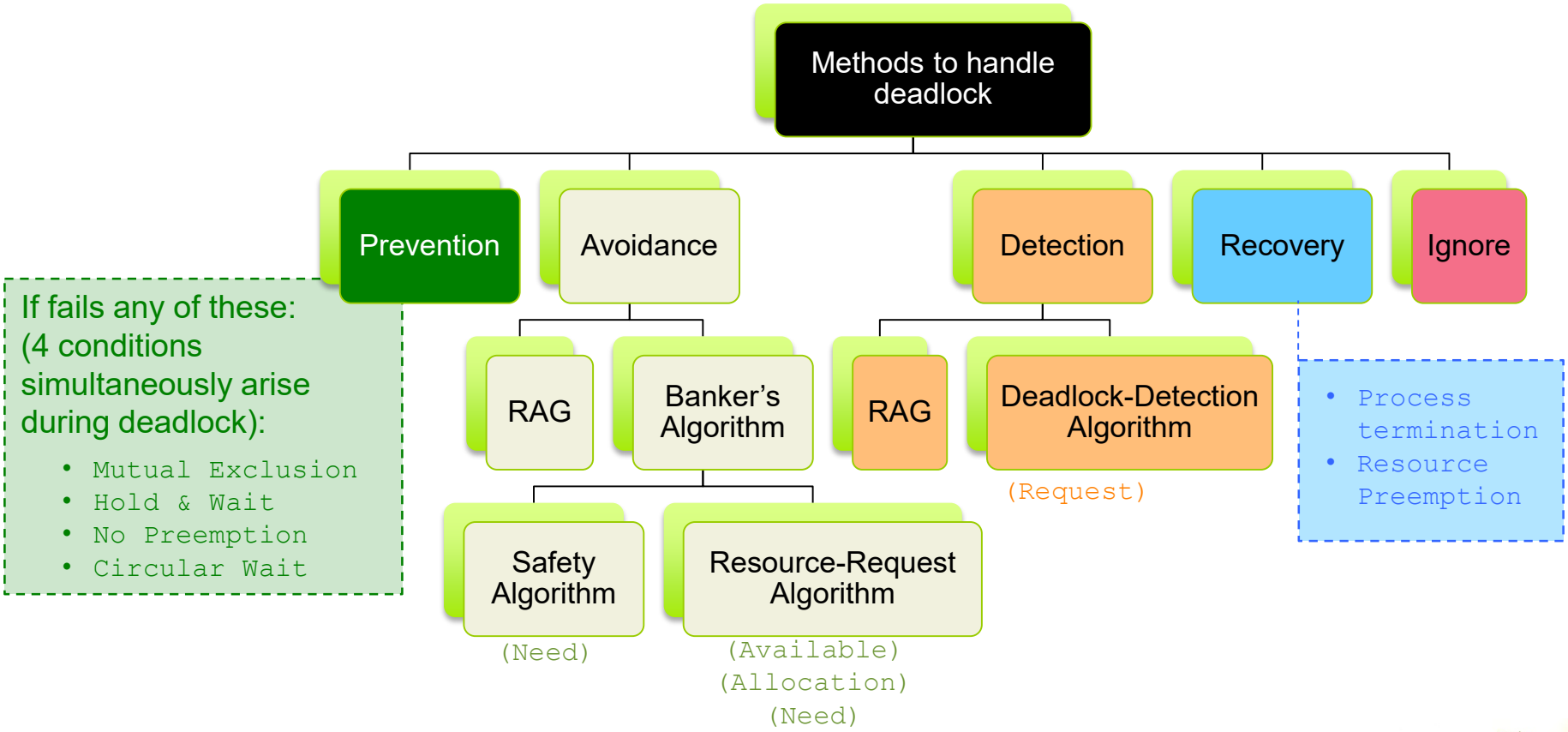
No more cycle:

7.3 Methods for Handling Deadlocks

- 1. Ensure that the system will ***never*** enter a deadlock state:
  - Deadlock prevention
  - Deadlock avoidance
- 2. Allow the system to enter a deadlock state and then recover
- 3. Ignore the problem and pretend that deadlocks never occur in the system; used by most operating systems, including UNIX

假裝

7.3 Methods for Handling Deadlocks



7.4 Deadlock Prevention

- Ensure that at least one of the necessary condition for deadlocks does not hold. Can be accomplished restraining the ways request can be made 实现办法

  - **Mutual Exclusion** – Must hold for non-sharable resources that can be accessed simultaneously by various processes. Therefore cannot be used for prevention.
  - **Hold and Wait** – must guarantee that whenever a process requests a resource, it does not hold any other resources
    - Require process to request and be allocated all its resources before it begins execution, or allow process to request resources only when the process has none allocated to it.
    - Low resource utilization; starvation possible

7.4 Deadlock Prevention

- **No Preemption** – not practical for most systems
  - If a process that is holding some resources requests another resource that cannot be immediately allocated to it, then all resources currently being held are released
  - Preempted resources are added to the list of resources for which the process is waiting
  - Process will be restarted only when it can regain its old resources, as well as the new ones that it is requesting
- **Circular Wait** – impose a total ordering of all resource types, and require that each process requests resources in an increasing order of enumeration. Can be used in practice.

7.4 Deadlock Prevention

- Two bank transactions 1 and 2 execute concurrently.
  - Transaction 1 transfers $25 from account A to account B
  - Transaction 2 transfers $50 from account B to account A
- Program

```
void transaction(Account from,
Account to, double amount)
{
   mutex lock1, lock2;
   lock1 = get_lock(from);
   lock2 = get_lock(to);
   acquire(lock1);
      acquire(lock2);
         withdraw(from, amount);
         deposit(to, amount);
      release(lock2);
   release(lock1);
}
```

Deadlock Example with Lock Ordering

7.5 Deadlock Avoidance

- Ensure that the system will ***never*** enter a deadlock state
  - What is a trivial way of avoiding deadlocks?
- Requires that the system to have some additional ***a priori*** information available on possible resource requests.
  - Simplest and most useful model requires that each process declare the ***maximum number*** of resources of each type that it may need
  - The deadlock-avoidance algorithm dynamically examines the resource-allocation state to ensure that there can never be a circular-wait condition
  - Resource-allocation *state* is defined by the number of available and allocated resources, and the maximum demands of the processes

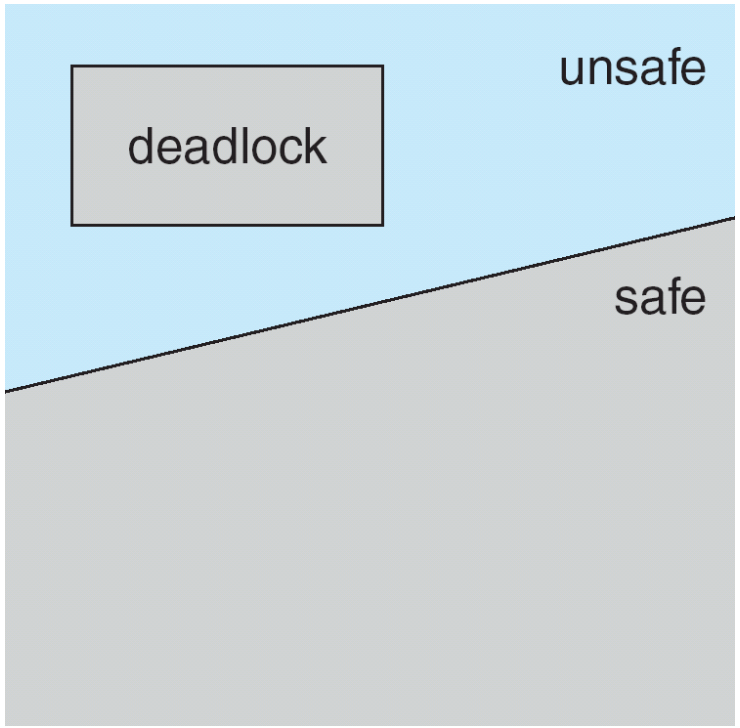
7.5 Deadlock Avoidance

Safe state

- System is in **safe state** if there exists a sequence $<P_1, P_2, \ldots, P_n>$ of ALL the processes in the systems such that for each P_i the resources that P_i can still request can be satisfied by currently available resources + resources held by all the P_j, with $j < i$
- That is:
  - If P_i resource needs are not immediately available, then P_i can wait until all processes P_j ($j < i$) have finished executing.
  - When they have finished executing they release all their resources and then P_i can obtain the needed resources, execute, return allocated resources, and terminate
  - When P_i terminates, P_{i+1} can obtain its needed resources, and so on

7.5 Deadlock Avoidance – Safe State

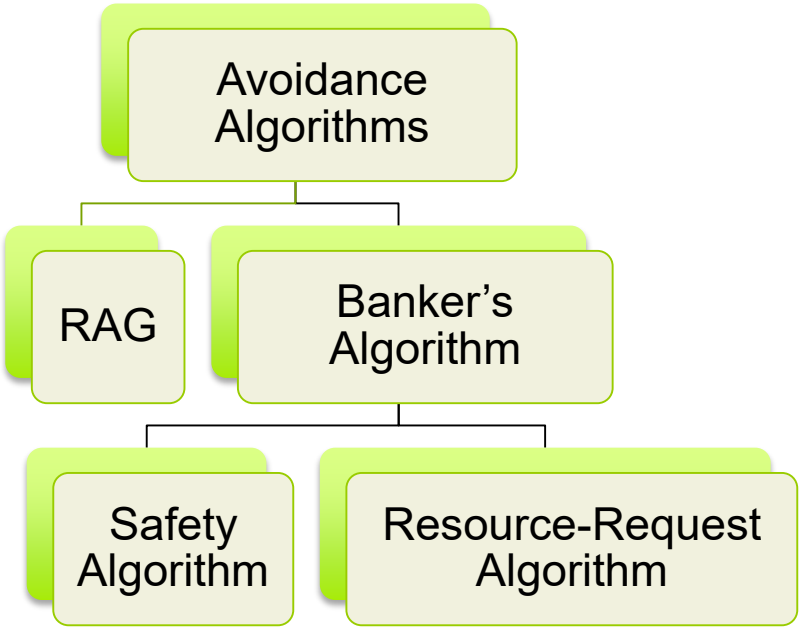
- System is in **safe state** if the system can allocate resources to each process (up to maximum) in some order and still avoid the deadlock.
- If a system is in safe state ⇒ no deadlocks
- If a system is in unsafe state ⇒ possibility of deadlock



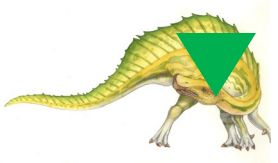
Basic Facts

Deadlock Avoidance

Algorithms



- All processes can be started any time.
- When a process requests for a resource, an **avoidance algorithm** is executed to check whether a deadlock may occur in future.
- If deadlock may occur, the **request is delayed**.



7.5 Deadlock Avoidance Algorithms

- Deadlock avoidance ⇒ ensure that a system will never enter an unsafe state.

① Single instance of a resource type
  - Use a variant of the resource-allocation graph

② Multiple instances of a resource type
  - Use the banker's algorithm

7.5 Deadlock Avoidance

(1) Resource Allocation Graph (RAG) scheme

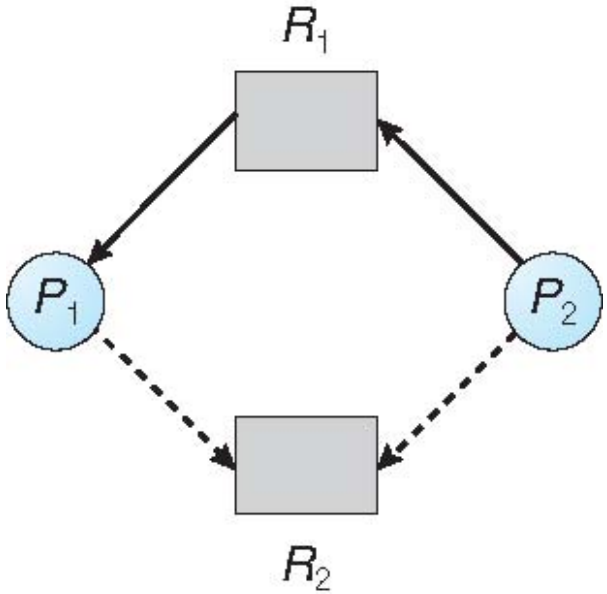
- Single instance of a resource type
- Each process must a priori claim maximum resource use
- Use a variant of the resource-allocation graph with claim edges.
- **Claim edge** $P_i \rightarrow R_j$ indicated that process P_j may request resource R_j; represented by a dashed line
- Claim edge converts to request edge when a process requests a resource
- Request edge converted to an assignment edge when the resource is allocated to the process
- When a resource is released by a process, assignment edge reconverts to a claim edge
- Resources must be claimed *a priori* in the system
- A cycle in the graph implies that the system is in unsafe state

Deadlock Avoidance

(1) Resource Allocation Graph (RAG) scheme

- P1 is holding resource R1 and has a claim on R2
- P2 is requesting R1 and has a claim on R2
- No cycle. So system is in a safe state.

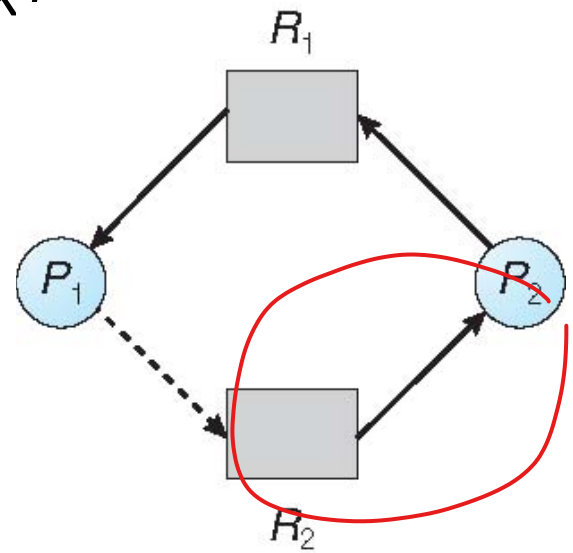
Resource-Allocation Graph with claim edges



Deadlock Avoidance

(1) Resource Allocation Graph (RAG) scheme

- The graph of slide before with a claim edge from P2 to R2 is changing to an assignment edge.
- There is a cycle in the graph ➔ unsafe state.
- Is there a deadlock?



Example of a Resource Allocation Graph

7.5 Deadlock Avoidance

(1) Resource Allocation Graph (RAG) scheme

- Suppose that process P_i requests a resource R_j
- The request can be granted only if converting the request edge to an assignment edge does not result in the formation of a cycle in the resource allocation graph
- Otherwise, the process must wait

7.5

Deadlock Avoidance

(2) Banker's algorithm

- Multiple instances of a resource type
- Each process must a priori claim maximum use
- When a process requests a resource it may have to wait
- When a process gets all its resources it must return them in a finite amount of time.

"The algorithm could be used in a banking system to ensure that the bank never allocated its available cash in such a way that it could no longer satisfy the needs of all its customers"

Deadlock Avoidance

(2) Banker's algorithm

Example 1: Bank Loan

- Bank with fixed amount of capital which operate as follows:
  - No customer will be granted a loan exceeding the bank's total capital
  - All customers will be given **maximum credit limit** when opening an account
  - The sum of all loan must not exceed the bank's total capital

7.5

Deadlock Avoidance

(2) Banker's algorithm

. . . Example 1a

- The bank total capital fund is RM100,000 and has 3 customers C_1, C_2, and C_3 each with credit limits RM40,000, RM 50,000, and RM80,000.

Customer	Loan Amount	Max Credit	Remaining Credit
C_1	0	40,000	40,000
C_2	20,000	50,000	30,000
C_3	40,000	80,000	40,000
Total loan	60,000		
Total capital fund	100,000	Available fund	40,000

- → safe state the bank has enough money left to satisfy customer's need (remaining credit)

Deadlock Avoidance

(2) Banker's algorithm

. . . Example 1b

- C_1 and C_2 made a loan of RM20,000 and RM10,000

Customer	Loan Amount	Max Credit	Remaining Credit
C_1	20,000	40,000	20,000
C_2	30,000	50,000	20,000
C_3	40,000	80,000	40,000
Total loan	90,000		
Total capital fund	100,000	Available fund	10,000

- → unsafe state, the bank doesn't have enough money left to satisfy customer's need (available fund < min remaining credit)

Deadlock Avoidance

(2) Banker's algorithm

Data structures of Banker's Algorithm

Let n = number of processes, and m = number of resources types.

- **Available**: Vector of length m.
  - If available [j] = k, then there are k instances of resource type R_j available
- **Max**: n x m matrix.
  - If Max [i,j] = k, then process P_i may request at most k instances of resource type R_j
- **Allocation**: n x m matrix.
  - If Allocation[i,j] = k then P_i is currently allocated k instances of R_j
- **Need**: n x m matrix.
  - If $Need$[i,j] = k, then P_i may need at most k more instances of R_j to complete its task.
    - $Need$ [i,j] = Max[i,j] – $Allocation$ [i,j]

Deadlock Avoidance

2a Safety algorithm

1. Let ***Work*** and ***Finish*** be vectors of length *m* and *n*, respectively. Initialize:

   Work* = *Available

   Finish* [*i*] = *false* for *i = 0, 1, …, n- 1

2. Find an ***i*** such that both:

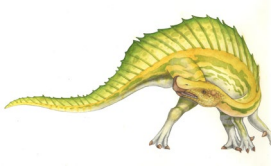
   (a) ***Finish* [*i*] = *false***

   (b) $Need_i \leq Work$

   If no such ***i*** exists, go to step 4

3. $Work = Work + Allocation_i$
   Finish*[*i*] = *true
   go to step 2

4. If ***Finish* [*i*] == *true*** for all ***i***, then the system is in a safe state. Otherwise, in an unsafe state.



Example of Safety Algorithm - 1

- 5 processes -- P_0 ….. P_4;
- 3 resource types:

  A (10 instances), B (5 instances), and C (7 instances)
- Snapshot at time T_0:

	Allocation	*Max*	*Available*
	A B C	*A B C*	*A B C*
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

Example of Safety Algorithm - 2

- The content of the matrix ***Need*** is defined to be ***Max – Allocation***

	Allocation	*Need*	*Available*
	A B C	*A B C*	*A B C*
P_0	0 1 0	7 4 3	3 3 2
P_1	2 0 0	1 2 2	
P_2	3 0 2	6 0 0	
P_3	2 1 1	0 1 1	
P_4	0 0 2	4 3 1	

Example of Safety Algorithm - 3

Step 1: **Work** = **Available** = (3, 3, 2)

Step 2: { **Need**(P_0) ≤ **Work** }

Step 2: { **Need**(P_1) ≤ **Work** }
{ (1, 2, 2) ≤ (3, 3, 2) }

Step 3: **Work** = **Work** + **Allocation**(P_1)
= (3, 3, 2) + (2, 0, 0)
= (5, 3, 2)

Step 2: { **Need**(P_2) ≤ **Work** }

Step 3: { **Need**(P_3) ≤ **Work** }
{ (0, 1, 1) ≤ (5, 3, 2) }
Work = **Work** + **Allocation**(P_3)
= (5, 3, 2) + (2, 1, 1)
= (7, 4, 3)

(……Repeat for all other processes)

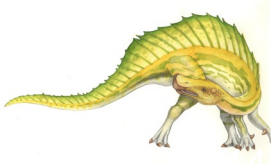
	Allocation			Max			Need		
	A	B	C	A	B	C	A	B	C
P_0	0	1	0	7	5	3	7	4	3
P_1	2	0	0	3	2	2	1	2	2
P_2	3	0	2	9	0	2	6	0	0
P_3	2	1	1	2	2	2	0	1	1
P_4	0	0	2	4	3	3	4	3	1

2, 4, 3

P_1 < Work

(1, 2, 3) ≤ (2, 4, 3)

2 4 3 + 1 1 0
3 5 3



Example of Safety Algorithm - 4

Step 2: { **Need**(P_4) ≤ **Work** }
{ (4, 3, 1) ≤ (7, 4, 3) }
Step 3: **Work** = **Work** + **Allocation**(P_4)
= (7, 4, 3) + (0, 0, 2)
= (7, 4, 5)

Step 2: { **Need**(P_2) ≤ **Work** }
{ (6, 0, 0) ≤ (7, 4, 5) }
Step 3: **Work** = **Work** + **Allocation**(P_2)
= (7, 4, 5) + (3, 0, 2)
= (10, 4, 7)

Step 2: { **Need**(P_0) ≤ **Work** }
{ (7, 4, 3) ≤ (10, 4, 7) }
Step 3: **Work** = **Work** + **Allocation**(P_0)
= (10, 4, 7) + (0, 1, 0)
= (10, 5, 7)

	Allocation			Max			Need		
	A	B	C	A	B	C	A	B	C
P_0	0	1	0	7	5	3	7	4	3
P_1	2	0	0	3	2	2	1	2	2
P_2	3	0	2	9	0	2	6	0	0
P_3	2	1	1	2	2	2	0	1	1
P_4	0	0	2	4	3	3	4	3	1

What is the safe sequences produced?

- The system is in a safe state since the sequence < P_1, P_3, P_4, P_2, P_0> satisfies safety criteria

Deadlock Avoidance

2b Resource-Request algorithm

Let $Request_i$ [...] be the request vector for process P_i.

$Request_i[j] = k$. Process P_i wants k instances of resource type R_j

1. If $Request_i \le Need_i$ go to step 2. Otherwise, raise error condition, since process has exceeded its maximum claim
2. If $Request_i \le Available$, go to step 3. Otherwise P_i must wait, since resources are not available
3. Pretend to allocate requested resources to P_i by modifying the state as follows:

$$Available = Available - Request_i$$

$$Allocation_i = Allocation_i + Request_i$$

$$Need_i = Need_i - Request_i$$

- If safe $\Rightarrow$ the resources are allocated to P_i
- If unsafe $\Rightarrow$ P_i must wait, and the old resource-allocation state is restored

Example Resource-Request Algorithm - 1

- 5 processes P_0 through P_4;

  3 resource types:

  A (10 instances), B (5 instances), and C (7 instances)

- Snapshot at time T_0:

	Allocation	*Max*	*Available*
	A B C	*A B C*	*A B C*
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

Example Resource-Request Algorithm - 2 (Determine Safe State)

Step 1: **Work** = **Available** = (3, 3, 2)

Step 2: { **Need**(P_0) ≤ **Work** }

Step 2: { **Need**(P_1) ≤ **Work** }

{ (1, 2, 2) ≤ (3, 3, 2) }

Step 3: **Work** = **Work** + **Allocation**(P_1)

= (3, 3, 2) + (2, 0, 0)

= (5, 3, 2)

Step 2: { **Need**(P_2) ≤ **Work** }

Step 3: { **Need**(P_3) ≤ **Work** }

{ (0, 1, 1) ≤ (5, 3, 2) }

Work = **Work** + **Allocation**(P_3)

= (5, 3, 2) + (2, 1, 1)

= (7, 4, 3)

(……Repeat for all other processes)

	Allocation			Max			Need		
	A	B	C	A	B	C	A	B	C
P_0	0	1	0	7	5	3	7	4	3
P_1	2	0	0	3	2	2	1	2	2
P_2	3	0	2	9	0	2	6	0	0
P_3	2	1	1	2	2	2	0	1	1
P_4	0	0	2	4	3	3	4	3	1

Example Resource-Request Algorithm – 3 (Determine Safe State)

Step 2: { **Need**(P_4) ≤ **Work** }
{ (4, 3, 1) ≤ (7, 4, 3) }
Step 3: **Work** = **Work** + **Allocation**(P_4)
= (7, 4, 3) + (0, 0, 2)
= (7, 4, 5)

Step 2: { **Need**(P_2) ≤ **Work** }
{ (6, 0, 0) ≤ (7, 4, 5) }
Step 3: **Work** = **Work** + **Allocation**(P_2)
= (7, 4, 5) + (3, 0, 2)
= (10, 4, 7)

Step 2: { **Need**(P_0) ≤ **Work** }
{ (7, 4, 3) ≤ (10, 4, 7) }
Step 3: **Work** = **Work** + **Allocation**(P_0)
= (10, 4, 7) + (0, 1, 0)
= (10, 5, 7)

	Allocation			Max			Need		
	A	B	C	A	B	C	A	B	C
P_0	0	1	0	7	5	3	7	4	3
P_1	2	0	0	3	2	2	1	2	2
P_2	3	0	2	9	0	2	6	0	0
P_3	2	1	1	2	2	2	0	1	1
P_4	0	0	2	4	3	3	4	3	1

What is the safe sequences produced?

- **The system is in a safe state since the sequence < P_1, P_3, P_4, P_2, P_0> satisfies safety criteria – Get using Safety Algorithm**

Example Resource-Request Algorithm - 4

Example: P_1 Request (1,0,2)

- Check that Request ≤ Available (that is, (1,0,2) ≤ (3,3,2) ⇒ true
- Since **Request$_1$ ≤ Need$_i$** & **Request$_1$ ≤ Available**, pretend this request has been fulfilled → execute *Resource-Request Algorithm*.

Available

= **Available** – **Request**(P_1)

= (3, 3, 2) – (1, 0, 2)

= **(2, 3, 0)**

Allocation(P_1)

= **Allocation**(P_1) + **Request**(P_1)

= (2, 0, 0) + (1, 0, 2)

= **(3, 0, 2)**

	Allocation			Max			Need		
	A	B	C	A	B	C	A	B	C
P_0	0	1	0	7	5	3	7	4	3
P_1	**3**	**0**	**2**	3	2	2	**0**	**2**	**0**
P_2	3	0	2	9	0	2	6	0	0
P_3	2	1	1	2	2	2	0	1	1
P_4	0	0	2	4	3	3	4	3	1

Available		
A	B	C
2	**3**	**0**

Need(P_1) = **Need**(P_1) – **Request**(P_1)

= (1, 2, 2) – (1, 0, 2)

= **(0, 2, 0)**

Example Resource-Request Algorithm - 5

Example: P_1 Request (1,0,2)

Step 1: **Work** = **Available** = (2, 3, 0)

Step 2: { **Need**(P_1) ≤ **Work** }
{ (0, 2, 0) ≤ (2, 3, 0) }

Step 3: **Work** = **Work** + **Allocation**(P_1)
= (2, 3, 0) + (3, 0, 2)
= (5, 3, 2)

Step 2: { **Need**(P_3) ≤ **Work** }
{ (0, 1, 1) ≤ (5, 3, 2) }

Step 3: **Work** = **Work** + **Allocation**(P_3)
= (5, 3, 2) + (2, 1, 1)
= (7, 4, 3)

	Allocation			Max			Need		
	A	B	C	A	B	C	A	B	C
P_0	0	1	0	7	5	3	7	4	3
P_1	**3**	**0**	**2**	3	2	2	**0**	**2**	**0**
P_2	3	0	2	9	0	2	6	0	0
P_3	2	1	1	2	2	2	0	1	1
P_4	0	0	2	4	3	3	4	3	1

Available		
A	B	C
2	**3**	**0**

Example Resource-Request Algorithm - 6
Example: P_1 Request (1,0,2)

Step 2: { **Need**(P_4) ≤ **Work** }
{ (4, 3, 1) ≤ (7, 4, 3) }
Step 3: **Work** = **Work** + **Allocation**(P_4)
= (7, 4, 3) + (0, 0, 2)
= (7, 4, 5)

Step 2: { **Need**(P_0) ≤ **Work** }
{ (7, 4, 3) ≤ (7, 4, 5) }
Step 3: **Work** = **Work** + **Allocation**(P_0)
= (7, 4, 5) + (0, 1, 0)
= (7, 5, 5)

Step 2: { **Need**(P_2) ≤ **Work** }
{ (6, 0, 0) ≤ (7, 5, 5) }
Step 3: **Work** = **Work** + **Allocation**(P_2)
= (7, 5, 5) + (3, 0, 2)
= (10, 5, 7)

	Allocation			Max			Need			Available		
	A	B	C	A	B	C	A	B	C	A	B	C
P_0	0	1	0	7	5	3	7	4	3	**2**	**3**	**0**
P_1	**3**	**0**	**2**	3	2	2	**0**	**2**	**0**			
P_2	3	0	2	9	0	2	6	0	0			
P_3	2	1	1	2	2	2	0	1	1			
P_4	0	0	2	4	3	3	4	3	1			

What is the new safe sequences ?

Executing algorithm shows that sequence < P_1, P_3, P_4, P_0, P_2> satisfies safety requirement

The system can grant the request of process P_1.

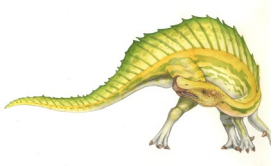
Example Resource-Request Algorithm - 7

Example: P_4 Request (3,3,0)

- Check that Request ≤ Available (that is, (3,3,0) ≤ (2,3,0) ⇒ false

	Allocation			Max			Need			Available		
	A	B	C	A	B	C	A	B	C	A	B	C
P_0	0	1	0	7	5	3	7	4	3	**2**	**3**	**0**
P_1	**3**	**0**	**2**	3	2	2	**0**	**2**	**0**			
P_2	3	0	2	9	0	2	6	0	0			
P_3	2	1	1	2	2	2	0	1	1			
P_4	0	0	2	4	3	3	4	3	1			

- **P_4 cannot be granted, since the resources are not available. $Request_4 \leq Available$ becomes FALSE, this request has been not fulfilled.**



Example Resource-Request Algorithm - 8

Example: P_0 Request (0,2,0)

- Check that Request ≤ Available (that is, (0,2,0) ≤ (2,3,0) ⇒ true
- Since **$Request_0 \leq Need_i$** & **$Request_1 \leq Available$**, pretend this request has been fulfilled → execute *Resource-Request Algorithm*.

Available

= **Available** – **Request**(P_0)

= (2, 3, 0) – (0, 2, 0)

= **(2, 1, 0)**

Allocation(P_0)

= **Allocation**(P_0) + **Request**(P_0)

= (0, 1, 0) + (0, 2, 0)

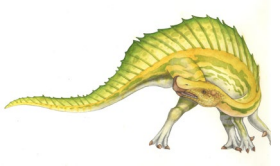
= **(0, 3, 0)**

	Allocation			Max			Need			Available		
	A	B	C	A	B	C	A	B	C	A	B	C
P_0	**0**	**3**	**0**	7	5	3	**7**	**2**	**3**	**2**	**1**	**0**
P_1	**3**	**0**	**2**	3	2	2	**0**	**2**	**0**			
P_2	3	0	2	9	0	2	6	0	0			
P_3	2	1	1	2	2	2	0	1	1			
P_4	0	0	2	4	3	3	4	3	1			

Need(P_0) = **Need**(P_0) – **Request**(P_0)

= (7, 4, 3) – (0, 2, 0)

= **(7, 2, 3)**



Example Resource-Request Algorithm - 9

Example: P_0 Request (0,2,0)

Step 1: **Work** = **Available** = (2, 1, 0)

	Allocation			Max			Need			Available		
	A	B	C	A	B	C	A	B	C	A	B	C
P_0	**0**	**3**	**0**	7	5	3	**7**	**2**	**3**	**2**	**1**	**0**
P_1	**3**	**0**	**2**	3	2	2	**0**	**2**	**0**			
P_2	3	0	2	9	0	2	6	0	0			
P_3	2	1	1	2	2	2	0	1	1			
P_4	0	0	2	4	3	3	4	3	1			

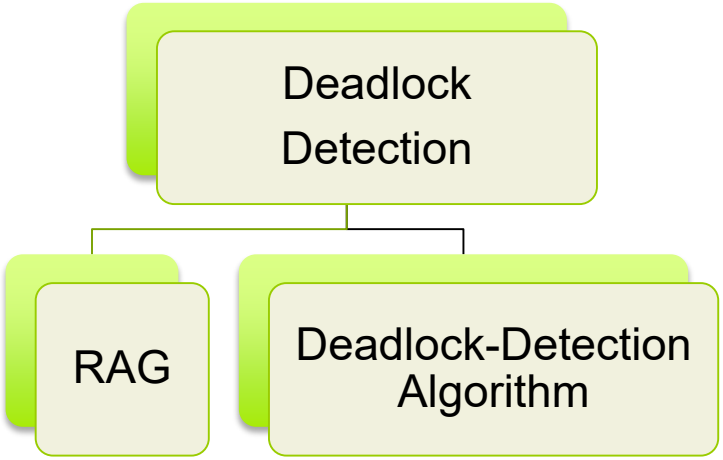
What is the new safe sequences produced?

The request results unsafe state, although resources are available.

System will not grant the request of process P_0

Deadlock Detection

- Allow system to enter deadlock state
- Detection algorithm
  - Single instance of a resource type
  - Multiple instances of a resource type.
- Recovery scheme

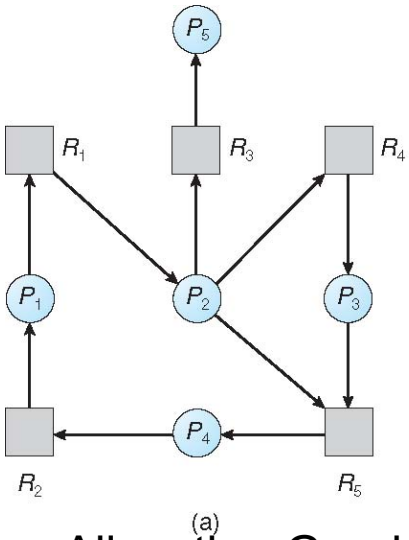


(almost similar to Safety Algorithm)

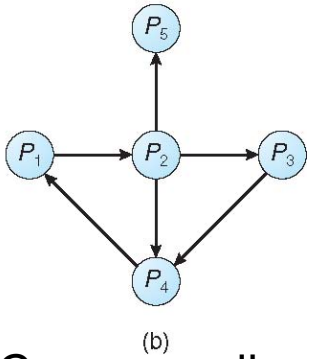
Deadlock Detection

Single Instance of Each Resource Type

- Deadlock can be detected by building *Resource-Allocation Graphs* (RAG)
- Maintain a **wait-for** graph
  - Nodes are processes
  - $P_i \rightarrow P_j$ if P_i is waiting for P_j
- Converting a resource-allocation graph to a wait-for graph.



Resource-Allocation Graph



Corresponding wait-for graph

Deadlock Detection

Single Instance of Each Resource Type

- Maintain a **wait-for** graph
- Periodically invoke an algorithm that searches for a cycle in the graph. If there is a cycle, there exists a deadlock
- An algorithm to detect a cycle in a graph requires an order of n^2 operations, where n is the number of vertices in the graph

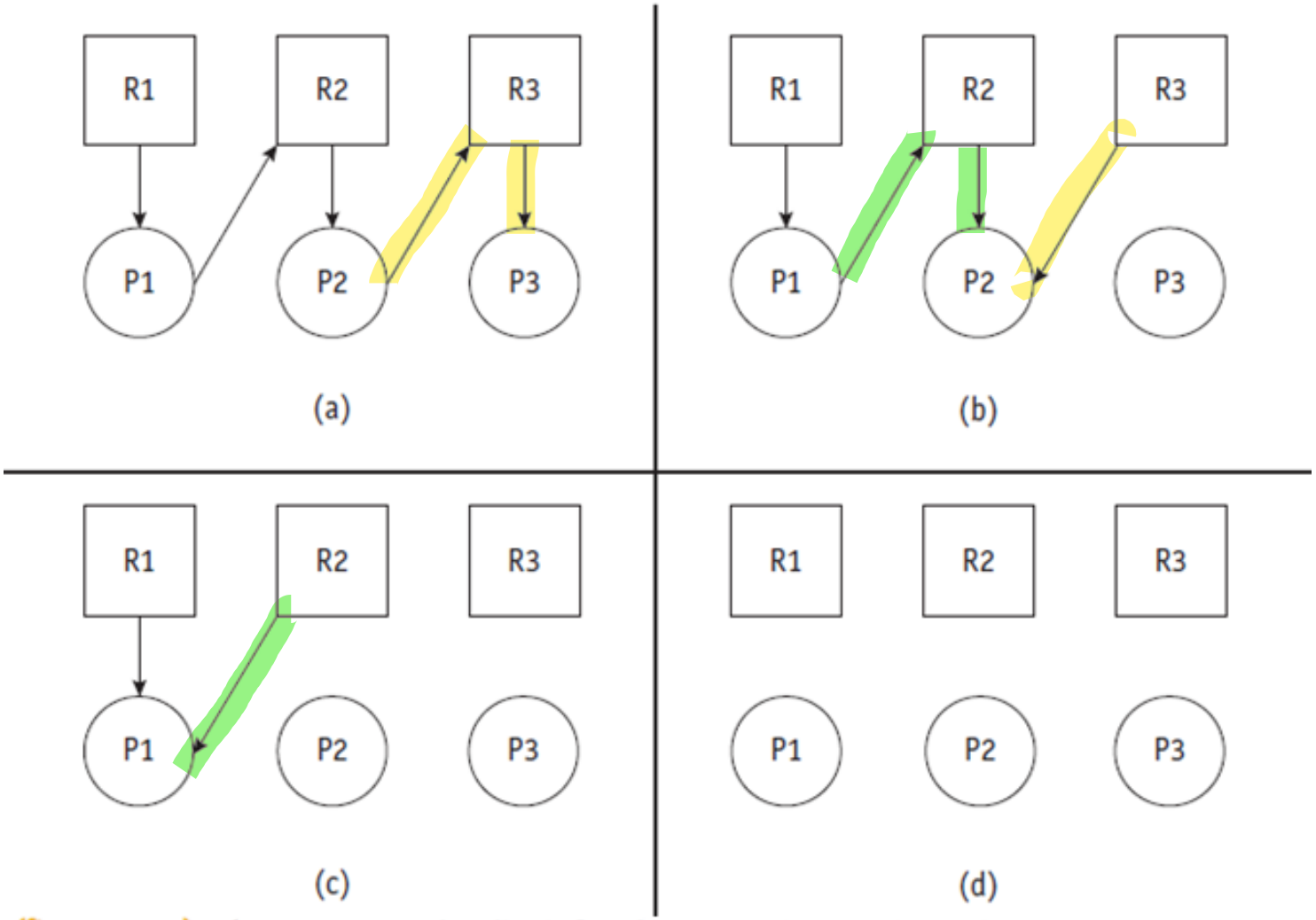
Deadlock Detection

Example

- The detection algorithm can be explained by using RAG and “reducing” them.
- The steps to reduce a graph:

1) Find process using a resource & not waiting for one (*Figure 5.12(b)*).
2) Find a process that’s waiting only for resource that aren’t fully allocated (*Figure 5.12(c)*).
3) Repeat step (1) and (2) until all lines removed (*Figure 5.12(d)*)

Deadlock Detection

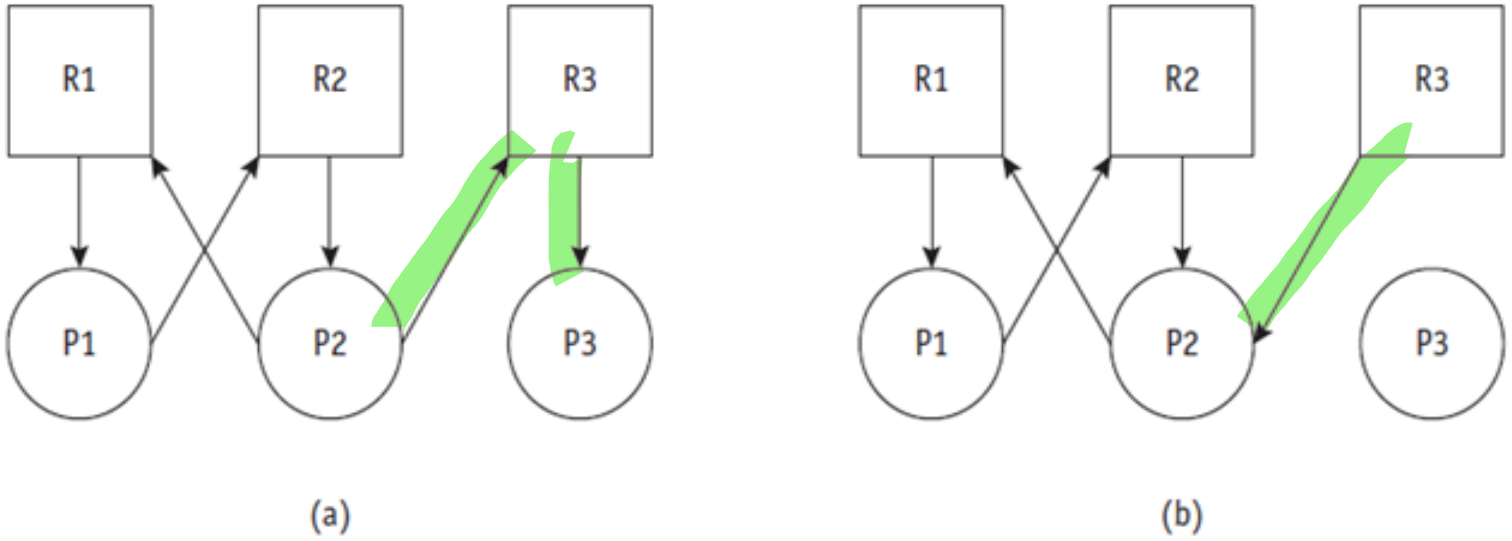


(figure 5.12) *This system is deadlock-free because the graph can be completely reduced, as shown in (d).*

Source: A.M. McHoes and I.M. Flyn, Understanding Operating Systems: 6th Edition, Course Technology, Cengage Learning, United State of America (2011)

Deadlock Detection

- Example a graph remaining the deadlock using those steps:



(figure 5.13) *Even after this graph (a) is reduced as much as possible (by removing the request from P3), it is still deadlocked (b).*

Source: A.M. McHoes and I.M. Flyn, Understanding Operating Systems: 6th Edition, Course Technology, Cengage Learning, United State of America (2011)

Deadlock Detection

Several/Multiple Instances of a Resource Type

Let n = number of processes, and m = number of resources types.

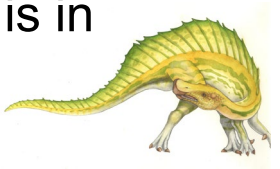
- **Available**: Vector of length m. If available [j] = k, then there are k instances of resource type R_j available
- **Allocation**: n x m matrix. If Allocation[i,j] = k, then P_i is currently allocated k instances of R_j
- **Request**: n x m matrix that indicates the current request of each process. If Request[i,j] = k, then process P_i is requesting k additional instances of resource type R_j.

7.6 Deadlock Detection

Detection Algorithm

Let ***Work*** and ***Finish*** be vectors of length ***m*** and ***n***, respectively Initialize:

1. Initialization
   (a) ***Work = Available***
   (b) For ***i*** = 1,2, …, n, if $\boldsymbol{Allocation_i \neq 0}$, then ***Finish*[i] = *false***; otherwise, ***Finish*[i] = *true***
2. Find an index ***i*** such that both:
   (a) ***Finish*[*i*] == *false***
   (b) $\boldsymbol{Request_i \leq Work}$
   If no such ***i*** exists, go to step 4
3. $\boldsymbol{Work = Work + Allocation_i}$
   Finish*[*i*] = *true
   go to step 2
4. If ***Finish[i] == false***, for some ***i***, $1 \leq i \leq n$, then the system is in deadlock state. Moreover, if ***Finish*[*i*] == *false***, then $\boldsymbol{P_i}$ is deadlocked



Example of Detection Algorithm

- Five processes P_0 through P_4; three resource types A (7 instances), *B* (2 instances), and *C* (6 instances)
- Snapshot at time T_0:

	Allocation	*Request*	*Available*
	A B C	*A B C*	*A B C*
P_0	0 1 0	0 0 0	0 0 0
P_1	2 0 0	2 0 2	
P_2	3 0 3	0 0 0	
P_3	2 1 1	1 0 0	
P_4	0 0 2	0 0 2	

- Sequence <P_0, P_2, P_3, P_1, P_4> will result in ***Finish[i] = true*** for all ***i***

Example of Detection Algorithm (Cont.)

- P_2 requests one additional instance of type *C*

	Allocation	*Request*	*Available*
	A B C	*A B C*	*A B C*
P_0	0 1 0	0 0 0	0 0 0
P_1	2 0 0	2 0 2	
P_2	3 0 3	0 0 1	
P_3	2 1 1	1 0 0	
P_4	0 0 2	0 0 2	

- State of system?
  - Can reclaim resources held by process P_0, but insufficient resources to fulfill other processes requests
  - Deadlock exists, consisting of processes P_1, P_2, P_3, and P_4

Deadlock Detection

Detection-Algorithm Usage

- If a deadlock is detected we must abort (rollback) some of the processes involved in the deadlock (see next slide)
- Need to decide when, and how often, to invoke the deadlock detection algorithm, which depends on:
  - How often a deadlock is likely to occur?
  - How many processes will need to be rolled back?
    - one for each disjoint cycle
- If detection algorithm is invoked arbitrarily, there may be many cycles in the resource graph and so we would not be able to tell which of the many deadlocked processes “caused” the deadlock.

7.7 Recovery from Deadlock

Process termination

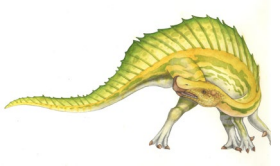
- Abort all deadlocked processes
- Abort one process at a time until the deadlock cycle is eliminated
- In which order should we choose to abort?
  - Priority of the process
  - How long process has computed, and how much longer to completion
  - Resources the process has used
  - Resources process needs to complete
  - How many processes will need to be terminated
  - Is process interactive or batch?

7.7

Recovery from Deadlock

Resource preemption

- **Selecting a victim** – minimize cost
- **Rollback** – return to some safe state, restart process for that state
- **Starvation** – same process may always be picked as victim, include number of rollback in cost factor



7.8 Summary

- A deadlock state occurs when two or more processes are waiting indefinitely for an event that can be caused only by one of the waiting processes.

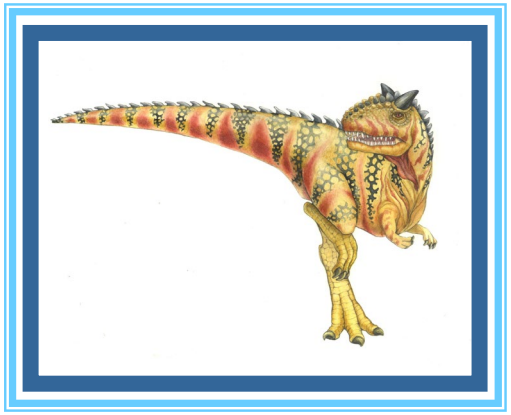
- Three principal methods for dealing with deadlocks:
  1. Use some protocol to prevent or avoid deadlocks.
  2. Allow the system to enter a deadlock state, detect it, and then recover.
  3. Ignore the problem altogether and pretend that deadlocks never occur in the system.

Exercises

From Silberchatz, Operating System Concepts
Chapter 7 Exercises (page: 335 - 338)

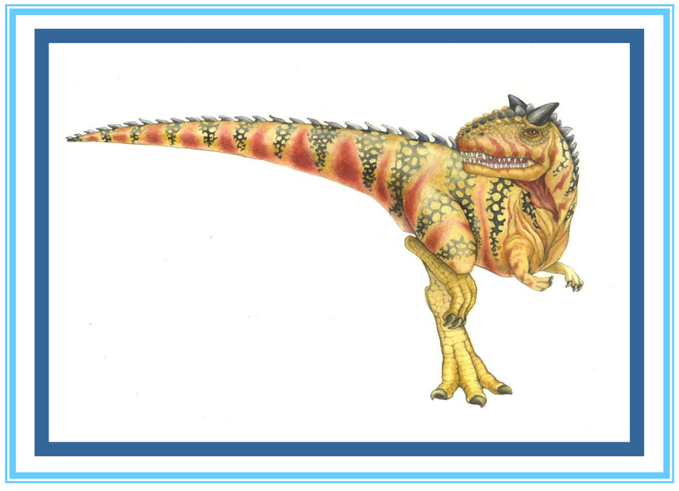
- 7.1
- 7.6
- 7.7
- 7.8
- 7.9
- 7.10
- 7.12
- 7.13
- 7.14

End of Chapter 7



Silberschatz, Galvin and Gagne ©2013

Chapter 8: Memory-Management Strategies



Silberschatz, Galvin and Gagne ©2013

Chapter 8: Memory Management Strategies

- 8.1 Background
- 8.2 Swapping
- 8.3 Contiguous Memory Allocation
- 8.4 Segmentation
- 8.5 Paging

- OBJECTIVES
- To provide a detailed description of various ways of organizing memory hardware
- To discuss various memory-management techniques, including paging and segmentation

Background

- Program must be brought (from disk) into memory and placed within a process for it to be run
- Main memory and registers are only storage CPU can access directly
- Memory unit only sees a stream of addresses + read requests, or address + data and write requests
- Register access in one CPU clock (or less)
- Main memory can take many cycles, causing a **stall**
- **Cache** sits between main memory and CPU registers
- Protection of memory required to ensure correct operation

Base and Limit Registers

- A pair of **base** and **limit registers** define the logical address space
- CPU must check every memory access generated in user mode to be sure it is between base and limit for that user

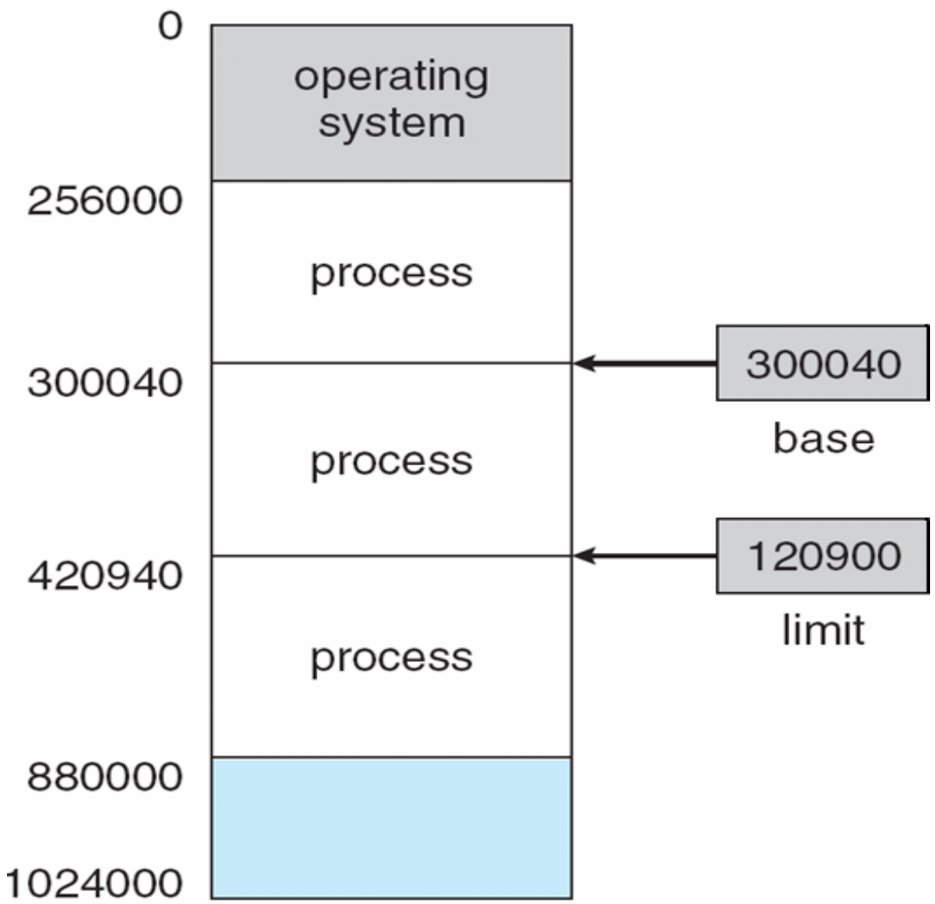


Figure 8.1 A base and a limit register define a logical address space.

Logical vs. Physical Address Space

- The concept of a logical address space that is bound to a separate **physical address space** is central to proper memory management
  - **Logical address** – generated by the CPU; also referred to as **virtual address**
  - **Physical address** – address seen by the memory unit

- Logical and physical addresses are the same in compile-time and load-time address-binding schemes; logical (virtual) and physical addresses differ in execution-time address-binding scheme
- **Logical address space** is the set of all logical addresses generated by a program
- **Physical address space** is the set of all physical addresses generated by a program

- 같음
compile time
load time

↓
execution time.

Memory-Management Unit (MMU)

- Hardware device that at run time maps virtual to physical address
- Many methods possible, covered in the rest of this chapter
- To start, consider simple scheme where the value in the relocation register is added to every address generated by a user process at the time it is sent to memory
  - Base register now called **relocation register**
  - MS-DOS on Intel 80x86 used 4 relocation registers
- The user program deals with *logical* addresses; it never sees the *real* physical addresses
  - Execution-time binding occurs when reference is made to location in memory
  - Logical address bound to physical addresses

- The user program deals with *logical addresses*; it never sees the ***real*** *physical addresses*.

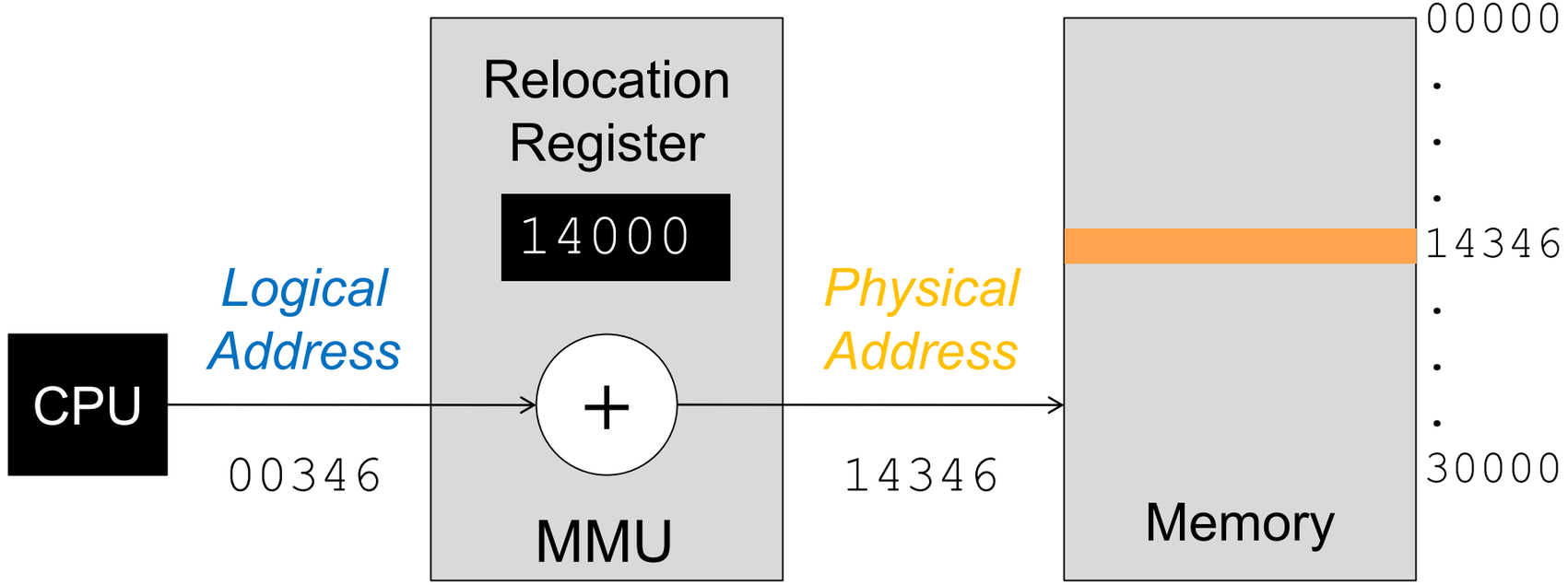


Figure 8.4

The concept of a **logical address space** (bound to separate **physical address space**) is central to proper memory management.

Swapping

- A process can be **swapped** temporarily out of memory to a backing store, and then brought back into memory for continued execution
  - Total physical memory space of processes can exceed physical memory
- **Backing store** – fast disk large enough to accommodate copies of all memory images for all users; must provide direct access to these memory images
- **Roll out, roll in** – swapping variant used for priority-based scheduling algorithms; lower-priority process is swapped out so higher-priority process can be loaded and executed
- Major part of swap time is transfer time; total transfer time is directly proportional to the amount of memory swapped
- System maintains a **ready queue** of ready-to-run processes which have memory images on disk
- Does the swapped out process need to swap back in to same physical addresses?
- Depends on address binding method
  - Plus consider pending I/O to / from process memory space
- Modified versions of swapping are found on many systems (i.e., UNIX, Linux, and Windows)
  - Swapping normally disabled
  - Started if more than threshold amount of memory allocated
  - Disabled again once memory demand reduced below threshold

Schematic View of Swapping

- fast disk large enough to accommodate copies of all memory images for all users
- provide direct access to these memory images.

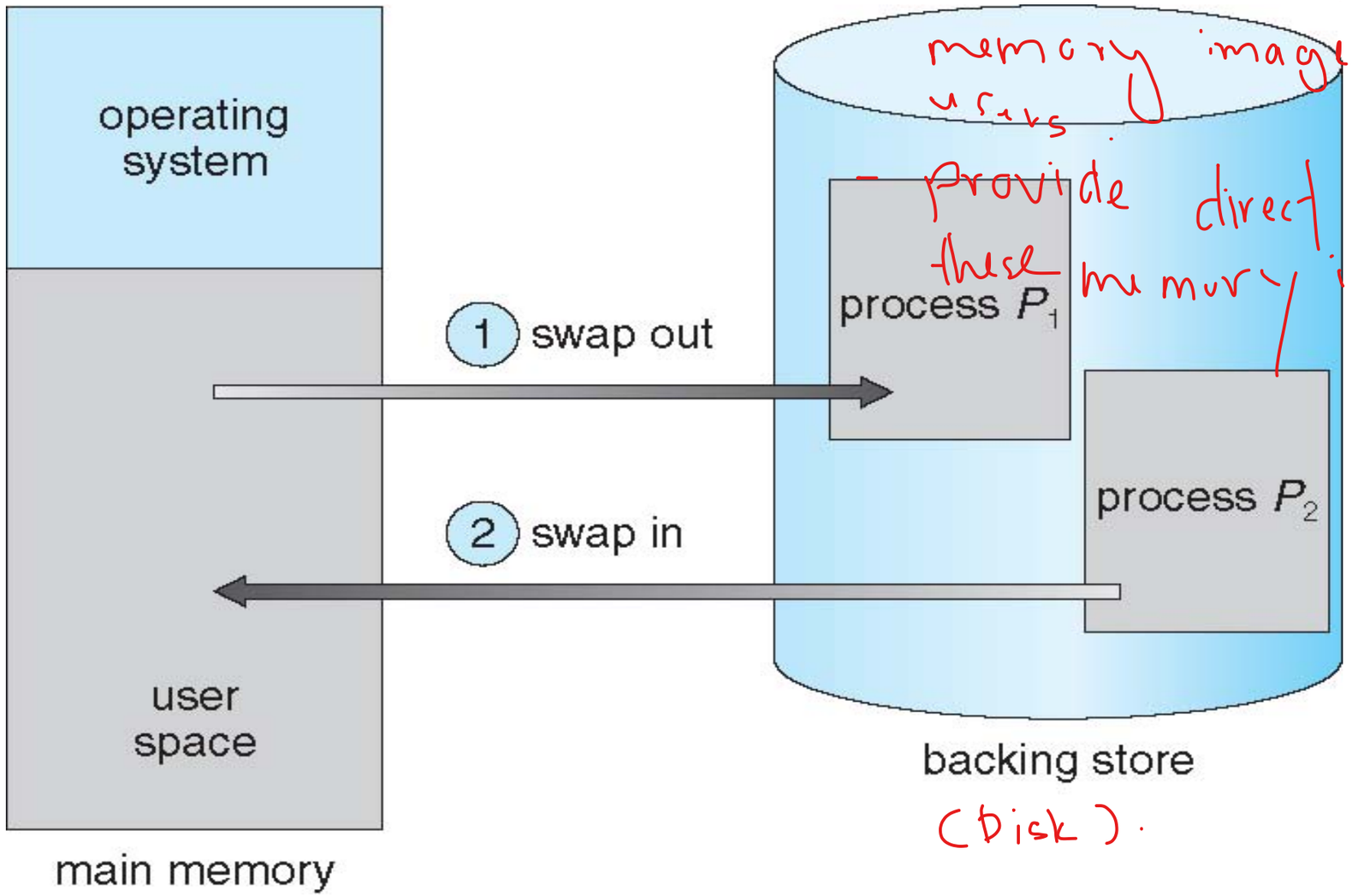
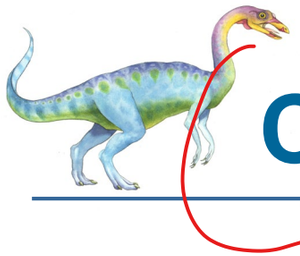


Figure 8.5 Swapping of two processes using a disk as a backing store.



Context Switch Time including Swapping

- If next processes to be put on CPU is not in memory, need to swap out a process and swap in target process
- Context switch time can then be very high
- 100MB process swapping to hard disk with transfer rate of 50MB/sec
  - Swap out time of 2000 ms
  - Plus swap in of same sized process
  - Total context switch swapping component time of 4000ms (4 seconds)
- Can reduce if reduce size of memory swapped – by knowing how much memory really being used
  - System calls to inform OS of memory use via `request memory()` and `release memory()`
- Other constraints as well on swapping
  - Pending I/O – can't swap out as I/O would occur to wrong process
  - Or always transfer I/O to kernel space, then to I/O device
    - Known as **double buffering**, adds overhead
- Standard swapping not used in modern operating systems
  - But modified version common
    - Swap only when free memory extremely low

Swapping on Mobile Systems

- Not typically supported
  - Flash memory based
    - Small amount of space
    - Limited number of write cycles
    - Poor throughput between flash memory and CPU on mobile platform
- Instead use other methods to free memory if low
  - iOS ***asks*** apps to voluntarily relinquish allocated memory
    - Read-only data thrown out and reloaded from flash if needed
    - Failure to free can result in termination
  - Android terminates apps if low free memory, but first writes **application state** to flash for fast restart
  - Both OSes support paging as discussed below

Memory Allocation

Contiguous

连续

– each process is contained in a single contiguous section of memory

- Fixed Partitions
- Dynamic/Variable Partitions

Non-Contiguous

– each process is contained in a non-sequential location of memory

- Paging
- Segmentation

Contiguous Memory Allocation

- Fixed partitions
- Dynamic / Variable Partitions.

Contiguous
– sharing a common border. 边界.
– next or together in sequence.

- Main memory must support both OS and user processes
- Limited resource, must allocate efficiently
- Contiguous allocation is one early method

- Main memory usually into two **partitions**:
  - Resident operating system, usually held in low memory with interrupt vector
  - User processes then held in high memory
  - Each process contained in single contiguous section of memory

Contiguous Memory Allocation (Cont..)

- Each partition may contain exactly one process.
- Degree of multiprogramming limited by number of partitions
- Multiple-partition allocation

- **Fixed-sized partitions**
  - Equal-size partitions & unequal-size partitions

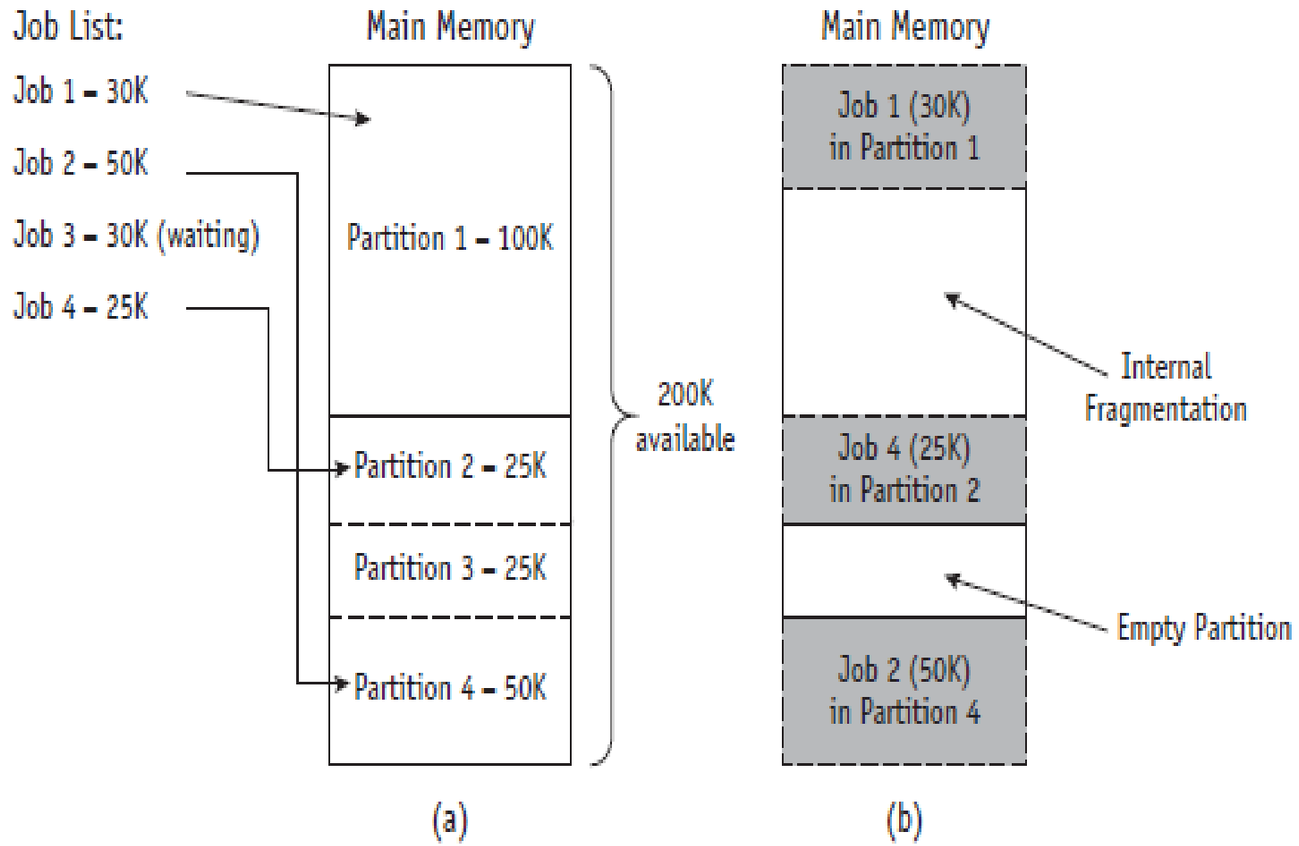
Example

Table A simplified fixed-partition memory table with the free partition shaded.

Partition Size	Memory Address	Access	Partition Status
100K	200K	Job 1	Busy
25K	300K	Job 4	Busy
25K	325K		Free
50K	350K	Job 2	Busy

Contiguous Memory Allocation (Cont..)

Example



(figure 2.3)

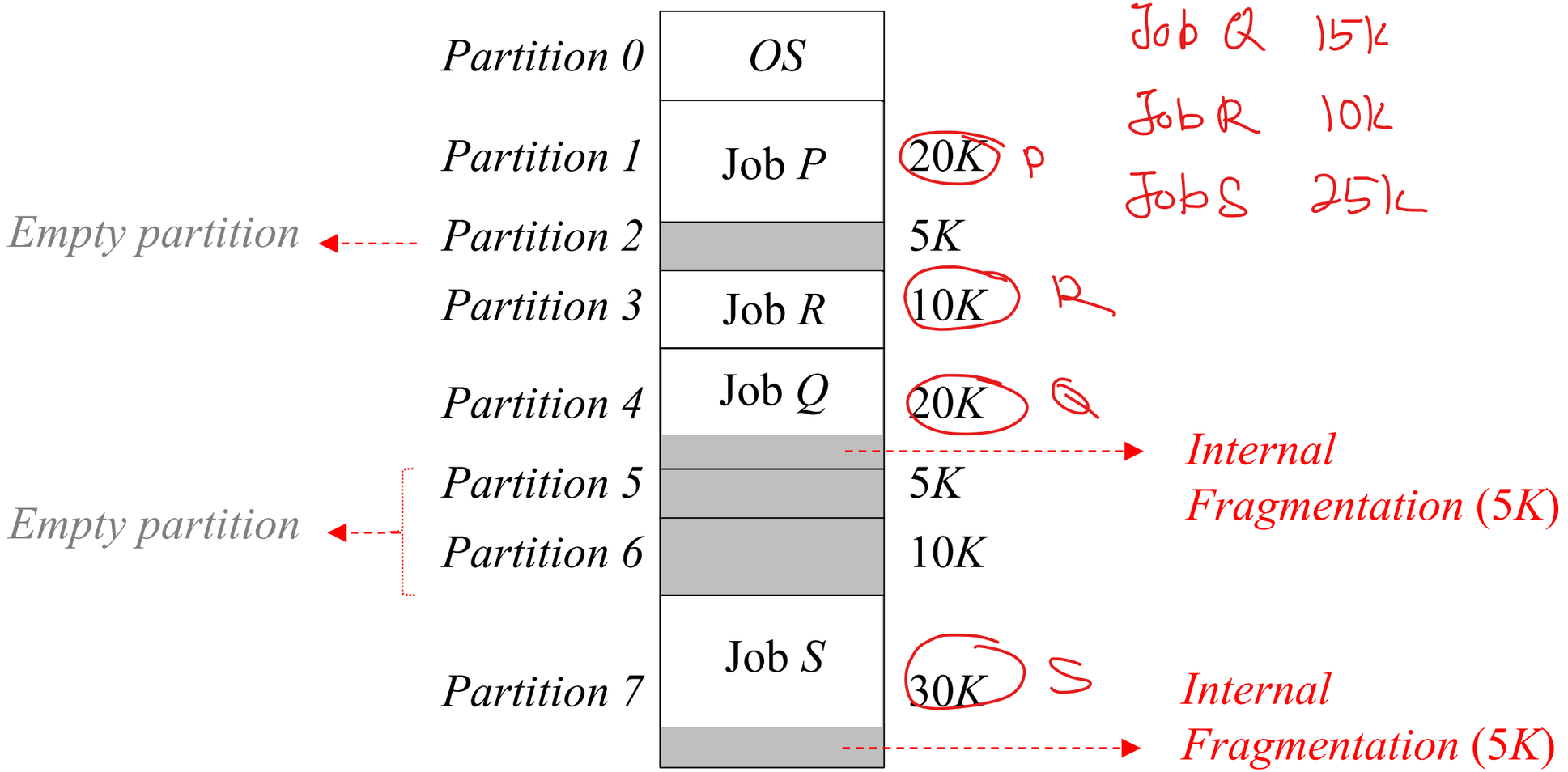
Main memory use during fixed partition allocation of Table 2.1. Job 3 must wait even though 70K of free space is available in Partition 1, where Job 1 only occupies 30K of the 100K available. The jobs are allocated space on the basis of "first available partition of required size."

Contiguous Memory Allocation (Cont..)

Exercise

The new jobs in the order: Job P (20K) – Job Q (15K) – Job R (10K) – Job S (25K) need to be allocated in a **fixed partition** memory as shown in the figure. Label all the jobs in which partition each will be allocated.

Job P 20K
Job Q 15K
Job R 10K
Job S 25K



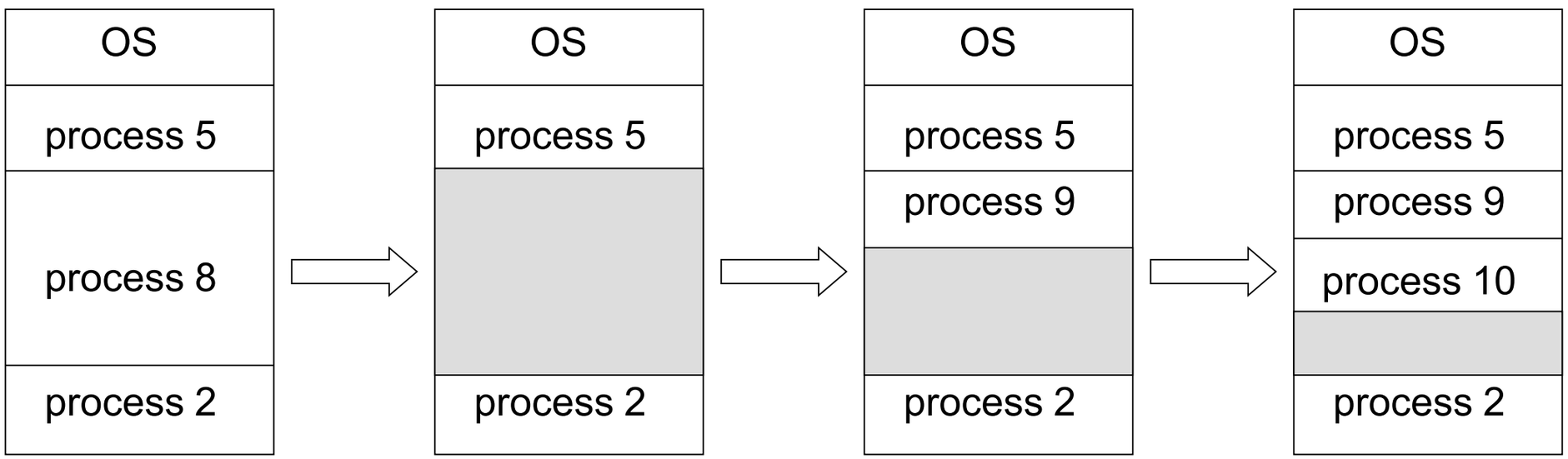
Contiguous Memory Allocation (Cont..)

- **Disadvantages Fixed Partitions**
  - Requires contiguous loading of entire program.
  - Job allocation method:
    → *First available partition with required size.*
  - Arbitrary partition size leads to undesired results:

▪ Partition too small → Large jobs have longer turnaround time.
▪ Partition too large → Memory waste: internal fragmentation.

Contiguous Memory Allocation (Cont..)

- **Dynamic/Variable-partition** sizes for efficiency (sized to a given process' needs)
  - **Hole** – block of available memory; holes of various size are scattered throughout memory
  - When a process arrives, it is allocated memory from a hole large enough to accommodate it
  - Process exiting frees its partition, adjacent free partitions combined
  - Operating system maintains information about:
    a) allocated partitions b) free partitions (hole)



Contiguous Memory Allocation (Cont..)

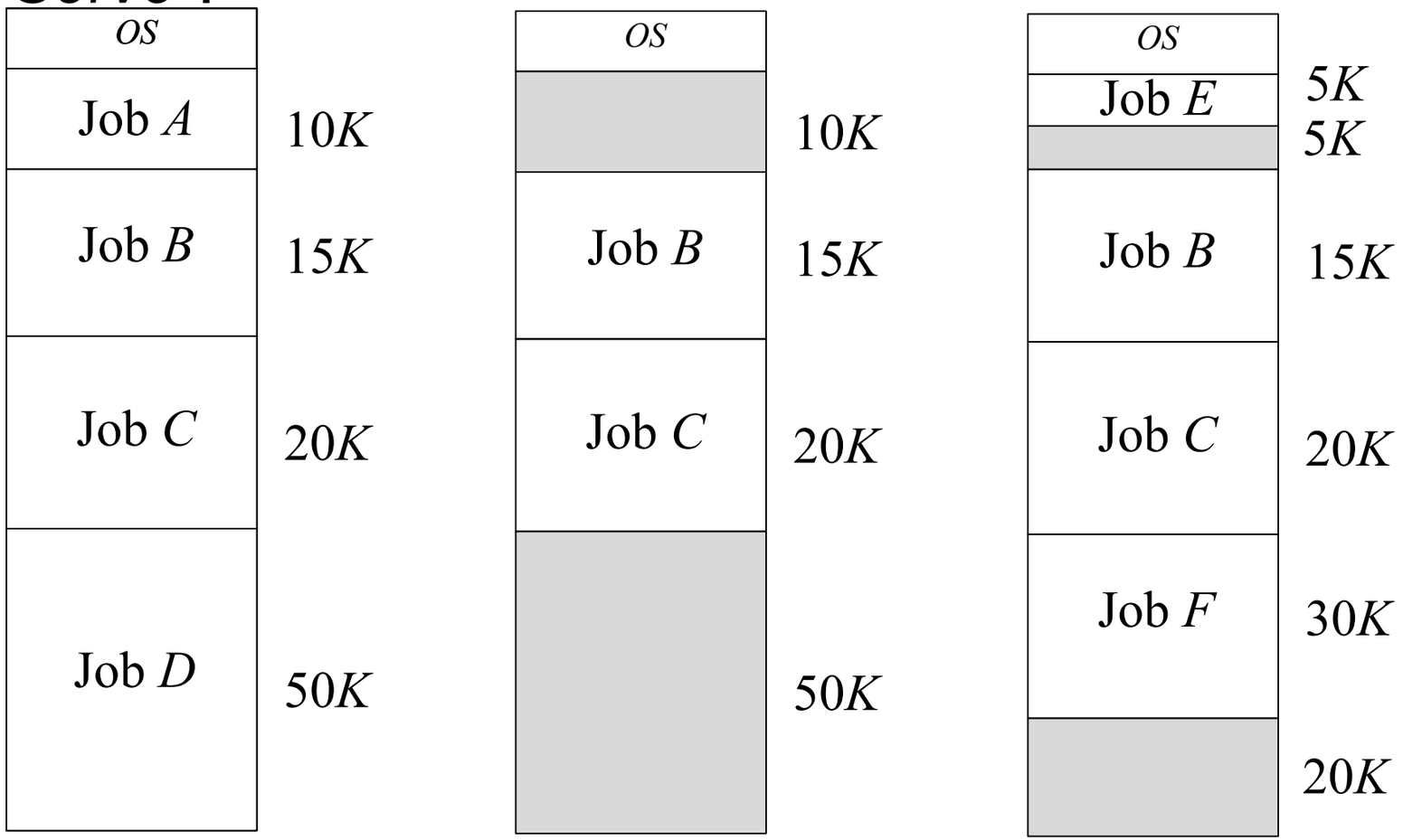
Example

Variable/Dynamic partition: Five snapshots (a – e) of main memory as 8 jobs submitted and allocated space on the basis "*First-Come, First-Serve*".

Init. jobs:
A (10*K*)
B (15*K*)
C (20*K*)
D (50*K*)

New jobs:
E (5*K*)
F (30*K*)

New jobs:
G (10*K*)
H (30*K*)



(a) Initial job entry memory allocation.

(b) After Job *A* and *D* have finished.

(c) After Job *E* and *F* have entered.

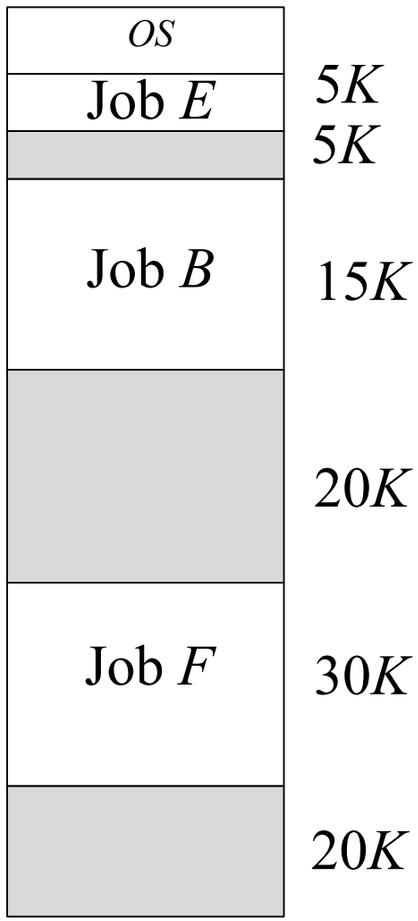
Contiguous Memory Allocation (Cont..)

Example

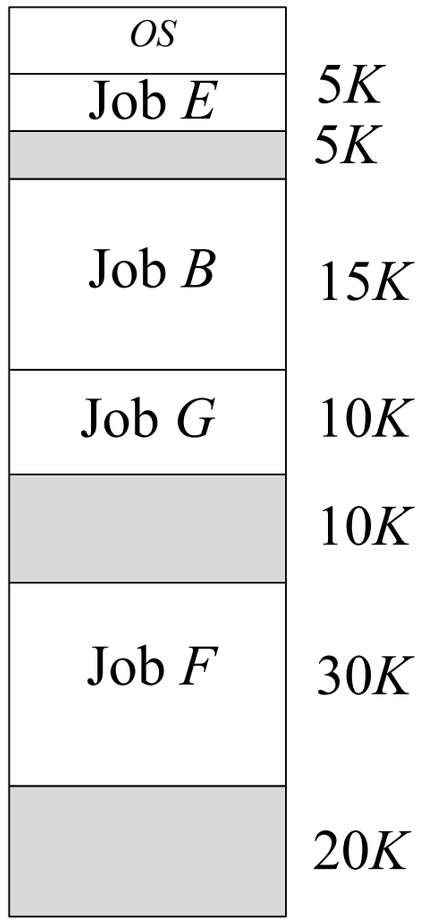
Init. jobs:
- *A* (10*K*)
- *B* (15*K*)
- *C* (20*K*)
- *D* (50*K*)

New jobs:
- *E* (5*K*)
- *F* (30*K*)

New jobs:
- *G* (10*K*)
- *H* (30*K*)



(d)
After Job *C* has finished.



(e)
After Job *G* has entered.

Job *H* has to wait even though there's enough free memory between partition to accommodate it.

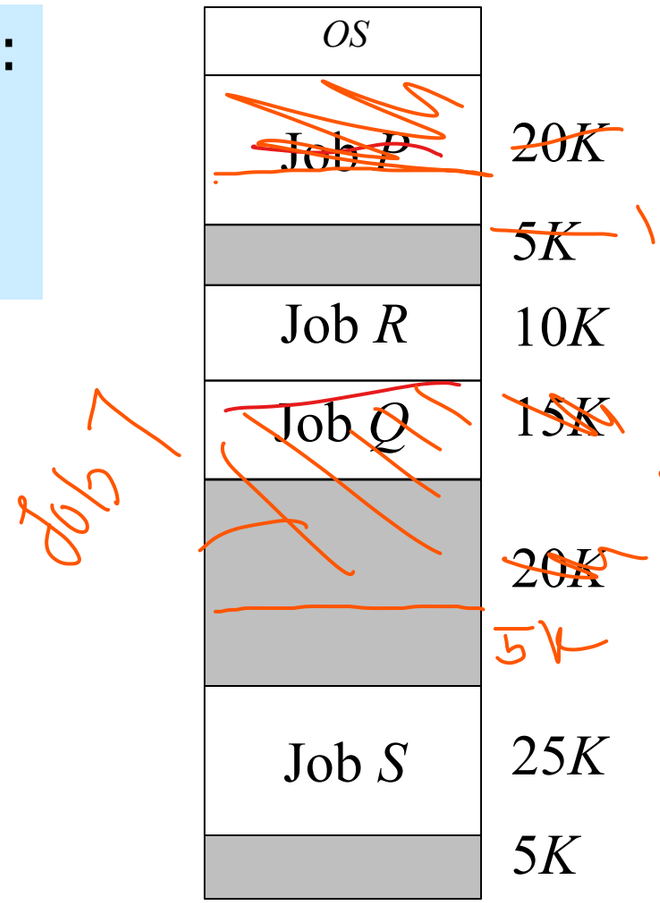
Contiguous Memory Allocation (Cont..)

Exercise

(Based Exercise 8.1)

Figure shows a few jobs have been allocated in a **dynamic partition** memory at time t_0. Suppose that job P and Q finished at t_1 and new jobs arrived at t_3. Allocate all the new jobs in the memory.

New jobs:
$T\,(30K)$
$U\,(35K)$
$V\,(15K)$



Contiguous Memory Allocation (Cont..)

Exercise

t_0: Initial jobs in memory.

t_1: After Job P and Q have finished.

t_3: After Job T and V have entered.

Job U need to wait.

New jobs:
T (30K)
U (35K)
V (15K)

Contiguous Memory Allocation (Cont..)

- Based on **examples before** Job H and U respectively have to wait until any job ends with enough size.

- **Disadvantages**:

  Produced through block of available memory (**holes**) with different sizes scattered out the memory.

- **Solution**:

  Relocatable Dynamic Partition

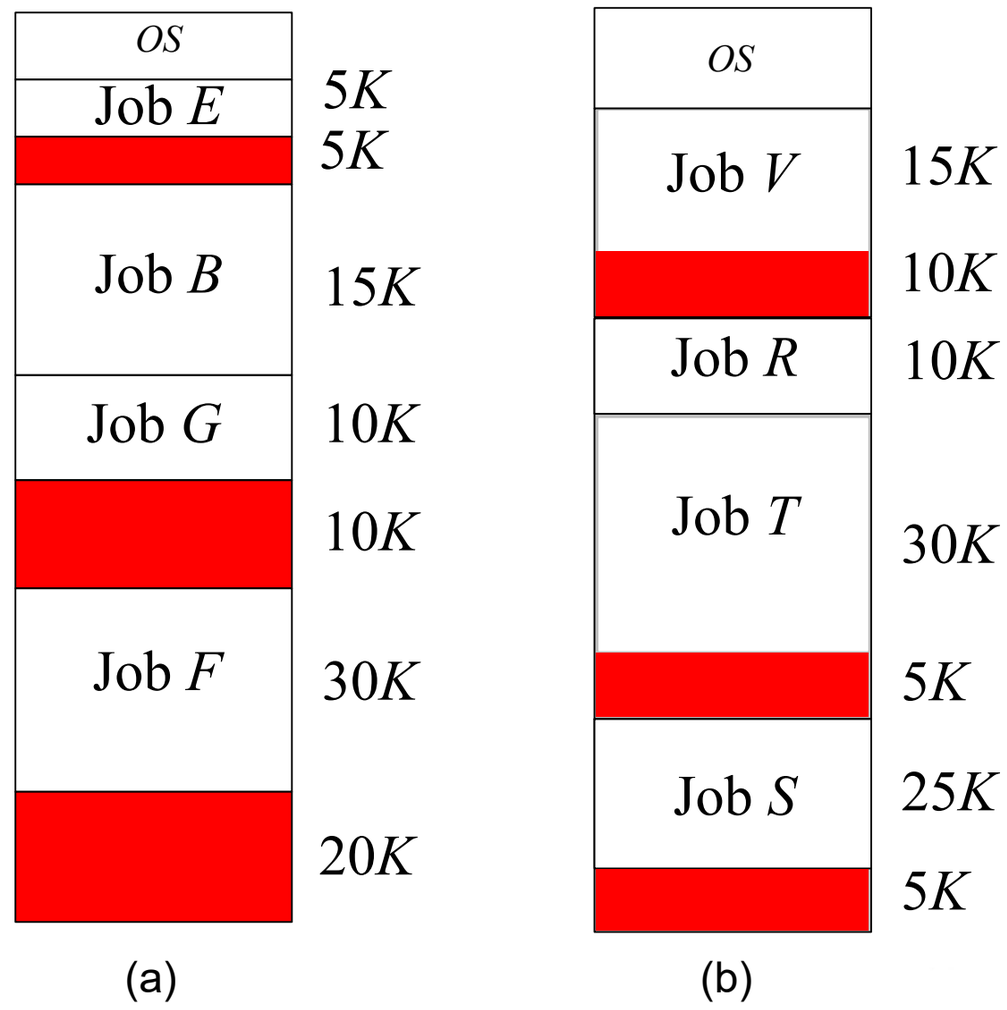
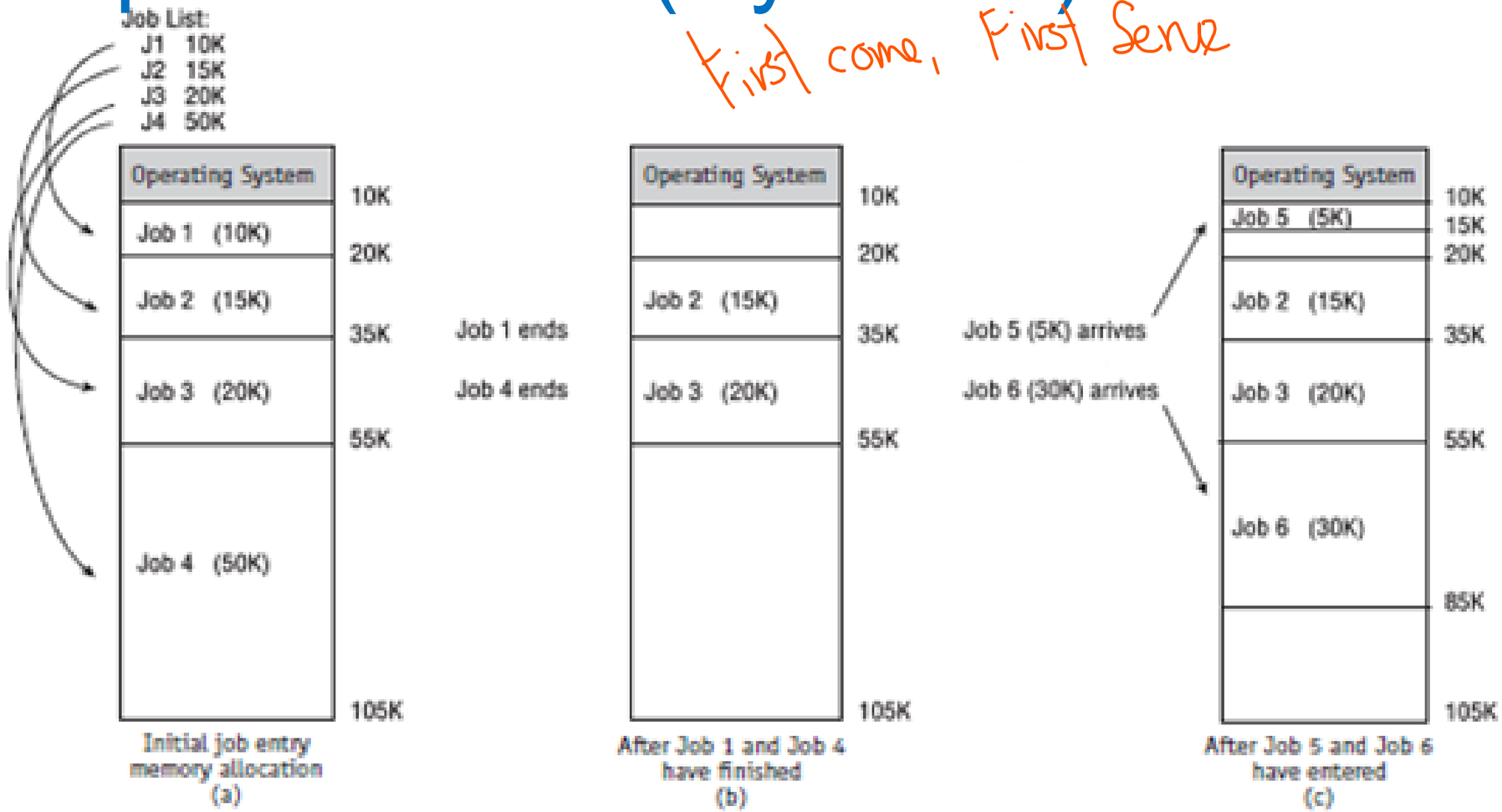


Figure Holes or external fragmentation examples before

Contiguous Memory Allocation (Cont..)

Example Variable (Dynamic) Partitions

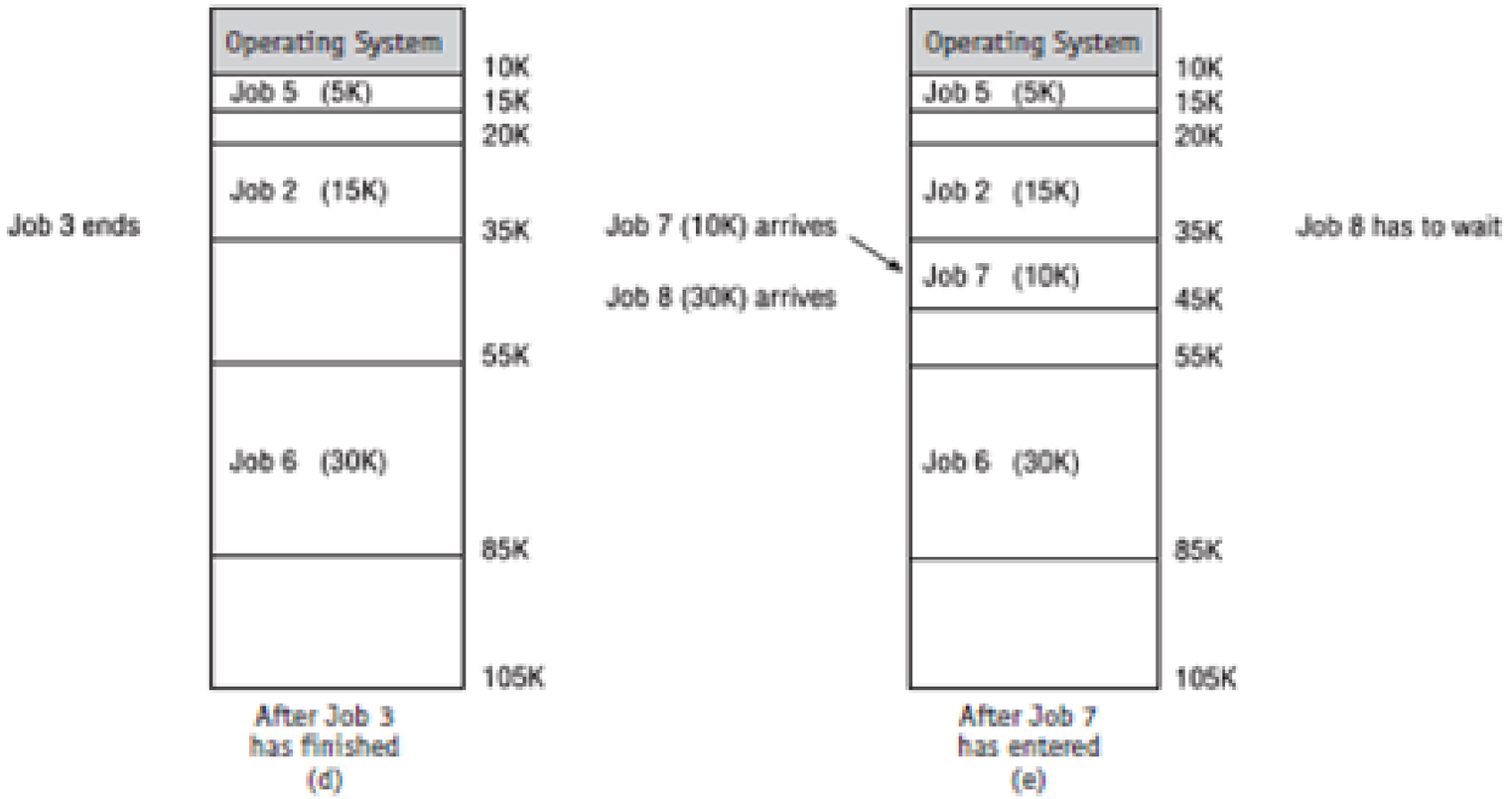
First come, First Serve



(figure 2.4)

Main memory use during dynamic partition allocation. Five snapshots (a-e) of main memory as eight jobs are submitted for processing and allocated space on the basis of "first come, first served." Job 8 has to wait (e) even though there's enough free memory between partitions to accommodate it.

Contiguous Memory Allocation (Cont..)



(figure 2.4)

Main memory use during dynamic partition allocation. Five snapshots (a-e) of main memory as eight jobs are submitted for processing and allocated space on the basis of "first come, first served." Job 8 has to wait (e) even though there's enough free memory between partitions to accommodate it.

Contiguous Memory Allocation (Cont..)

Fragmentation

- There are two types of fragmentation:

① **Internal fragmentation**

- allocated memory may be slightly larger than requested memory; the difference of memory internal to a partition that not being used;
- Results from **Fixed partition allocation**.

② **External fragmentation**

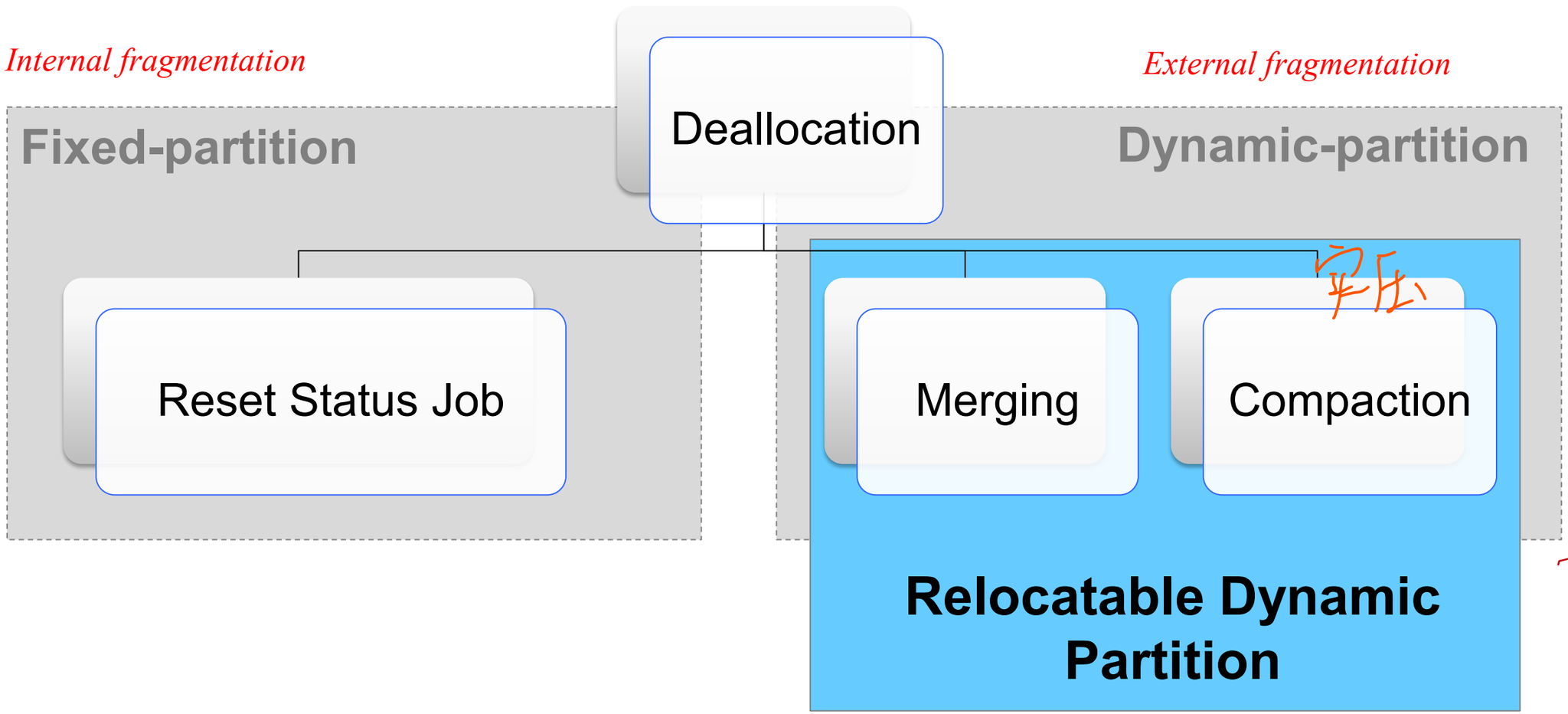
- total memory space exists to satisfy a request, but it is not contiguous.
- Results from **Dynamic/Variable partition allocation.**

Contiguous Memory Allocation (Cont..)

Deallocation

- Solution to reduce fragmentation

  → **Deallocation** : freeing allocated memory spaces.



Contiguous Memory Allocation (Cont..)

Relocatable Dynamic Partitions

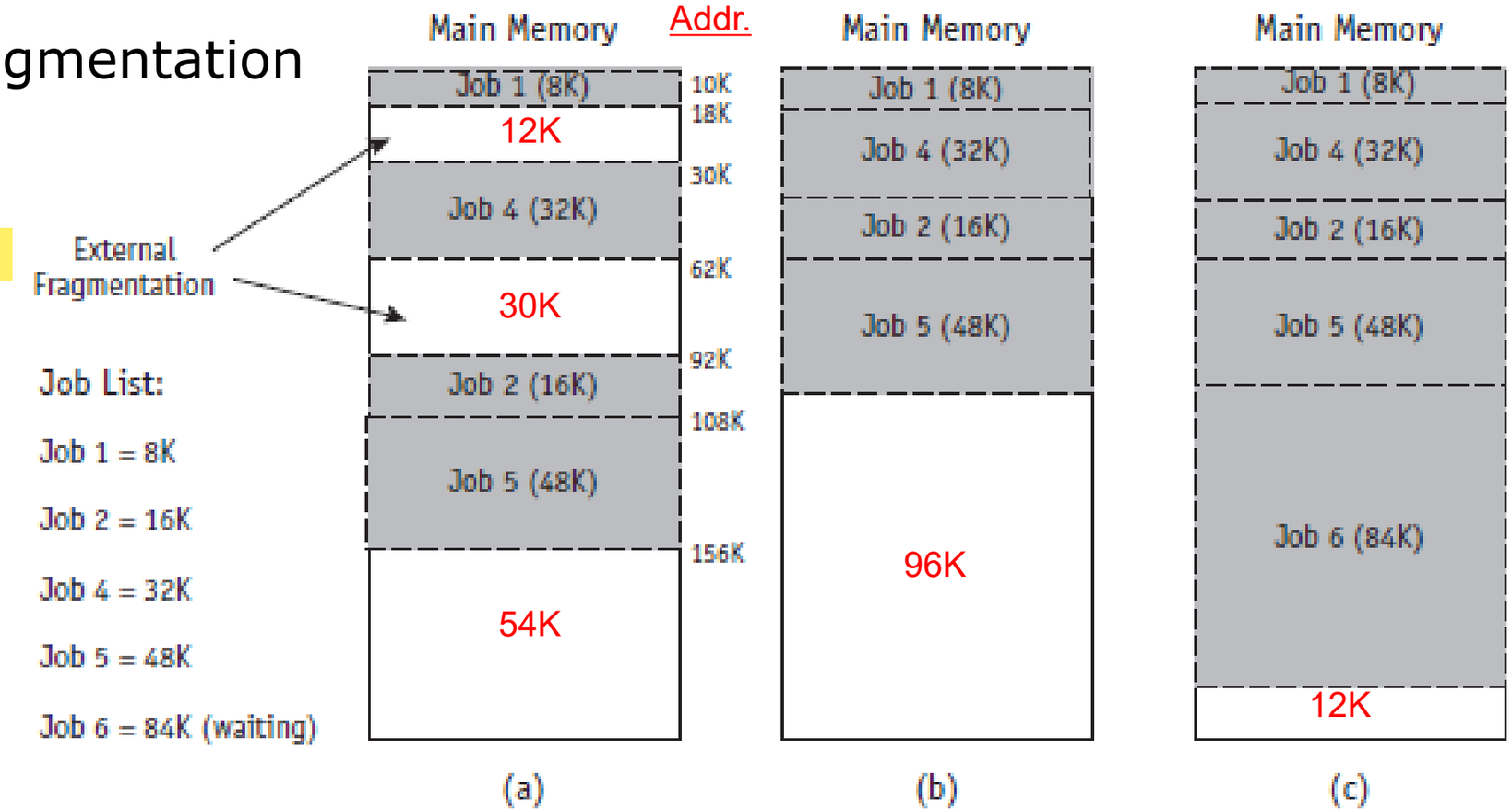
- Reduce external fragmentation by merging and **compaction**
  - **Compaction**: reclaiming fragmented sections of memory space. Every program in memory must be relocated.
  - Shuffle memory contents to place all free memory together in one large block
- Compaction is possible *only* if relocation is dynamic, and is done at execution time

Contiguous Memory Allocation (Cont..)

Relocatable Dynamic Partition

Solution to External Fragmentation
– **Compaction**

(i) Merge or combine all scattered holes at one location.



(figure 2.9)

Three snapshots of memory before and after compaction with the operating system occupying the first 10K of memory. When Job 6 arrives requiring 84K, the initial memory layout in (a) shows external fragmentation totaling 96K of space. Immediately after compaction (b), external fragmentation has been eliminated, making room for Job 6 which, after loading, is shown in (c).

Contiguous Memory Allocation (Cont..)

Relocatable Dynamic Partition

Example

(Based Example 2)

Relocatable Dynamic partition: Three snapshots (a – c) of main memory (dynamic partition system) before and after **compaction**.

Jobs:
B (15*K*)
E (5*K*)
F (30*K*)
H (30*K*) - waiting

External fragmentation total 45*K*.

Compaction and **merge** to make a large empty space.

Job *H* entered the memory.

Contiguous Memory Allocation (Cont..)

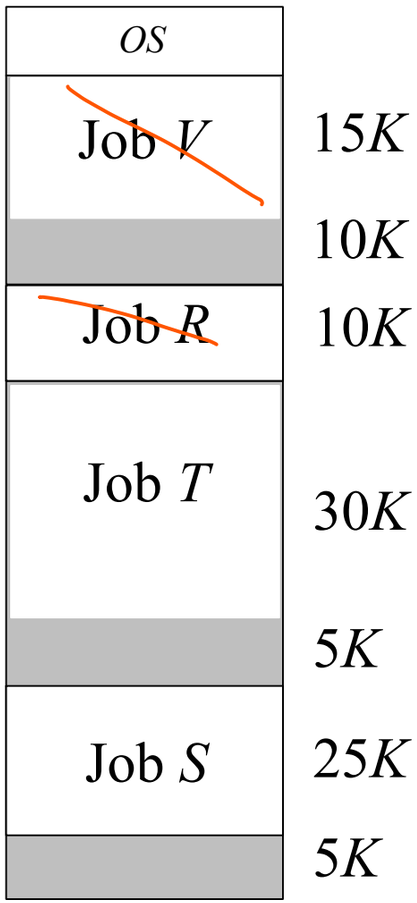
Relocatable Dynamic Partition

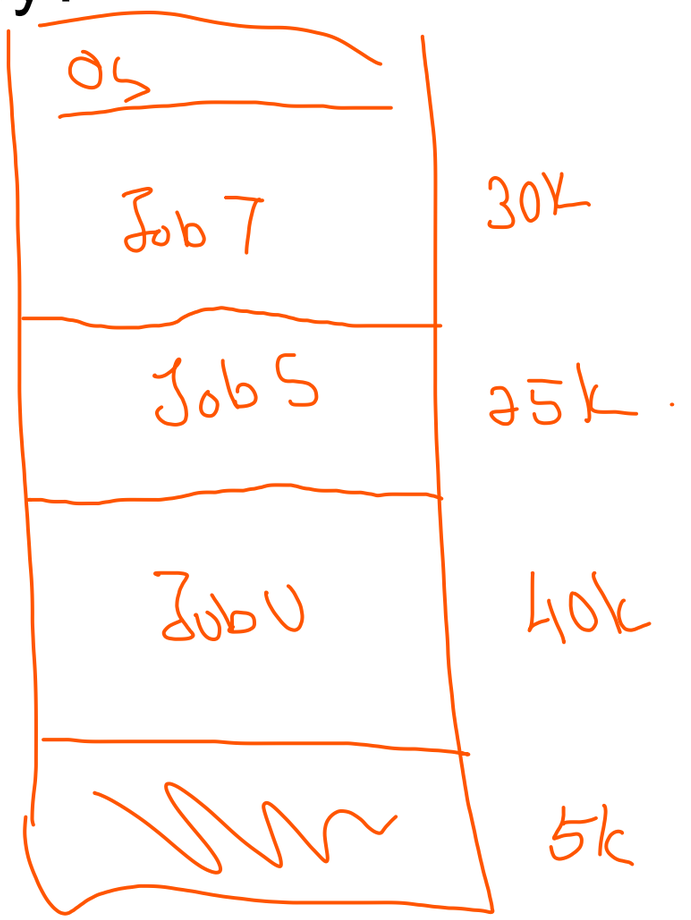
Exercise

(Based Exercise 8.2)

Suppose a system with dynamic partition memory. If Job V and R finished, how the Job U can enter the memory?

Jobs:
R $(10K)$
S $(25K)$
T $(30K)$
U $(40K)$ - waiting
V $(15K)$

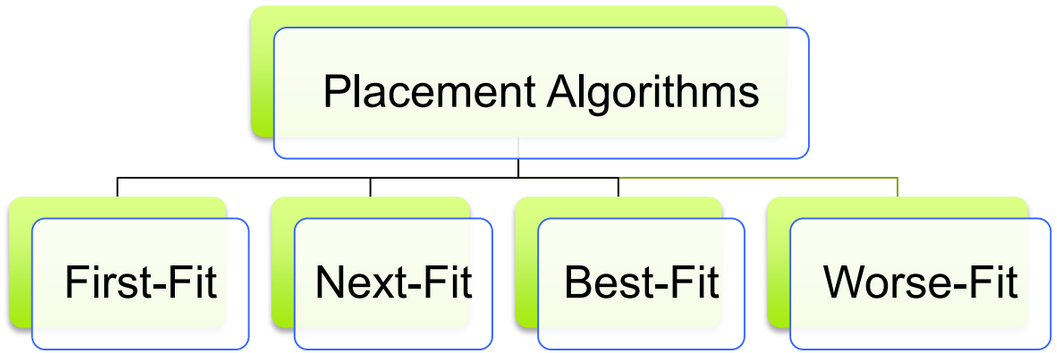




Contiguous Memory Allocation (Cont..)
Dynamic Storage-Allocation Problem

How to satisfy a request of size ***n*** from a list of free holes?

- Use the **Placement Algorithms** for selecting among free blocks of main memory.



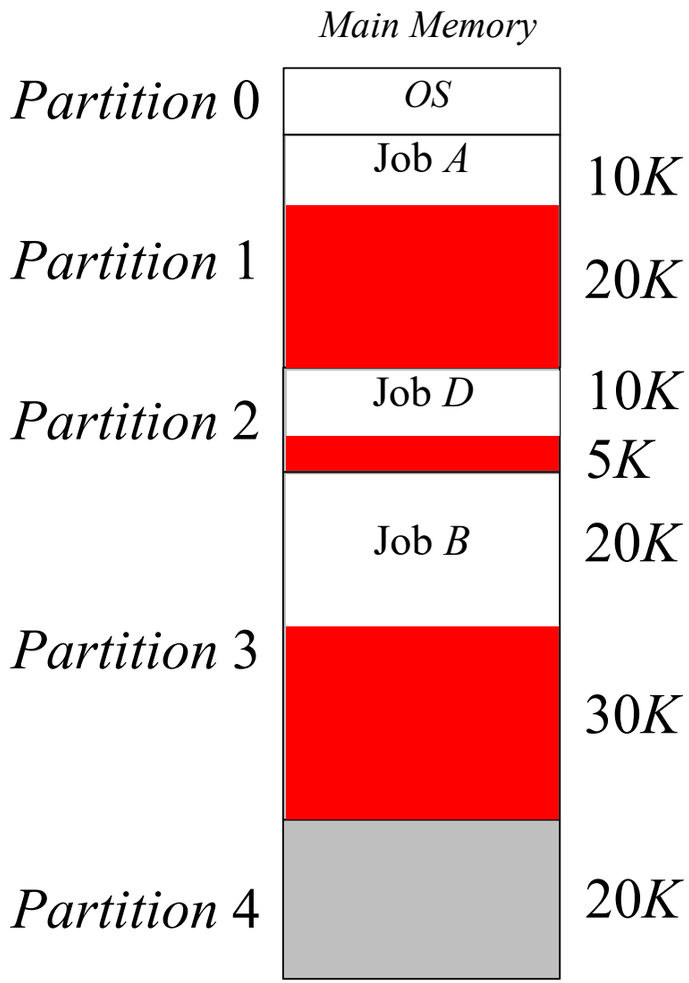
- **First-fit**: Allocate the ***first*** hole that is big enough. Search start at the beginning of memory.
- Next-fit: Similar to first-fit, except the search starts at the last hole allocated.
- **Best-fit**: Allocate the ***smallest*** hole that is big enough; must search entire list, unless ordered by size
  - Produces the smallest leftover hole
- **Worst-fit**: Allocate the ***largest*** hole; must also search entire list
  - Produces the largest leftover hole

Contiguous Memory Allocation (Cont..)

Example

First-fit allocation: (从 Partition 1 开始)

(Assume fixed partition are used)



Init. jobs:
A (10*K*)
B (20*K*)
C (30*K*)
D (10K)

- Job *C* has to wait until a large block become available, even though there is 75*K* unused memory space.

Unused partition (20*K*)

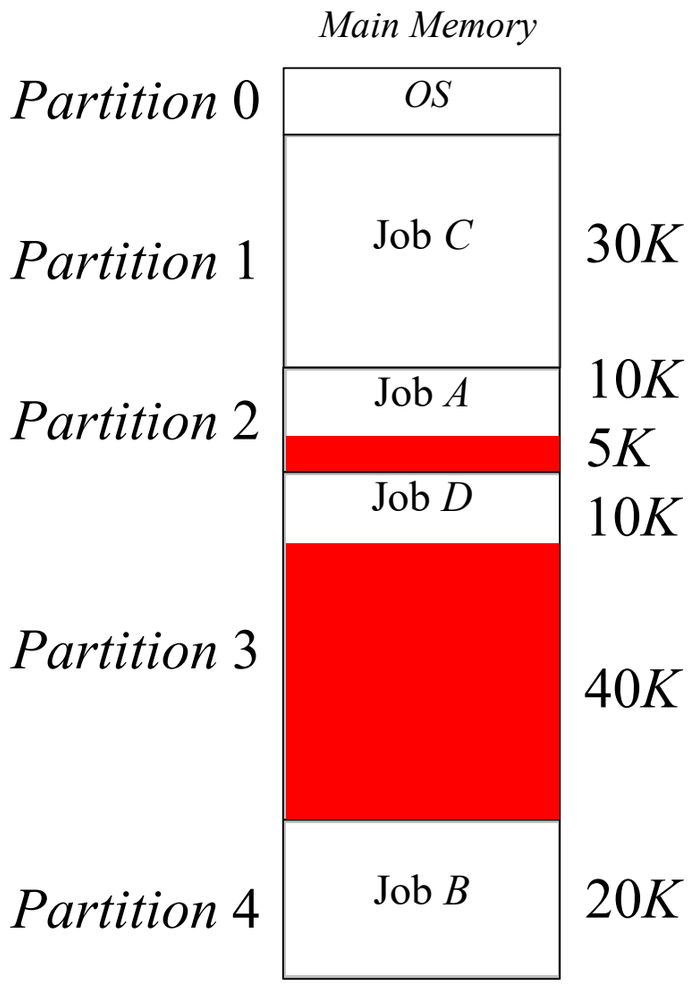
Internal Fragmentation (55*K*)

Contiguous Memory Allocation (Cont..)

Example

Best-fit allocation: (选 最大适合的)(刚刚好)

(Assume fixed partition are used)



Init. jobs:
A (10*K*)
B (20*K*)
C (30*K*)
D (10*K*)

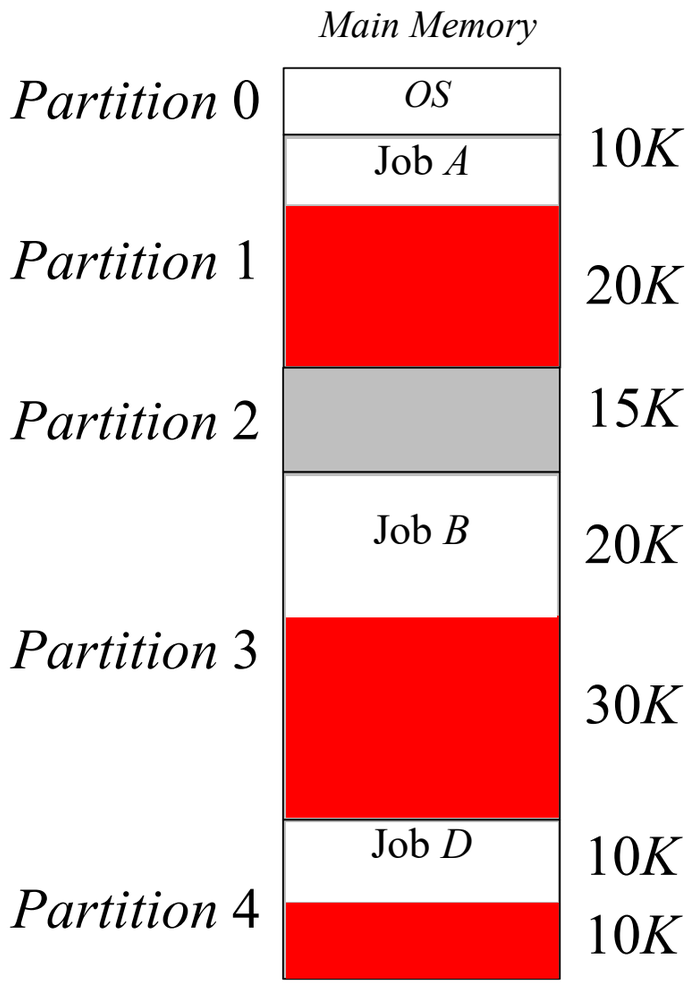
- ✓ All jobs can be allocated in the memory.
- ✓ Use the memory efficiently but slower implementation.

Internal Fragmentation (45*K*)

Contiguous Memory Allocation (Cont..)

Example

Next-fit allocation: (从 Partition 4 开始)
(Assume fixed partition are used)



Init. jobs:
A (10*K*)
B (20*K*)
C (30*K*)
D (10*K*)

- Job *C* has to wait until a any job exit and has enough 30*K* memory space.

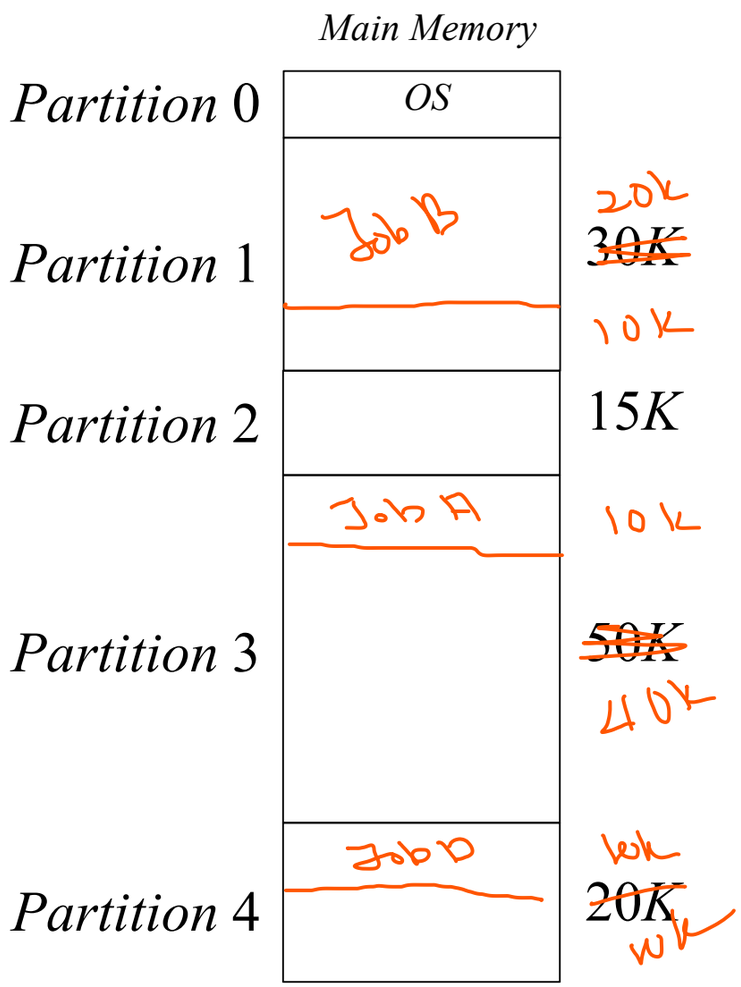
- ✓ This scheme also use the memory efficiently but slower implementation.

Unused partition (15*K*)
Internal Fragmentation (60*K*)

Contiguous Memory Allocation (Cont..)

Example

Worst-fit allocation:
(Assume fixed partition are used)



Init. jobs:
A (10*K*)
B (20*K*)
C (30*K*)
D (10*K*)

Contiguous Memory Allocation (Cont..)

Example 7 : **Worst-fit allocation**: 选最大的
(Assume fixed partition are used)

Init. jobs:
A (10*K*)
B (20*K*)
C (30*K*)
D (10*K*)

- Job *C* has to wait until a large block become available, even though there is 75*K* unused memory space.

Unused partition (15*K*)
Internal Fragmentation (60*K*)

Contiguous Memory Allocation (Cont..)

Best-Fit versus First-Fit Allocation

- **First-fit memory allocation**
  - Advantage: faster in making allocation
  - Disadvantage: leads to memory waste

- **Best-fit memory allocation**
  - Advantage: makes the best use of memory space
  - Disadvantage: slower in making allocation

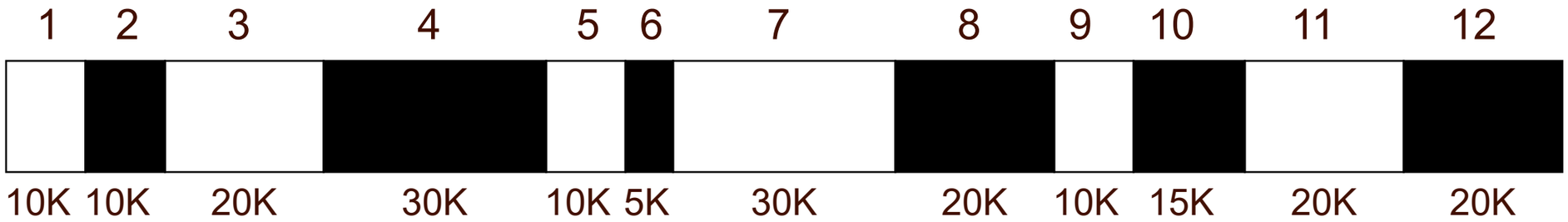
First-fit and Best-fit better than Worst-fit in terms of speed and storage utilization.

Contiguous Memory Allocation (Cont..)

Exercise:

P_1 - 20k
P_2 - 10k
P_3 - 5k

Given memory is allocated as specified below:



Assume partition requests are in the following order:

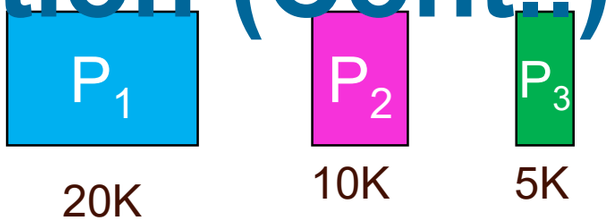
P_1 – 20K, P_2 – 10K, P_3 – 5K

Determine which partition each request will be allocated if the following strategy is used:

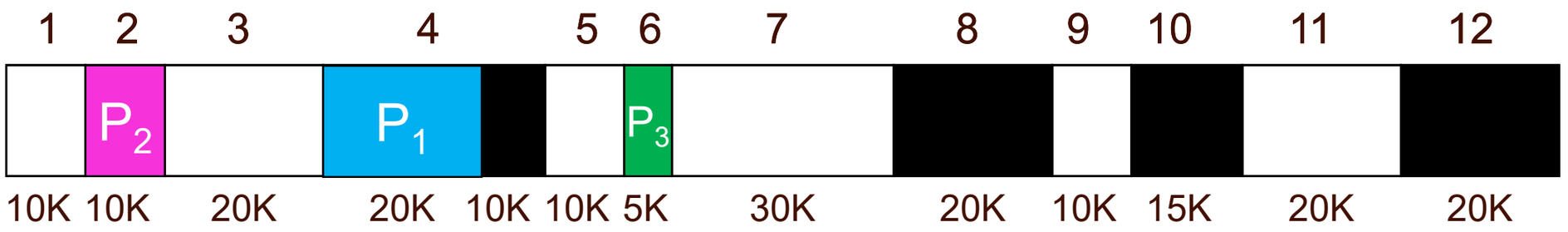
- (i) First-fit (ii) Next-fit (iii) Best-fit (iv) Worst-fit

Contiguous Memory Allocation (Cont..)

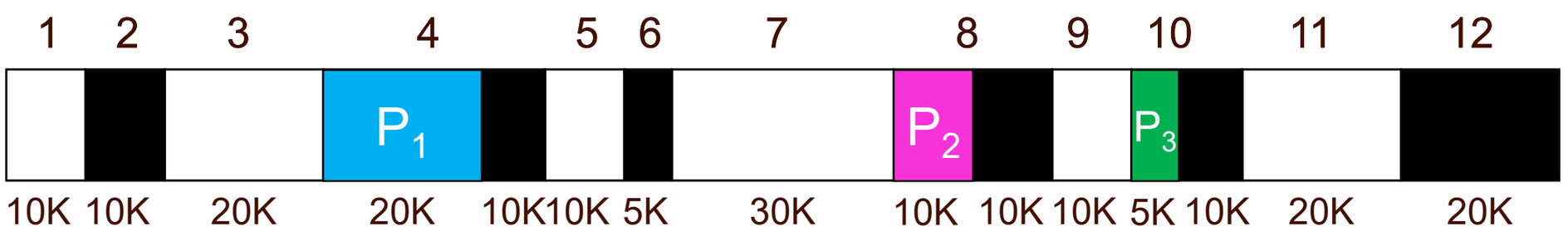
Answers



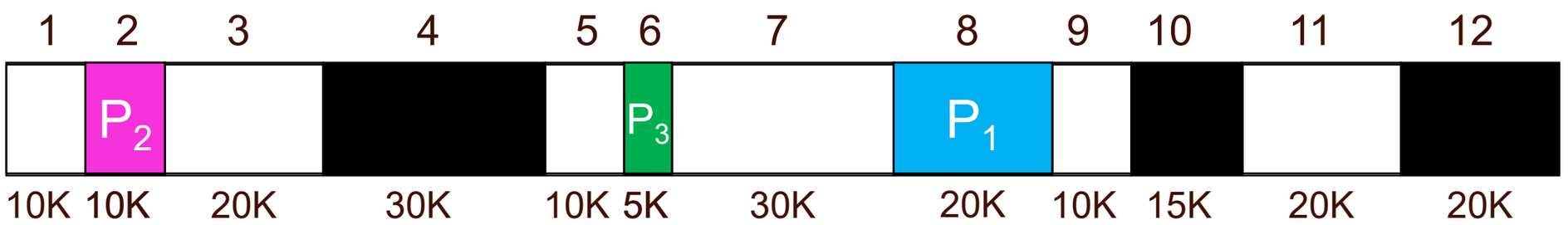
First-fit



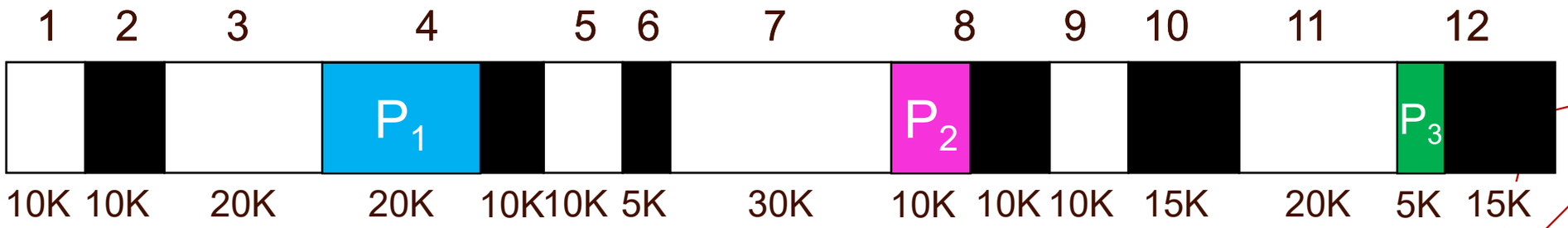
Next-fit



Best-fit



Worst-fit



Summary of Contiguous Memory Allocation

- Early memory management techniques
  - Fixed partitions, dynamic partitions, and relocatable dynamic partitions

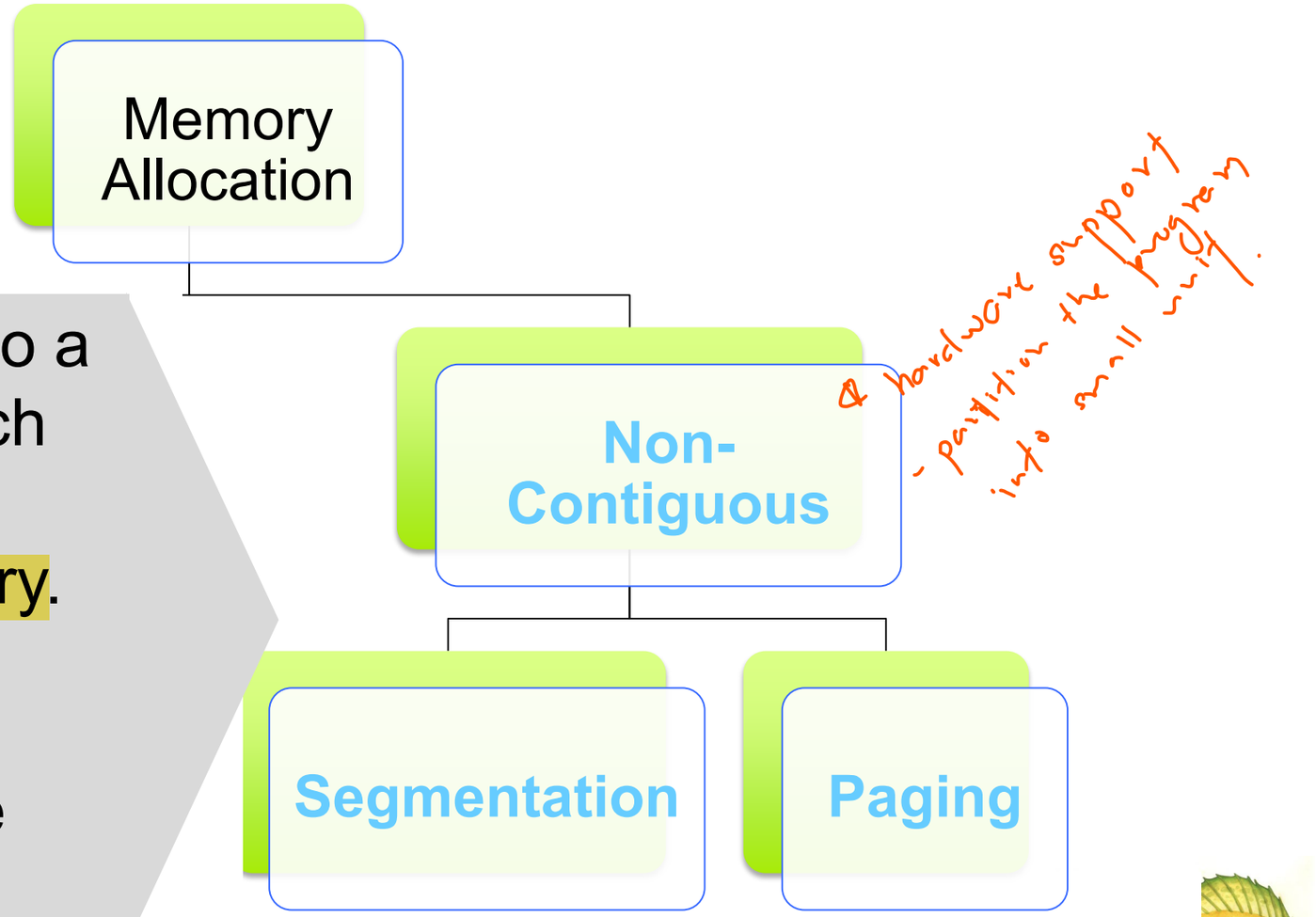
- Common requirements of four memory management techniques
  - Entire program loaded into memory
  - Contiguous storage
  - Memory residency until job completed

- Each places severe restrictions on job size
- Sufficient for first three generations of computers

- New modern memory management trends
  - Common characteristics of memory schemes
    - Programs are not stored in contiguous memory
    - Not all segments reside in memory during job execution

Non Contiguous Memory Allocation

- Partition the a program into a number of small units, each of which can reside in a different part of the memory.
- Need hardware support.
- Various methods to do the partitions:



Non Contiguous : Segmentation

Overview

- Memory-management scheme that supports user view of memory.
- A program is a collection of segments.

Program
- collection of segments
- logical unit

- A segment is a logical unit such as:

main program,	*global variables,*
procedure,	*common block,*
function,	*stack, local variables,*
method,	*symbol table,*
object	*arrays*

Non Contiguous : Segmentation

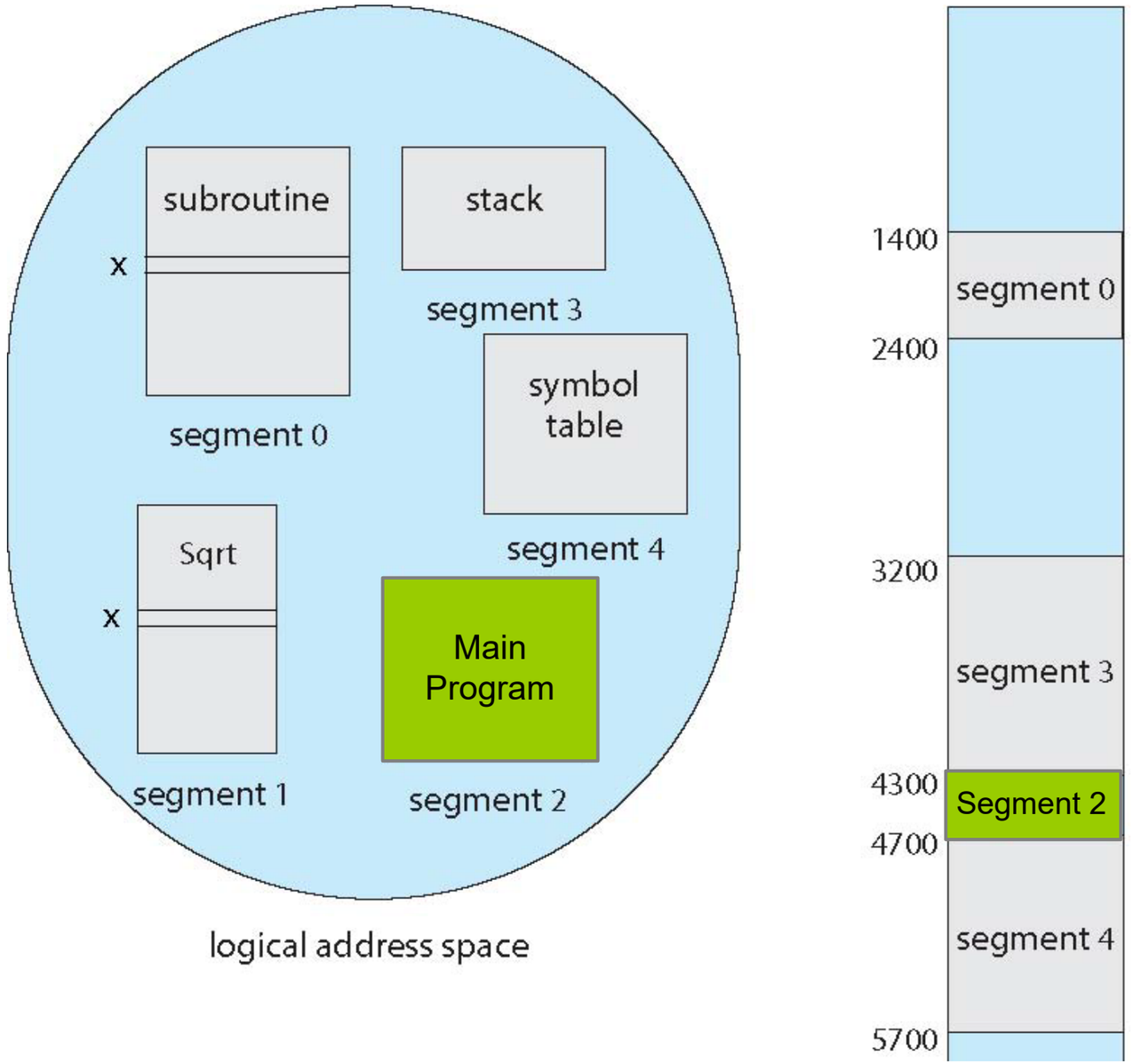


Figure 8.2 Logical and Physical Memory

Non Contiguous :Segmentation

Segmented Memory Allocation

- Each job divided into several segments.
  - Segments are different sizes.
  - One for each module containing related functions.
- Main memory is allocated dynamically.
- Program's structural modules determine segments.
  - Each segment numbered when compiled/assembled.
  - Need **Segment Table**.

Non Contiguous : Segmentation

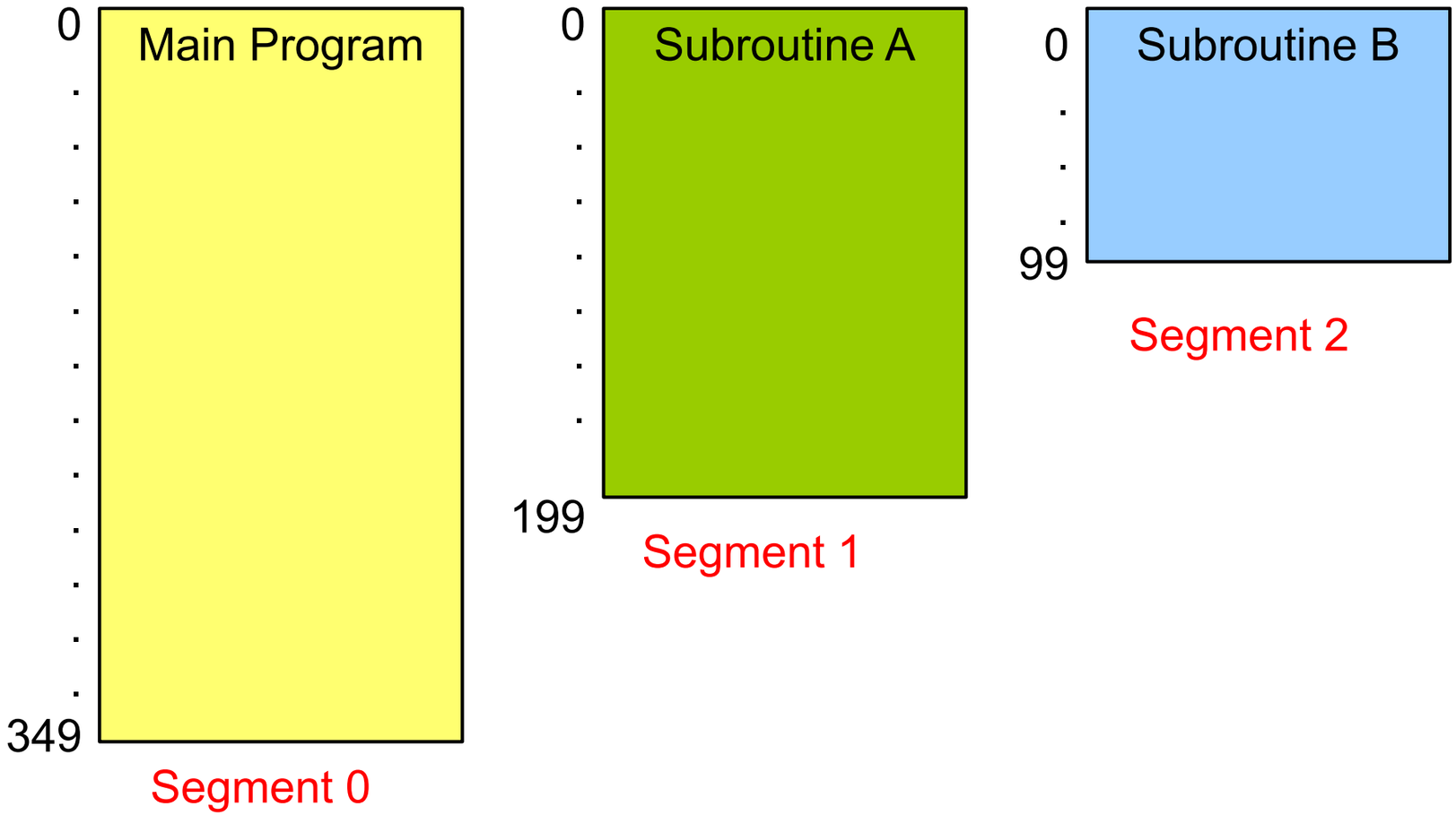
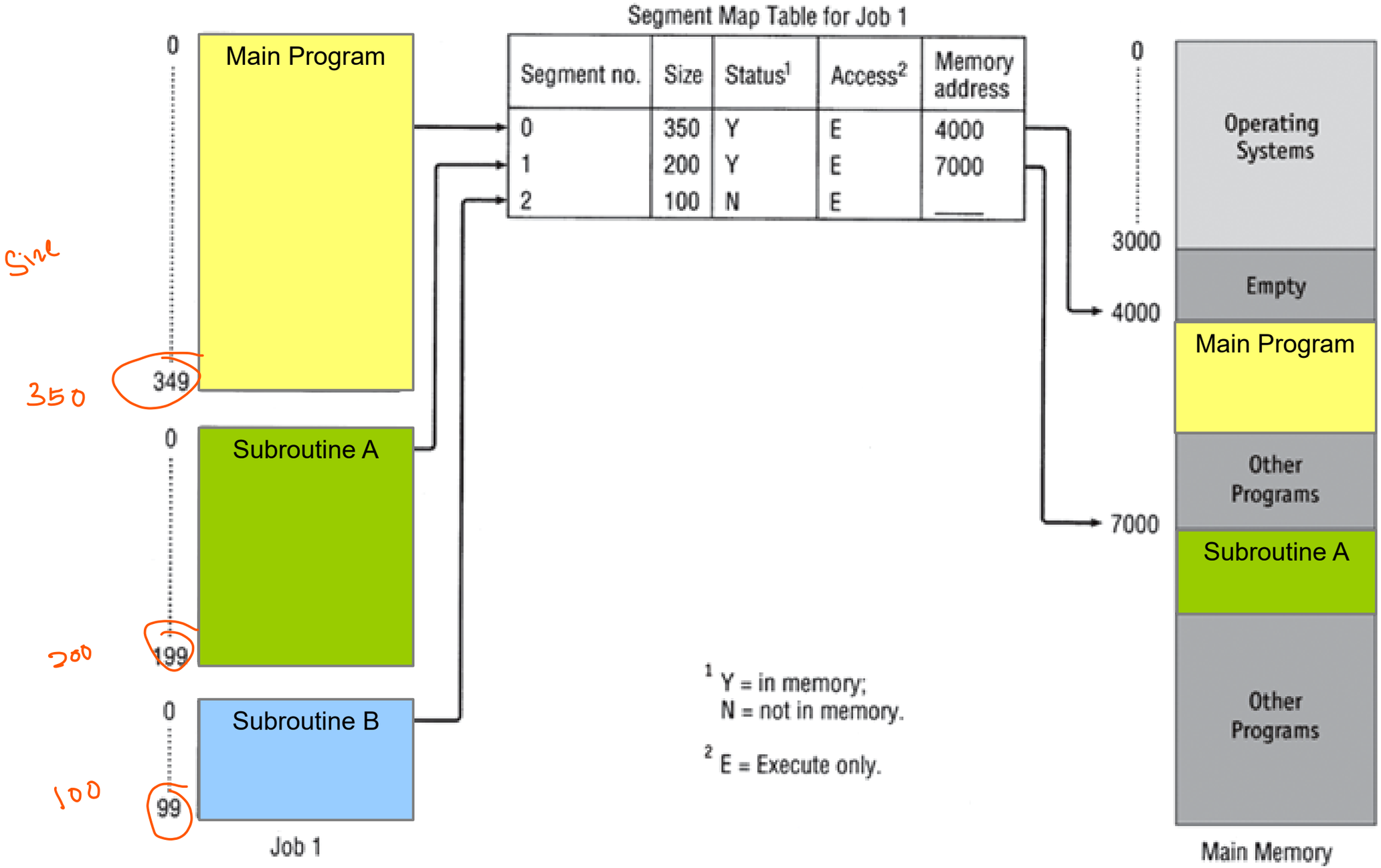


Figure Segmented memory allocation: The job includes a main program, Subroutine A, and Subroutine B. It is one job divided into three segments.

Non Contiguous : Segmentation



(figure 3.14) *The Segment Map Table tracks each segment for Job 1.*

Non Contiguous : Segmentation

- Addressing scheme require Segment number and displacement / offset;

- **Advantages**
  - Internal fragmentation is removed.
  - Memory allocated dynamically.

- **Disadvantages**
  - Difficulty managing variable-length segments in secondary storage.
  - External fragmentation.

Non Contiguous : Segmentation

Segmentation hardware

- Logical address consists of a two tuple:

  <segment-number, offset> displacement

- Need to map a two-dimensional logical addresses to a one-dimensional physical address. Done via **Segment table**:
  - **base** – contains the starting physical address where the segments reside in memory
  - **limit** – specifies the length of the segment.

Non Contiguous : Segmentation

Segmentation hardware

- Segment table is kept in memory:

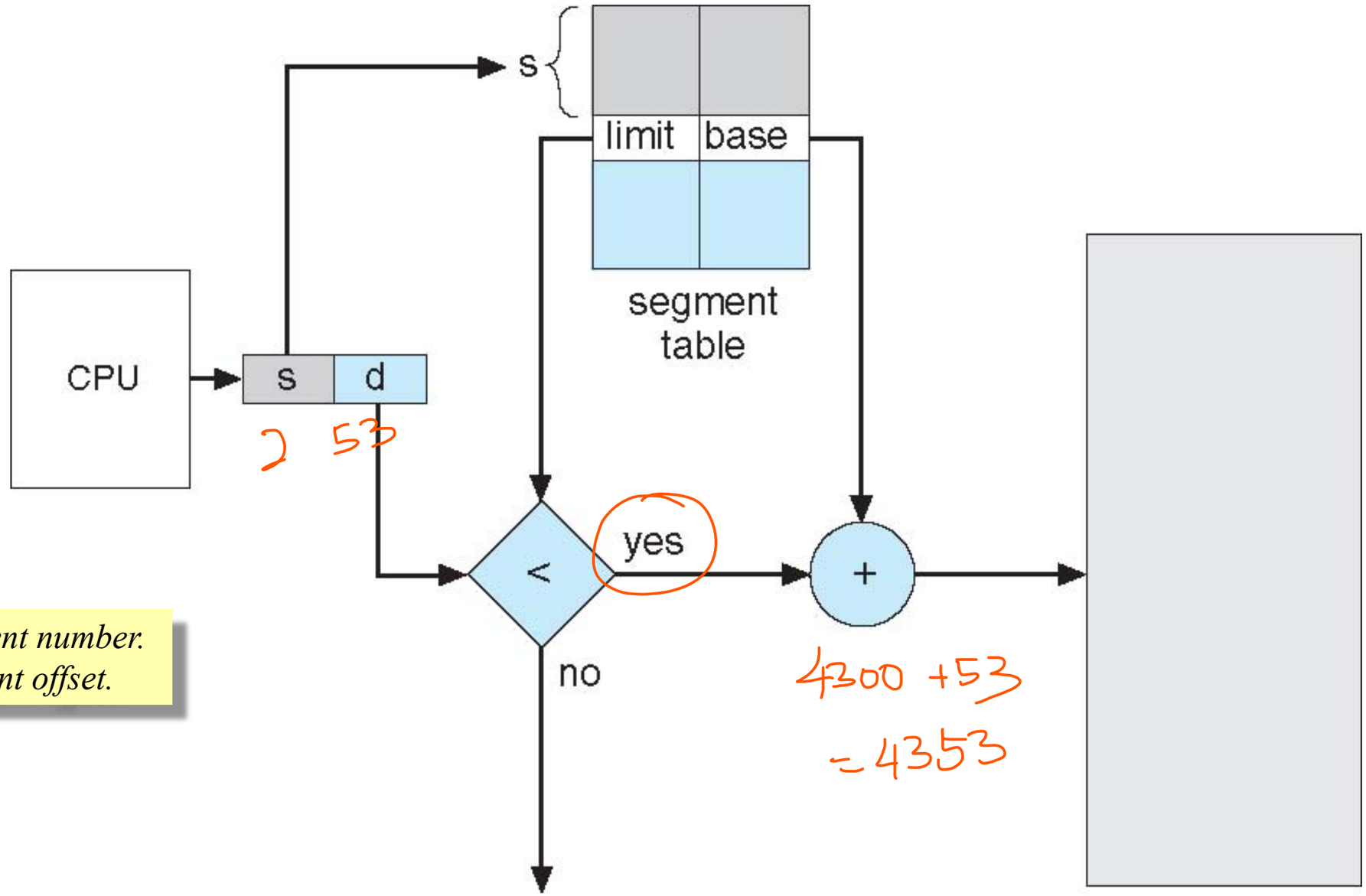
  - **Segment-table base register (STBR)** points to the segment table's location in memory.

  - **Segment-table length register (STLR)** indicates number of segments used by a program;

    ✓ segment number s is legal if $s < \text{STLR}$

Non Contiguous : Segmentation

Segmentation hardware



s : Segment number.
d : Segment offset.

Figure 8.8 Segmentation hardware.

Non Contiguous : Segmentation

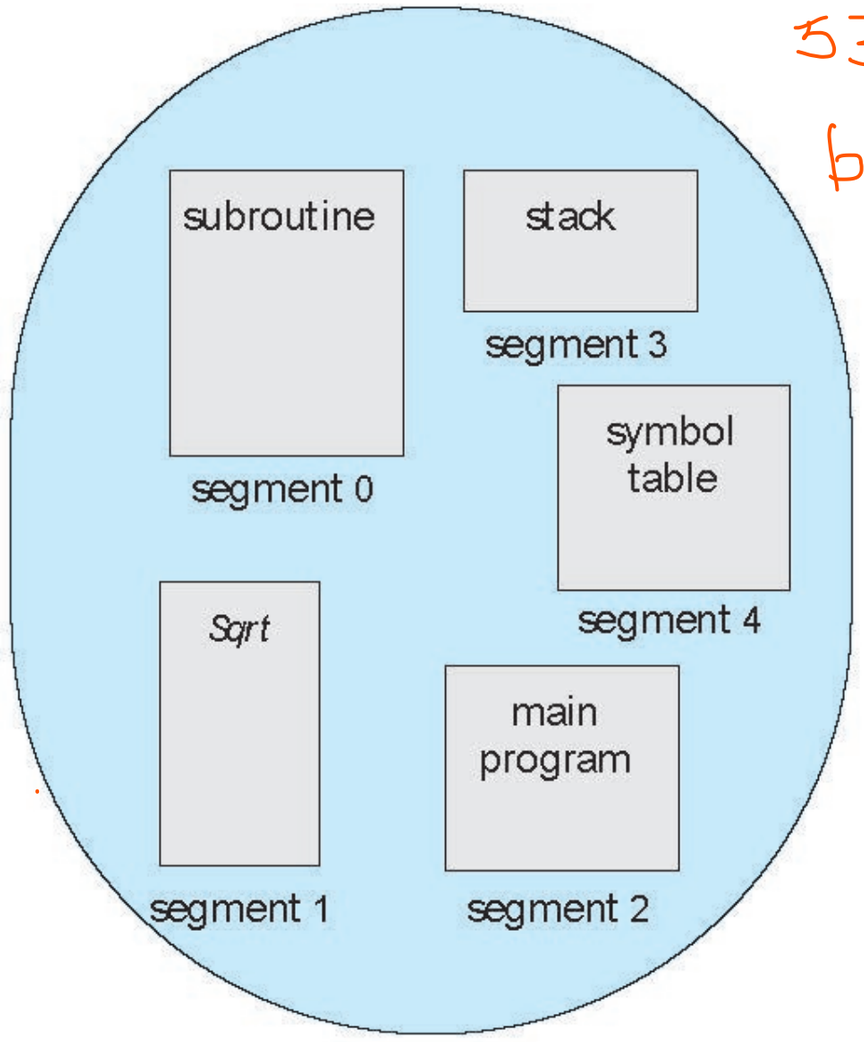
Example

What is the physical memory address for the logical address 2 : 53?

(Page: 360)

Segment 2
= 400 bytes long

Segment 2
53 < limit (400)
base (4300) + 53 = 4353



	limit	base
0	1000	1400
1	400	6300
2	400	4300
3	1100	3200
4	1000	4700

segment table

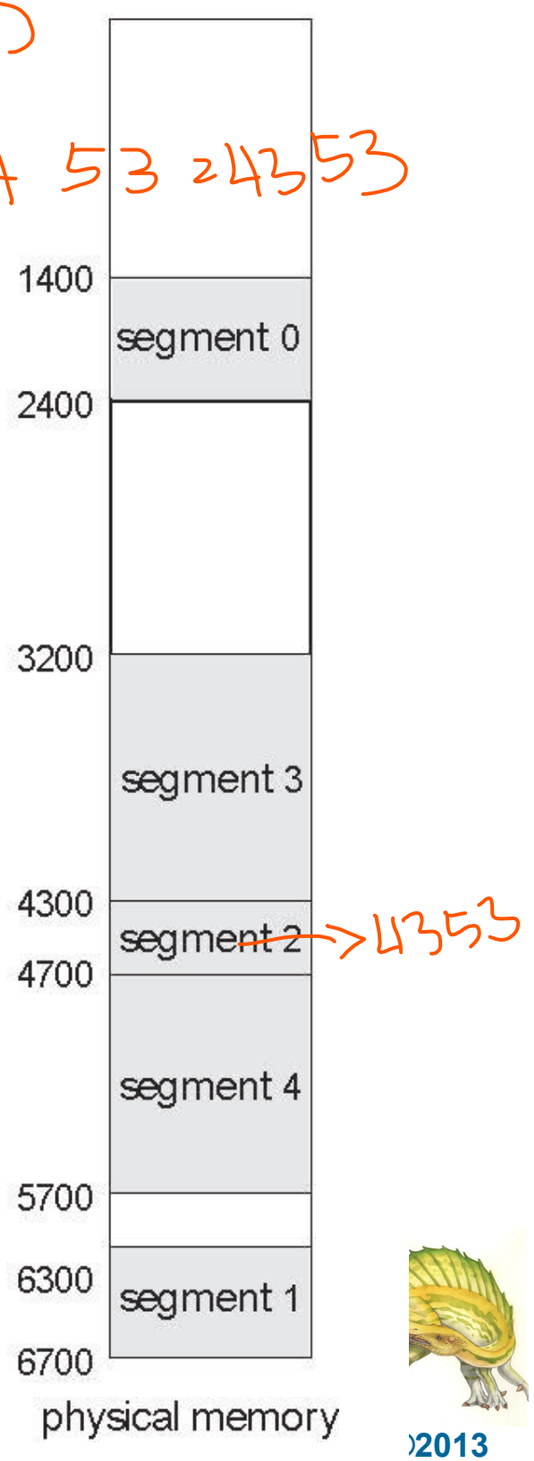


Figure 8.9 Example of segmentation.

Non Contiguous : Segmentation

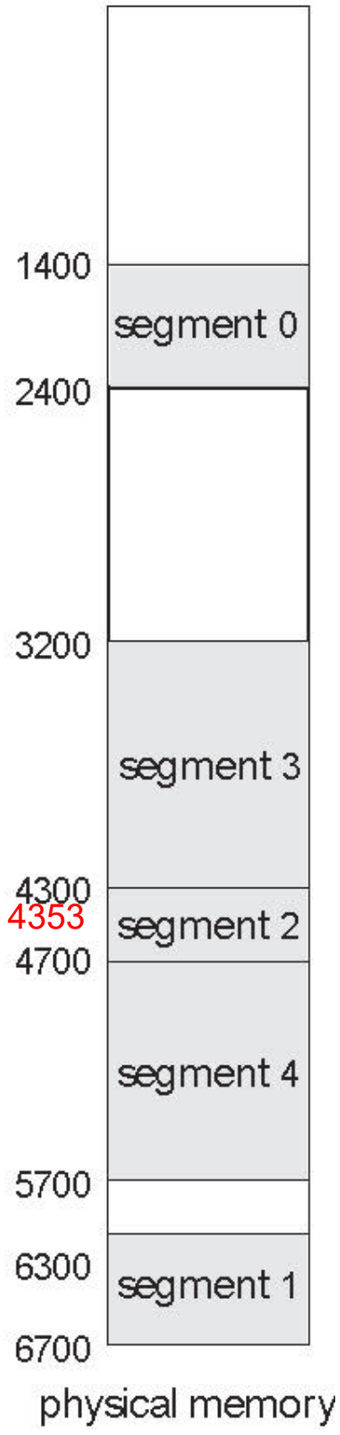
Solution

Segment 2 is 400 bytes long, and begins at location 4300.

→ Physical address = 4300 + 53 = **4353**

	limit	base
0	1000	1400
1	400	6300
2	400	4300
3	1100	3200
4	1000	4700

segment table



Non Contiguous : Segmentation

Exercise

What is the physical memory address for the logical address?

a) 3 : 852

b) 0 : 1222

Segment 3
852 < 1100
3200 + 852 = 4052

Segment 0
1222 > 1000

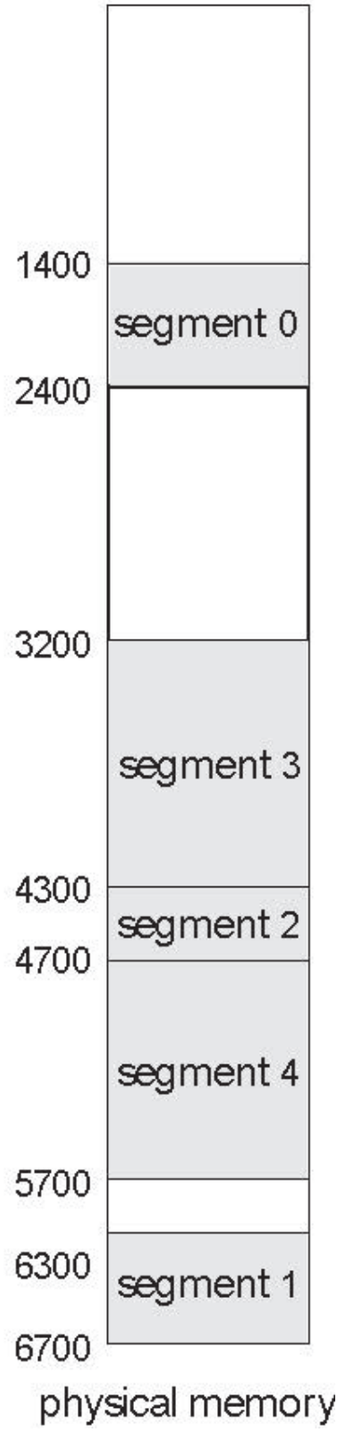
(Page: 360)

- **Answer:**
- **A reference to segment 3, byte 852, is mapped to Physical memory address: 3200 (the base of segment 3) + 852 = 4502.**
- **A reference to byte 1222 of segment 0 would result in a trap to the OS, as this segment is only 1000 bytes long.**

(byte long)

	limit	base
0	1000	1400
1	400	6300
2	400	4300
3	1100	3200
4	1000	4700

segment table



Non Contiguous : Paging

Overview

- Physical address space of a process can be **non-contiguous**.

- Process is divided into **fixed-size** blocks, each of which may reside in a different part of physical memory.

- Divide physical memory into **fixed-size** blocks called **frames**:
  - Size of a frame is power of 2 between 512 Bytes and 8192 bytes (2^9 - 2^{13}).

- Divide logical memory into blocks of same size as frames called **pages**.

Non Contiguous : Paging

Overview

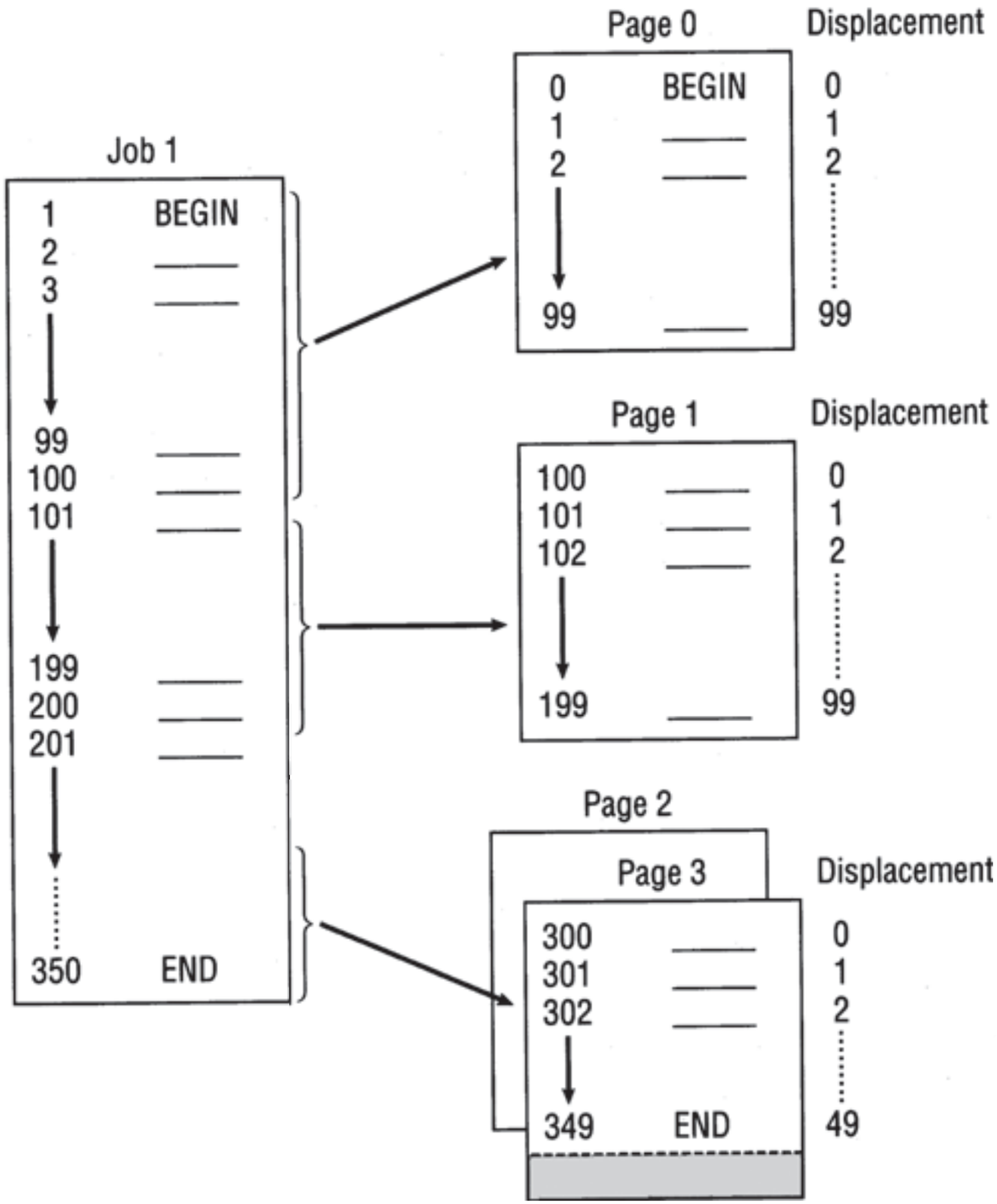
- Backing store, where the program is permanently residing, is also split into storage units (called **blocks**), which are the same size as the **frames** and **pages**.

- Physical memory allocated whenever the latter is available:
  - Avoids external fragmentation.
  - Still have Internal fragmentation.

Non Contiguous : Paging

Overview

- Program is 350 lines long and is divided into 4 page frames of 100 lines each.



Non Contiguous : Paging

Overview

Consider a program with 350 lines long. Each page frame in a system can hold 100 lines. What happen in paged memory allocation?

Job 1

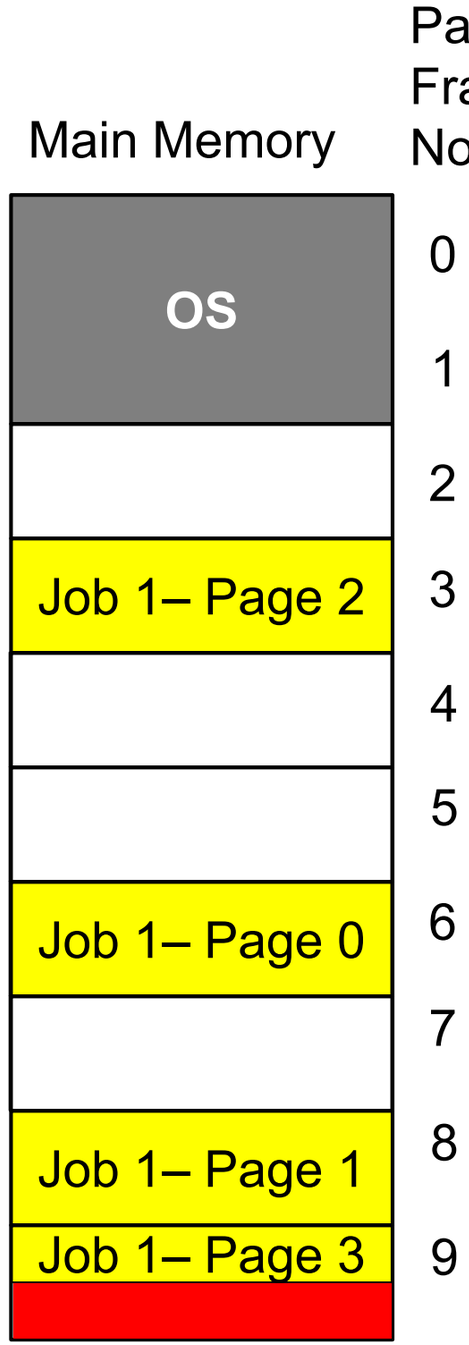
100 lines	*Page 0*
100 lines	*Page 1*
100 lines	*Page 2*
50 lines	*Page 3*

Page size = Frame size

Page Table	
0	6
1	8
2	3
3	9

Page# → Frame#

Internal Fragmentation



Non Contiguous : Paging

Overview

- **Advantages**:
  - Allows job allocation in non-contiguous memory.
  - Efficient memory use.

- **Disadvantages**:
  - Increased overhead from address resolution.
  - Internal fragmentation in last page.

- Page size selection is crucial:
  - **Too small**: generates very long page table.
  - **Too large**: excessive internal fragmentation.

Non Contiguous : Paging

Basic method

- Paging keeps track of all free frames.
- To run a program of size N pages, need to find N free frames and load program from backing store.
- Set up a **page table** to translate logical to physical addresses.
- Page table is kept in memory:
  - **Page-table base register (PTBR)** points to the page table.
  - **Page-table length register (PTLR)** indicates size of the page table.

Non Contiguous : Paging

Address translation scheme

- Assume the logical address space is 2^m.
- Assume page size is 2^n
- Address generated by CPU is divided into:
  - **Page number** (***p***) – used as an index into a **page table** which contains base address of each page in physical memory. Size of ***p*** is "m – n"
  - **Page offset** (***d***) – combined with base address to define the physical memory address that is sent to the memory unit. Size of ***d*** is "n".

Non Contiguous : Paging

Example

Considered size of logical address is 16 MB with it page size 1 KB. Find the p bits for the page number of the address.

Solution

– size of logical address = 16 MB = 16 x 2^{20} Bytes
= 2^4 x 2^{20} = 2^{24}

– page size = 1 KB = 1024 Byte = 2^{10}

Page number | Page offset

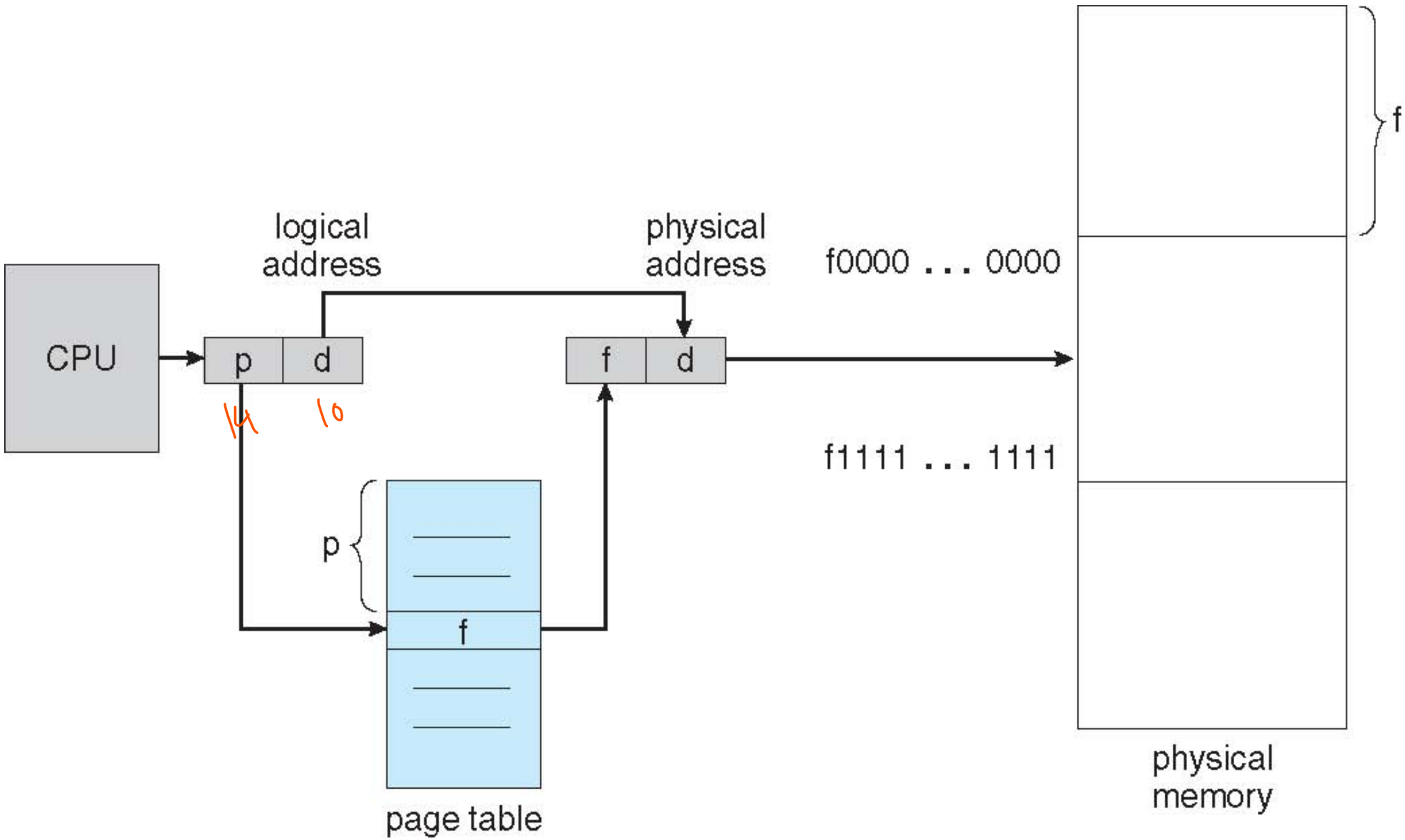
p	d

$= m - n$
$= 24 - 10$
$= 14$ bits

$= n$
$= 10$ bits

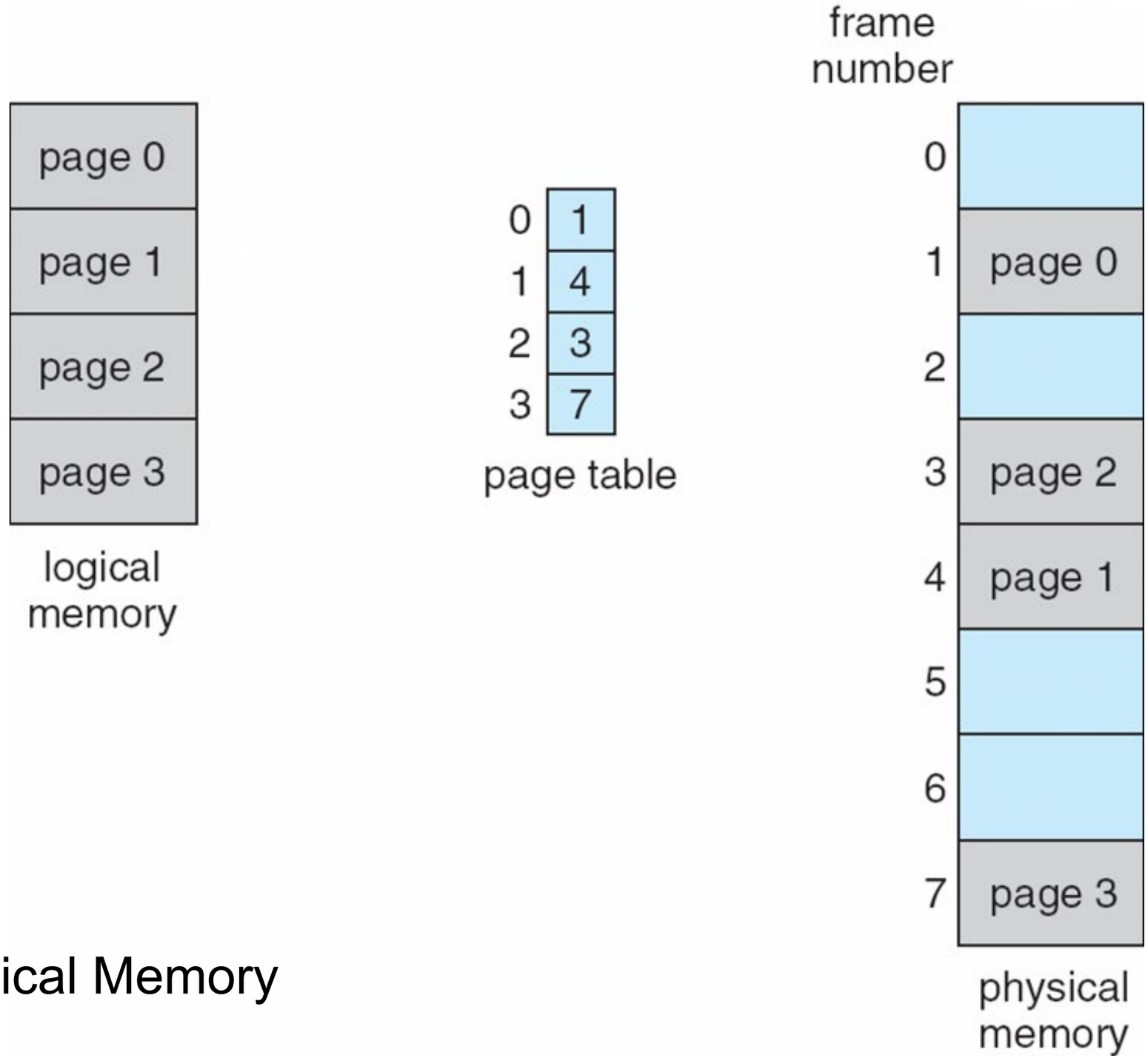
Non Contiguous : Paging

Paging hardware



Non Contiguous : Paging

Example :



Paging Model:
Logical and Physical Memory

Non Contiguous : Paging

Example

n=2 and *m*=4

Page size = frame size = 2^n = 4-byte pages

Logical memory = Total page x Page size

= 4 x 4 = 16 Bytes = $2^m = 2^4$

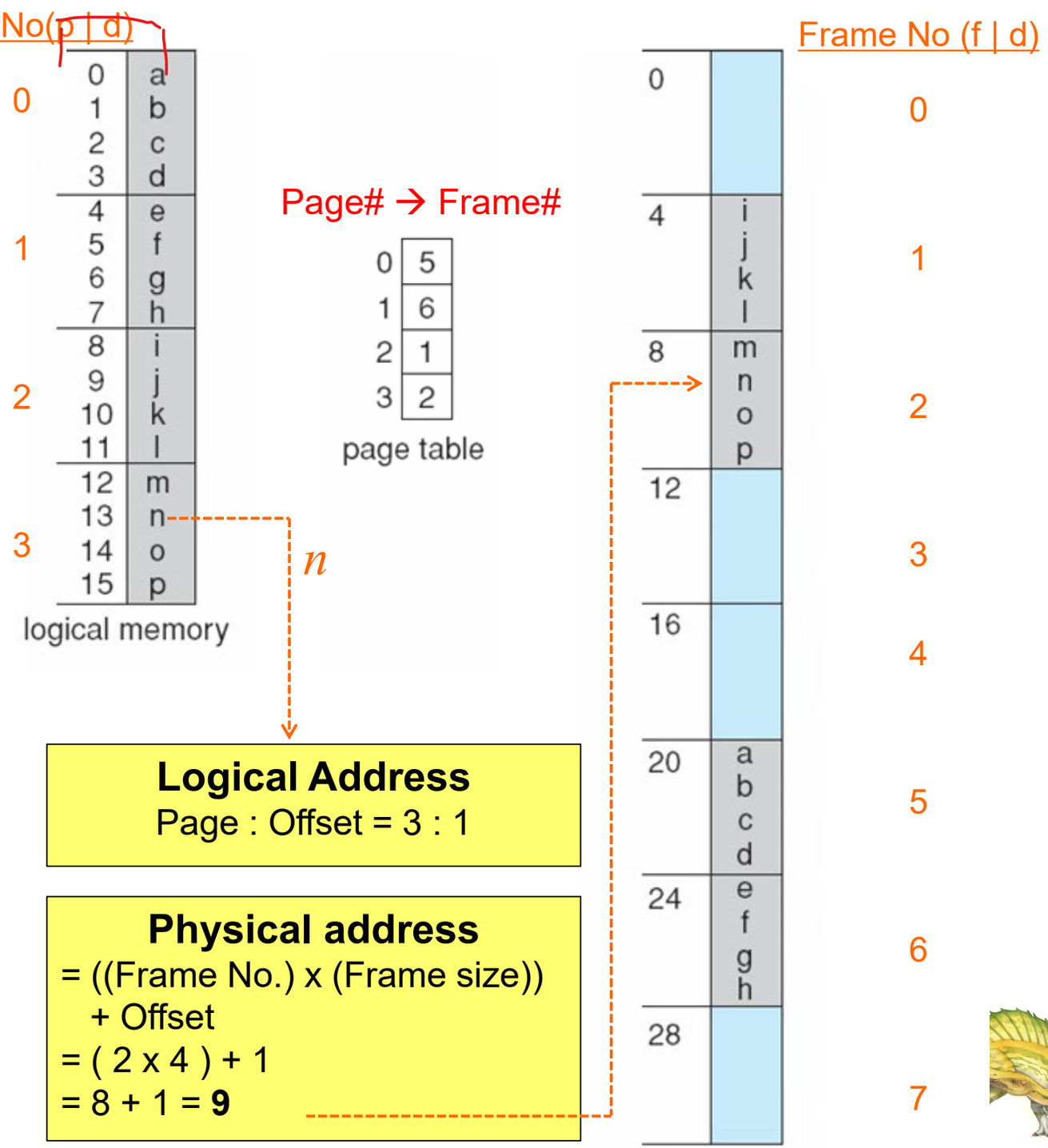
Physical memory = Total Frame x Frame size

= 8 x 4 = 32 Bytes

Physical address = (Frame No x Frame Size) + Offset

Logical address = (Page No x Page Size) + Offset

Get the physical address for *n*.



Non Contiguous : Paging

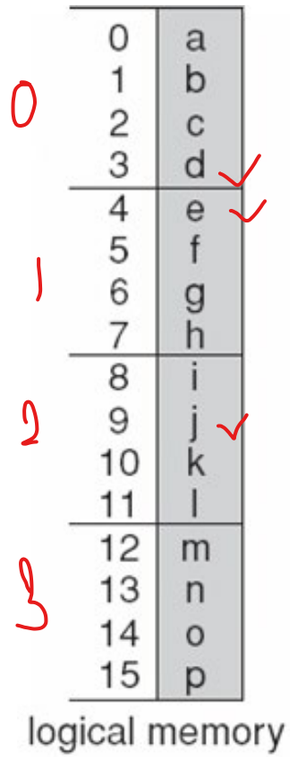
Exercise

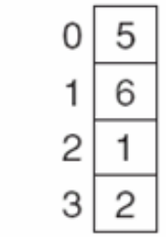
What is the physical address for the following logical addresses with given the number of the page and the offset?

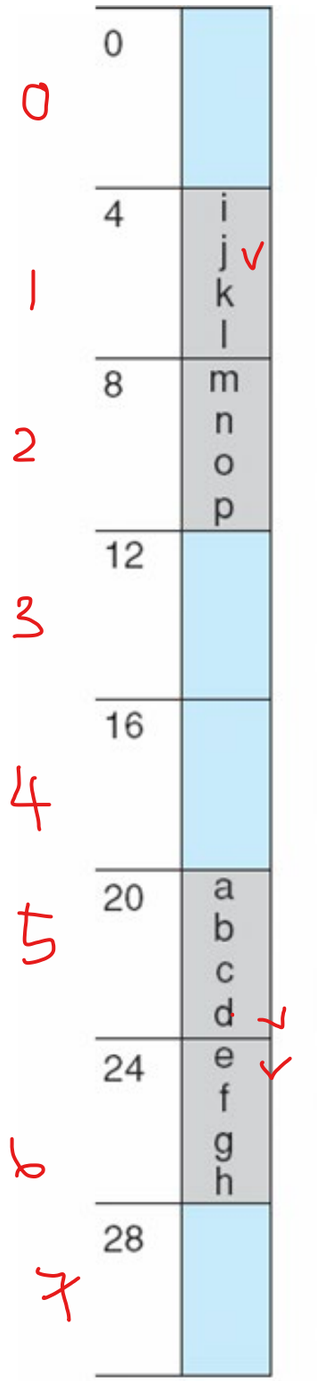
a) 0 : 3 $20+3=23$

b) 1 : 0 $24+0=24$

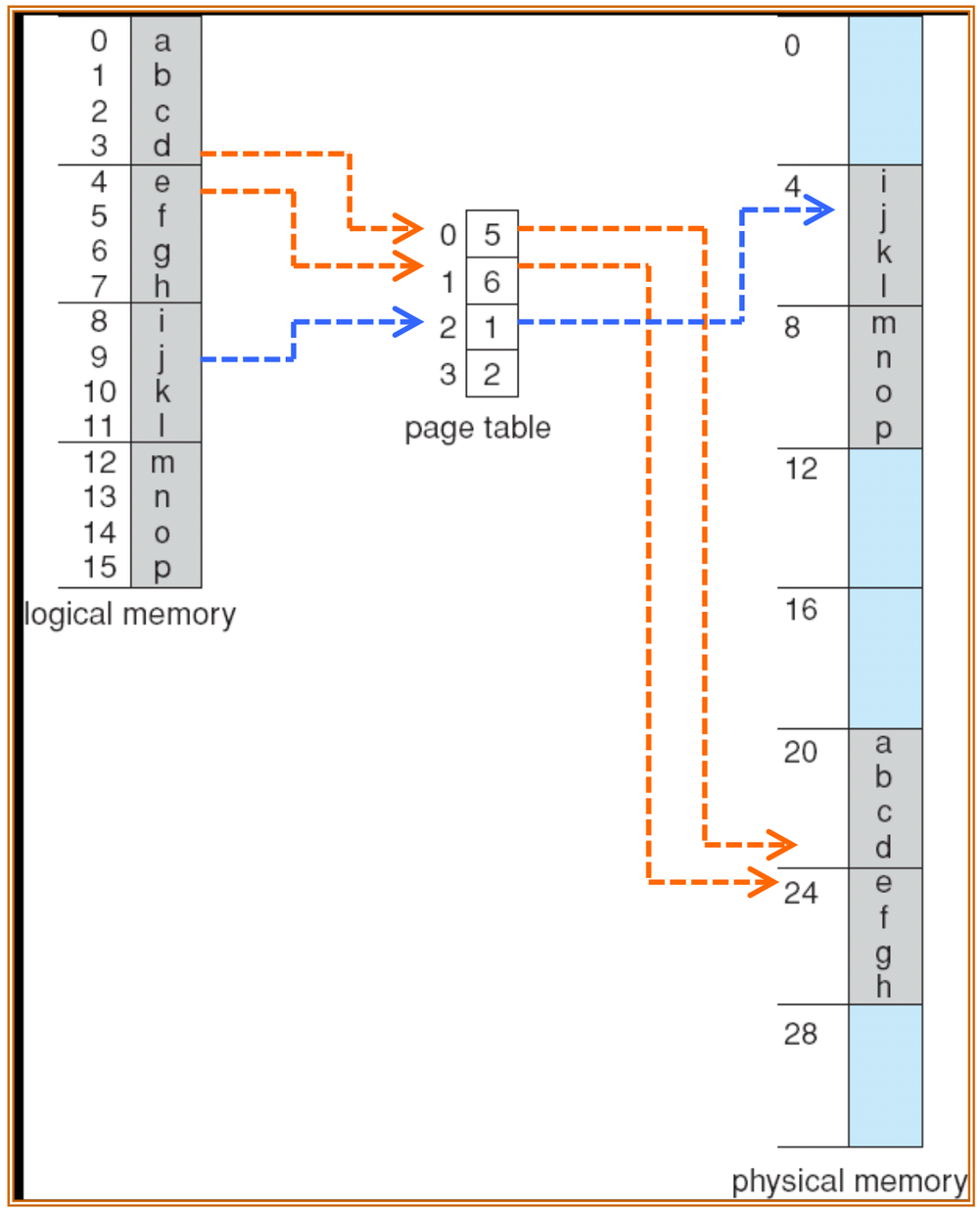
c) 2 : 1 $4+1=5$







Non Contiguous : Paging



32-Byte memory and 4-Byte pages

Page	Offset
0	3

=(5 x 4) + 3
= 20 + 3
= **23**

Page	Offset
1	0

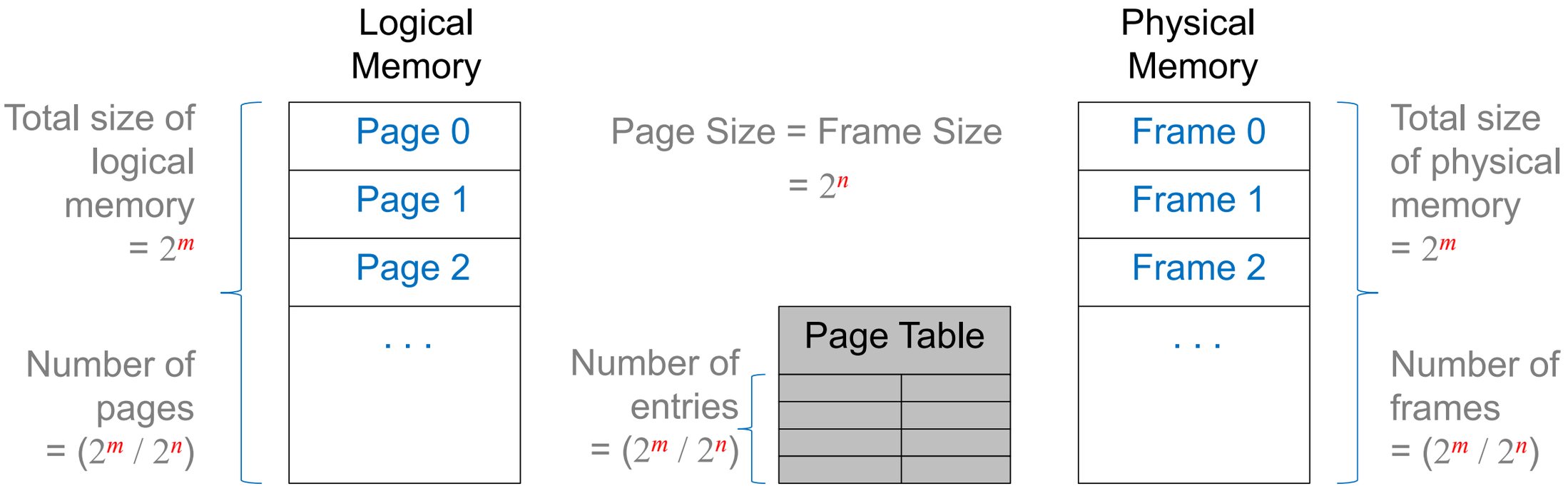
= (6 x 4) + 0
= 24 + 0
= **24**

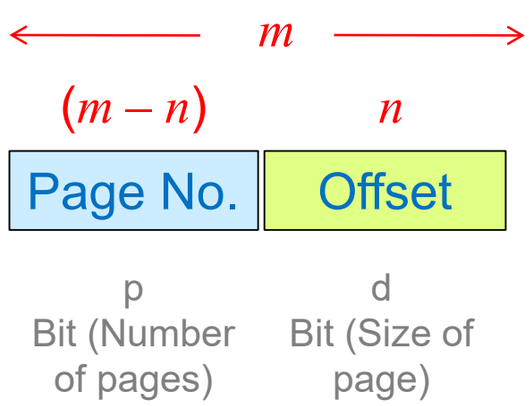
Page	Offset
2	1

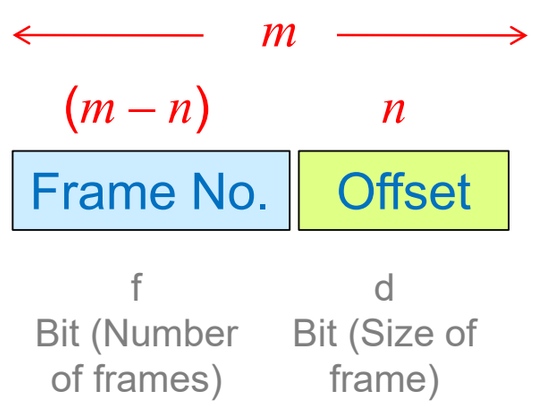
= (1 x 4) + 1
= 4 + 1
= **5**

Non Contiguous : Paging

Summary
Paging System







Non Contiguous : Paging

Exercise

Consider a paging system with 4 pages of logical address space, a page size of 4 bytes and a physical memory of 64 bytes as depicted in the following diagram.

Logical Memory

Logical address	Content
0	A
1	B
2	C
3	D
.	.
.	.
8	G
9	H
10	I
11	J
12	K
13	L
14	M
15	N

a) What is the frame size of the physical memory?

b) How many frames of the physical memory?

c) List the logical addresses for the following page.

Page No.	Logical Address
1	4, 5, 6, 7
3	12, 13, 14, 15

Non Contiguous : Paging

d) Calculate the page offset for logical memory location that stores the data *H*.

ב : ו

e) Suppose a page table for the above paging system is as follows:

0	1
1	3
2	8
3	10

Calculate the physical address in binary for data *H*.

Non Contiguous : Paging

Solutions:

Logical Memory

Logical address	Content
0	A
1	B
2	C
3	D
.	.
.	.
8	G
9	H
10	I
11	J
12	K
13	L
14	M
15	N

Page 0 (4 bytes)

Page 1

Page 2

Page 3

Question (a):

Frame Size = Page Size = 4 bytes = 2^2

Physical Memory

Frame 0

. . .

Frame 2

Frame 3

Size Physical memory = 64 bytes = 2^6

Question (b):

Number of frames = (2^6 / 2^2) = 2^4 = 16

Non Contiguous : Paging

(c) List the logical addresses for the following pages: [2 mark]

Question (c):

Page No.	Logical addresses
1	4, 5, 6, 7
2	8, 9, 10, 11

(d) Calculate the page offset for logical memory location that stores data **H**. [1 mark]

Logical Memory

Logical address	Content
0	**A**
1	**B**
2	**C**
3	**D**
. .	. .
8	**G**
9	**H**
10	**I**
11	**J**
12	**K**
13	**L**
14	**M**
15	**N**

Page 0

Page 1

Page 2

Page 3

Question (d):

H : logical address = 9

Logical address = (Page# x Page size) + $offset_H$

$Offset_H$ = 9 – (2 x 4)
= 9 – 8 = 1

Non Contiguous : Paging

(e) Suppose a page table for the above paging system is as follows:

Page	Frame
0	1
1	3
2	8
3	10

Calculate the physical address in binary for data **H** [3 marks]

Logical Memory

Logical address	Content
0	**A**
1	**B**
2	**C**
3	**D**
.	.
.	.
8	**G**
9	**H**
10	**I**
11	**J**
12	**K**
13	**L**
14	**M**
15	**N**

Page 0 | Page 1 | Page 2 | Page 3

Offset: 0, 1, 2, 3

Page → Frame

Page	Frame
0	1
1	3
2	8
3	10

Physical address :
= (Frame# x Frame size) + $offset_H$
= (8 x 4) + 1
= 32 + 1 =
= 33_{10} → 100001_2

Non Contiguous : Paging

Exercise

Consider a paging system with 2^{10} pages of logical address space, a page size of 256 bytes and a physical memory of 2^{24} bytes. Answer the following questions. [8 *marks*]

a) What is the total size of the logical address space?

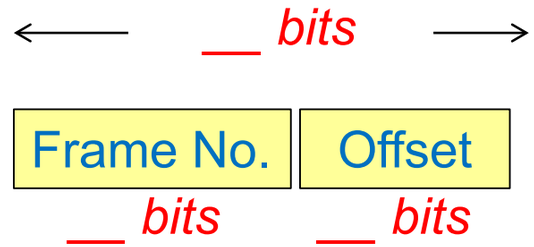
b) What is the frame size of the physical memory?

c) How many frames of the physical memory?

d) How many number of entry in the page table?

e) Calculate the offset bits for a page of virtual memory.

f) Draw the page/frame offset of the logical/physical memory address.



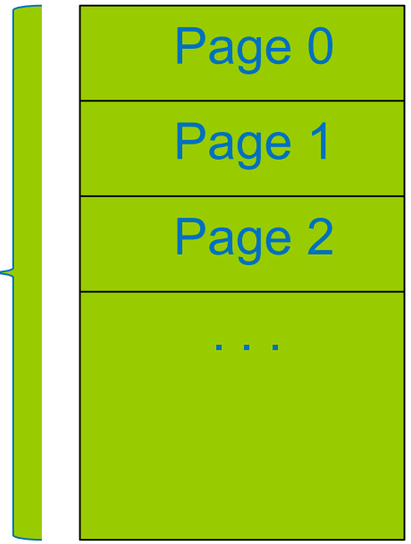
Non Contiguous : Paging

Solutions:

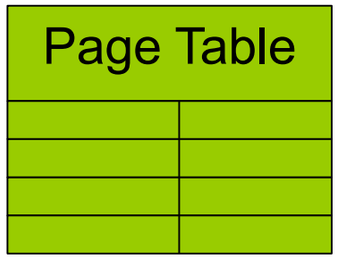
Space of logical memory or number of pages = 2^{10}

Total size of logical Memory = 2^m = (No.Pages * page size) = (2^{10} * 2^8) = 2^{18}

Logical Memory

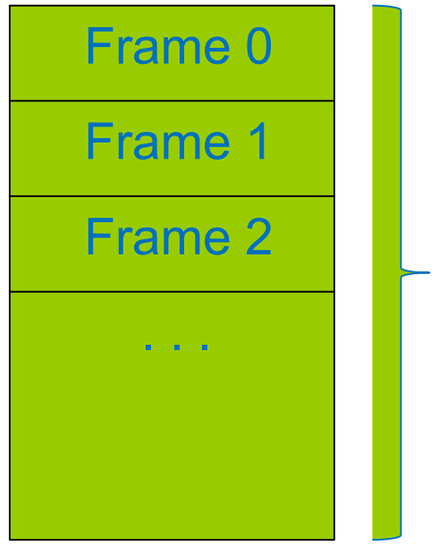


Page Size = Frame Size = 2^n = 256 bytes = 2^8



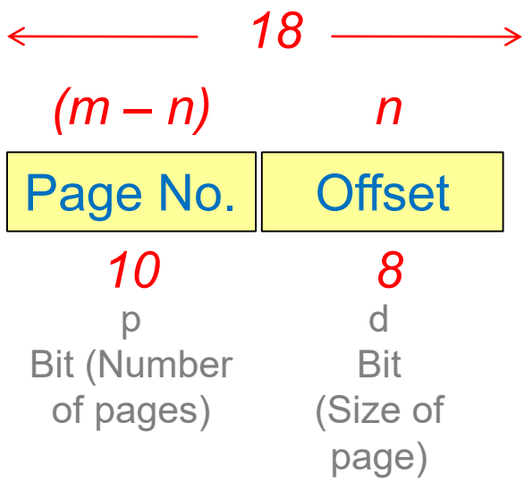
Number of entries = Number of pages = 2^{10} = 1024 entries

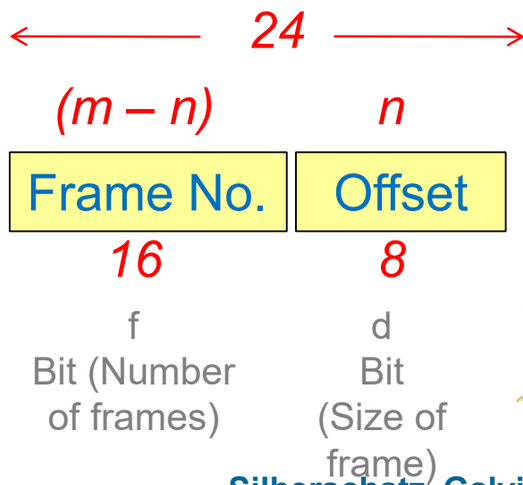
Physical Memory



Size of physical memory = 2^m = 2^{24}

Number of frames = (2^{24} / 2^8) = 2^{16}





Non Contiguous : Paging

Memory protection

- Memory protection implemented by associating **protection bits** with each frame to indicate if "*read-only*" or "*read-write*" access is allowed.
  - Can also add more bits to indicate "*execute-only*" and so on.
- **Valid-invalid** bit attached to each entry in the page table:
  - "*valid*" indicates that the associated page is in the process' logical address space, and is thus is a legal page.
  - "*invalid*" indicates that the page is not in the process' logical address space.
  - Or use **Page-Table Length Register (PTLR)**
- Any violations result in a trap to the kernel.

Non Contiguous : Paging

Example

(Page: 370)

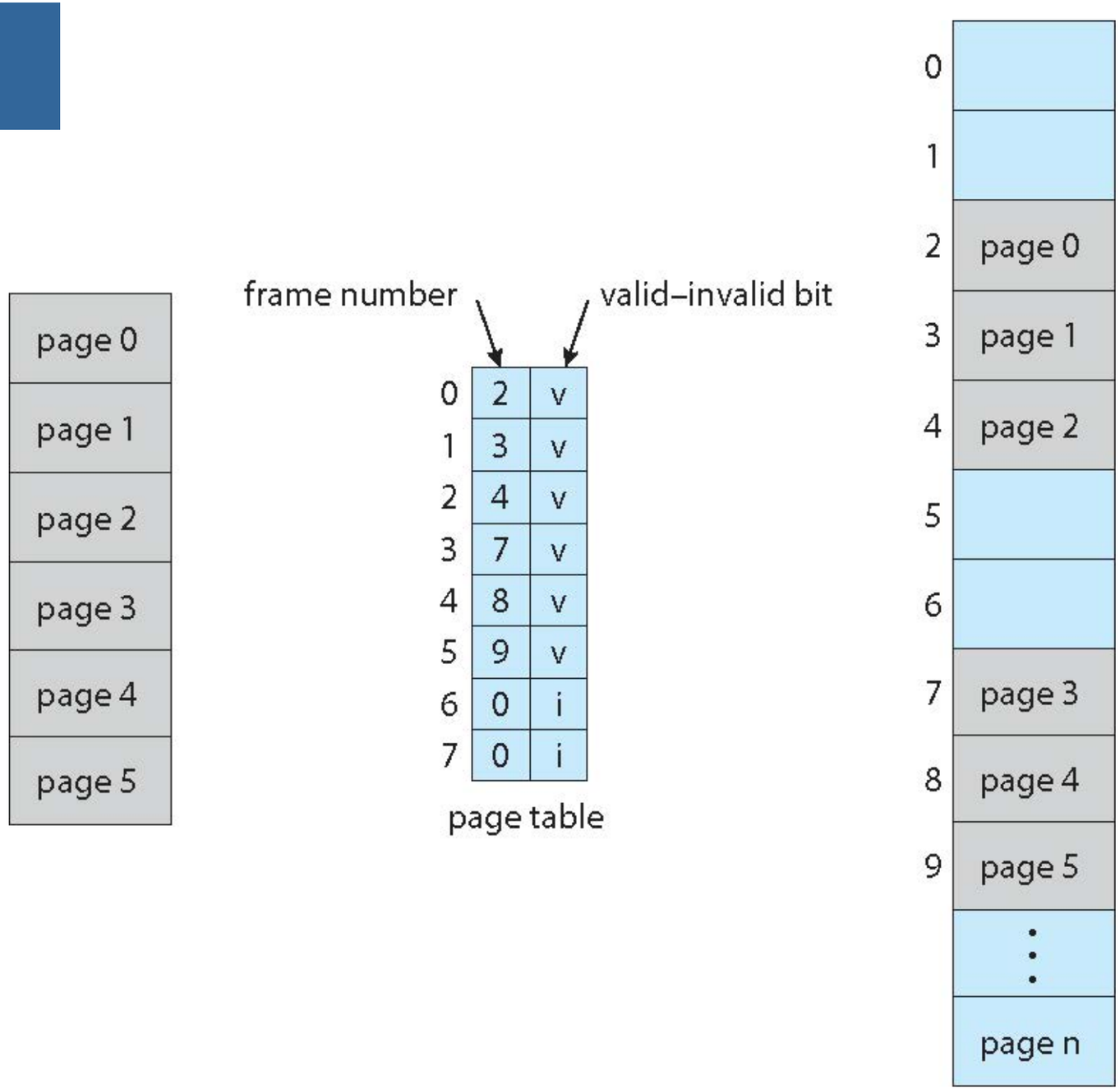


Figure 8.15 Valid (v) or Invalid (i) Bit In A Page Table.

Summary

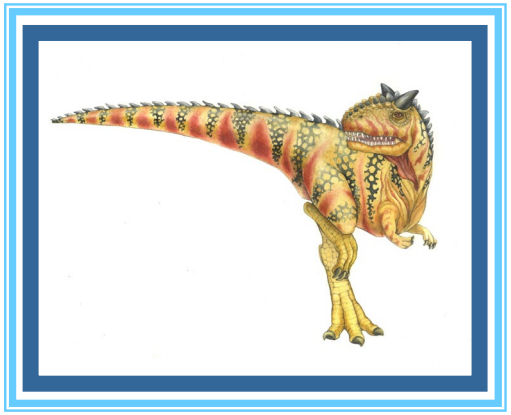
- Memory-management algorithms for multiprogrammed operating systems range from the simple single-user system approach to segmentation and paging.

- The most important determinant of the method used is the **hardware** provided.
  - ✓ Every memory address generated by the CPU must be **checked** for legality and possibly mapped to a physical address.
  - ✓ Cannot be implemented in software.

Summary

- The various memory-management algorithms for multiprogramming differ in many aspects:
  - ✓ Hardware support
  - ✓ Performance
  - ✓ Fragmentation
  - ✓ Relocation
  - ✓ Swapping
  - ✓ Sharing
  - ✓ Protection

Chapter 9: Virtual Memory



Silberschatz, Galvin and Gagne ©2013

Chapter 9: Virtual Memory

- 9.1 Background
- 9.2 Demand Paging
- 9.4 Page Replacement
- 9.5 Allocation of Frames
- 9.6 Thrashing
- 9.11 Summary

Objectives

- To describe the benefits of a virtual memory system.
- To explain the concepts of demand paging, page-replacement algorithms, and allocation of page frames.
- To discuss the principle of the working-set model.
- To examine the relationship between shared memory and memory-mapped files
- To explore how kernel memory is managed.

9.1 Background

- Code needs to be in memory to execute, but entire program rarely used.
- Entire program code not needed at same time.
- Consider ability to execute partially-loaded program:
  - ✓ Program no longer constrained by limits of physical memory.
  - ✓ Each program takes less memory while running; hence more programs run at the same time.
    - Increased CPU utilization and throughput with no increase in response time or turnaround time.
  - ✓ Less I/O needed to load or swap programs into memory; hence, each user program runs faster.

Background

Concepts

Virtual memory – separation of user logical memory from physical memory:

- Only part of the program needs to be in memory for execution.
- Logical address space can therefore be much larger than physical address space
- Allows address spaces to be shared by several processes.
- Allows for more efficient process creation.
- More programs running concurrently.
- Less I/O needed to load or swap processes.

Background

Virtual address space of a process refers to the logical (or virtual) view how it stored in memory.

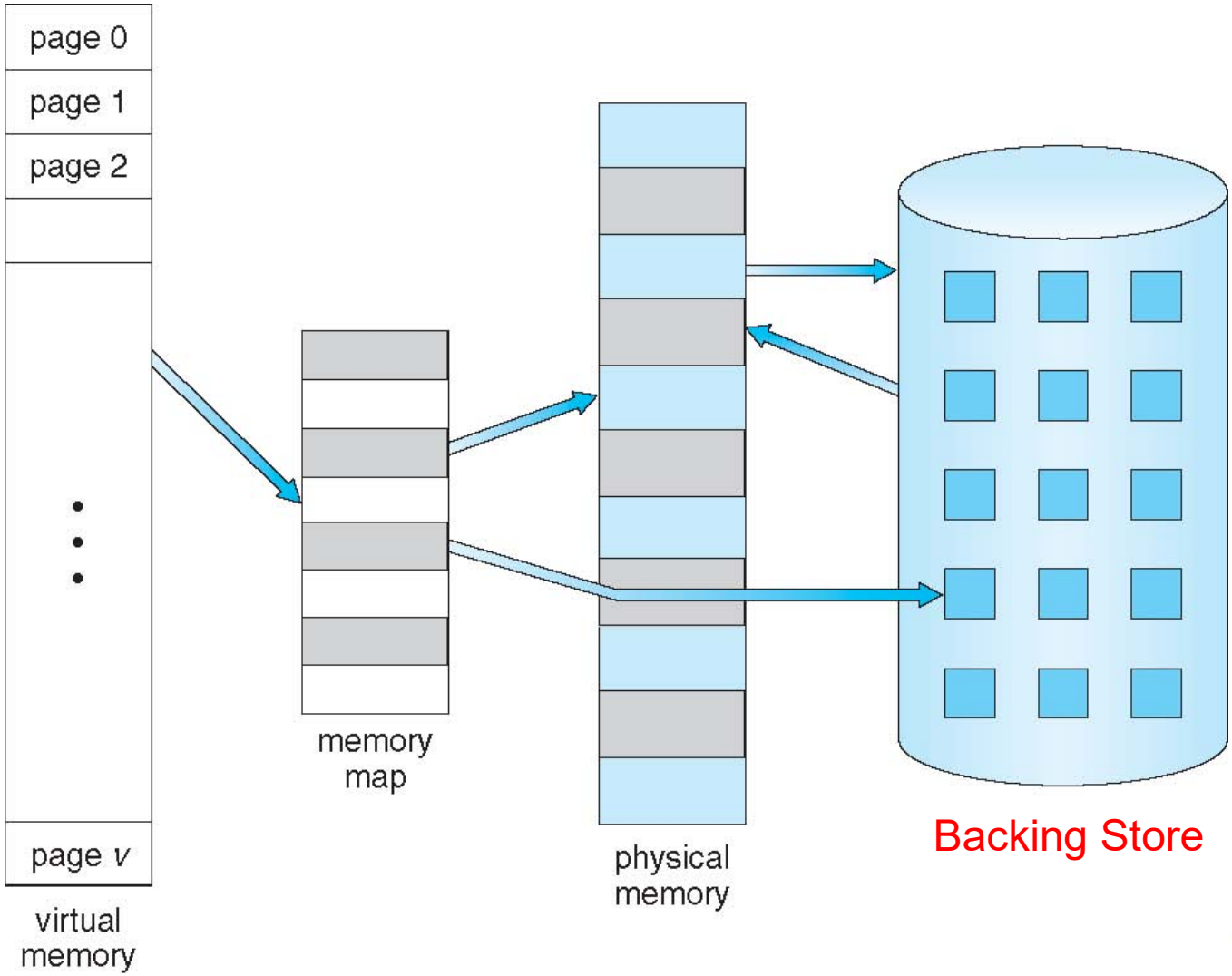
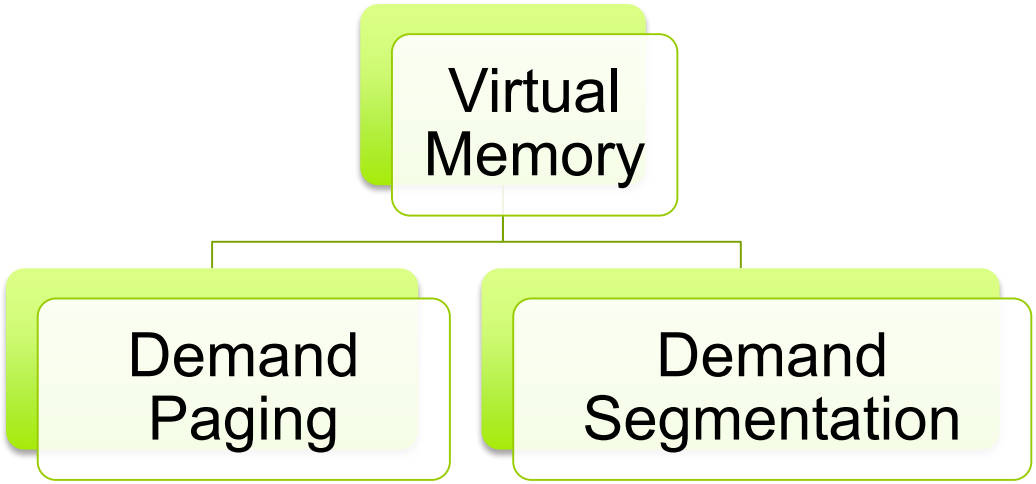


Figure 9.1 Virtual Memory That is Larger Than Physical Memory.

Background

- **Virtual memory** can be implemented via TWO approaches:



(Not cover in this course)

Virtual Memory: Demand Paging

Demand Paging

Overview

- Allowed for wide availability of **virtual memory** concept:
  - Provides appearance of almost infinite or nonfinite physical memory.
  - Jobs run with less main memory than required in paged memory allocation scheme.
- **Disadvantages:**
  - Increased overhead caused by tables and page interrupts.
  - Requires high-speed direct access storage device.

Solution: Swapping

- How and when pages passed in memory?
- Depends on predefined policies.

9.2 Demand Paging

A strategy to implement virtual memory

- Bring a page into memory only when it is needed:
  - Less I/O needed
  - Less memory needed
  - Faster response
  - More users

Lazy swapper

- never swaps a page into memory unless page will be needed
- Swapper that deals with pages is a **pager**

- **Page** is needed → **reference** to it
  - invalid reference → abort
  - not-in-memory → bring to memory

Demand Paging

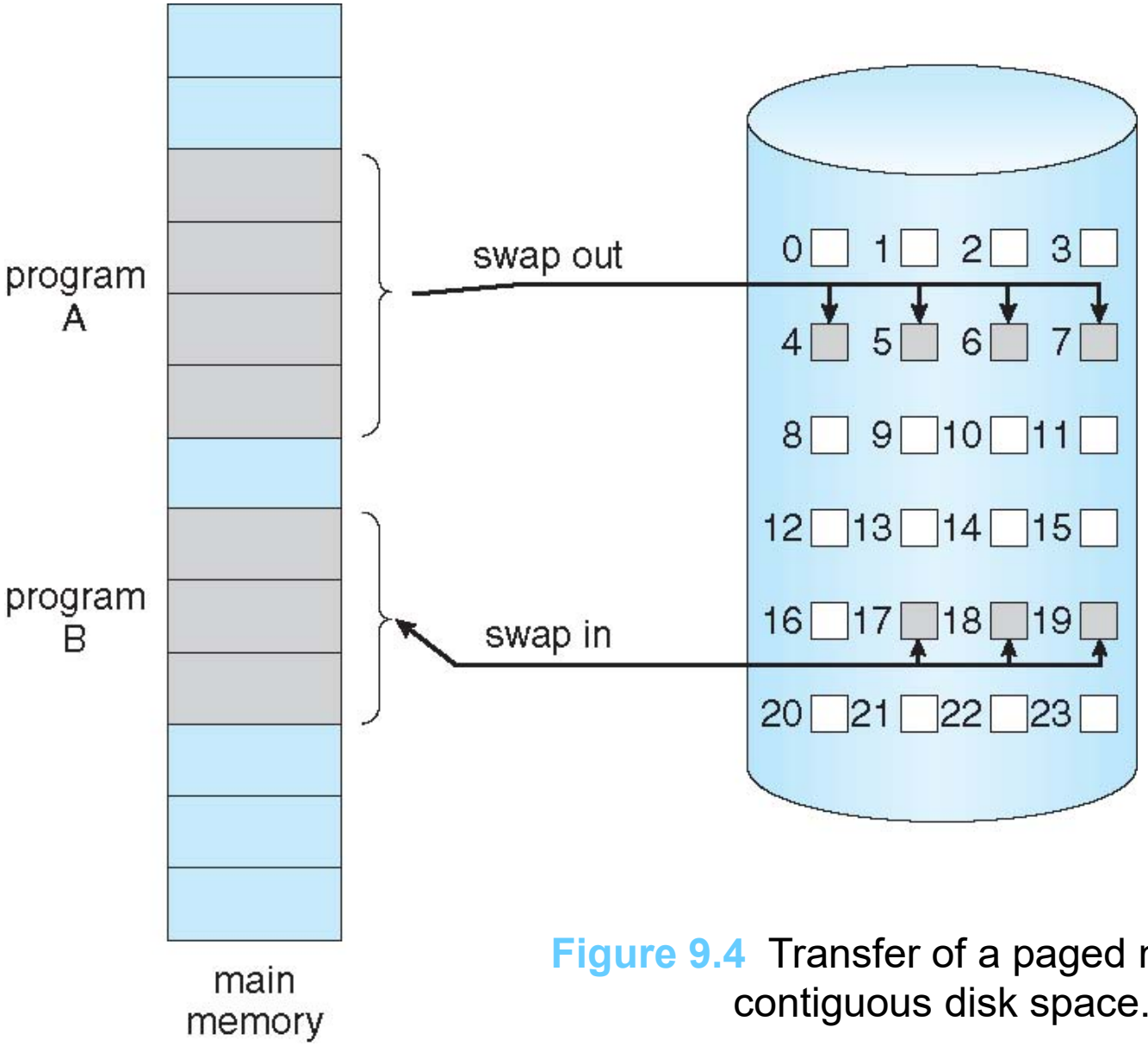


Figure 9.4 Transfer of a paged memory to contiguous disk space.

Demand Paging

Basic concepts

- When a process is to be swapped in, the pager guesses which pages will be used before the process is swapped out again.
- Instead of swapping in a whole process, the pager brings in only those "guessed" pages into memory.
- Need new MMU functionality to implement demand paging.

  - ✓ Need to distinguish between the pages that are in memory and the pages that are on the disk.
  - ✓ Use a variation of the valid-invalid scheme used for protection (see next slide).

Demand Paging

- If pages needed are already **memory resident.**
  - No difference from non demand-paging.

- If page needed and **not memory resident**, need to detect and load the page into memory from storage.
  - Without changing program behavior.
  - Without programmer needing to change code.

Demand Paging

Valid-Invalid Bit

- With each page table entry a **valid**–**invalid** bit is associated:

(v → in-memory

i → not-in-memory)

- Initially valid–invalid bit is set to i on all entries.

Demand Paging

Invalid → the page either:

- **not valid** (not in the logical address space of the process), or
- **valid** but is currently on the disk.

Access to a page marked invalid causes a page fault.

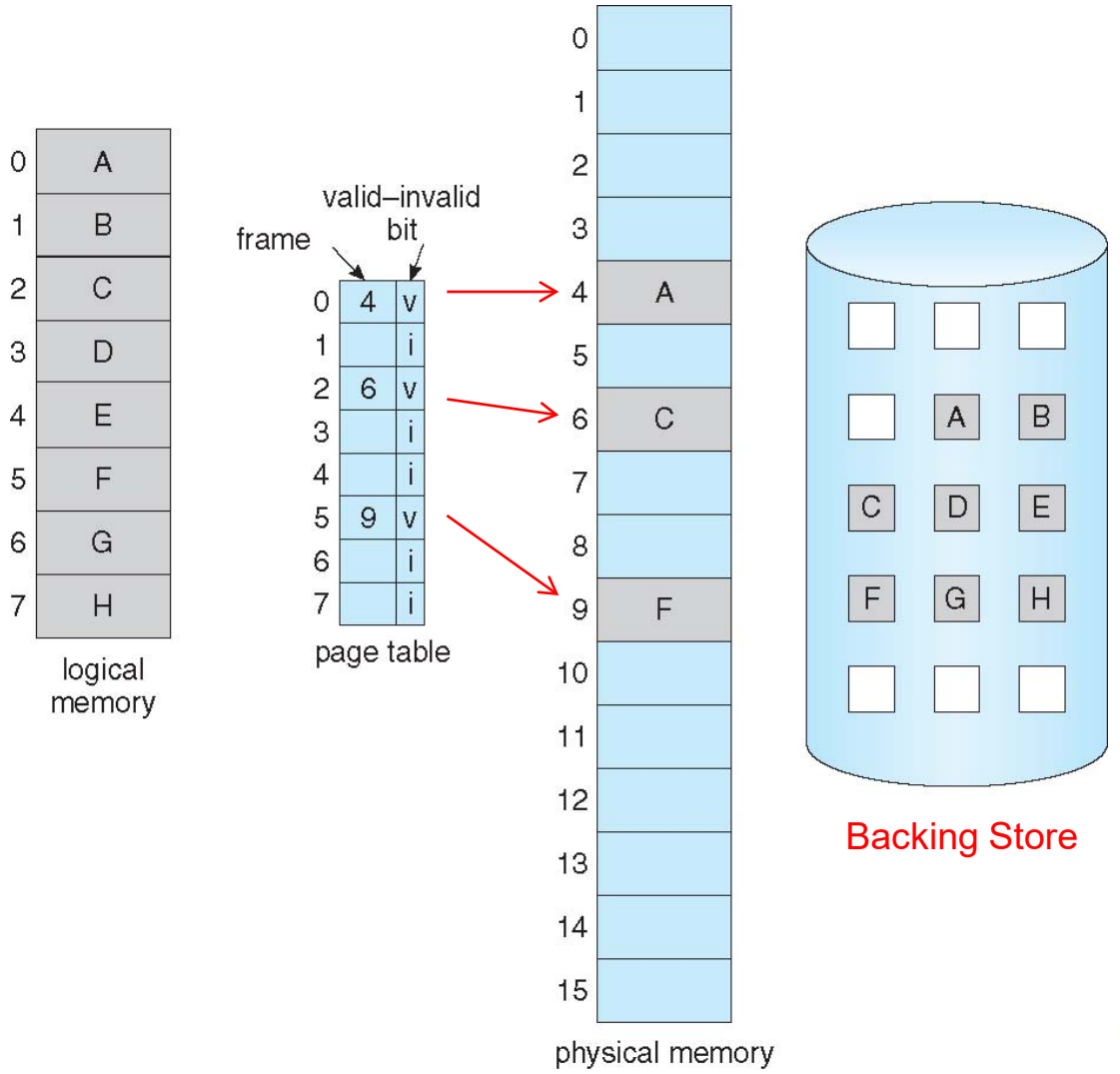


Figure 9.5 Page Table When Some Pages are Not in Main Memory

Demand Paging

Page Fault

- If there is a reference to a page, first reference to that page will trap to operating system → page fault
- Actions when a page fault occurs:

① Operating system looks at another table to decide:
   - **Invalid** reference → abort

② **Valid** but not in memory → Go to step (3).

③ Find **free frame** in memory;

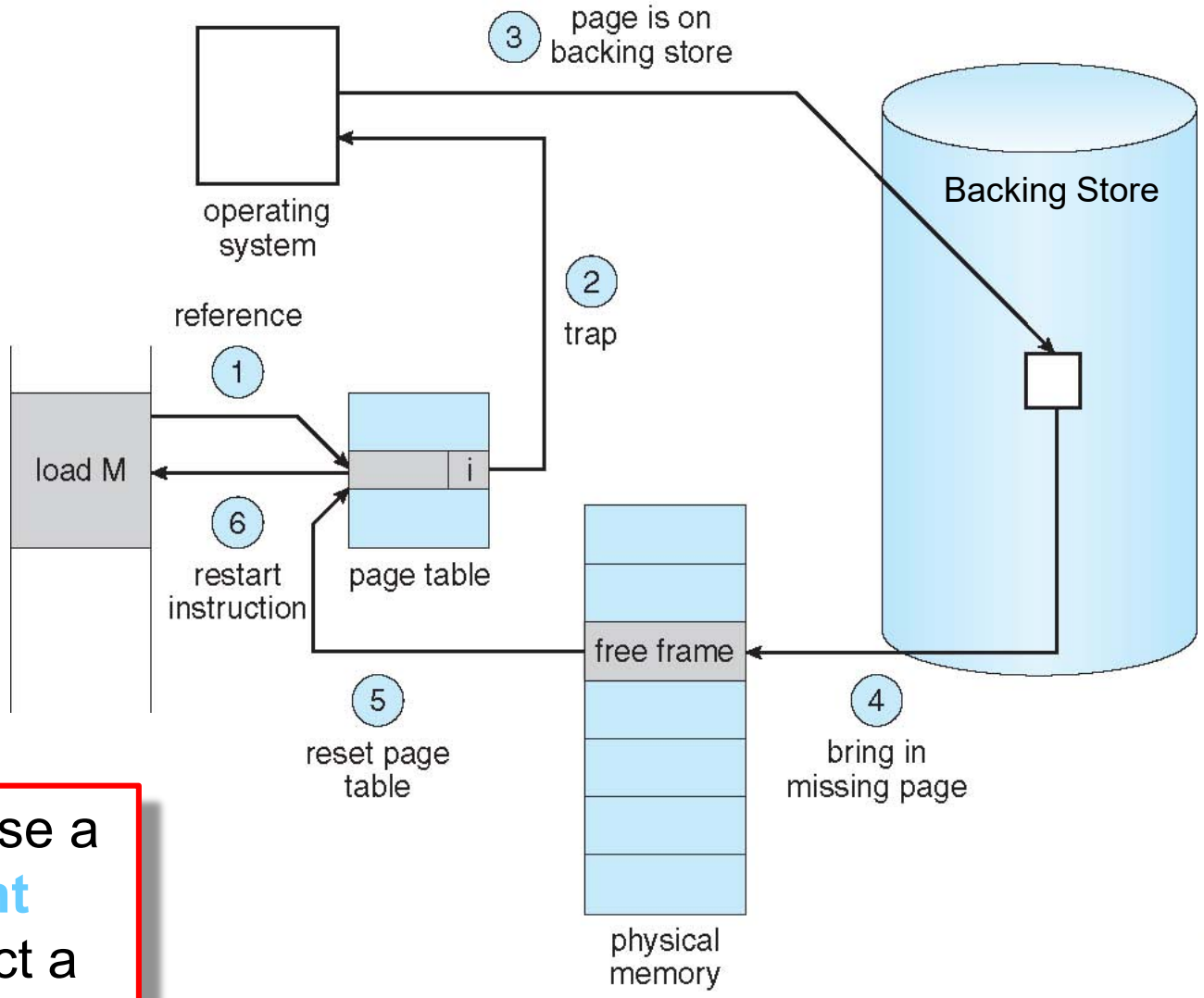
④ **Swap page** into frame via scheduled disk operation;

⑤ **Reset tables** to indicate page now in memory;
Set validation bit = v;

⑥ **Restart** the instruction.

Demand Paging

Trap → result of the OS's failure to bring the desired page into memory.



If no free frame, use a **page replacement algorithm** to select a **victim** frame.

Figure 9.6 Steps in Handling a Page Fault.

Page Replacement

Overview

- **Page replacement** occurs when:
  - A **page fault** occurs and we need to bring the desired page into memory.
  - There are NO free frames. to select a victim frame

- **Page replacement** – find page in memory, but not really in use, page it out:
  - Algorithm – decide which victim frame to free.
  - Performance – need an algorithm which will result in minimum number of page faults.

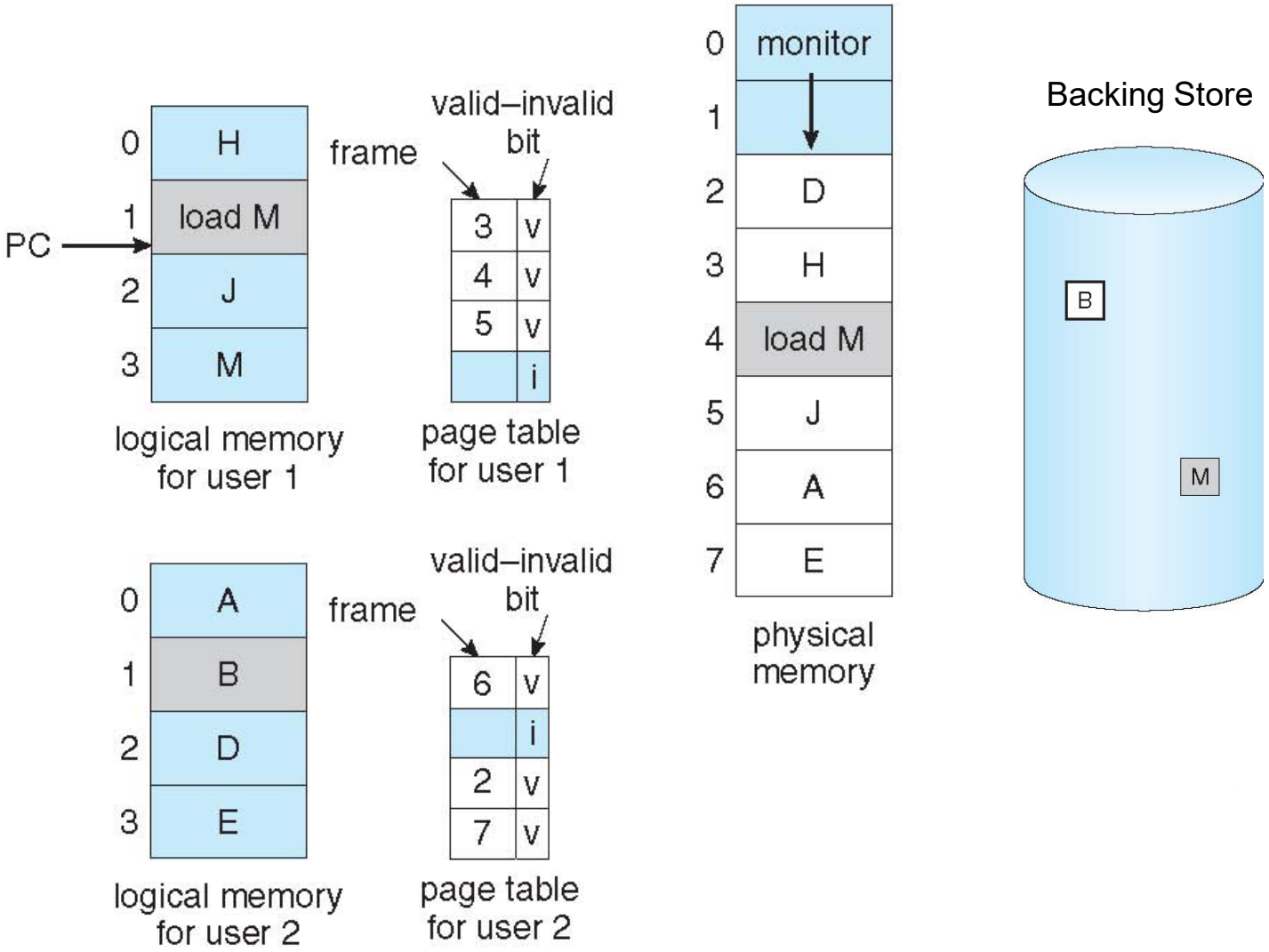
Same page may be brought into memory several times.

Page Replacement

Example

All frames are used, no free fames, user 2 needs B.

(Page: 402)



Page Replacement

Basic page replacement

① Find the location of the desired page on disk;

② Find a free frame:
- If there is a free frame, use it;
- If there is no free frame, use a page replacement algorithm to
  - Select a **victim** frame;
  - Write victim frame to disk if dirty

③ Bring the desired page into the (newly) free frame; update the page and frame tables

④ Restart the process that caused the page fault;

Page Replacement

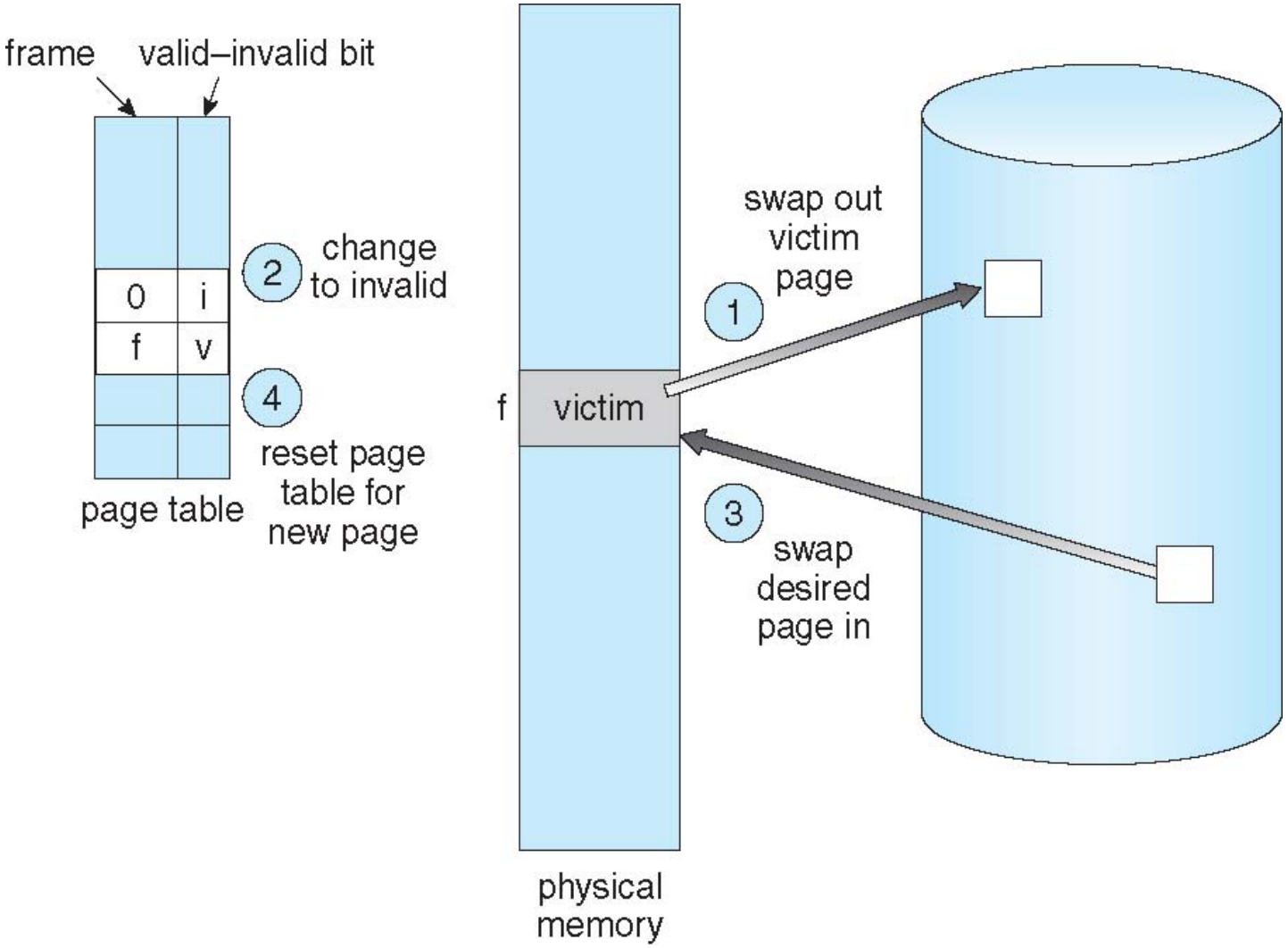


Figure 9.10 Page replacement.

Page Replacement

Problems: Overhead

If no free frames → TWO page transfers (one out one in):

- ✓ Doubles the page-fault service time.
- ✓ Increases access time.

Solution:

Use **modify bit** (or **dirty bit**) to reduce overhead of page transfers:

- ✓ only modified pages are written to back to disk.

- **Page replacement** completes separation between logical memory and physical memory
  - large virtual memory can be provided on a smaller physical memory.

Page Replacement

Page-Replacement Algorithms

- **Frame-allocation algorithm:** determines
  - ✓ How many frames to give each process? 几个要给.
  - ✓ Which frames to replace? 几个代替

- **Page-replacement algorithm:**
  - ✓ need an algorithm which will result in minimum number of **page faults**.

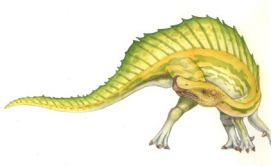
Algorithm
- minimum number of page faults.

Page Replacement

Page-Replacement Algorithms

- Evaluate algorithm by running it on a particular string of memory references (**reference string**) and compute the number of **page faults** on that string.

  - String is just page numbers, not full addresses.
  - Repeated access to the same page does not cause a **page fault**.
  - Results depend on **number of frames** available.



Page Replacement

If the number of frames increases, then the number of page faults decreases.

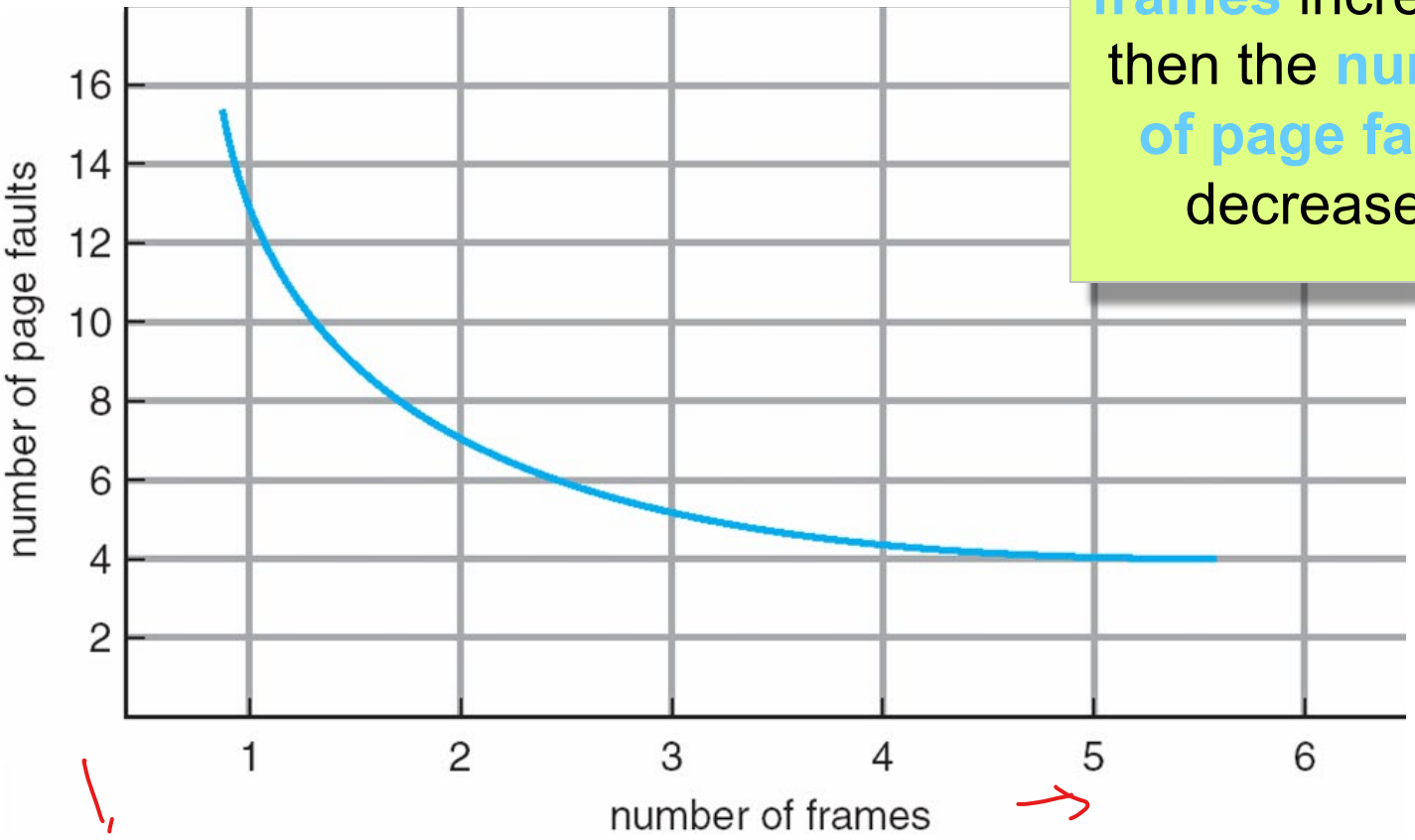


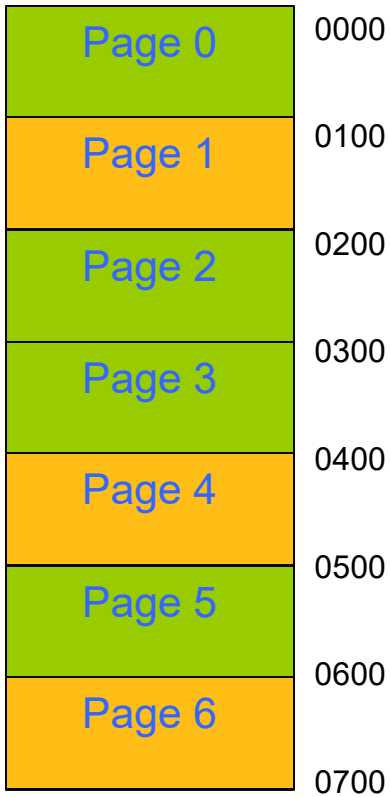
Figure 9.11 Graph of Page Faults Versus the Number of Frames.

Page Replacement

Reference String

- If CPU accesses the following addresses:

0100	**0432**	**0101**	**0612**	**0102**
0103	**0104**	**0101**	**0611**	**0105**
0103	0104	0101	0610	0107
0103	0104	0101	0609	0101



0 4 1 6 1 1 1 1 6 1 1

If page size = 100 byte, reference string will be:

1 4 1 6 1 6 1 6 1 6 1

Page Replacement

Example

- Reference String:

  1 4 1 6 1 6 1 6 1 6 1

- Number of frames = 3
- What is the total number of page fault?

Solution

	1	4	1	6	1	6	1	6	1	6	1
Frame #0	1	1		1							
Frame #1		4		4							
Frame #2				6							

- Number of page fault = 3

9.4 Page Replacement

Exercise

Given a reference string:

1 4 1 6 1 6 1 6 1 6 1

If the number of frame is 1 and 2 respectively, define how many number of page faults will be generated for each of them. Show your works.

	1	4	1	6	1	6	1	6	1	6	1
Frame #0	1	4	1	6	1	6	1	6	1	6	1

– Number of page fault = 11

1 4 1 6 1 6 1 6 1 6 1

Frame #0: 1 1 H 6 6 H H H H

Frame #1: 4 4 1 H H H

– No. of page fault = 4

Page Replacement

- Policy to select page removal:
  - Crucial to system efficiency.

- Page replacement polices:
  - **First-In First-Out (FIFO)** policy

    → Best page to remove is one in memory longest.
  - **Optimal Page Replacement (OPT)** policy

    → Best page to remove is one not be accessed longest.
  - **Least Recently Used (LRU)** policy

    → Best page to remove is least recently accessed.

9.4 Page Replacement

- Another Page Replacement Policies:
  - **Random Page Replacement -** Replace page at random.
  - **LRU Approximation Algorithms**
    - Additional Reference Bits Algorithm
    - Second Chance Algorithm
    - Enhanced Second Chance Algorithm
  - **Counting-Based Replacement**
    - Least Frequently Used (LFU) - replaces page with smallest count
    - Most Frequently Used (MFU) -based on the argument that the page with the smallest count was probably just brought in and has yet to be used

Page Replacement

(a) First-In First-Out (FIFO)

- Removes the page that in memory with the longest of time;
- **Efficiency**
  - Ratio of page interrupts to page requests.
  - Not so good
- **FIFO anomaly**
  - More memory does not lead to better performance.

Page Replacement

Example

(Page: 406)

- FIFO.
- Reference String:

7 0 1 2 0 3 0 4 2 3 0 3 0 3 2 1 2 0 1 7 0 1

- 3 frames = 3 (3 pages can be in memory at a time per process)
- What is the total number of page fault?

	7	**0**	**1**	**2**	**0**	**3**	**0**	**4**	**2**	**3**	**0**	**3**	**2**	**1**	**2**	**0**	**1**	**7**	**0**	**1**
Frame #0	7	7	7	2		2	2	4	4	4	0			0	0			7	7	7
Frame #1		0	0	0		3	3	3	2	2	2			1	1			1	0	0
Frame #2			1	1		1	0	0	0	3	3			3	2			2	2	1

- Number of page fault = 15

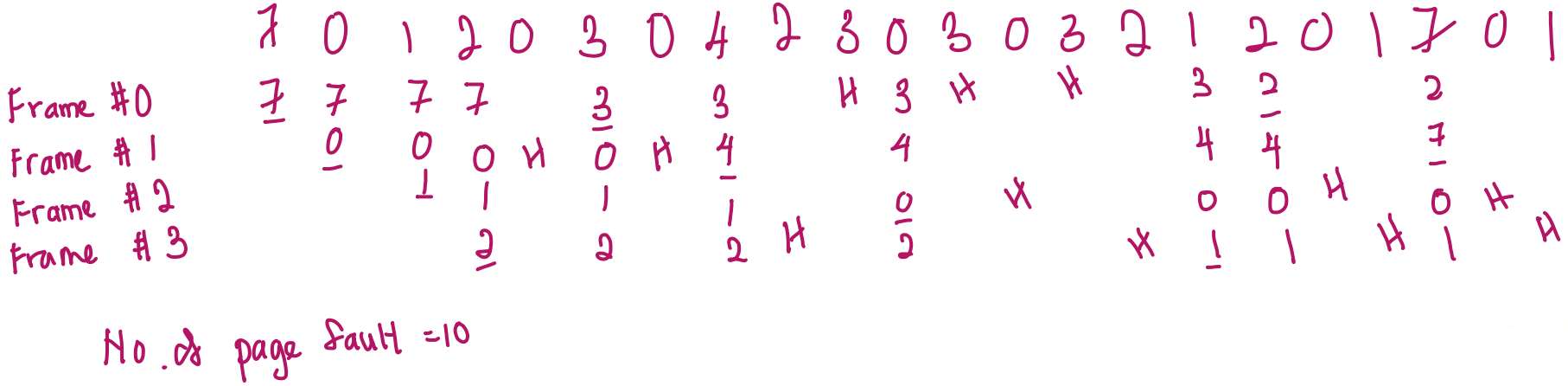
Page Replacement

Exercise

- FIFO.
- Reference String:

7 0 1 2 0 3 0 4 2 3 0 3 0 3 2 1 2 0 1 7 0 1

- Number of frames = 4
- What is the total number of page fault?



Page Replacement

Number of Frames vs Page Faults

- One would expect that the more frames are allocated to a process the fewer page faults.

Exercise

Consider the reference string:

1 2 3 4 1 2 5 1 2 3 4

a) How many page faults if we have 3 frames?

b) How many page faults if we have 4 frames?

Reference String: 1 2 3 4 1 2 5 1 2 3 4

- 3 frames (3 pages can be in memory at a time per process)
- How many page faults?

Solution: Number of Page faults = 9

Frame #	1	2	3	4	1	2	5	1	2	3	4
1	1	1	1	4	4	4	5			5	5
2		2	2	2	1	1	1			3	3
3			3	3	3	2	2			2	4

Reference String: 1 2 3 4 1 2 5 1 2 3 4

- 4 frames (4 pages can be in memory at a time per process)
- How many page numbers?

Solution: Number of Page faults = 9

Frame #	1	2	3	4	1	2	5	1	2	3	4
1	1	1	1	1			5	5	5	5	4
2		2	2	2			2	1	1	1	1
3			3	3			3	3	2	2	2
4				4			4	4	4	3	3

Page Replacement

Belady's Anomaly

- If we use FIFO for page replacement, we can encounter the anomaly that more frames may lead to more page faults.
- FIFO Illustrating **Belady's Anomaly**

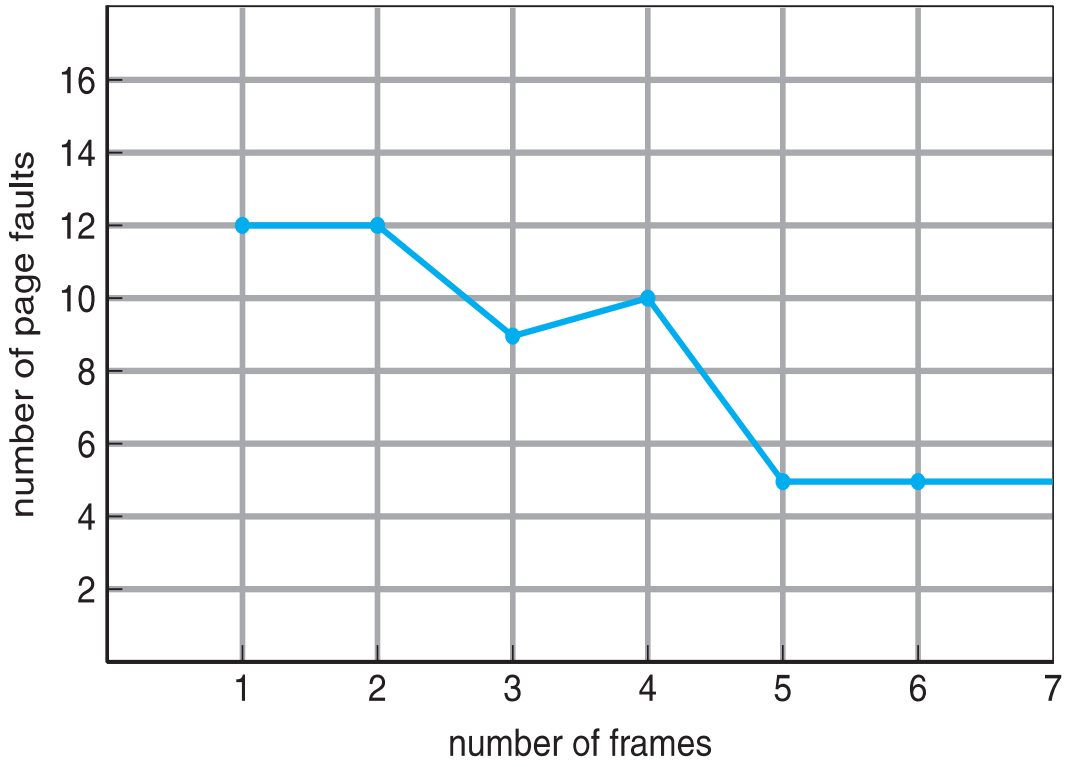


Figure 9.13 Page-fault curve for FIFO replacement on a reference string.

9.4 Page Replacement

(b) Optimal Page Replacement (OPT)

假设有用到的块 (有多项)

- Replace page that will not be accessed for longest period of time.

- **Q**: How do you know which page will not be accessed for longest period of time?
  - Can't read the future
- Used mainly for measuring how well a given algorithm performs.

Page Replacement

Example

(Page: 407)

- OPT.
- Reference String:

7 0 1 2 | 0 3 0 4 | 2 3 0 3 | 0 3 2 1 | 2 0 1 7 | 0 1

- 3 frames = 3 (3 pages can be in memory at a time per process)
- What is the total number of page fault?

	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
Frame #0	7	7	7	2		2		2			2			2				7		
Frame #1		0	0	0		0		4			0			0				0		
Frame #2			1	1		3		3			3			1				1		

- Number of page fault = 9

Page Replacement

Exercise

- OPT.
- Reference String:

7 0 1 2 0 3 0 4 2 3 0 3 0 3 2 1 2 0 1 7 0 1

- Number of frame = 4
- What is the total number of page fault?

7 0 1 2 0 3 0 4 2 3 0 3 0 3 2 1 2 0 1 7 0 1

Frame #0	7	7	7	7		3		3							1			1		
Frame #1		0	0	0		0		0							0			0		
Frame #2			1	1		1		4							4			7		
Frame #3				2		2		2							2			2		

No. of page fault = 8

9.4 Page Replacement

(c) Least Recently Used (LRU)

有前瞻

- Use past knowledge rather than future.
- Replace page that has not been used for the longest period of time.
- Associate time of last use with each page.

Page Replacement

Example

(Page: 407)

- LRU.
- Reference String:

7 0 1 2 0 3 0 4 2 3 0 3 0 3 2 1 2 0 1 7 0 1

- 3 frames = 3 (3 pages can be in memory at a time per process)
- What is the total number of page fault?

	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
Frame #0	7	7	7	2		2		4	4	4	0			1		1		1		
Frame #1		0	0	0		0		0	0	3	3			3		0		0		
Frame #2			1	1		3		3	2	2	2			2		2		7		

- Number of page fault = 12

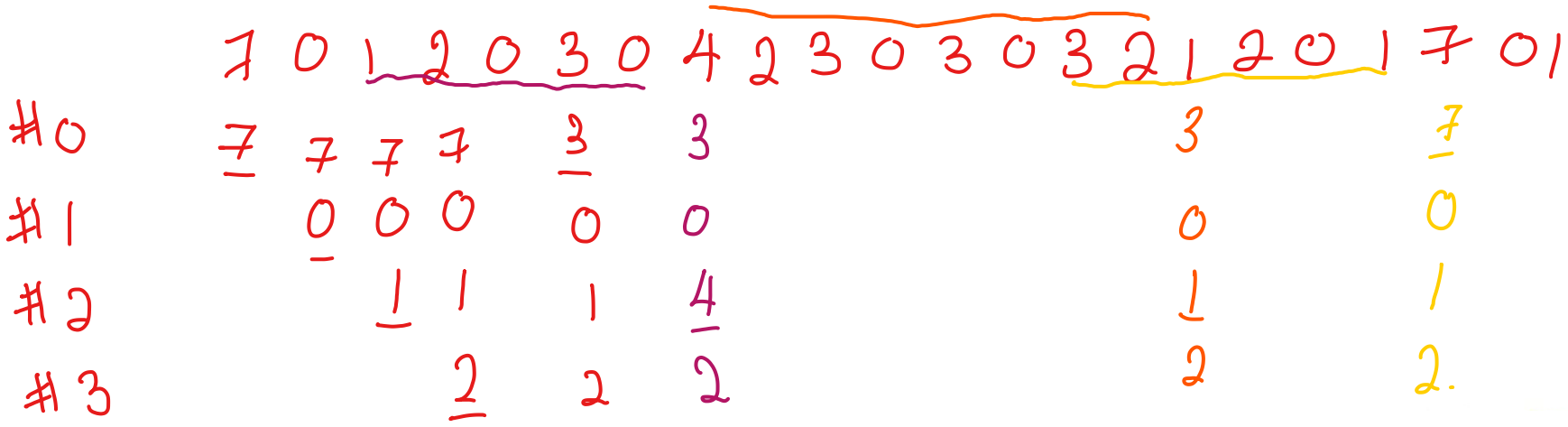
Page Replacement

Exercise

- LRU.
- Reference String:

7 0 1 2 0 3 0 4 2 3 0 3 0 3 2 1 2 0 1 7 0 1

- Number of frame = 4
- What is the total number of page fault?



Page Replacement

LRU Implementation

- **Problem**: To determine an order for the frames defined by the time of last use.
- How to implement LRU algorithm?
- Two implementations:

① Counter

- Every page table entry has a counter associated with it;
- Every time page is referenced through this entry, the content of the clock if copied into the counter
- We replace the page with the smallest time value – search through table needed

Page Replacement

② Stack

- keep a stack of page numbers in a double link form: (*Header pointer* & *Tail pointer*)
- When a page is referenced:
  - move page to the **top**
  - Therefore, most recently used page is on **top** while least recently used is always at the **bottom**.
- No search for replacement

LRU needs special hardware and is still slow
LRU and OPT are cases of **stack algorithms** that don't suffer from Belady's Anomaly

Page Replacement

Example

(Page: 409)

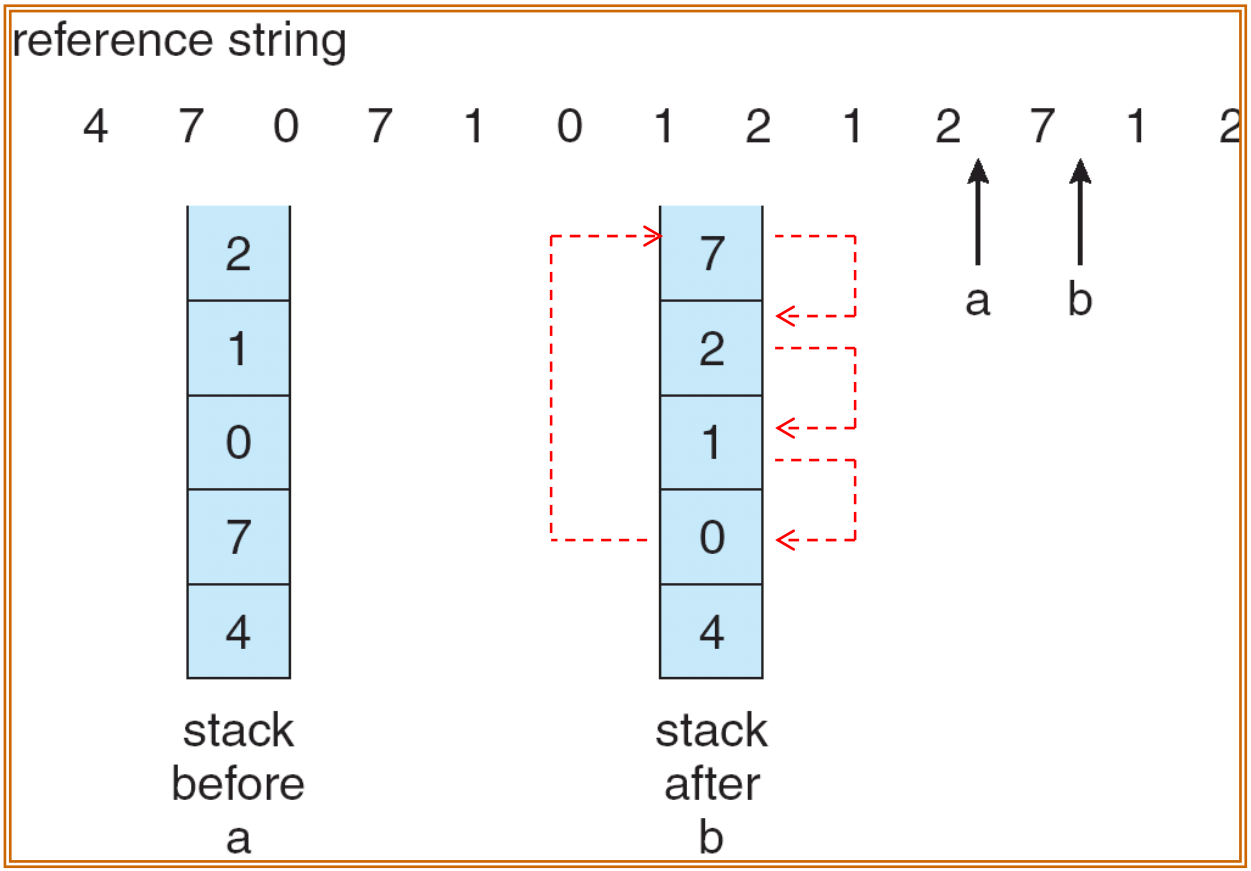


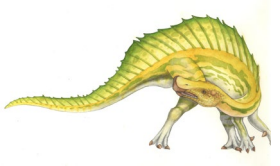
Figure 9.16 Use of a stack to record the most recent page references.

Virtual Memory: Demand Paging

(Some Issues)

Page Replacement

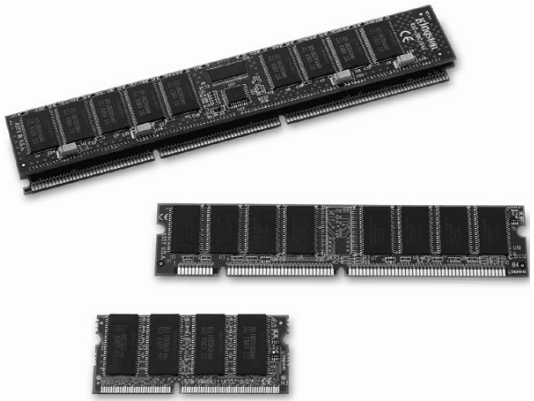
- **Global Replacement:**
  - Choose frames from all sets of frames in the system;
  - Process cannot control page-fault rate by itself;
- **Local Replacement:**
  - Choose frames already allocated to the process;
  - Effect – a process may keep pages that are seldom used 很少使用 while other process that need more frames has a high page fault rate;
- Generally, global replacement results in greater system throughput and is therefore the more common method.



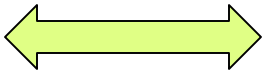
Thrashing

Overview

- An excessive amount of page swapping or paging activity (in & out) between **main memory** & **secondary storage**;
- A process is thrashing if it spends more time on paging than executing;



Main memory: DRAM



Secondary storage: Hard disk

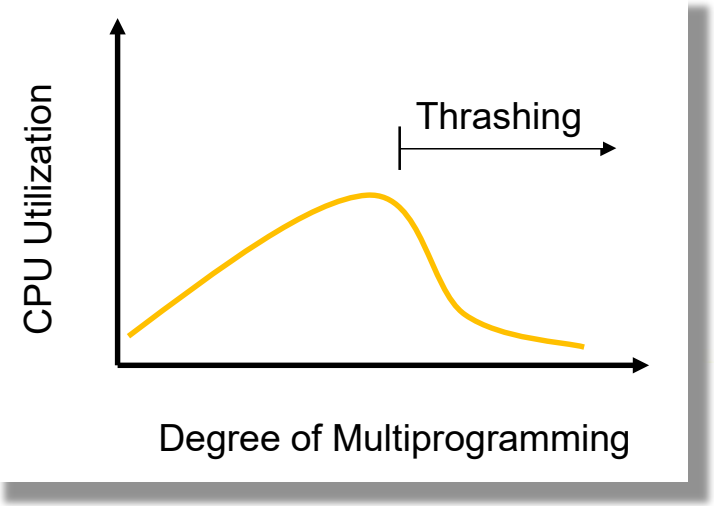
Thrashing

Why does this happen?

- If CPU utilization is low, OS increase the degree of multiprogramming;
- The number of programs increases while the number of frames remains the same;
- Therefore, page fault will occur more frequently;

Consequences:

- that leads to low CPU utilization;
- inefficient operation of system;



Thrashing

Example

Page fault due to program structure.

- Program structure
  - `Int[128,128] data;`
  - Each row is stored in one page
  - **Program 1**:

```
for (j = 0; j <128; j++)
     for (i = 0; i < 128; i++)
          data[i,j] = 0;
```

128 x 128 = 16,384 page faults

  - **Program 2**:

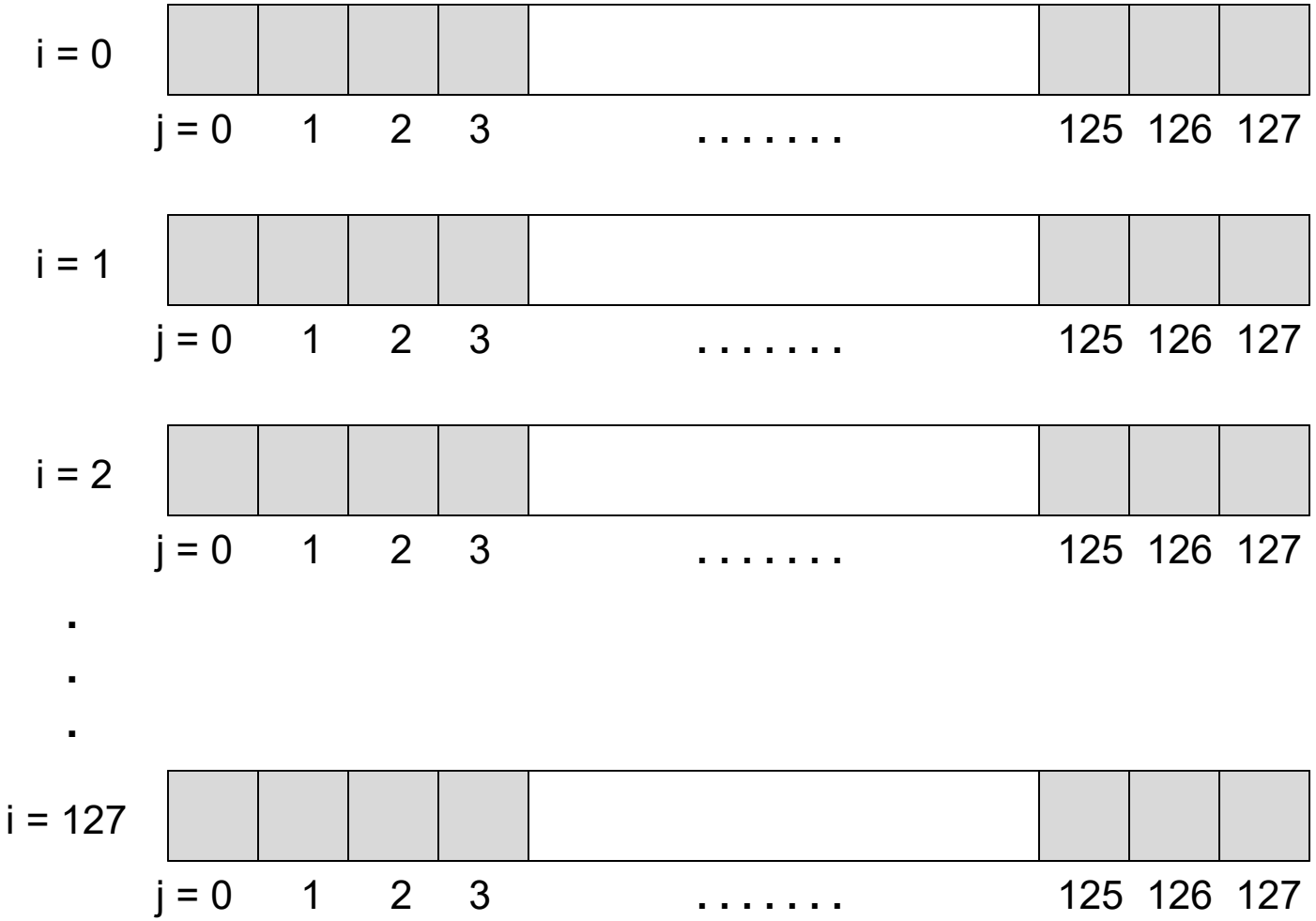
```
for (i = 0; i < 128; i++)
     for (j = 0; j < 128; j++)
          data[i,j] = 0;
```

128 page faults

Thrashing

Program 1:

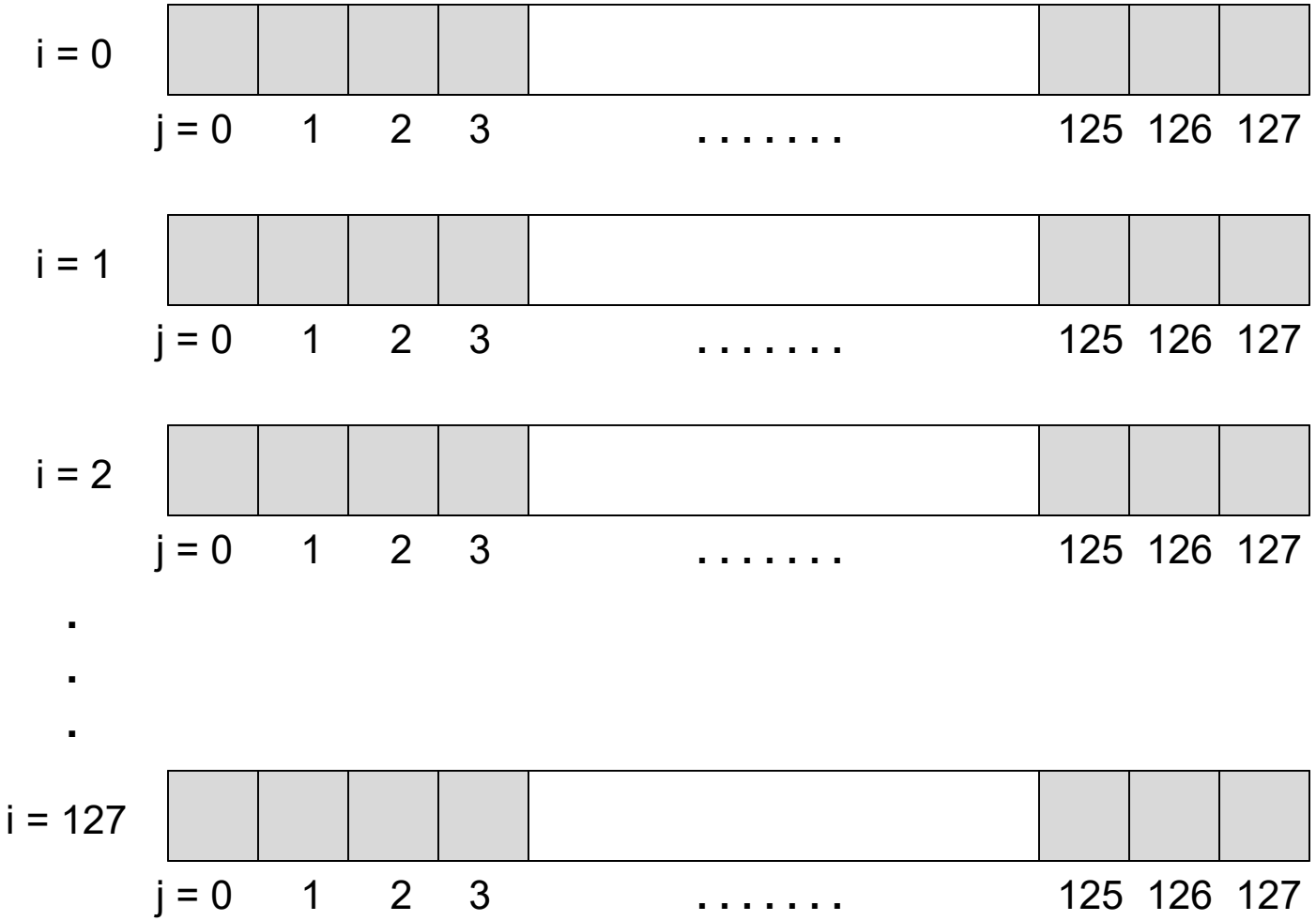
Page Fault(s) = **16384**



Thrashing

Program 2:

Page Fault(s) = **128**



Thrashing

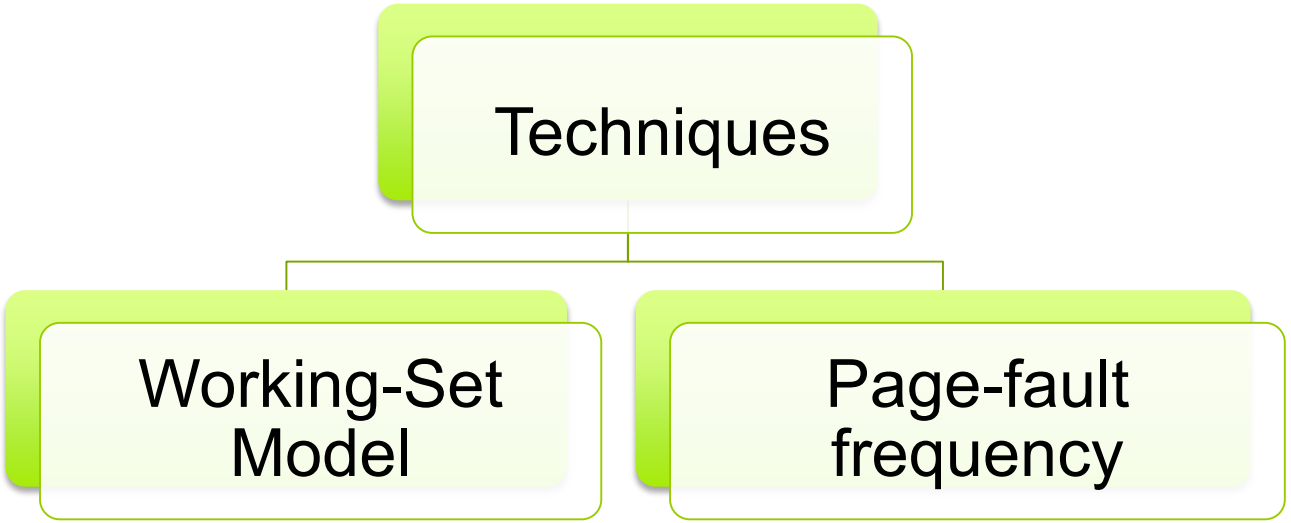


Figure Two techniques to prevent thrashing

Thrashing

(a) Working-Set (WS) Model

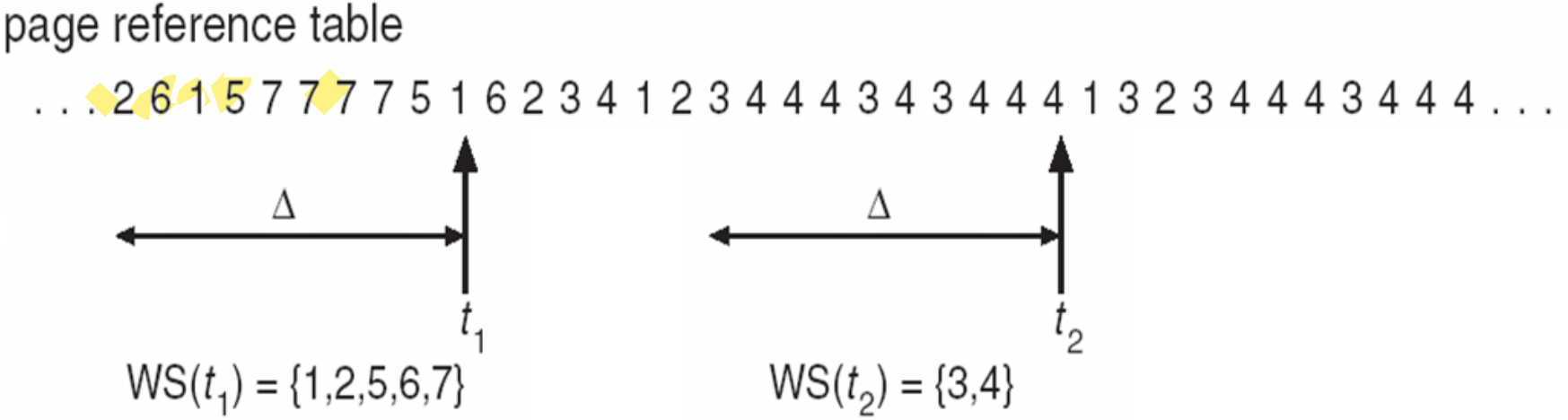
- Based on the assumption of locality;
- Uses a parameter, Δ , to define the **working-set window**;
- Δ is a fixed number of page references
- **Working-Set (WS)** → is defined to be the total number of pages referenced in the most recent Δ

Thrashing

Example

(Page: 420)

- Given a sequence of memory references below;
- If Δ = 10memory references,



Solution

- The working set at t_1 is $\{1, 2, 5, 6, 7\}$
- By the time t_2, the working set has changed to $\{3, 4\}$

Thrashing

- System decides:
  - **Number of pages** comprising working set;
  - **Maximum number of pages** allowed for a working set;

- The accuracy of a working-set depends on the selection of Δ (its size)
  - If Δ too **small** → not encompass the entire locality;
  - If Δ too **large** → may overlap page several localities;
  - if $\Delta = \infty \Rightarrow$ will encompass entire program

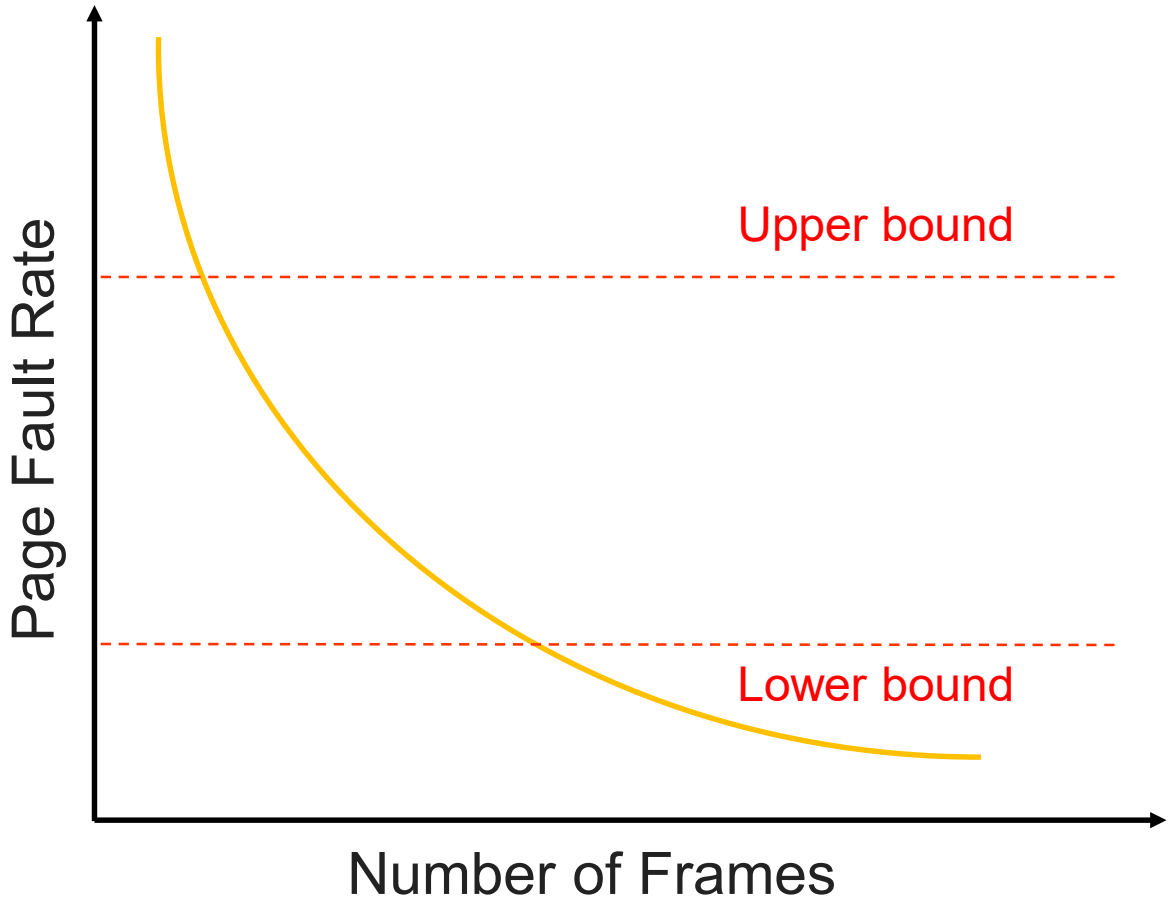
Thrashing

(b) Page-Fault Frequency (PFF)

- Establish "acceptable" **Page-Fault Frequency** rate and use local replacement policy:
  - If actual rate **too low**, process loses frame.
  - If actual rate **too high**, process gains frame.

- Provide **upper** and **lower** bound of page-fault.
- If **page-fault** rate > **upper** bound, allocate more frames.
- If **page-fault** rate < **lower** bound, reduce frame allocation.

Thrashing



Increase number of frames if the page fault rate is higher.

Decrease number of frames if the page fault rate is lower.

Figure 9.21 Page-fault frequency (PFF)

Summary

- It is able to execute a process with the **logical address space** larger than the available **physical address space**.
- **Virtual memory** allows :
  - ✓ to run extremely large processes.
  - ✓ to raise the degree of multiprogramming.
  - ✓ to increase CPU utilization.
- Virtual memory:
  - Programs execute if not stored entirely in memory.
  - Job's size no longer restricted to main memory size.

Summary

- **Demand paging** can be used to reduce the number of frames allocated to a process
- If total memory requirements exceed the capacity of physical memory, replace the pages from memory to free frames for new pages.
  - ✓ Various **page-replacement algorithms** are used.

  - ✓ **FIFO**: easy to program but suffers from Belady's anomaly.
  - ✓ **OPT**: requires future knowledge.
  - ✓ **LRU**: approximates the OPT, but difficult to implement.

Exercise

Consider the following page reference string:

1 2 3 4 2 1 5 6 2 1 2 3 7 6 3 2 1 2 3 6

How many **page faults** would occur for the following replacement algorithms, assuming **1, 3** and **5 frames**? Remember that all frames are initially empty.

a) FIFO replacement.

b) LRU replacement.

c) OPT replacement.

Answer(s): FIFO

1 frame: 20 page faults

1	2	3	4	2	1	5	6	2	1	2	3	7	6	3	2	1	2	3	6
1	2	3	4	2	1	5	6	2	1	2	3	7	6	3	2	1	2	3	6

3 frames: 16 page faults

1	2	3	4	2	1	5	6	2	1	2	3	7	6	3	2	1	2	3	6
1	1	1	4		4	4	6	6	6		3	3	3		2	2		2	6
	2	2	2		1	1	1	2	2		2	7	7		7	1		1	1
		3	3		3	5	5	5	1		1	1	6		6	6		3	3

Answer(s): FIFO

5 frames: 10 page faults

1	2	3	4	2	1	5	6	2	1	2	3	7	6	3	2	1	2	3	6
1	1	1	1			1	6		6	6	6	6							
	2	2	2			2	2		1	1	1	1							
		3	3			3	3		3	2	2	2							
			4			4	4		4	4	3	3							
						5	5		5	5	5	7							

Answer(s): LRU

1 frame: 20 page faults

1	2	3	4	2	1	5	6	2	1	2	3	7	6	3	2	1	2	3	6
1	2	3	4	2	1	5	6	2	1	2	3	7	6	3	2	1	2	3	6

3 frames: 15 page faults

1	2	3	4	2	1	5	6	2	1	2	3	7	6	3	2	1	2	3	6
1	1	1	4		4	5	5	5	1		1	7	7		2	2			2
	2	2	2		2	2	6	6	6		3	3	3		3	3			3
		3	3		1	1	1	2	2		2	2	6		6	1			6

Answer(s): LRU

5 frames: 8 page faults

1	2	3	4	2	1	5	6	2	1	2	3	7	6	3	2	1	2	3	6
1	1	1	1			1	1				1	1		1					
	2	2	2			2	2				2	2		2					
		3	3			3	6				6	6		6					
			4			4	4				3	7		7					
						5	5				5	5		3					

Answer(s): OPT

1 frame: 20 page faults

1	2	3	4	2	1	5	6	2	1	2	3	7	6	3	2	1	2	3	6
1	2	3	4	2	1	5	6	2	1	2	3	7	6	3	2	1	2	3	6

3 frames: 11 page faults

1	2	3	4	2	1	5	6	2	1	2	3	7	6	3	2	1	2	3	6
1	1	1	1			1	1				3	3			3	3			3
	2	2	2			2	2				2	7			2	2			2
		3	4			5	6				6	6			6	1			6

Answer(s): OPT

5 frames: 7 page faults

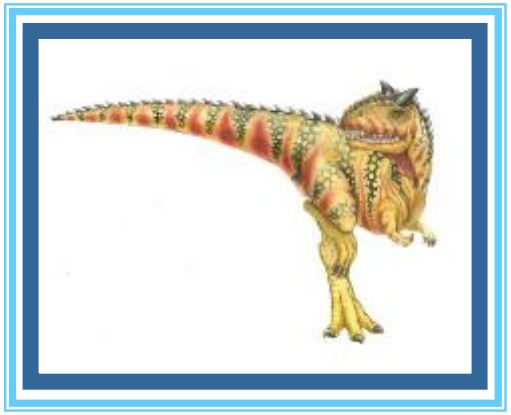
1	2	3	4	2	1	5	6	2	1	2	3	7	6	3	2	1	2	3	6
1	1	1	1			1	1					1							
	2	2	2			2	2					2							
		3	3			3	3					3							
			4			4	6					6							
						5	5					7							

Exercises

From Silberchatz, Operating System Concepts
Chapter 9 Exercises (page: 441 - 446)

- 9.5
- 9.6
- 9.8
- 9.9
- 9.11
- 9.12
- 9.17
- 9.19
- 9.20
- 9.21

Chapter 10: File-System



Silberschatz, Galvin and Gagne ©2013

Chapter 10: File-System

10.1 File Concept

10.2 Access Methods

10.3 Directory and Disk Structure

10.6 Protection

10.7 Summary

Objectives

- To explain the function of file systems.
- To describe the interfaces to file systems.
- To discuss file-system design tradeoffs, including access methods, file sharing, file locking, and directory structures.
- To explore file-system protection.

10.1

File Concept

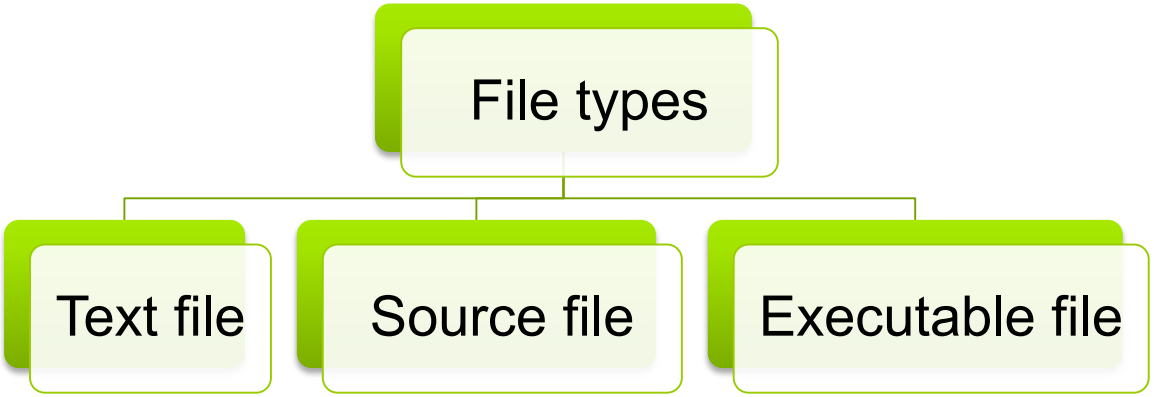
Overview

- Computers can store information on various storage media:
  - *e.g.* magnetic disks, magnetic tapes, and optical disks.
- Operating system:
  - provides a uniform logical view of stored information.
  - abstracts from physical properties of storage devices to define a logical storage unit → File.
  - maps the file onto physical devices.

A **file** is a named collection of related information that is recorded on secondary storage.

File Concept

- Contents (many types) is defined by file's creator.



- **Text file** : A sequence of characters organized into lines (and possibly pages)
- **Source file** : A sequence of functions the followed by executable statements.
- **Executable file** : A series of code sections can be loaded in memory and execute.

File Concept

Definitions

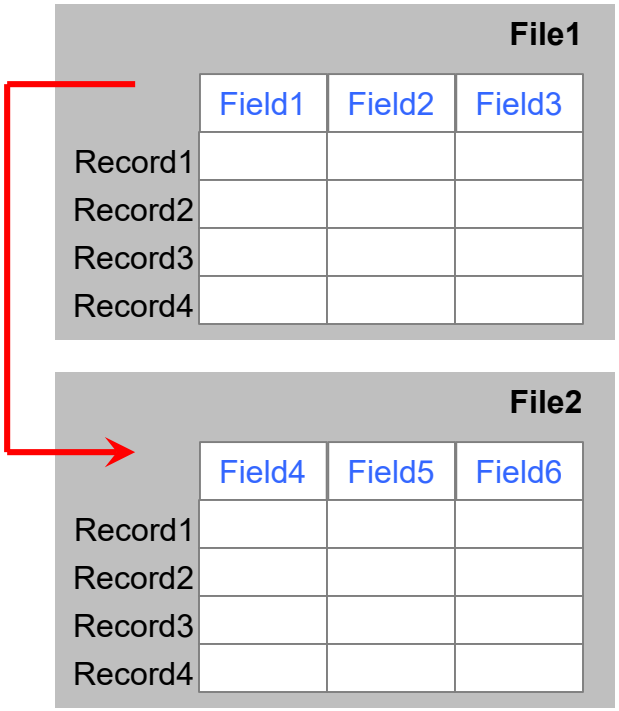
- **Field**
  - Group of related bytes.
  - Identified by user (name, type).
- **Record**
  - Group of related fields.
- **File**
  - Group of related records that is stored in secondary storage.
  - Contain information used by specific application programs to generate reports.
  - Sometimes known as *flat file* because it has no connections to other files.

File1

	Field1	Field2	Field3
Record1			
Record2			
Record3			
Record4			

File Concept

- **Databases**
  - Groups of related files that are interconnected at various levels to give users flexibility of access to stored data.



- **Program files**
  - Contain instructions.

- **Directories**
  - Listings of filenames and their attributes.
  - Attributes are information related to a files.
  - Organize files into a logical structure.

File Concept

File attributes

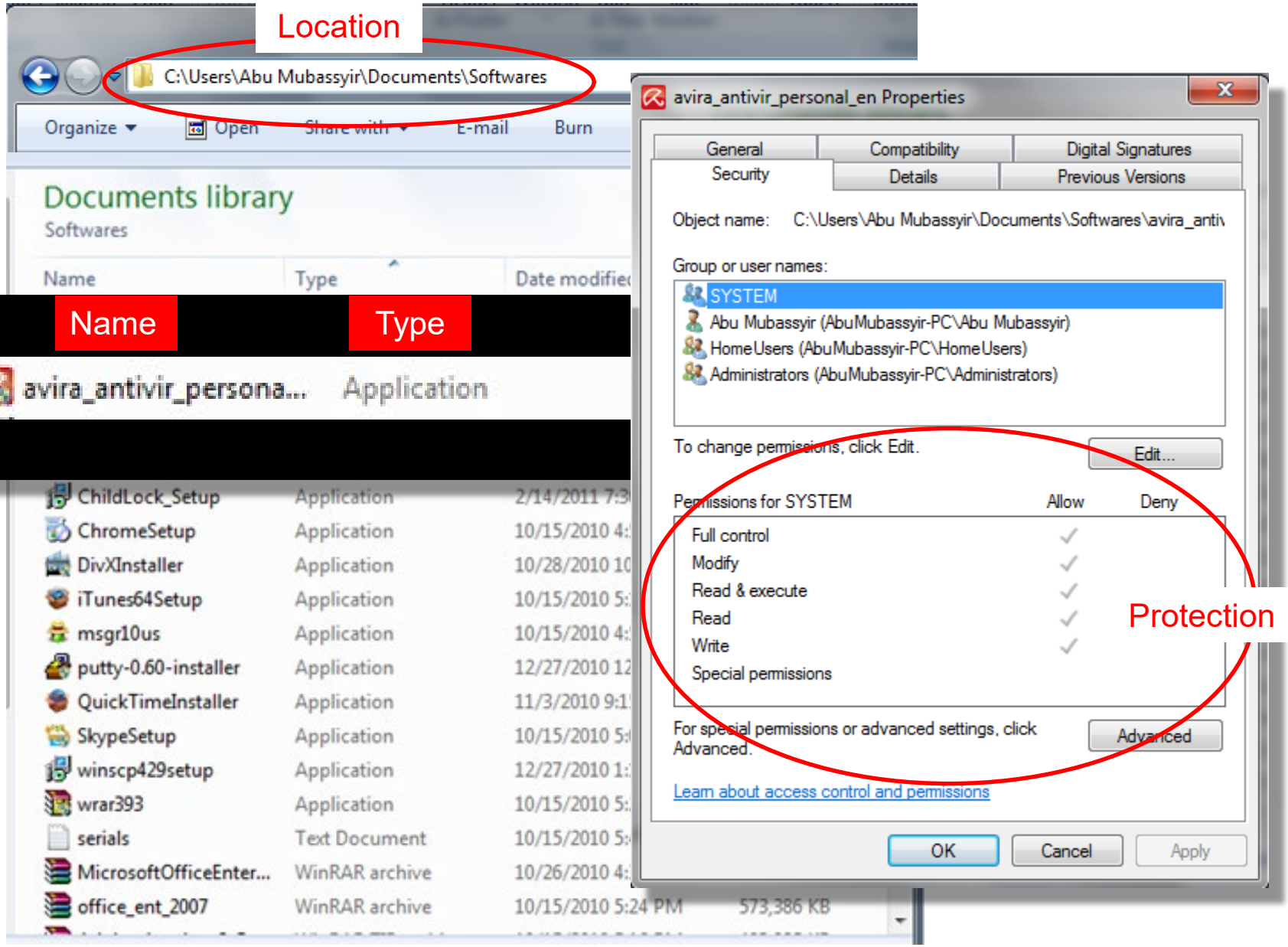
- File attributes are stored in a directory.
- Attributes that are commonly used are:
  - **Name** – only information kept in human-readable form.
  - **Identifier** – unique tag (number) identifies file within file system.
  - **Type** – needed for systems that support different types.
  - **Location** – pointer to file location on the device (disk).
  - **Size** – current file size.

File Concept

- **Protection** – controls who can do reading, writing, executing
- **Time, date, and user identification** – information kept for creation time, last modification time, and last use time.
  - Useful for data for protection, security, and usage monitoring.

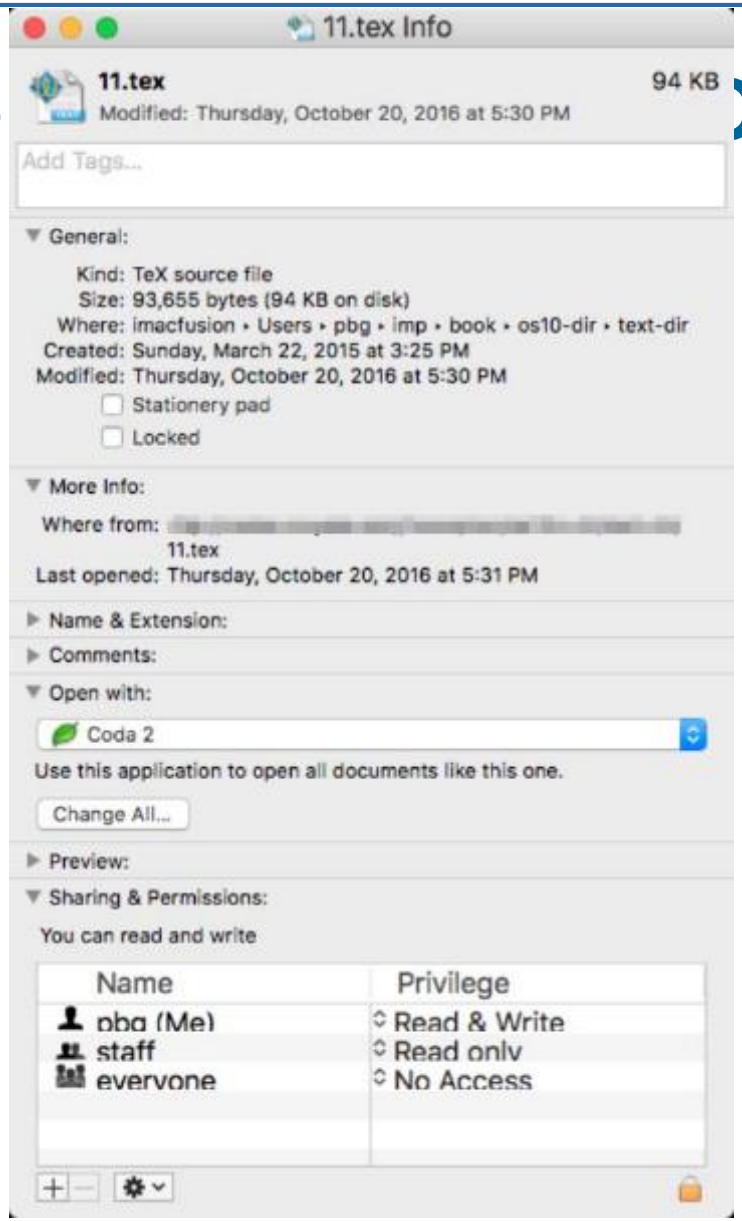
- Many variations, including extended file attributes such as file checksum.
- Information kept in the directory structure (on disk), which consists of “inode” entries for each of the files in the system.

10.1 File Concept



File Concept

File info



OS X

File Concept

File operations

- **Create**
- **Write –** at write pointer location
- **Read –** at read pointer location
- **Reposition within file -** seek
- **Delete**
- **Truncate**
- `Open` (F_i) – search the directory structure on disk for inode entry F_i, and move the content of the entry to memory
- `Close` (F_i) – move the content of inode entry F_i in memory to directory structure on disk.

File Concept

File types (name, extension)

- **Extensions:**
  - Appended to relative filename.
    - Two to three characters.
    - Separated by period.
    - Identifies file type or contents.

- Example: `TUNE.MP3,Menu.doc, slide.ppt`
- Unknown extension:
  - Requires user intervention.

File Concept

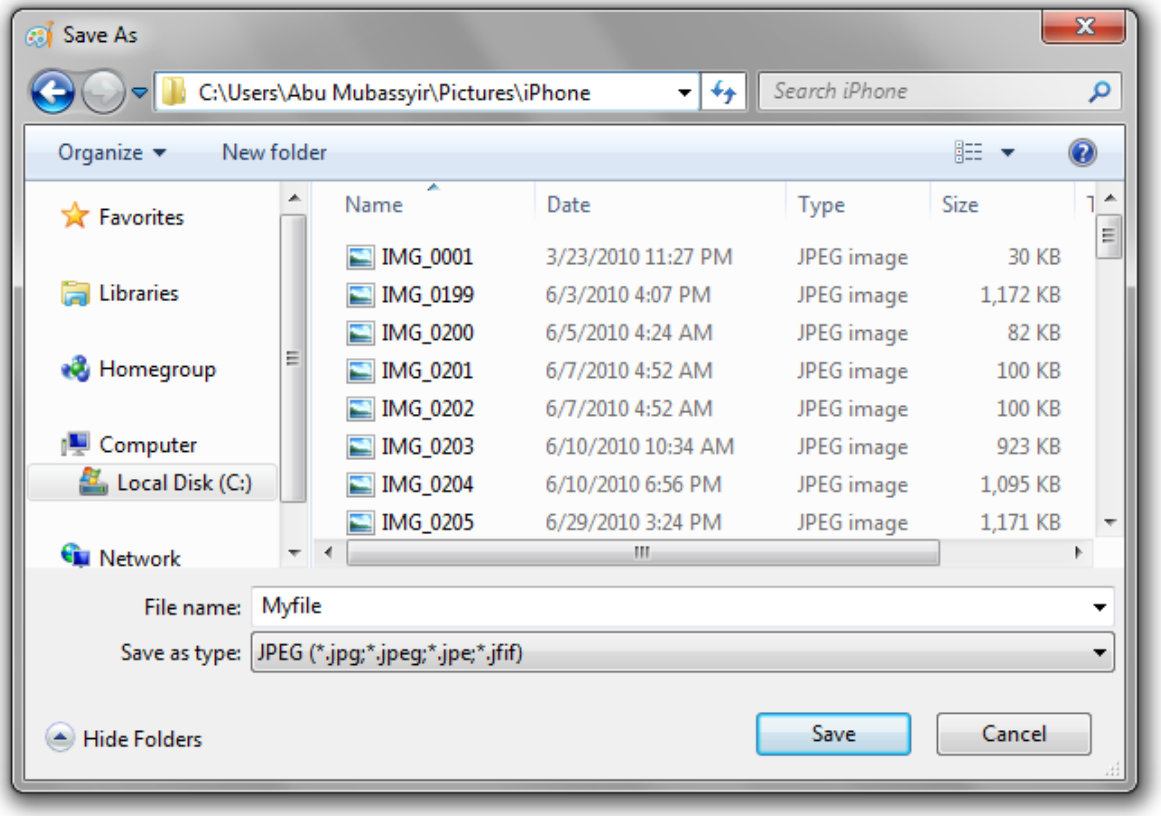
File type	Usual extension	Function
Executable	*exe, com, bin or none*	ready-to-run machine-language program
Object	*obj, o*	complied, machine language, not linked
Source code	*c, p, pas, 177, asm, a*	source code in various languages
Batch	*bat, sh*	commands to the command interpreter
Text	*txt, doc*	textual data documents
Word processor	*wp, tex, rrf, etc.*	various word-processor formats
Library	*lib, a*	libraries of routines for programmers
Print or view	*ps, dvi, gif*	ASCII or binary file
Archive	*arc, zip, tar*	related files grouped into one file, sometimes compressed.
Multimedia	*mpeg, mov, mp3, mp4, avi*	binary file containing audio or A/V information

Figure 10.3 Common file types

File Concept

Example 1 :

- Operating system specifics.
- **Windows :** Drive label and directory name, relative name, and extension.



File Concept

File structure

- None - sequence of words, bytes.

Simple structure

- ✓ Lines
- ✓ Fixed length
- ✓ Variable length

Complex structure

- ✓ Formatted document
- ✓ Relocatable load file

- Who decides:
  - ✓ Operating system
  - ✓ Program

Access Methods

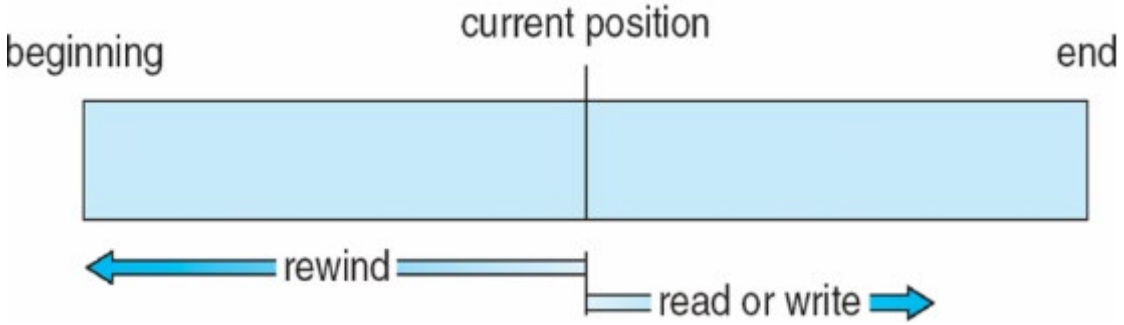
Overview

- Files store information will be accessed and read into computer memory.
- The files can be accessed in several ways for a particular application → major design problem.

Access Methods

(a) Sequential access

- General structure:



- Operations:
  - `read_next()` – reads the next portion of the file and automatically advances a file pointer.
  - `write_next()` – append to the end of the file and advances to the end of the newly written material (the new end of file).
  - `reset` – back to the beginning of the file.

Access Methods

(b) Direct access

- File is made up of fixed-length logical records that allow programs to read and write records rapidly in no particular order.
- File is viewed as a numbered sequence of blocks or records. For example, can read block 14, then read block 53, and then write block 7.
- Operations:
  - `read(n)` – reads relative block number n.
  - `write(n)` – writes relative block number n.
- Relative block numbers allow OS to decide where file should be placed.

Access Methods

- *cp* is a pointer to the next block to be read/written.

sequential access	implementation for direct access
reset	*cp* = 0;
read next	*read cp*; *cp* = *cp* + 1;
write next	*write cp*; *cp* = *cp* + 1;

Figure 10.5 Simulation of Sequential Access on Direct-access File.

Access Methods

(c) Other access methods

- Can be built on top of the base methods.
- Generally -- involve creation of an **index** for the file.

- Keep index in memory for fast determination of location of data to be operated on (consider UPC code plus record of data about that item).
- If **too large**, keep index (in memory) of the main index (on disk).

Access Methods

- IBM indexed sequential-access method (ISAM)
  - Small master index, points to disk blocks of secondary index.
  - File kept sorted on a defined key.
  - All done by the OS.
- VMS operating system provides index and relative files as another example (see next slide).

Access Methods

Example 2 :

(Page: 468)

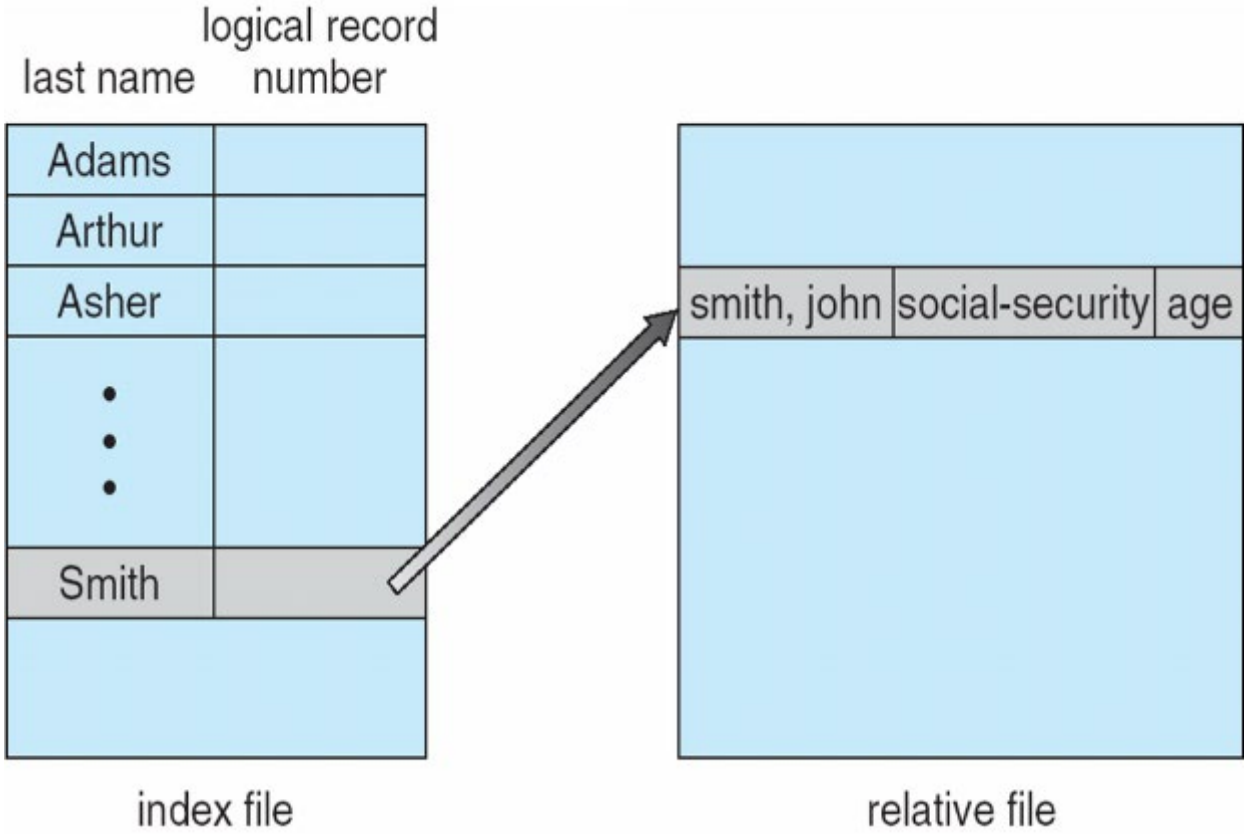


Figure 10.6 Index and Relative Files.

10.3 Directory and Disk Structure

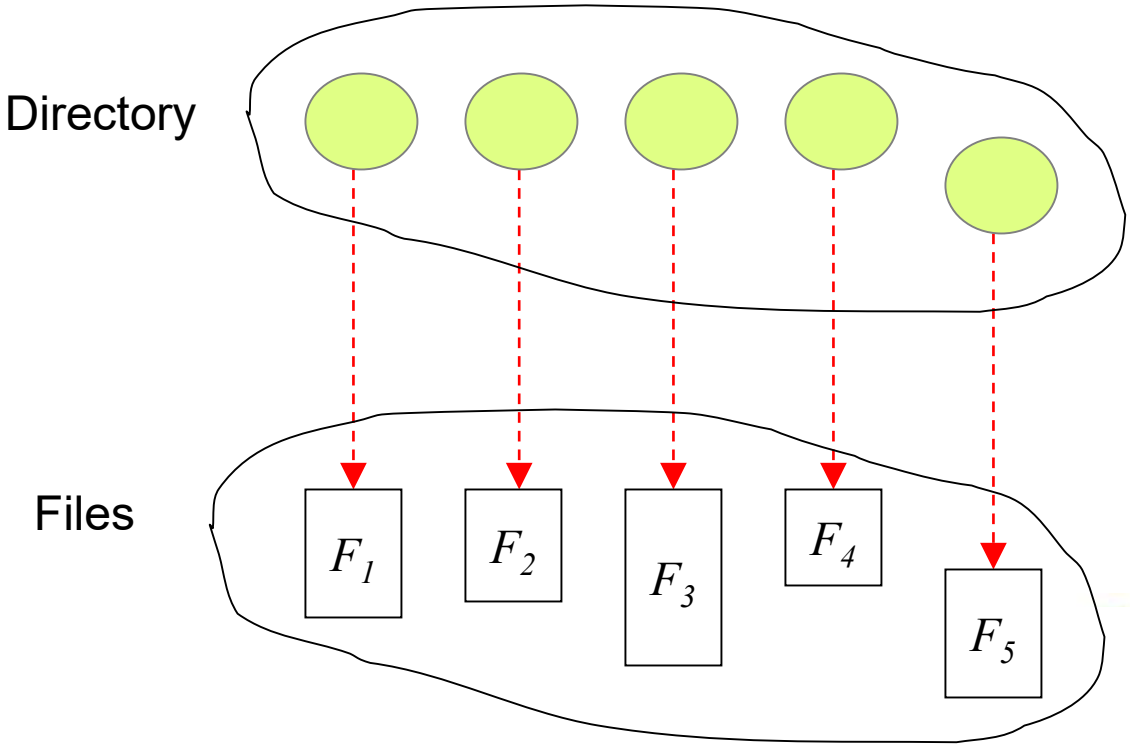
Directory structure

- Files are stored on random-access storage devices (hard disks, optical disks, solid-state disks).
- A storage devices:
  - can be used in it entirety for a file system,
  - can be subdivided into partition that hold a separate file system.
  - can be collected together into RAID sets.
- Any entity containing a file system is known as **volume**.

10.3 Directory and Disk Structure

- Each **volume** contains a file system and information about each files in the system.
  → **Device directory** / volume table of contents.

- A collection of nodes containing information about all **files**.
- Both the **directory structure** and the **files** reside on disk.



10.3 Directory and Disk Structure

Typical Volume Configuration

Volume:

- **Secondary storage unit**
  - Removable – *e.g.* CD, DVD, USB
  - Non-removable – *e.g.* hard disks
- **Multi-file volume**
  - Contains many files.
- **Multi-volume files**
  - Extremely large files spread across several volumes.

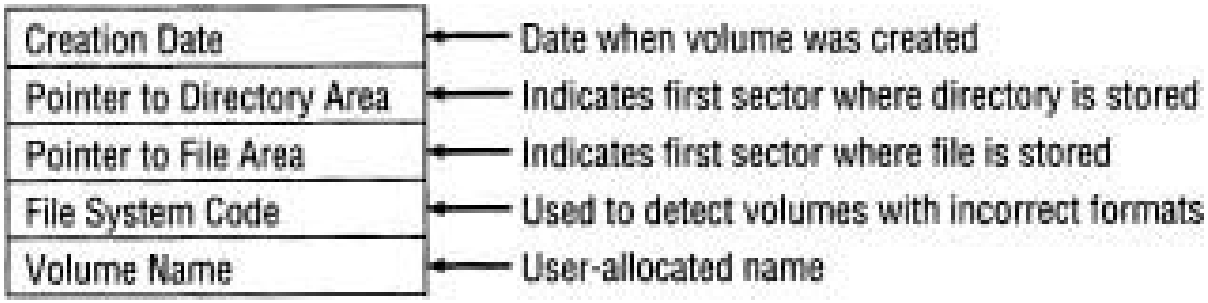
Removable storages unit

Non-Removable storages unit

10.3 Directory and Disk Structure

Volume Name:

- File manager writes the **volume name** and other descriptive information on **easily accessible area** on each unit.
  - ✓ *e.g.* Innermost part of CD, beginning of tape, first sector of outermost track.



(figure 8.3)

The volume descriptor, which is stored at the beginning of each volume, includes this vital information about the storage unit.

10.3 Directory and Disk Structure

Example 3 :

Hard disk components.

http://ojelhtc.blogspot.com/2010/09/tutorial-cara-instal-windows-xp-pada.html

10.3 Directory and Disk Structure

Reading / writing **heads** that constantly move on a ray of the disks to read / write the data.

A set of disks called **platters** which store data on tracks

- All disk surface is covered by these heads because of their **spinning**.
- Typically a fast hard disk uses a **motor** that spins the platters at more than 7200 rotations per minute.

10.3 Directory and Disk Structure

Operation performed on directory

- Search for a file.
- Create a file.
- Delete a file.
- List a directory.
- Rename a file.
- Traverse the file system.

10.3 Directory and Disk Structure

Directory organization

The directory is organized logically to obtain:

- **Efficiency** – locating a file quickly.
- **Naming** – convenient to users.
  - Two users can have same name for different files
  - The same file can have several different names
- **Grouping** – logical grouping of files by properties (*e.g.*, all Java programs, all games, …).

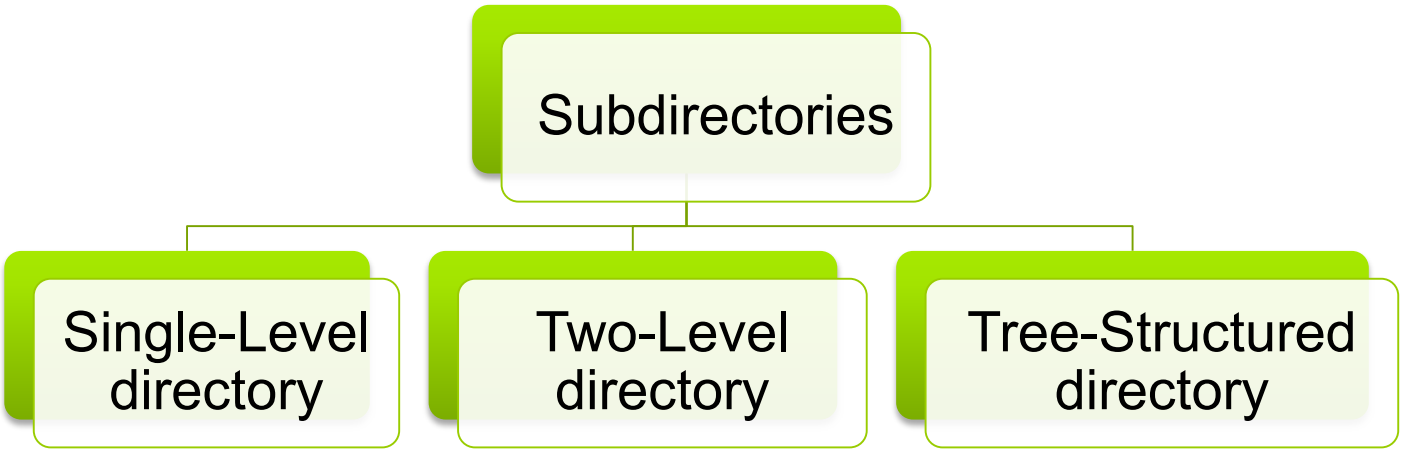
10.3 Directory and Disk Structure

Introducing subdirectories

- File Managers create an **MFD** (**Master File Directory**) for each volume.
- MFD is stored immediately after volume descriptor.
- It lists:
  - Names and characteristics of every file in volume.
    - File names (program files, data files, system files)
  - Subdirectories.
    - If supported by file manager.
  - Remainder of volume.
    - Used for file storage.

FAT (File Allocation Table)
NTFS (New Technology File System)

10.3 Directory and Disk Structure



10.3 Directory and Disk Structure

(a) Single-Level directory

- A single directory for all users.

Single directory per volume

Supported by early operating systems

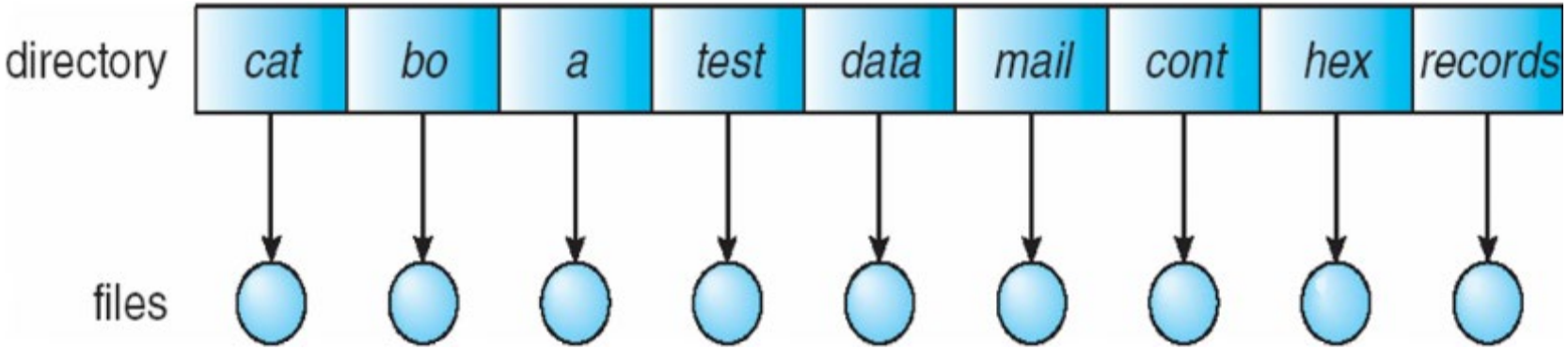
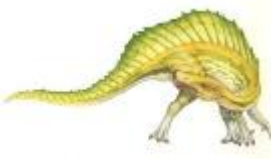


Figure 10.9 Single-level directory.

10.3 Directory and Disk Structure

Disadvantages:

- Long search time for individual file.
- Users cannot create subdirectories.
- Users cannot safeguard their files because entire directory was freely available to every user.
- Each program needs unique name.



(b) Two-Level directory

- Separate directory for each user.

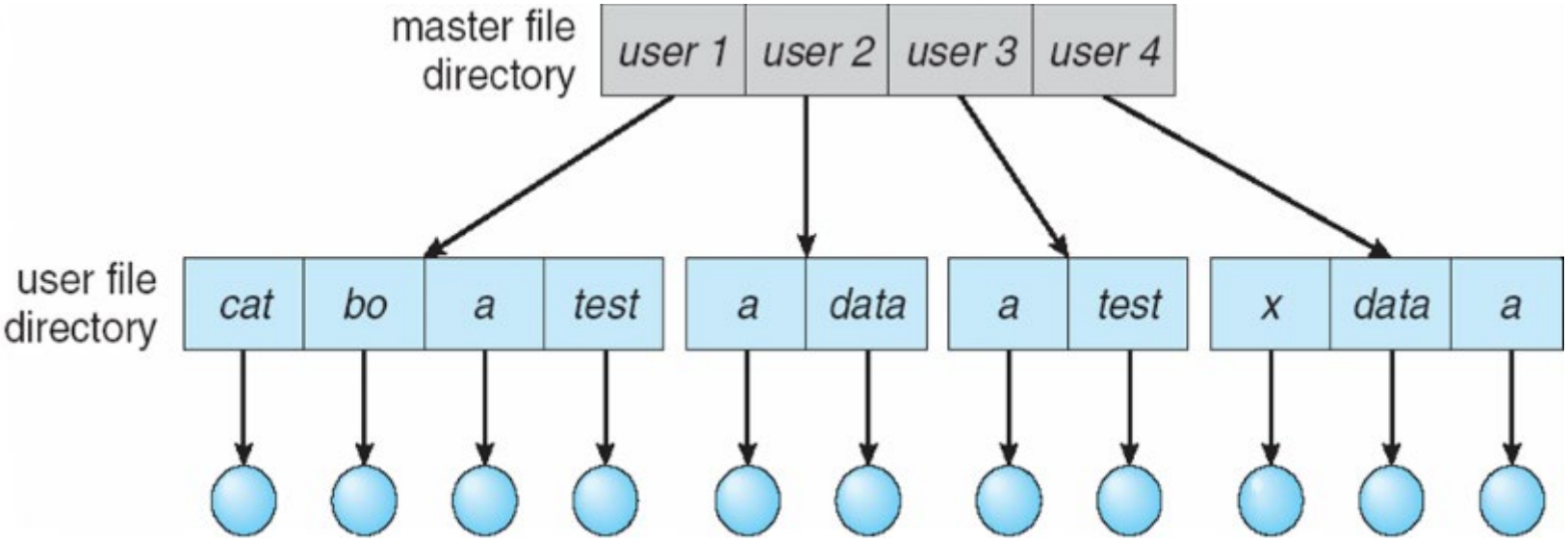


Figure 10.10 Two-level directory.

10.3 Directory and Disk Structure

- **Advantages**:
  - ✓ Path name.
  - ✓ Can have the same file name for different user.
  - ✓ Efficient searching.
- **Disadvantage**:
  - ✓ Problem remain → No logically file grouping capability

10.3 Directory and Disk Structure

(c) Tree-Structured directory

- File managers today.
- Users create own subdirectories (**folders**).
  - ✓ Related files are grouped together.
- Implemented as **upside-down tree.**
  - ✓ Efficient system searching of individual directories.
  - ✓ May require several directories to reach file.

10.3 Directory and Disk Structure

(c) Tree-Structured directory

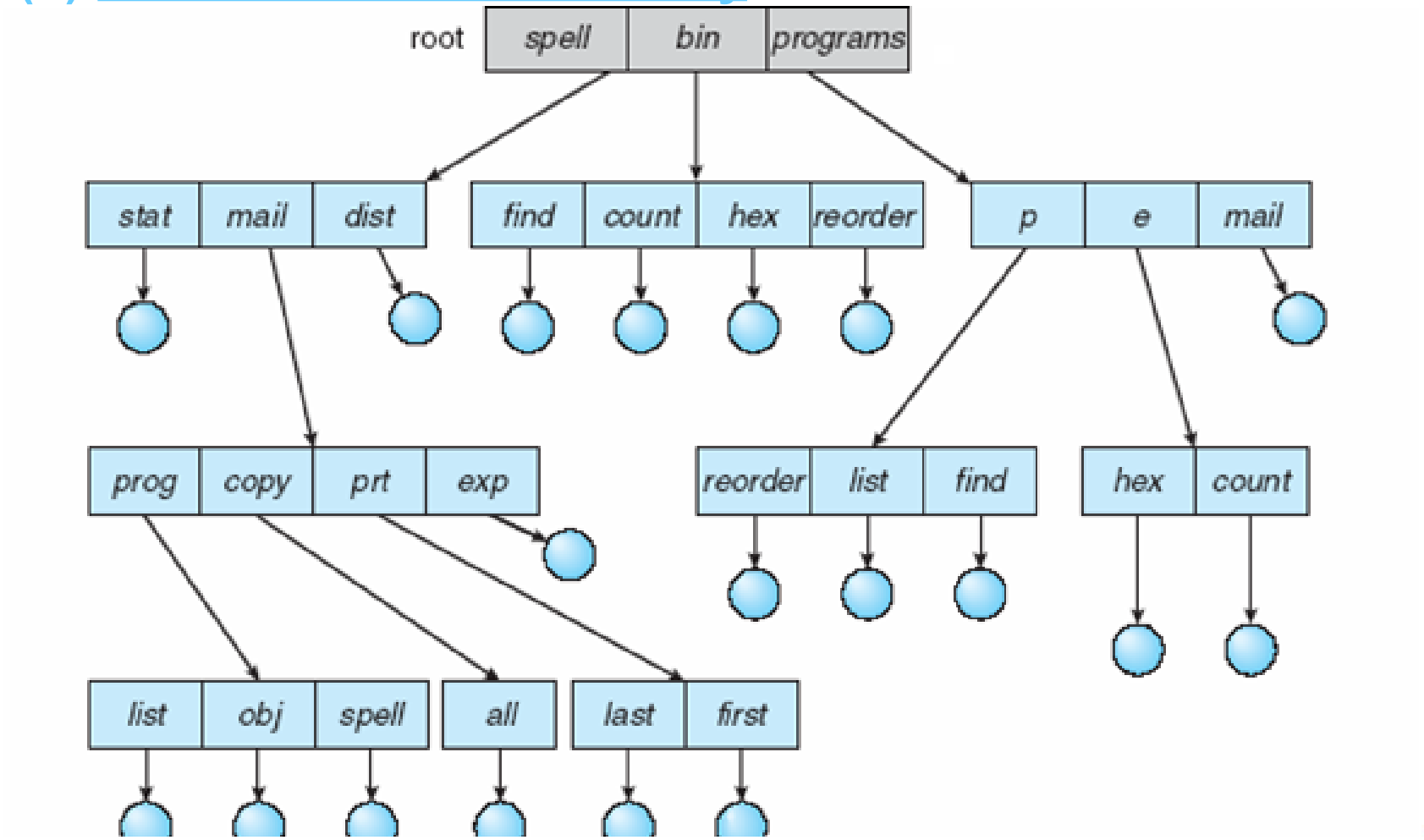


Figure 10.11 Tree-structured directory.

10.3 Directory and Disk Structure

(c) Tree-Structured directory

- **Advantages**:
  - ✓ Efficient searching.
  - ✓ Grouping Capability.

- Current directory (working directory):

Command = cd.

```
cd /spell/mail/prog
type list
```

10.3 Directory and Disk Structure

(c) Tree-Structured directory: Path name

- **Absolute** or **relative** path name.
- Creating a new file is done in current directory.

- Delete a file: Remove (rm)

```
rm <file-name>
```

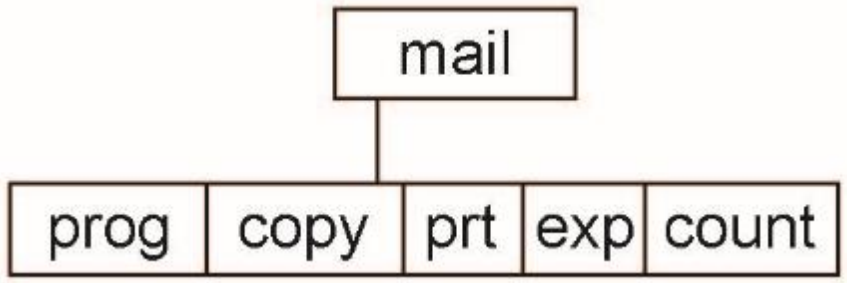
- Creating a new subdirectory is done in current directory

```
mkdir <dir-name>
```

10.3 Directory and Disk Structure

Example 4 : Suppose the current directory is: -- "`/mail`"

```
mkdir count
```

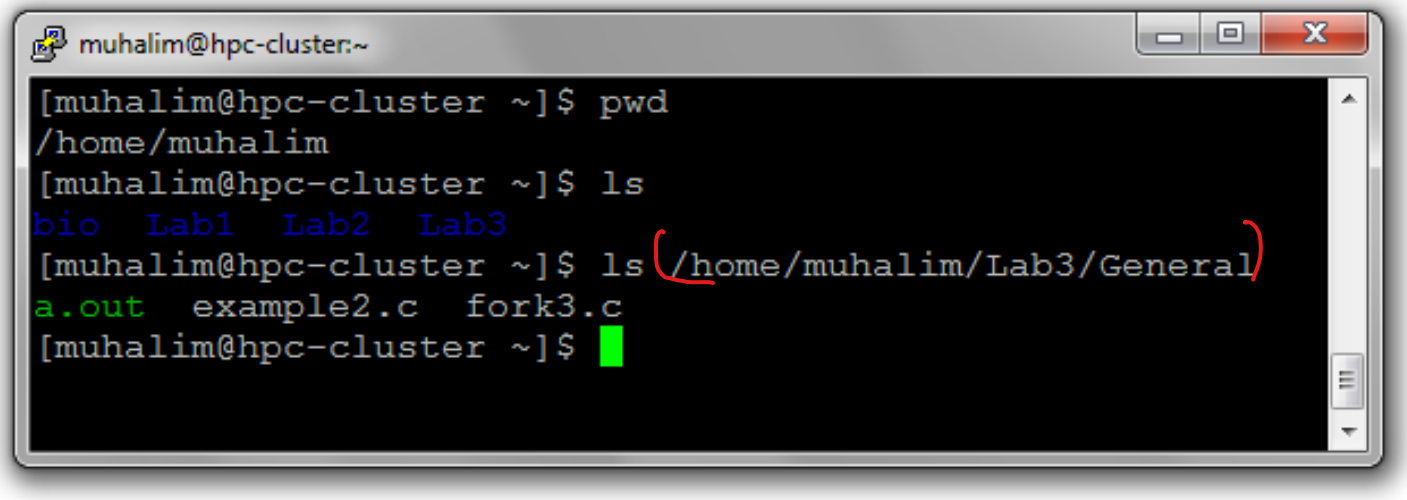


- Deleting "`mail`" → deleting the entire subtree rooted by "`mail`"

Directory and Disk Structure

Example 5a :

- Filename components:
  - Relative filename and extension
- Complete filename (**Absolute path name**)
  - begins at the root and follows a path down to the specified file
  - *e.g.* `root/spell/mail/prt/first`

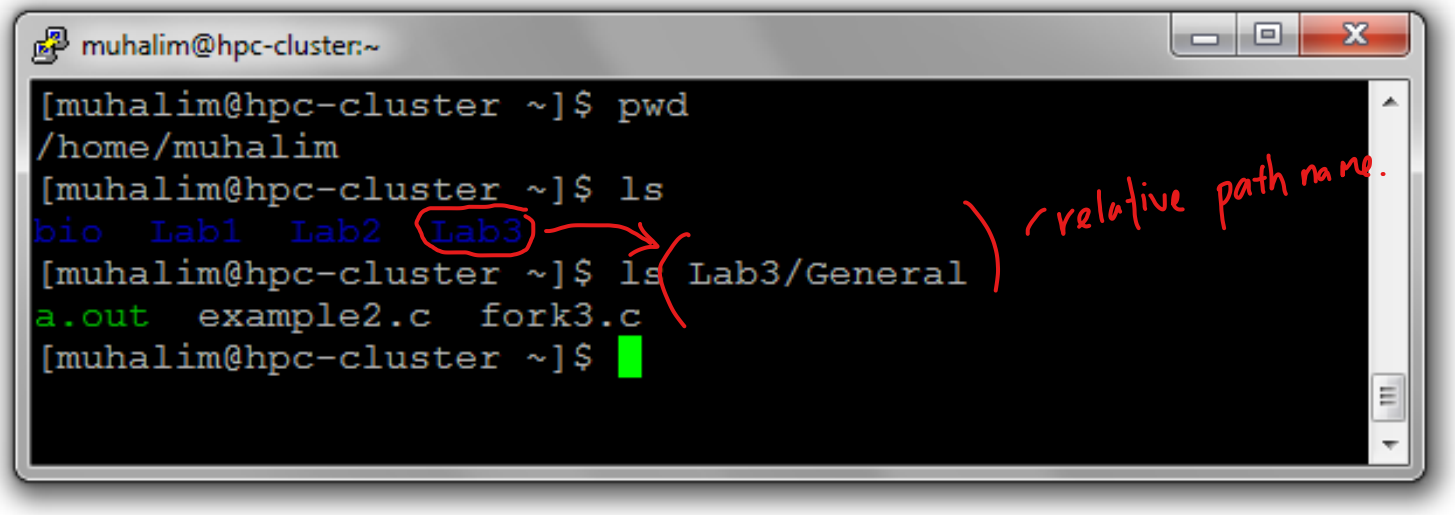


10.3 Directory and Disk Structure

Example 5b :

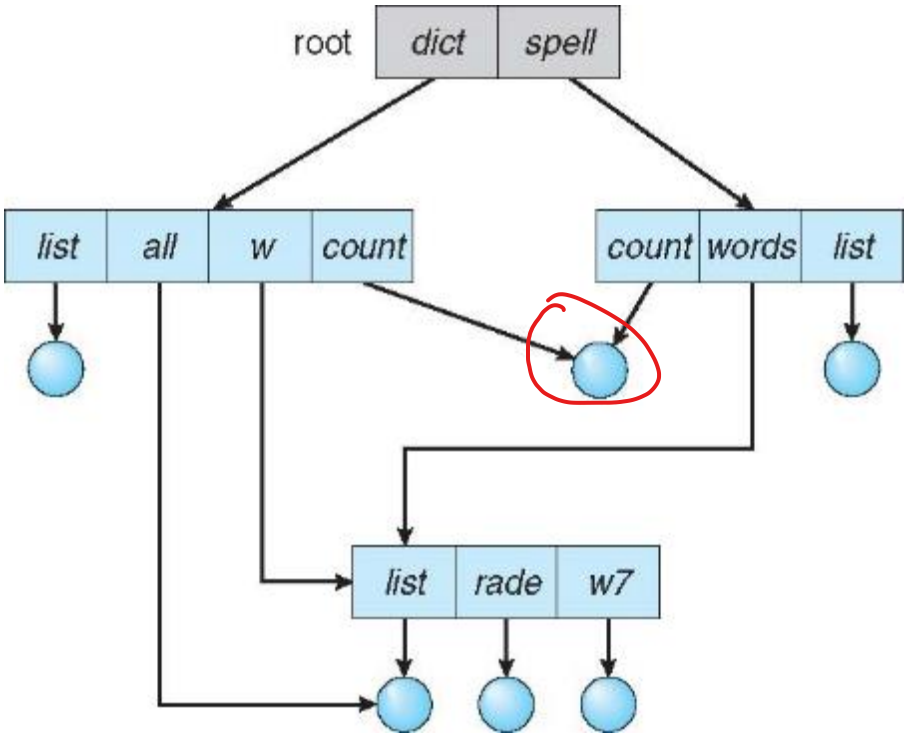
Relative path name

- Name without path information
  - Defines a path from the current directory
  - *e.g.* Current directory: `root/spell/mail`, relative path: `prt/first`



Acyclic-Graph Directories

Have shared subdirectories and files



Acyclic-Graph Directories (Cont.)

- Files/subdirectories have two different names (aliasing)
  - Only one actual file exists, so any changes made by one person are immediately visible to the other.
- How do we implement shared files and subdirectories?
  - Can duplicate the *inode* information about the shared file/subdirectory in each of the subdirectories that "point' to the shared structure.
    - If some information changes about the file it had to be updated in several places.
  - Have a new directory entry type:
    - **Link** – another name (pointer) to an existing file or subdirectory.

Acyclic-Graph Directories -- Links

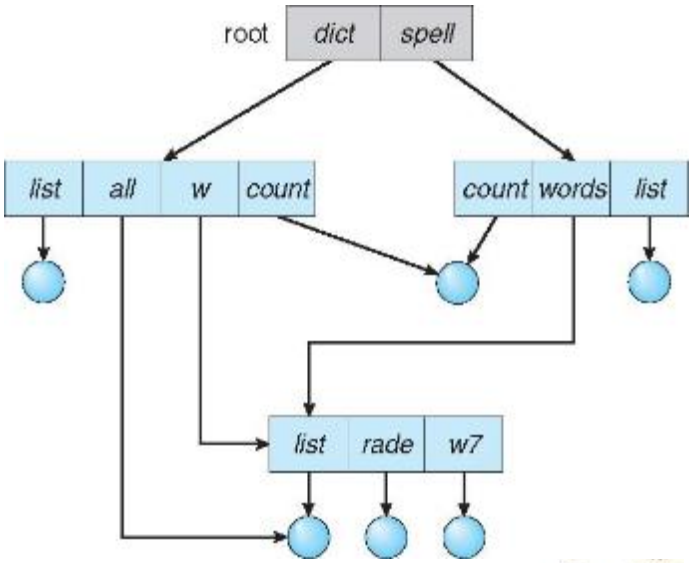
- A link may be implemented as an **absolute** or a **relative** path name.
- When a reference to a file is made, the directory is searched. If the directory entry is marked as a link, then the name of the real file is included in the link information.
- We **resolve** the link by using that path name to locate the real file.
- Links are easily identified by their format in the directory entry (or by having a special type on systems that support types) and are effectively indirect pointers.
- The operating system ignores these links when traversing directory trees to preserve the acyclic structure of the system.

Acyclic-Graph Directories -- Deletion

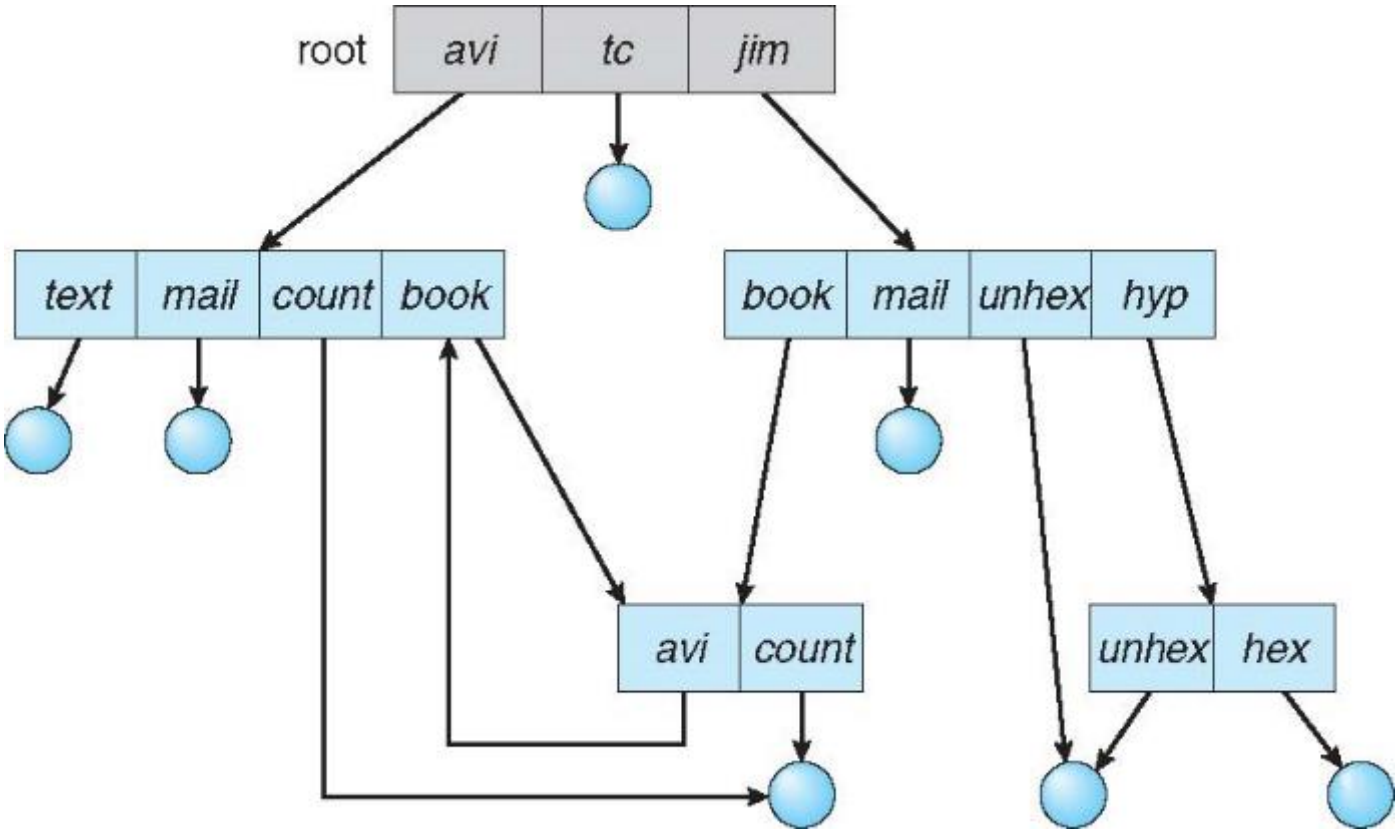
- If ***dict /count*** is deleted ⇒ dangling pointer

  Solutions:

  - Backpointers -- so we can delete all pointers.
    - Variable size records a problem
  - Backpointers using a daisy-chain organization
  - Entry-hold-count solution



General Graph Directory



General Graph Directory (Cont.)

- How do we guarantee no cycles?
  - Allow only links to file not subdirectories
  - **Garbage collection**
  - Every time a new link is added use a cycle detection algorithm to determine whether it is OK

Protection

Overview

- File owner/creator should be able to control:
  - What can be done?
  - By whom?

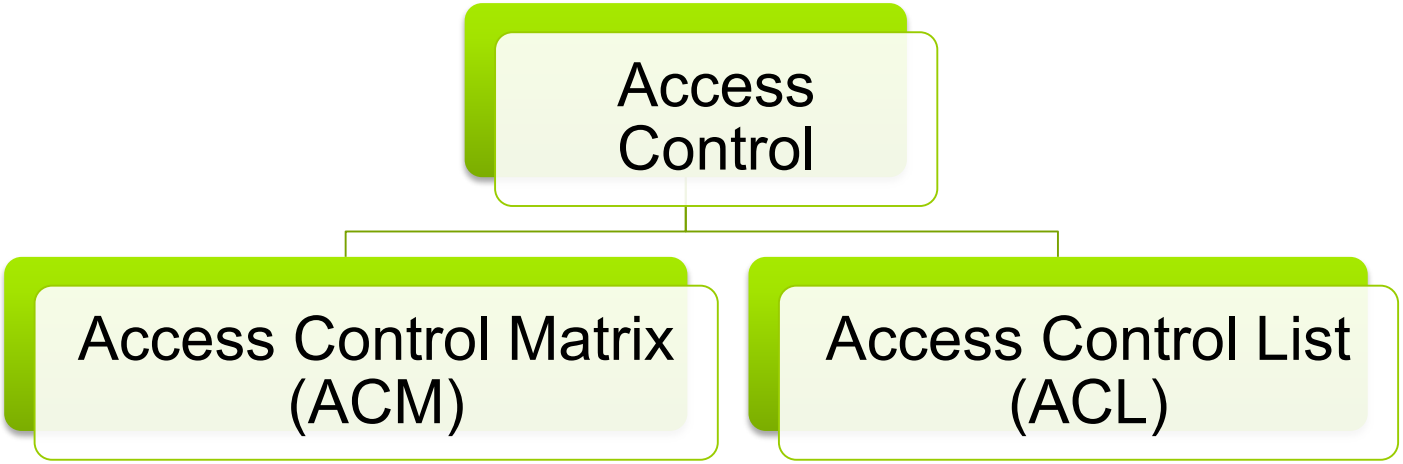
- Types of access:
  - `read`
  - `write`
  - `execute`
  - `append` 附加
  - `delete`
  - `list`

Protection

Access control verification module

- **File sharing**
  - Data files, user-owned program files, system files.
  - **Advantages**
    - Save space, synchronized data updates, system's resource efficiency (I/O operations can be reduced).
  - **Disadvantage** 文件完整性. 访问控制
    - Need to protect file integrity → **access control**

Protection



(a) Access Control Matrix (ACM)

- An **access control matrix** abstracts the state relevant to access control.
- This method shows the access rights for each user for each file.

Protection

Example 6 :

- User 1 is allowed unlimited access to File 1 but is allowed only read and execute File 4 and is denied access to the three other files (File 2, 3 and 5).

R = Read Access
W = Write Access
E = Execute Access
D = Delete Access
– = Access Not Allowed

	User 1	User 2	User 3	User 4	User 5
File 1	RWED	R–E–	----	RWE–	--E–
File 2	----	R–E–	R–E–	--E–	----
File 3	----	RWED	----	--E–	----
File 4	R–E–	----	----	----	RWED
File 5	----	----	----	----	RWED

Protection

- **Advantages:** (Access control matrix)
  - Easy to implement.
  - Works well in system with few files, users.
- **Disadvantages:**
  - As files and users increase, matrix size increases.
    - Possibly beyond main memory capacity
  - Wasted space: due to null entries.

Protection

Example 6b :

- The five access codes for User 2 from **Example 6a**.
- The resulting code for each file is created by assigning a '1' for each checkmark, and a '0' for each blank space.

R = Read Access
W = Write Access
E = Execute Access
D = Delete Access
– = Access Not Allowed

Access	R	W	E	D	Resulting Code	
R–E–	√		√		1010	*File 1*
R–E–	√		√		1010	*File 2*
RWED	√	√	√	√	1111	*File 3*
----					0000	*File 4*
----					0000	*File 5*

Protection

(b) Access Control List (ACL)

- An **access control list** showing which users are allowed to access each file.

- It is the most general scheme to implement independent access is to associate with
  - ✓ each **file**, and
  - ✓ **directory** the specifying user names and the types of access allowed for each user.

Protection

Example 7 :

- ACL method requires less storage space than an access control matrix.

R = Read Access
W = Write Access
E = Execute Access
D = Delete Access
– = Access Not Allowed

File	Access
File 1	USER1 (RWED), USER2 (R-E-), USER4 (RWE-), USER5 (--E-), WORLD (----)
File 2	USER2 (R-E-), USER3 (R-E-), USER4 (--E-), WORLD (----)
File 3	USER2 (RWED), USER4 (--E-), WORLD (----)
File 4	USER1 (R-E-), USER5 (RWED), WORLD (----)
File 5	USER5 (RWED), WORLD (----)

Protection

- Contains user names granted file access with ACL.
  - User denied access grouped under “WORLD”
- Shorten list by categorizing users:

  - **SYSTEM**
    - Personnel with unlimited access to all files.
  - **OWNER**
    - Absolute control over all files created in own account.
  - **GROUP**
    - All users belonging to appropriate group have access.
  - **WORLD**
    - All other users in system.

Protection

Example: UNIX

- Solve the problem of directory size by classifying users into 3 classes:
  - **Owner** – A user who creates the file.
  - **Group** – A set of users who are sharing the file and need similar access.
  - **Universe/public** – All users.
- Defines 3 bits to represent each class: `R W X`
  - *e.g.* `RW- R-- ---` indicates owner has `RW` access, group has `R` access and public has no access.

Protection

Example 8a :

R=Read access
W=Write access
X=Execute access.

With each file and subdirectory we keep 9 protection bits, 3 for owner, 3 for group, 3 for public.

```
RWX R-- R-X
```

- Owner access 7 → RWX
  111
- Group access 4 → R--
  100
- Public access 5 → R-X
  101

Protection

Example 8b :

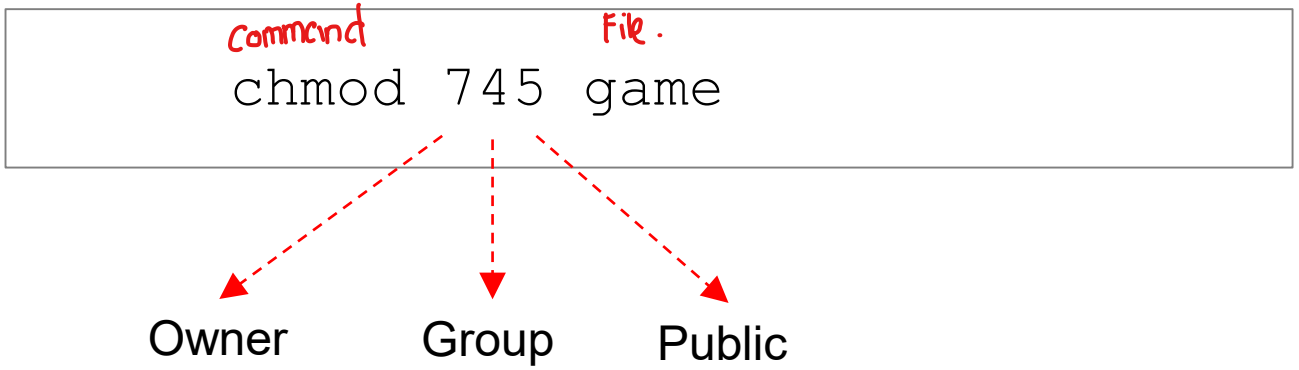
UNIX: Set protection for file "`game`" using the command "`chmod`".

7 111 RWX owner

4 100 R-- group

5 101 R-X public.

- ✓ All user classes can read the file.
- ✓ Only owner can write the file.
- ✓ Owner and public can execute the file.



Protection

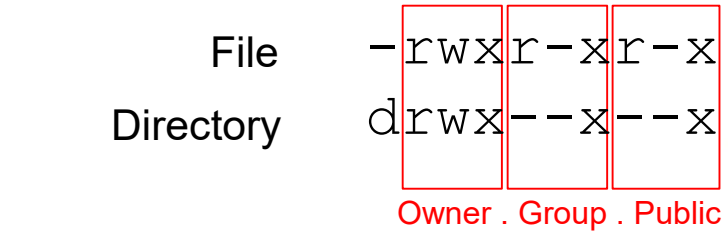
Example 9a : A sample UNIX directory listing.

-rw-rw-r--	1 pbg	staff	31200	Sep 3 08:30	intro.ps
drwx------	5 pbg	staff	512	Jul 8 09.33	private/
drwxrwxr-x	2 pbg	staff	512	Jul 8 09:35	doc/
drwxrwx---	2 pbg	student	512	Aug 3 14:13	student-proj/
-rw-r--r--	1 pbg	staff	9423	Feb 24 2003	program.c
-rwxr-xr-x	1 pbg	staff	20471	Feb 24 2003	program
drwx--x--x	4 pbg	faculty	512	Jul 31 10:31	lib/
drwx------	3 pbg	staff	1024	Aug 29 06:52	mail/
drwxrwxrwx	3 pbg	staff	512	Jul 8 09:35	test/

Owner Group. Subdirectory / File. Subdirectory. Subdirectory (d).

Explanation:

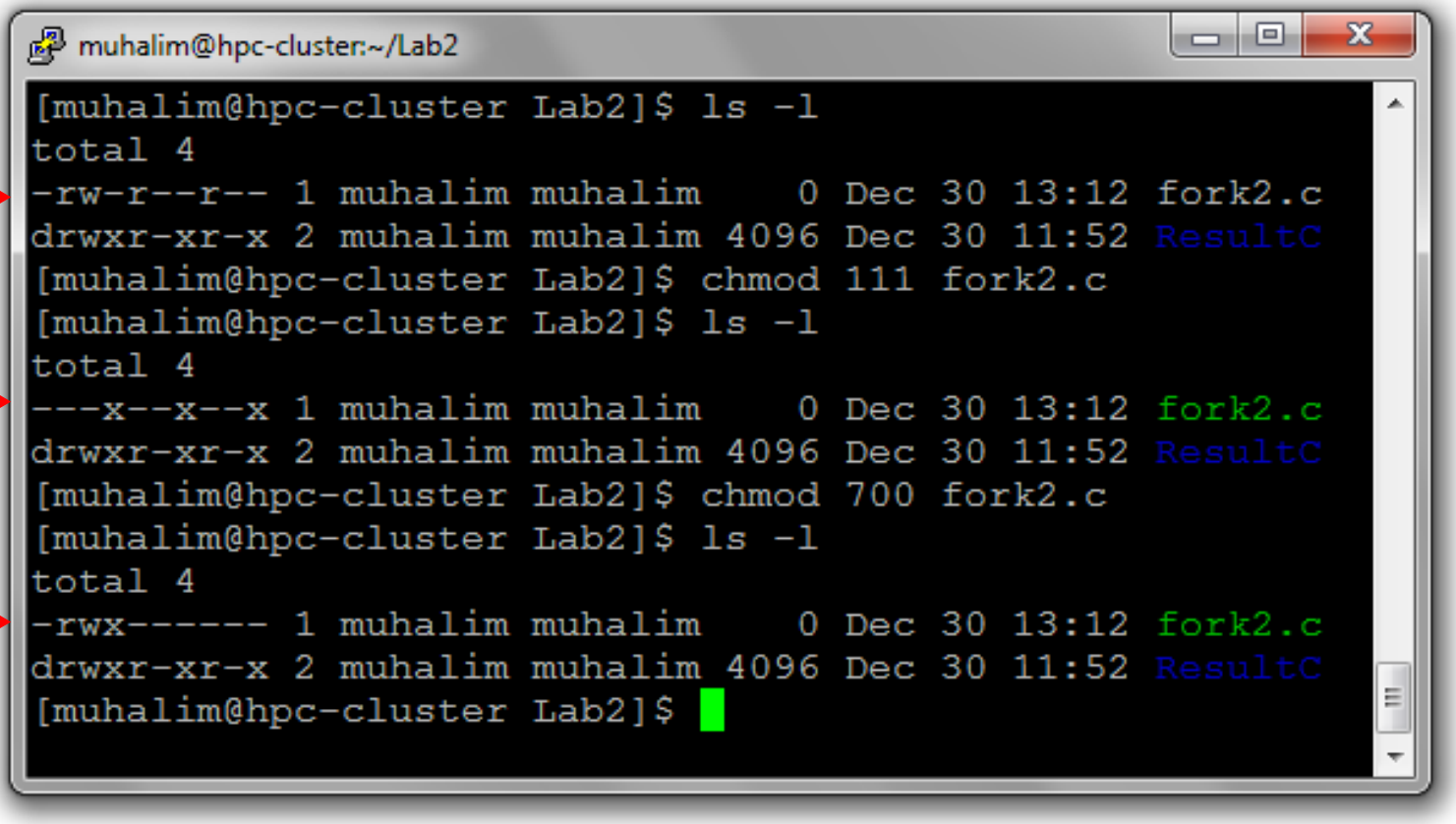
- "`into.ps`" is a file owned by "`pbg`" with group "`staff`"
- "`lib`" is a subdirectory owned by "`pbg`" with group "`faculty`"



Protection

All users can READ the file; cannot EXECUTE.
Only Owner can WRITE.

Owner group Public
--- --- ---



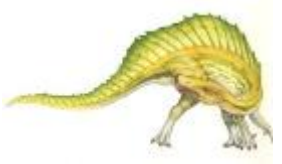
All users cannot READ, WRITE; but they can EXECUTE the file.

Only Owner can READ, WRITE, EXECUTE.
Group and Public cannot access the file.

File -rw-r--r--

Directory drwxr-xr-x

Owner Group Public



Protection

Exercise 1 : Given a sample UNIX directory listing.

-rw-rw-r--	1 pbg	staff	31200	Sep 3 08:30	intro.ps
drwx------	5 pbg	staff	512	Jul 8 09.33	private/
drwxrwxr-x	2 pbg	staff	512	Jul 8 09:35	doc/
drwxrwx---	2 pbg	student	512	Aug 3 14:13	student-proj/
-rw-r--r--	1 pbg	staff	9423	Feb 24 2003	program.c
-rwxr-xr-x	1 pbg	staff	20471	Feb 24 2003	program
drwx--x--x	4 pbg	faculty	512	Jul 31 10:31	lib/
drwx------	3 pbg	staff	1024	Aug 29 06:52	mail/
drwxrwxrwx	3 pbg	staff	512	Jul 8 09:35	test/

3 6

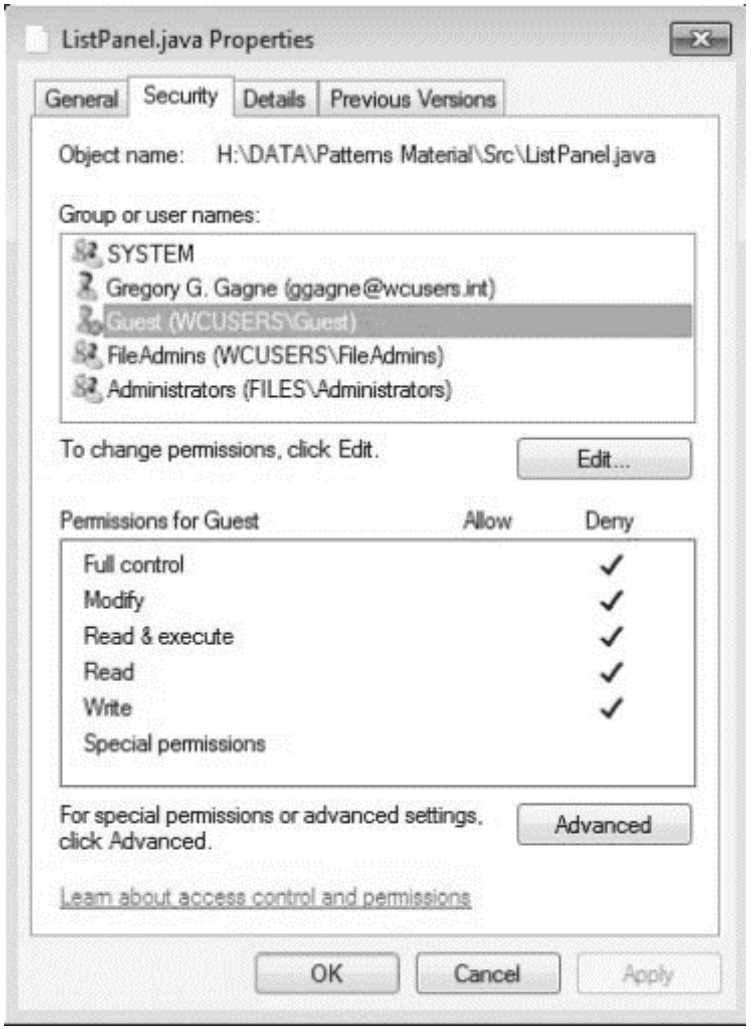
(a) How many files and directories exist in the listing?

(b) Define the access control of all files.

Access Control List (ACL)

(c) Who can access the `program.c`? Justify your answer.

Windows 7 Access-Control List Management



Summary

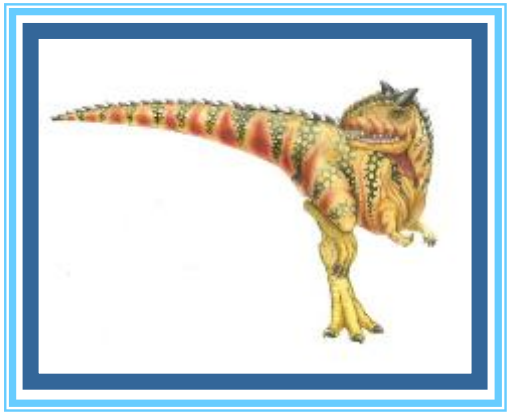
- A **file** is an abstract data type defined and implemented by the operating system (OS).
- A file is a sequence of **logical records**.
- An OS needs to **map** the logical file concept onto physical storage devices.
- A **tree-structured directory** allows a user to create subdirectories to organize files.
- **Access** to files can be controlled separately for each type of access (`read`, `write`, `execute`, `delete`)

Exercises

From Silberchatz, Operating System Concepts
Chapter 10 Exercises (page: 491 - 492)

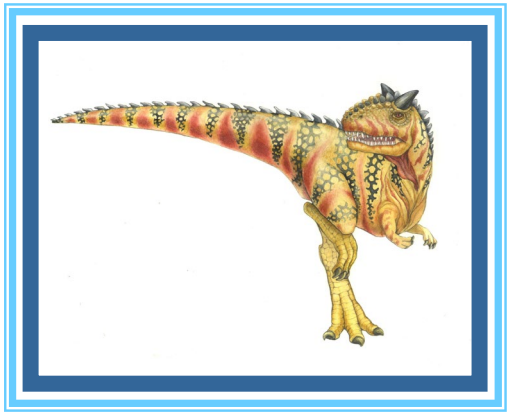
- 10.1
- 10.4
- 10.5
- 10.6
- 10.7
- 10.8
- 10.9
- 10.10

End of Chapter 10



Silberschatz, Galvin and Gagne ©2013

Chapter 12: Mass-Storage Structure



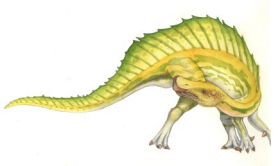
Silberschatz, Galvin and Gagne ©2013

Chapter 12: Mass-Storage Structure

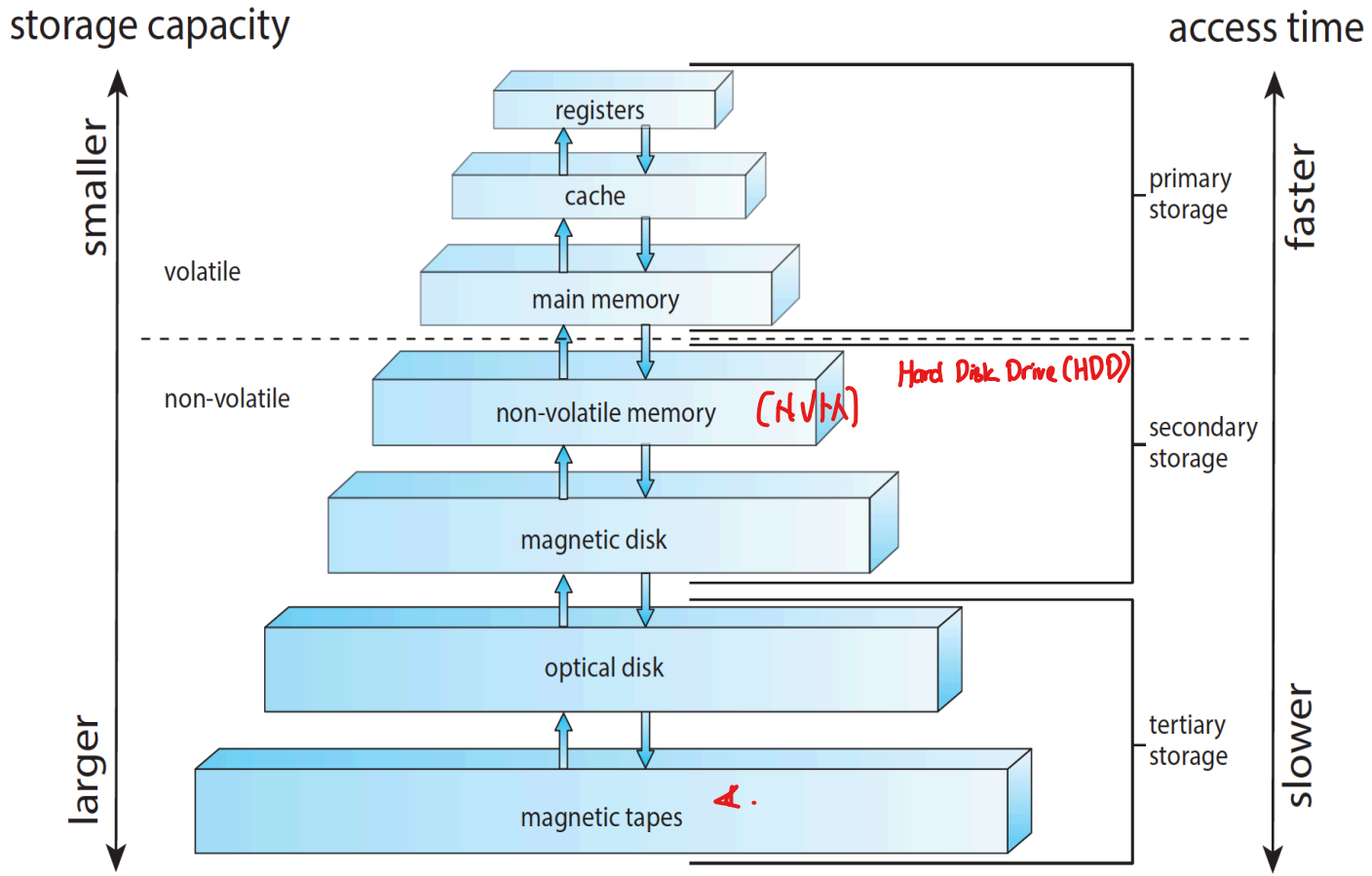
- 12.1 Overview of Mass Storage Structure
- 12.4 Disk Scheduling
- 12.7 RAID Structure

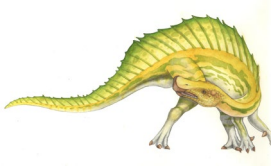
OBJECTIVE

- To describe the physical structure of secondary storage devices and its effects on the uses of the devices
- To explain the performance characteristics of mass-storage devices
- To evaluate I/O scheduling algorithms
- To discuss operating-system services provided for mass storage, including RAID



Storage-Device Hierarchy



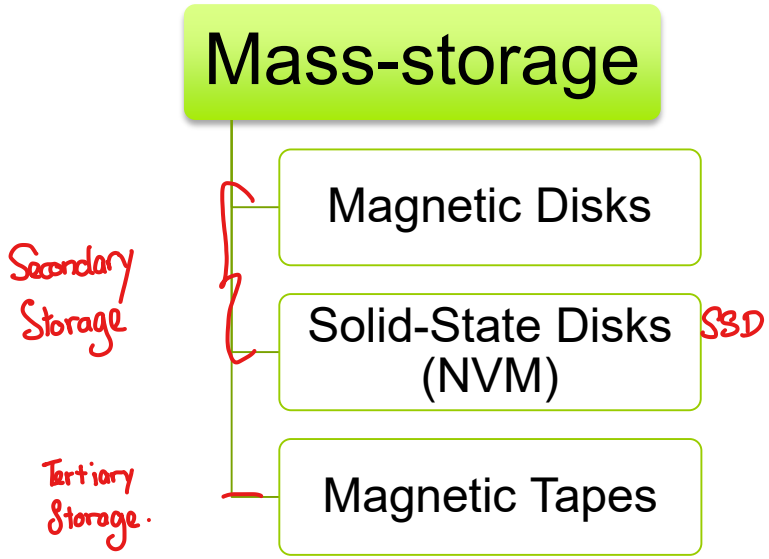


Overview of Mass-Storage Structure

- The bulk of secondary storage for modern computer are
  - Hard disk drives (HDD)
  - Nonvolatile Memory (NVM)
- The most common tertiary storage is magnetic tape.
- In this chapter we describe:
  - The basic mechanism of those devices
  - How operating systems translate their physical properties to logical storage via address mapping.

Mass-Storage Structure

Overview



Mass-Storage Structure

Secondary Storage.

Magnetic disks

- **Magnetic disks** provide bulk of secondary storage of modern computers.
- Disk **platter**: each one has a flat circular shape.
  - Diameter range: 1.8” to 3.5” (inches).
- The two surfaces of a platter are covered with a magnetic materials.

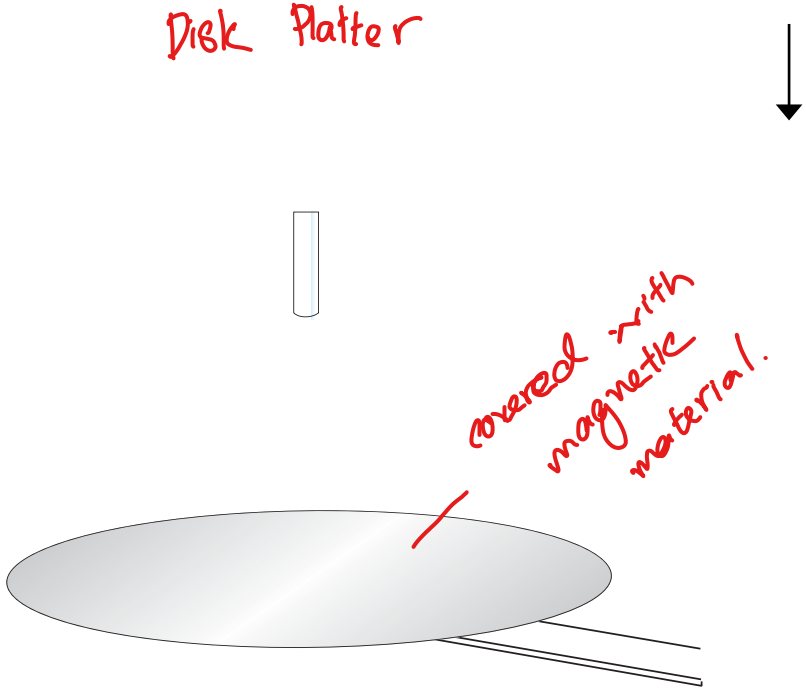


Figure 12.1 Moving-head Disk Mechanism.

Mass-Storage Structure

Magnetic disks

- Storage capacity measured in *gigabytes*: Range from 30*GB* to 3*TB* per drive.
- The read-write **heads** attached to a **disk arm** that moves all the heads as a unit.
- The surface of a platter is logically divided into circular **tracks**, which subdivided into **sectors**.

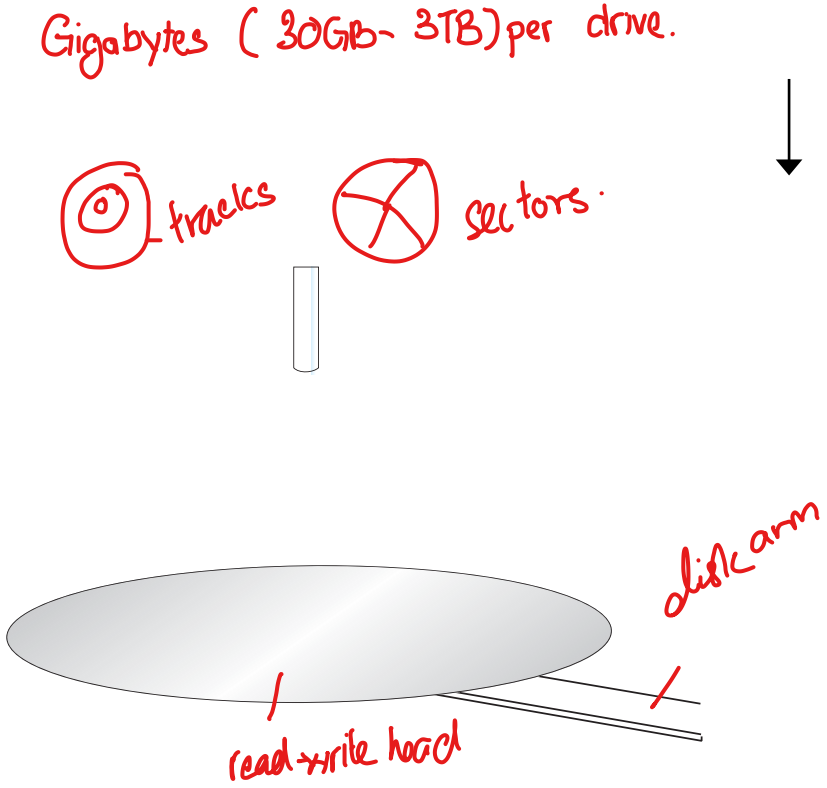


Figure 12.1 Moving-head Disk Mechanism.

Mass-Storage Structure

Magnetic disks

- The sets of tracks at one arm position makes up a **cylinder**.
- A drive motor spins (rotate) it at high speed: 60 to 250 times per second.
- Specified in term of **Rotations Per Minute** (**RPM**) → 5400, 7200, 10000, and 15000 RPM.

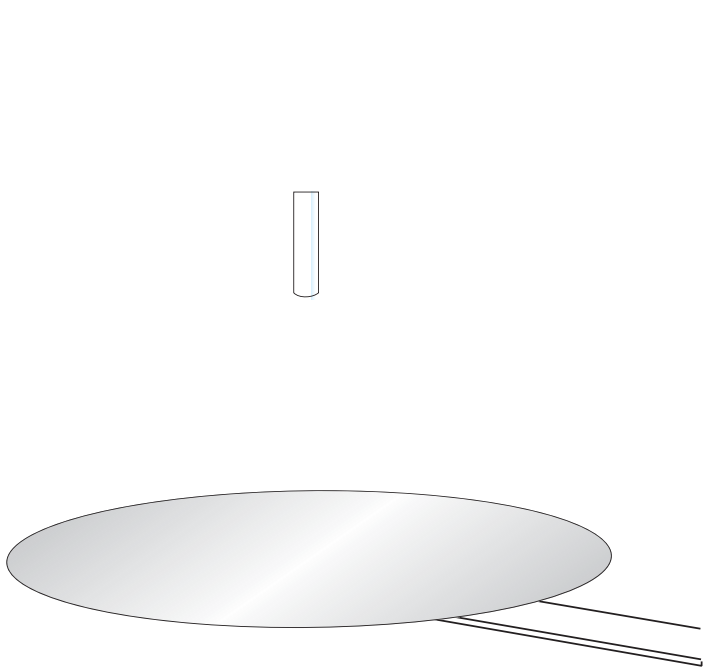
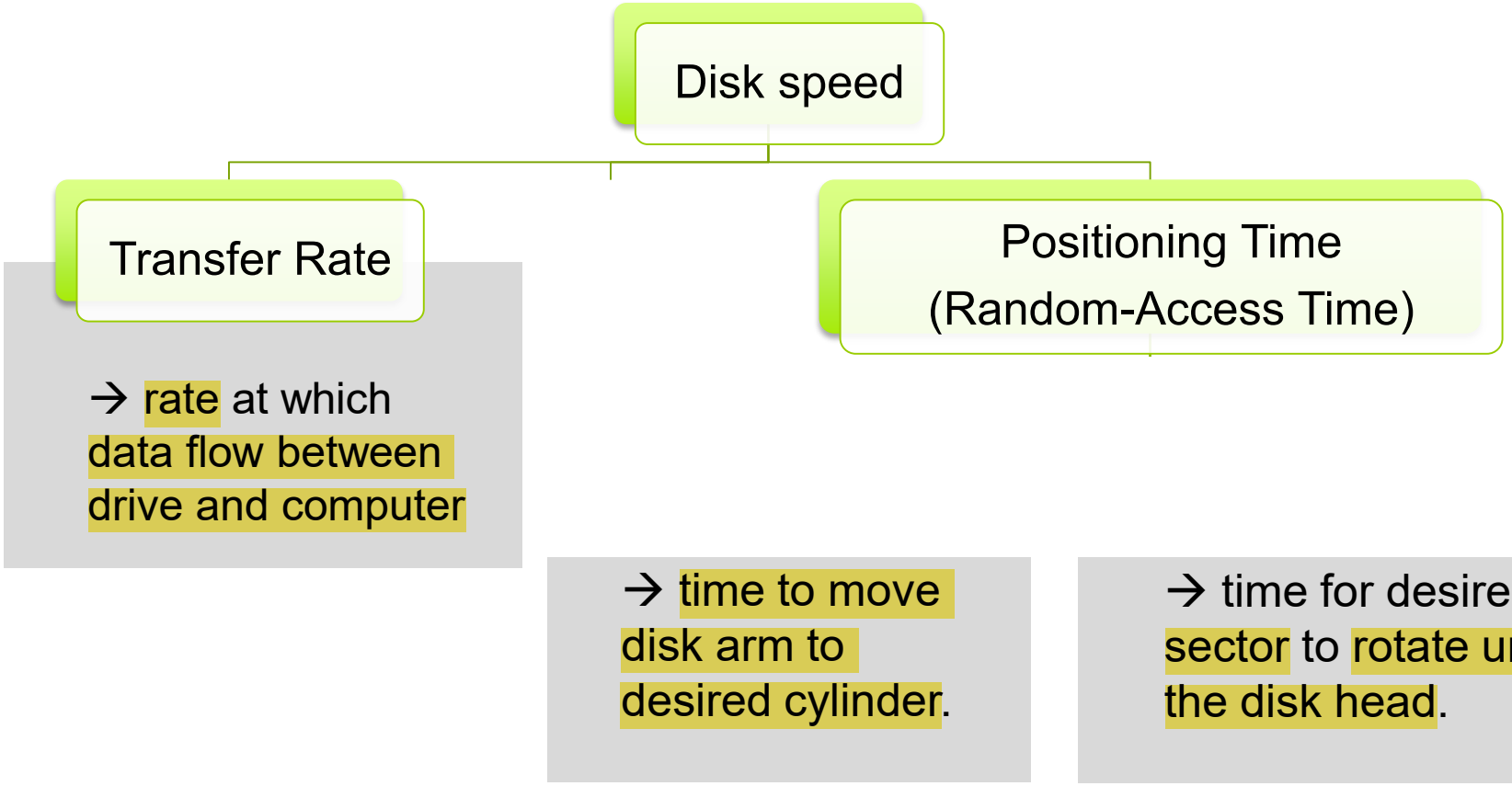


Figure 12.1 Moving-head Disk Mechanism.

Mass-Storage Structure

Magnetic disks

- Two parts of disks speed:



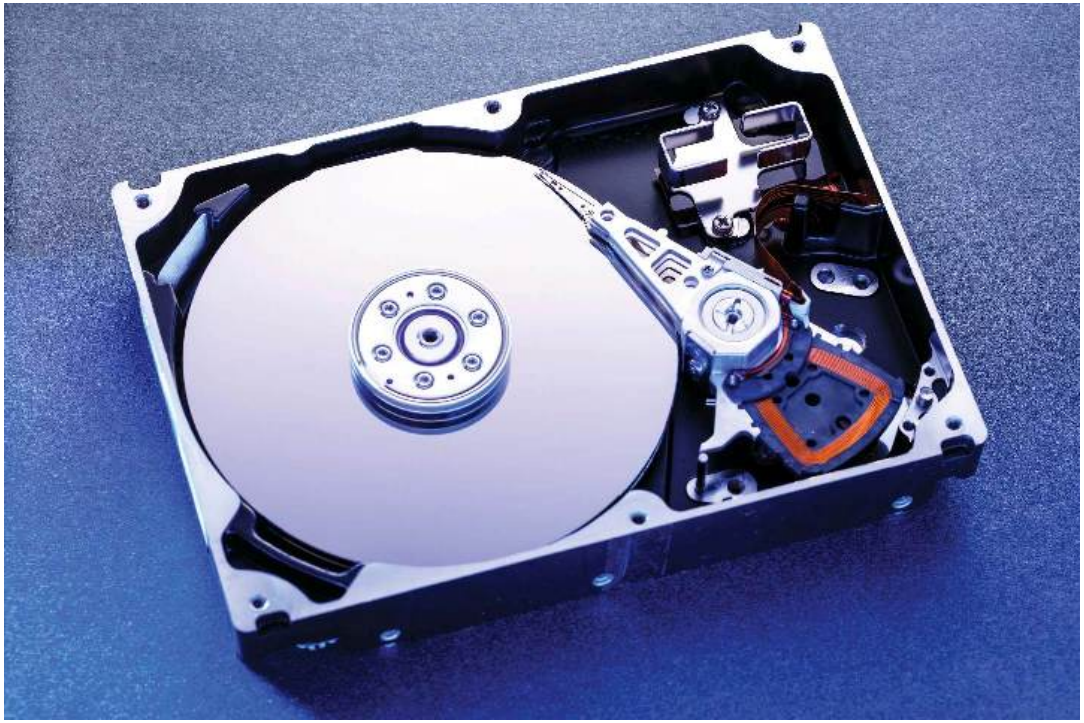
Mass-Storage Structure

Magnetic disks

- **Head crash** results from disk head making contact with the disk surface
  - That's bad
- Disks can be removable

A 3.5" HDD with Cover Removed.

Magnetic disks



The First Commercial Disk Drive

1956
IBM RAMDAC computer included the IBM Model 350 disk storage system

5 million (7 bit) characters
50 x 24” platters
Access time = < 1 second

Performance of Magnetic Disks

- Performance
  - Transfer Rate – theoretical – 6 Gb/sec
  - Effective Transfer Rate – real – 1Gb/sec
  - Seek time from 3ms to 12ms – 9ms common for desktop drives
  - Average seek time measured or calculated based on 1/3 of tracks
  - Latency based on spindle speed
    - 1 / (RPM / 60) = 60 / RPM
  - Average latency = ½ latency
- From Wikipedia

Spindle [rpm]	Average latency [ms]
4200	7.14
5400	5.56
7200	4.17
10000	3
15000	2

Mass-Storage Structure

Magnetic disks

Example

Calculate the average latency for a spindle speed 7200 RPM .

Solution

$$7200\,RPM = 60s$$

$$1RPM = \left(\frac{60}{7200}\right)/2$$

$$= 0.00833s/2$$

$$= 0.00417s = 4.17ms$$

Performance of Magnetic Disks (Cont.)

Magnetic disks

- **Access Latency** = **Average access time** = average seek time + average latency
  - For fastest disk 3ms + 2ms = 5ms
  - For slow disk 9ms + 5.56ms = 14.56ms
- Average I/O time = average access time + (amount to transfer / transfer rate) + controller overhead
- For example to transfer a 4KB block on a 7200 RPM disk with a 5ms average seek time, 1Gb/sec transfer rate with a .1ms controller overhead =
  - 5ms + 4.17ms + 0.1ms + transfer time
  - Transfer time = 4KB / 1Gb/s * 8Gb / GB * 1GB / 1024^2KB = 32 / (1024^2) = 0.031 ms
  - Average I/O time for 4KB block = 9.27ms + .031ms = 9.301ms

Secondary Storage Connection Methods

- Host-attached storage accessed through I/O ports talking to I/O busses
- Several bus technologies including **advanced technology attachment** (ATA), **serial ATA** (SATA), **eSATA**, **universal serial bus** (USB), **fibre channel** (FC), **serial attached SCSI** (SAS)
- Data transfers are carried out by special electronic processors: **controllers**
  - Host controller is in computer, device controller built into storage device
  - Talk to each other, usually via memory-mapped I/O ports
- Device controllers have built in caches
  - Data transfer from media into cache, then over bus to host (and vice versa)

Solid-State Disks: SSD (NVM)

- Nonvolatile Memory (**NVM)** is electrical vs. mechanical (HDD)
- Commonly composed of a controller and flash NAND die semiconductor chips (A **die** is a small block of semiconducting material, on which a given functional circuit is fabricated
  - Other forms include DRAM with battery backup, non-NAND die like 3D XPoint
- Flash-memory-based NVM is frequently used in disk-drive like container -> **solid-state disk (SSD)**

- Also can be in other formats like **USB drive**

SSD (Cont.)

- In all forms, acts and treated the same way
- More reliable than HDD (no moving parts), can be faster (no seek time or latency), consumes less power
- More expensive per MB, lower capacity, may have shorter lifespan (writes wear it out)
- Higher speed means new connection methods
  - Direct to PCIe bus (called NVMe)
- Can be used in variety of ways – replacement for disk, caching tier, etc

Magnetic Tape

- Was early secondary-storage medium
  - Evolved from open spools to cartridges
- Relatively permanent and holds large quantities of data
- Access time slow
- Random access ~1000 times slower than disk
- Mainly used for backup, storage of infrequently-used data, transfer medium between systems

Magnetic Tape (Cont)

- Kept in spool and wound or rewound past read-write head
- Once data under head, transfer rates comparable to disk
  - 140MB/sec and greater
- GB to TB typical storage
- Common technologies are LTO-{5,6}, SDLT, 4, 8, and 19mm, and ¼ and ½" widths

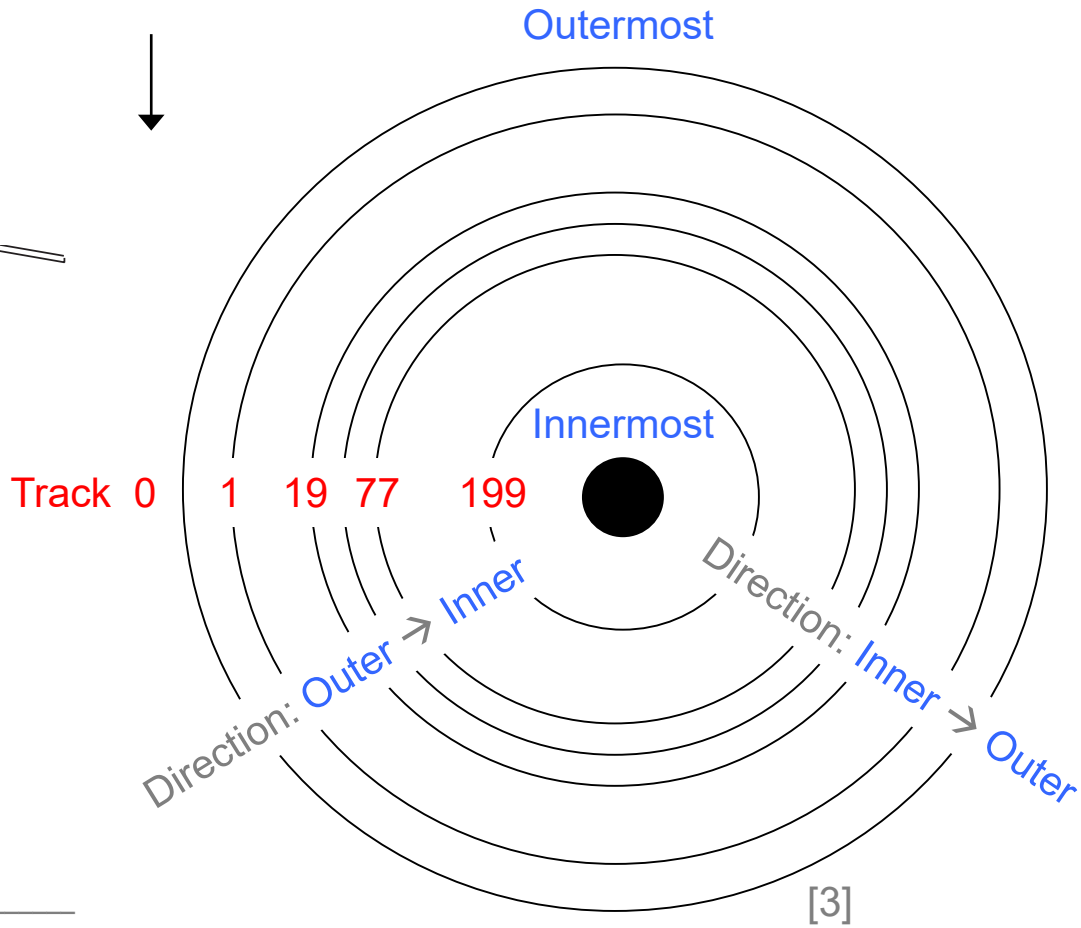
Disk Scheduling

- The operating system is responsible for using hardware efficiently — for the disk drives, this means having a fast access time and disk bandwidth
- Minimize seek time
  - Seek time ≈ seek distance
- Disk **bandwidth** is the total number of bytes transferred, divided by the total time between the first request for service and the completion of the last transfer

Disk Scheduling (Cont.)

- There are many sources of disk I/O request
  - OS
  - System processes
  - Users processes
- I/O request includes input or output mode, disk address, memory address, number of sectors to transfer
- OS maintains queue of requests, per disk or device
- Idle disk can immediately work on I/O request, busy disk means work must queued.
  - **Optimization algorithms only make sense when a queue exists**

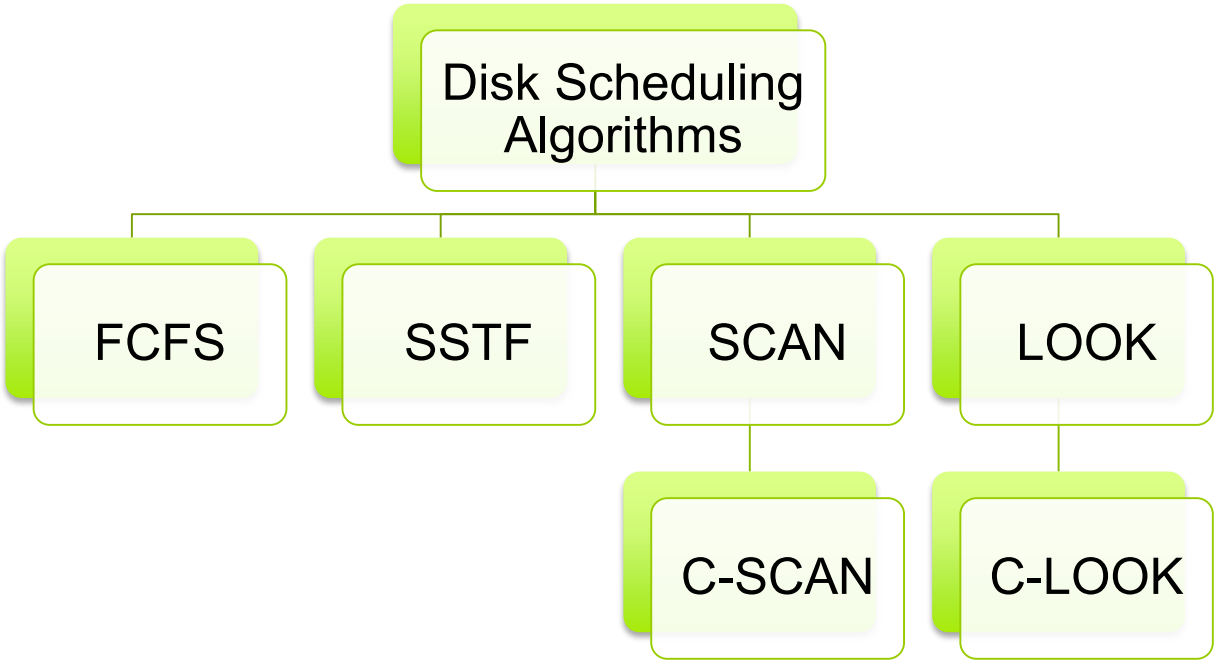
Disk Scheduling (Cont.)



Resources:
[3] http://cnx.org/content/m27262/latest/

Disk Scheduling (Cont)

- Note that drive controllers have small buffers and can manage a queue of I/O requests (of varying “depth”)
- Several algorithms exist to schedule the servicing of disk I/O requests



First-Come, First-Serve (FCFS)
Shortest-Seek-Time-First (SSTF)
Circular SCAN (C-SCAN)
Circular LOOK (C-LOOK)

REFER TO PART B

Selecting a Disk-Scheduling Algorithm

- SSTF is common and has a natural appeal
- SCAN and C-SCAN perform better for systems that place a heavy load on the disk
  - Less starvation
- Performance depends on the number and types of requests
- Requests for disk service can be influenced by the file-allocation method
  - And metadata layout

Disk-Scheduling Algorithm (Cont.)

- The disk-scheduling algorithm should be written as a separate module of the operating system, allowing it to be replaced with a different algorithm if necessary
- Either SSTF or LOOK is a reasonable choice for the default algorithm
- What about rotational latency?
  - Difficult for OS to calculate
- How does disk-based queuing effect OS queue ordering efforts?

Summary

Strategy	Advantages	Disadvantages
FCFS	• Easy to implement • Sufficient for light loads	• Doesn't provide best average service • Doesn't maximize throughput
SSTF	• Throughput better than FCFS • Tends to minimize arm movement • Tends to minimize response time	• May cause starvation of some requests • Localizes under heavy loads
SCAN/LOOK	• Eliminates starvation • Throughput similar to SSTF • Works well with light to moderate loads	• Needs directional bit • More complex algorithm to implement • Increased overhead
N-Step SCAN	• Easier to implement than SCAN	• The most recent requests wait longer than with SCAN
C-SCAN/C-LOOK	• Works well with moderate to heavy loads • No directional bit • Small variance in service time • C-LOOK doesn't travel to unused tracks	• May not be fair to recent requests for high-numbered tracks • More complex algorithm than N-Step SCAN, causing more overhead

(table 7.9)

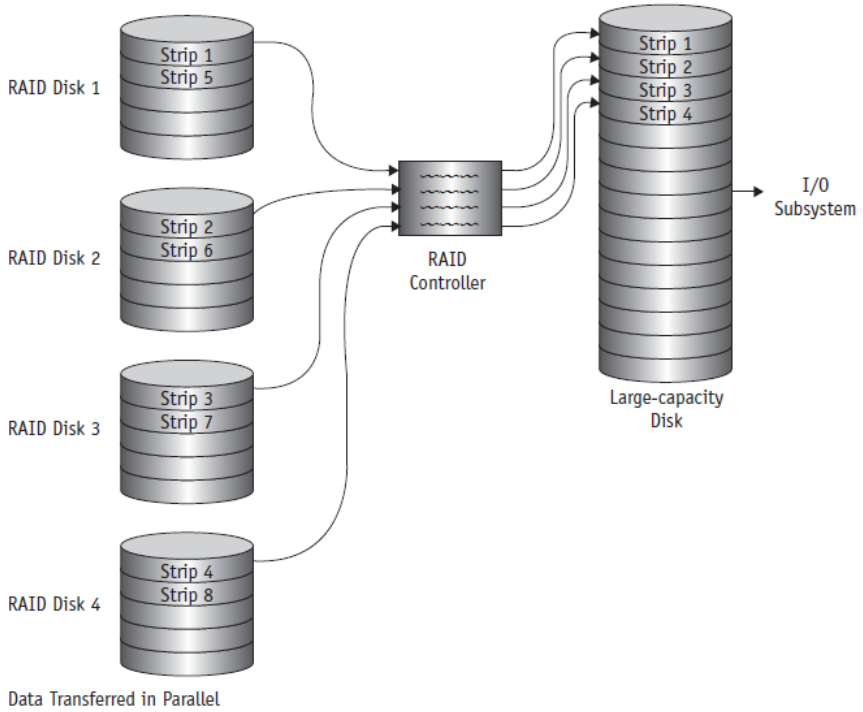
Comparison of DASD seek strategies discussed in this chapter.

Redundant Array of Independent Disks (RAID)

12.7 RAID

- Stands for Redundant Arrays of Independent Disks
- Commonly used to address the performance and reliability issues.
- Frequently used on **servers**, but not generally used on PCs
- Normally consists of 2 or more disks working in parallel but appear as one drive to the user – **disk striping**.

- **Disk striping** can speed up data transfer because transfer can be done in parallel.



(figure 7.19)

Data being transferred in parallel from a Level o RAID configuration to a large-capacity disk. The software in the controller ensures that the strips are stored in correct order.

冗余

- RAID provides **reliability** through **redundancy**
  - *Mirroring* or *shadowing* keeps duplicate of each disk.
  - *Block interleaved parity* uses much less redundancy.

文错

Data Striping:

- A technique for spreading data over multiple disks
- Data striping speeds up operations that retrieve data from disk storage
- Data on disks are broken into units (blocks) and these are spread across the available disks
- Implementations allow selection of data units size, or *stripe width*

Data Mirrowing (Shadowing):

- A technique in which data are written to two duplicate disks simultaneously
- If one disk fails, the system can instantly switch to the other disk without loss of data or service
- Used commonly in on-line database systems where it is critical that data are accessible at all times

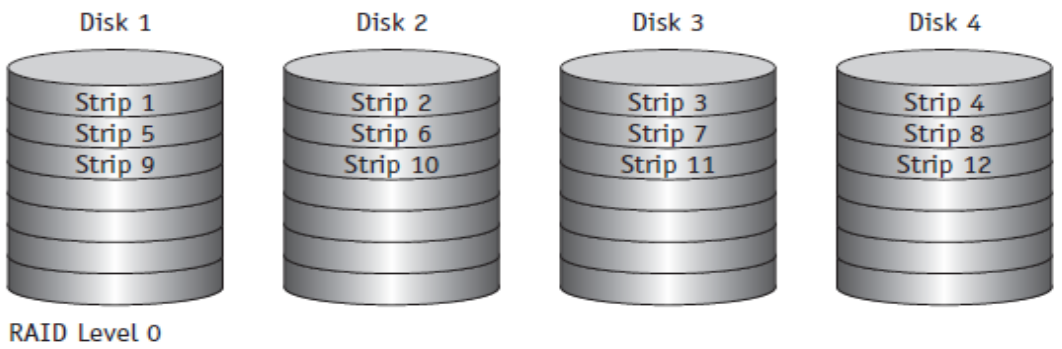
12.7.3 RAID Levels

Level Zero

- Uses only data striping (not considered true RAID)
  - No parity and error corrections
  - No error correction/redundancy/recovery
- Benefits
  - Devices appear as one logical unit
  - Best for large data quantity non-critical data

(figure 7.20)

RAID Level 0 with four disks in the array. Strips 1, 2, 3, and 4 make up a stripe. Strips 5, 6, 7, and 8 make up another stripe, and so on.

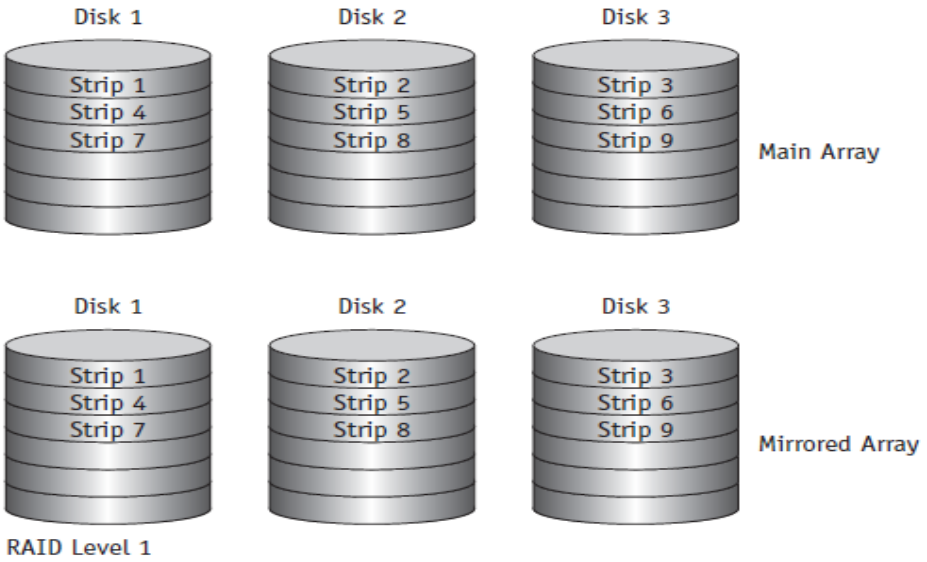


Level One

- Uses data striping and mirroring (considered true RAID)
  - Mirroring configuration (backup)
    - Duplicate set of all data (expensive)
  - Provides redundancy and improved reliability

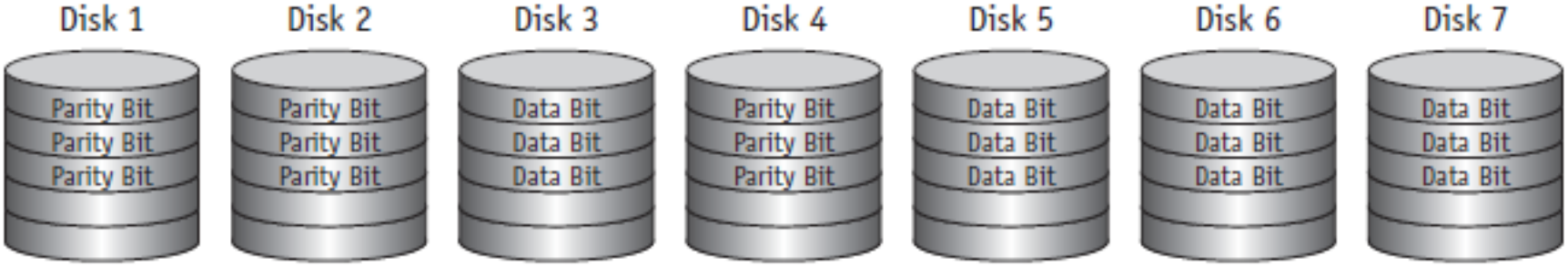
(figure 7.21)

RAID Level 1 with three disks in the main array and three corresponding disks in the backup array, the mirrored array.



Level Two

- Uses small stripes (bit level) (considered true RAID)
- **Hamming code**: error detection and correction
- Expensive and complex
  - Size of strip determines number of array disks



RAID Level 2 - Hamming Code

(figure 7.22)

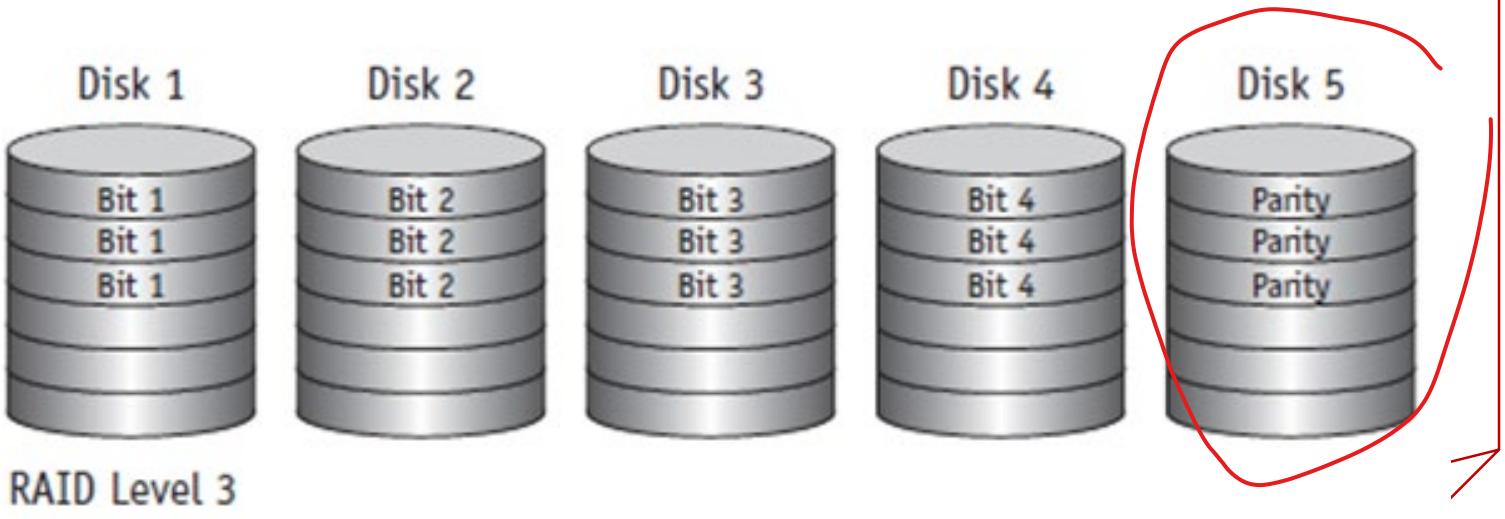
RAID Level 2. Seven disks are needed in the array to store a 4-bit data item, one for each bit and three for the parity bits. Each disk stores either a bit or a parity bit based on the Hamming code used for redundancy.

Level Three

- Modification of Level 2
  - Requires one disk for redundancy
    - Small strip (bit level) for data bits
    - Block strip for parity bits

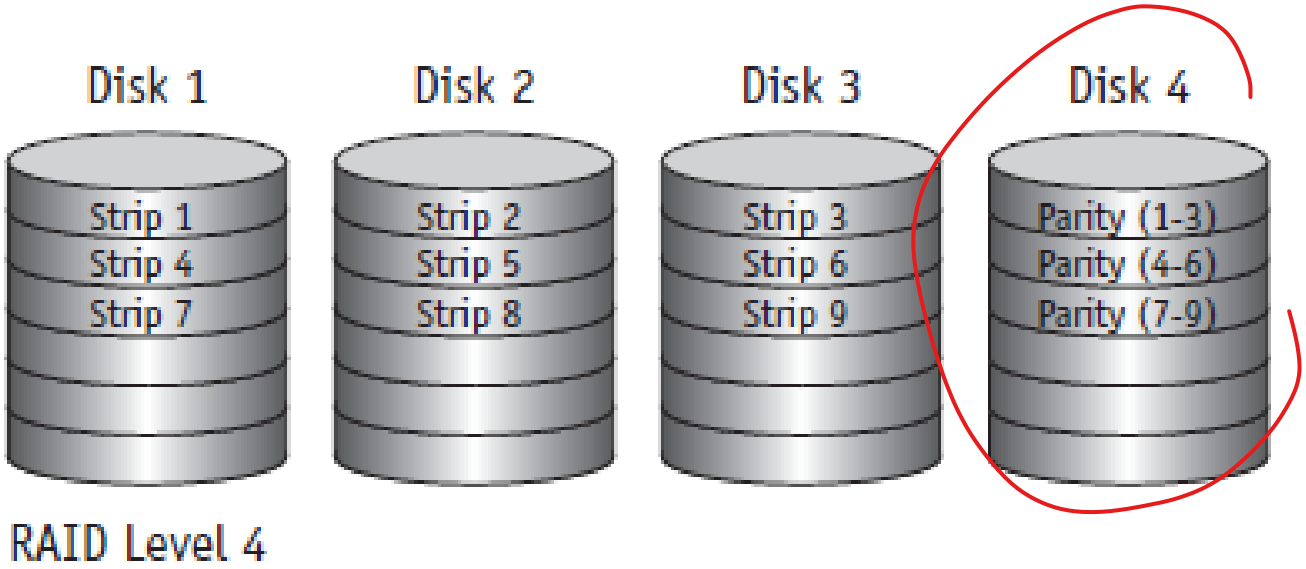
(figure 7.23)

RAID Level 3. A 4-bit data item is stored in the first four disks of the array. The fifth disk is used to store the parity for the stored data item.



Level Four

- Same strip scheme as Levels 0 and 1
  - Computes parity for each strip
  - Stores parities in corresponding strip
    - Has designated parity disk

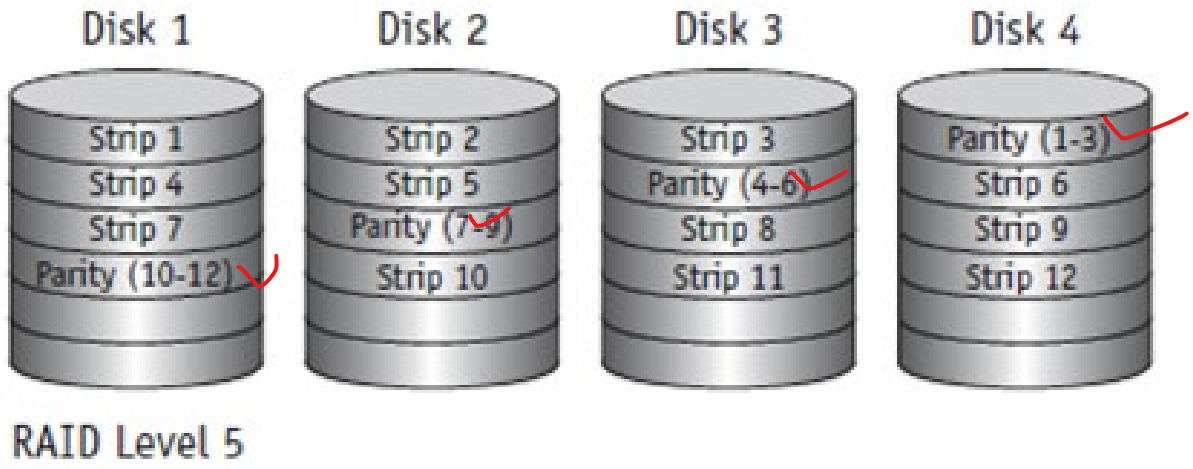


(figure 7.24)

RAID Level 4. The array contains four disks: the first three are used to store data strips, and the fourth is used to store the parity of those strips.

Level Five

- Modification of Level 4
- Distributes parity strips across disks
  - Avoids Level 4 bottleneck
- Disadvantage
  - Complicated to regenerate data from failed device

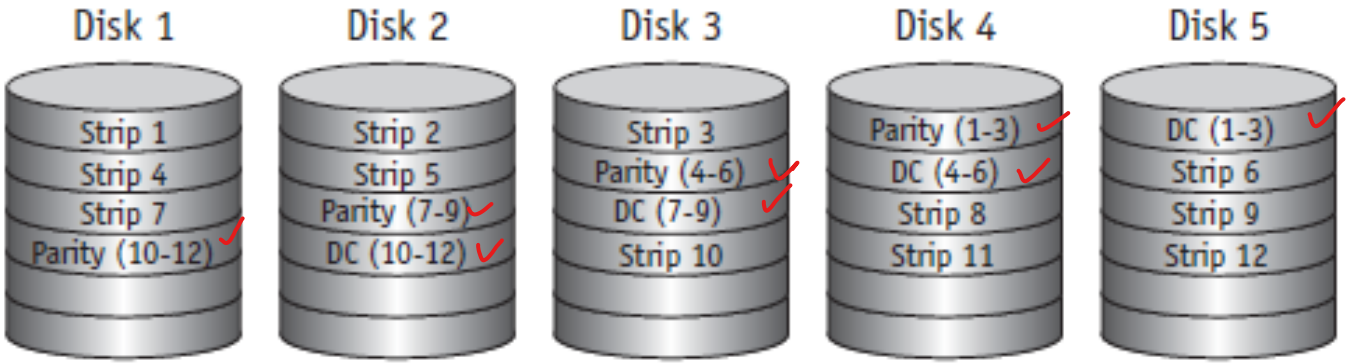


(figure 7.25)

RAID Level 5 with four disks. Notice how the parity strips are distributed among the disks.

Level Six

- Provides extra degree of error protection/correction
  - Two different parity calculations (double parity)
    - Same as level four/five and independent algorithm
  - Parities stored on separate disk across array
    - Stored in corresponding data strip
- Advantage: data restoration even if two disks fail



RAID Level 6

(figure 7.26)

RAID Level 6. Notice how parity strips and data check (DC) strips are distributed across the disks.

RAID

(table 7.7) *The seven standard levels of RAID provide various degrees of error correction. Cost, speed, and the system's applications are significant factors to consider when choosing a system.*

RAID Level	Error Correction Method	I/O Request Rate	Data Transfer Rate
0	None	Excellent	Excellent
1	Mirroring	Read: Good Write: Fair	Read: Fair Write: Fair
2	Hamming code	Poor	Excellent
3	Word parity	Poor	Excellent
4	Strip parity	Read: Excellent Write: Fair	Read: Fair Write: Poor
5	Distributed strip parity	Read: Excellent Write: Fair	Read: Fair Write: Poor
6	Distributed strip parity and independent data check	Read: Excellent Write: Poor	Read: Fair Write: Poor

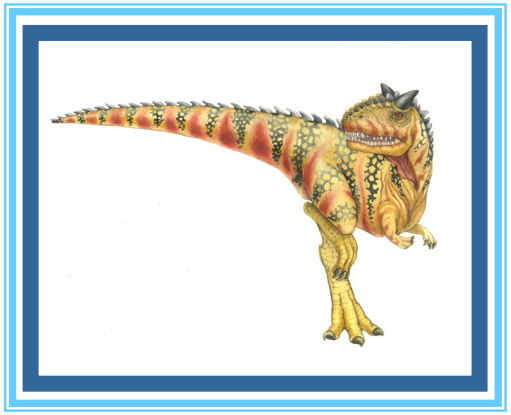
Summary

- Disk performance (disk access time) is based on seek time, latency and transfer time
- Disk scheduling algorithms
  - FCFS is a disk seek algorithm that is based on first come first served.
  - SSTF – serve request with track closest to one being served
  - SCAN and C-SCAN – serve request from outer to inner track and serve every request on way back to inner track. C-SCAN serves during inward sweep only
  - LOOK and C-LOOK – similar to SCAN and C-SCAN respectively except does not scan until the edge of cylinder

Summary

- RAID
  - Data striping is used to increase performance
  - Data mirroring and parity are techniques used for reliability since these techniques provide redundancy of data

End of Chapter 12



Silberschatz, Galvin and Gagne ©2013

12.4

Disk Scheduling

(a) FCFS

- The simplest form of disk scheduling.
- Generally does not provide the fastest service.

Example 3 :

(Page: 546)

We illustrate the **FCFS** disk scheduling algorithms with the following information to get the total distance (in cylinders) :

- ✓ A disk drive with 200 cylinders (0-199);
- ✓ A request queue :

  98, 183, 37, 122, 14, 124, 65, 67
- ✓ Head pointer 53.

12.4

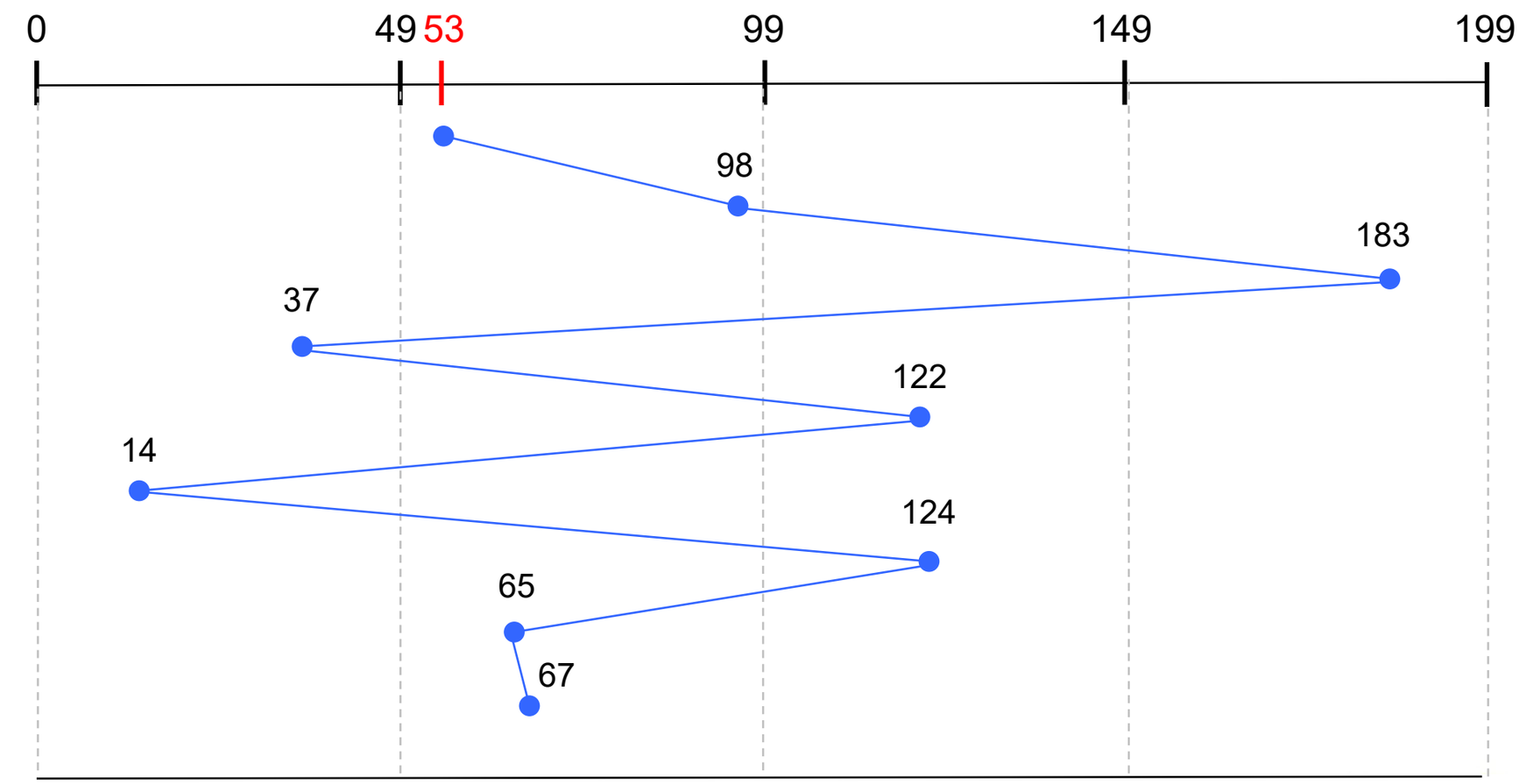
Disk Scheduling

Solution 3 :

(Page: 546)

- A request queue :

98, 183, 37, 122, 14, 124, 65, 67



Total distance =

$(98-53)+$

$(183-98)+$

$(183-37)+$

$(122-37)+$

$(122-14)+$

$(124-14)+$

$(124-65)+$

$(67-65)$

$=45+85+146+85+108+110+59+2=640$ cylinders.

Disk Scheduling

Shortest Seek Time First (SSTF)

(b) SSTF

- This algorithm selects the request with the minimum seek time from the current head position.
- SSTF scheduling is a form of SJF scheduling; may cause starvation of some requests.
- Next example illustrates a total head movement of 236 cylinders.

12.4

Disk Scheduling

Example 4 :

(Page: 547)

We illustrate the **SSTF** disk scheduling algorithms with the following information:

- ✓ A disk drive with 200 cylinders (0-199);
- ✓ A request queue :

  98, 183, 37, 122, 14, 124, 65, 67
- ✓ Head pointer 53.

Calculate the total distance (in cylinders) that the disk arm moves to satisfy the requests.

12.4

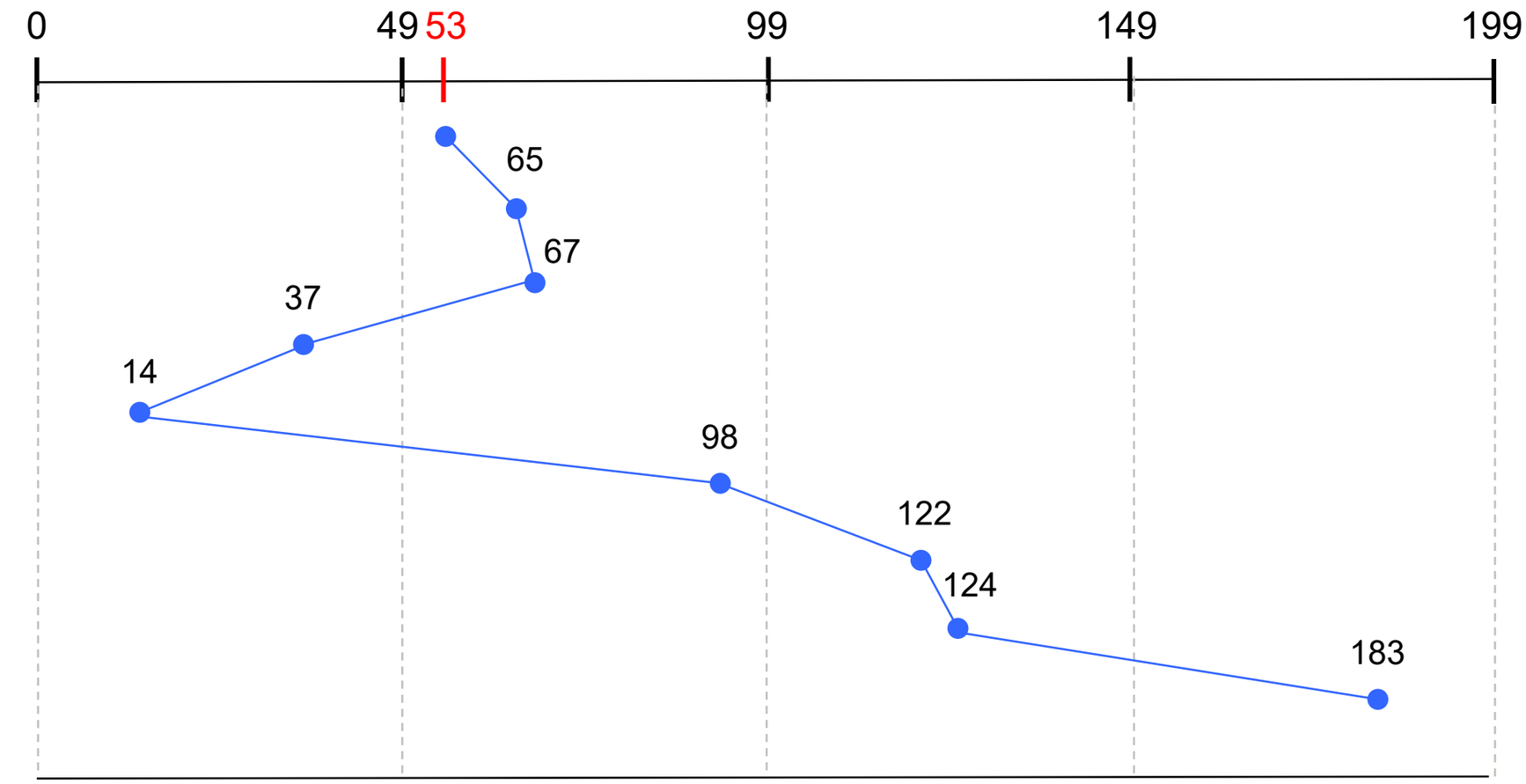
Disk Scheduling

Solution 4 :

(Page: 547)

- A request queue :

98, 183, 37, 122, 14, 124, 65, 67



Total distance =

$$(65-53)+$$
$$(67-65)+$$
$$(67-37)+$$
$$(37-14)+$$
$$(98-14)+$$
$$(122-98)+$$
$$(124-122)+$$
$$(183-124)$$

$=12+2+30+23+84+24+2+59=236$ cylinders (head movement).

Disk Scheduling

- Although **SSTF** algorithm is substantial improvement over **FCFS** algorithm, it is not optimal.
- We can do better by moving the head from 53 to 37, even though the latter is not closest.
- This strategy reduces the total head movement to 208 cylinders.

Exercise 12.1 : Based on previous example, redraw the entire head movement of the disk if the head moving from 53 to 37 instead of 53 to 65.

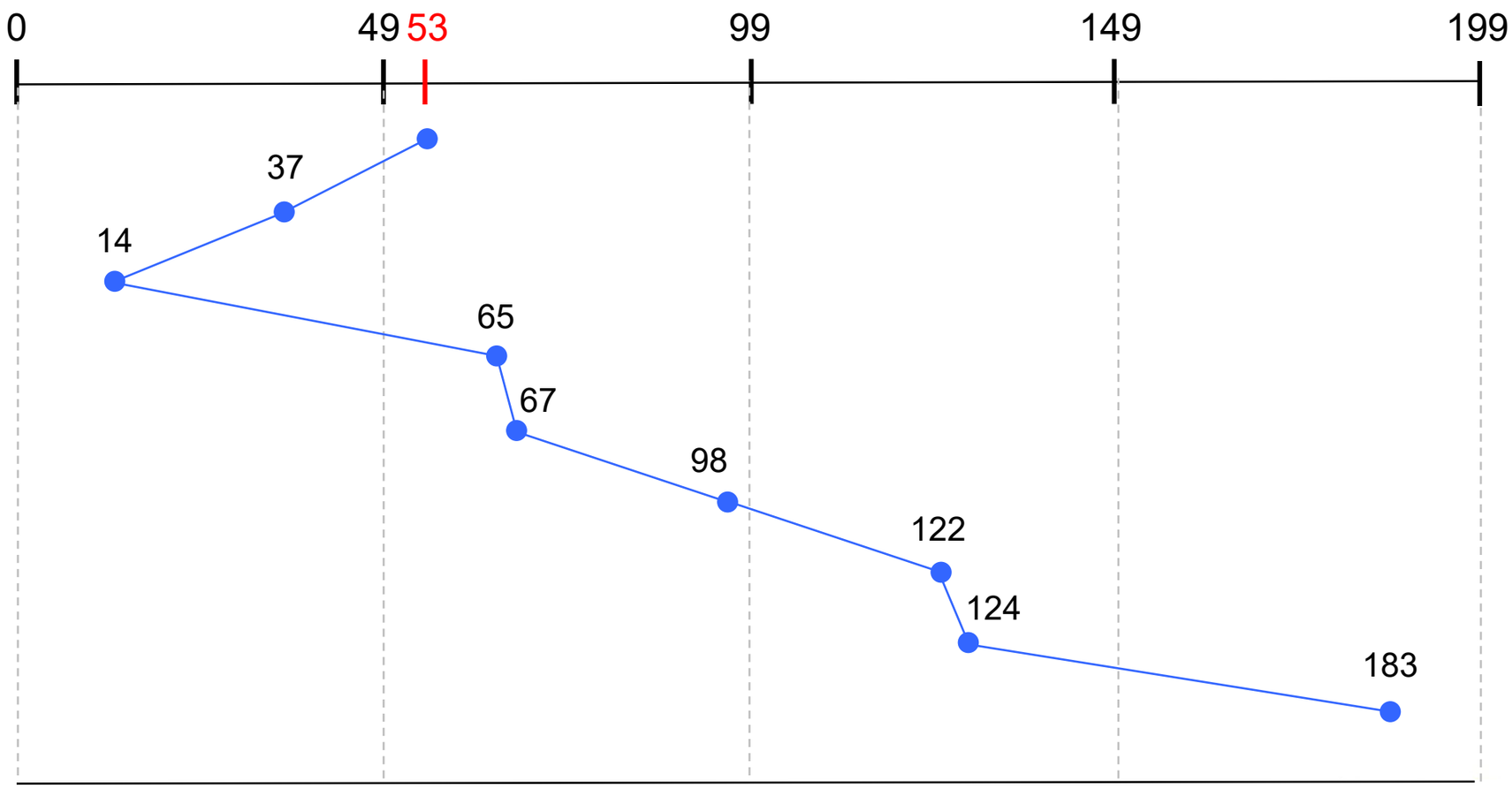
12.4

Disk Scheduling

Solution 12.1

- A request queue :

98, 183, 37, 122, 14, 124, 65, 67



Total distance =

$$(53-37)+$$
$$(37-14)+$$
$$(65-14)+$$
$$(67-65)+$$
$$(98-67)+$$
$$(122-98)+$$
$$(124-122)+$$
$$(183-124)$$

$$=16+23+51+2+31+24+2+59=208$$ cylinders (head movement).

Disk Scheduling

(c - i) SCAN

- The disk arm starts at one end of the disk, and moves **toward** the other end, servicing requests until it gets to the other end of the disk, where the head movement is **reversed** and servicing continues.
- **SCAN algorithm** sometimes called the **elevator algorithm.**

- Next example illustrates total head movement of 208 cylinders.
- But note that if requests are uniformly dense, largest density at other end of disk and those wait the longest.

Disk Scheduling

Example 5 :

(Page: 548)

We illustrate the **SCAN** disk scheduling algorithms with the following information:

- ✓ A disk drive with 200 cylinders (0-199);
- ✓ A request queue :

  98, 183, 37, 122, 14, 124, 65, 67
- ✓ Head pointer 53.
- ✓ **Direction**: the disk arm is moving toward 0.

Calculate the total distance (in cylinders) that the disk arm moves to satisfy the requests.

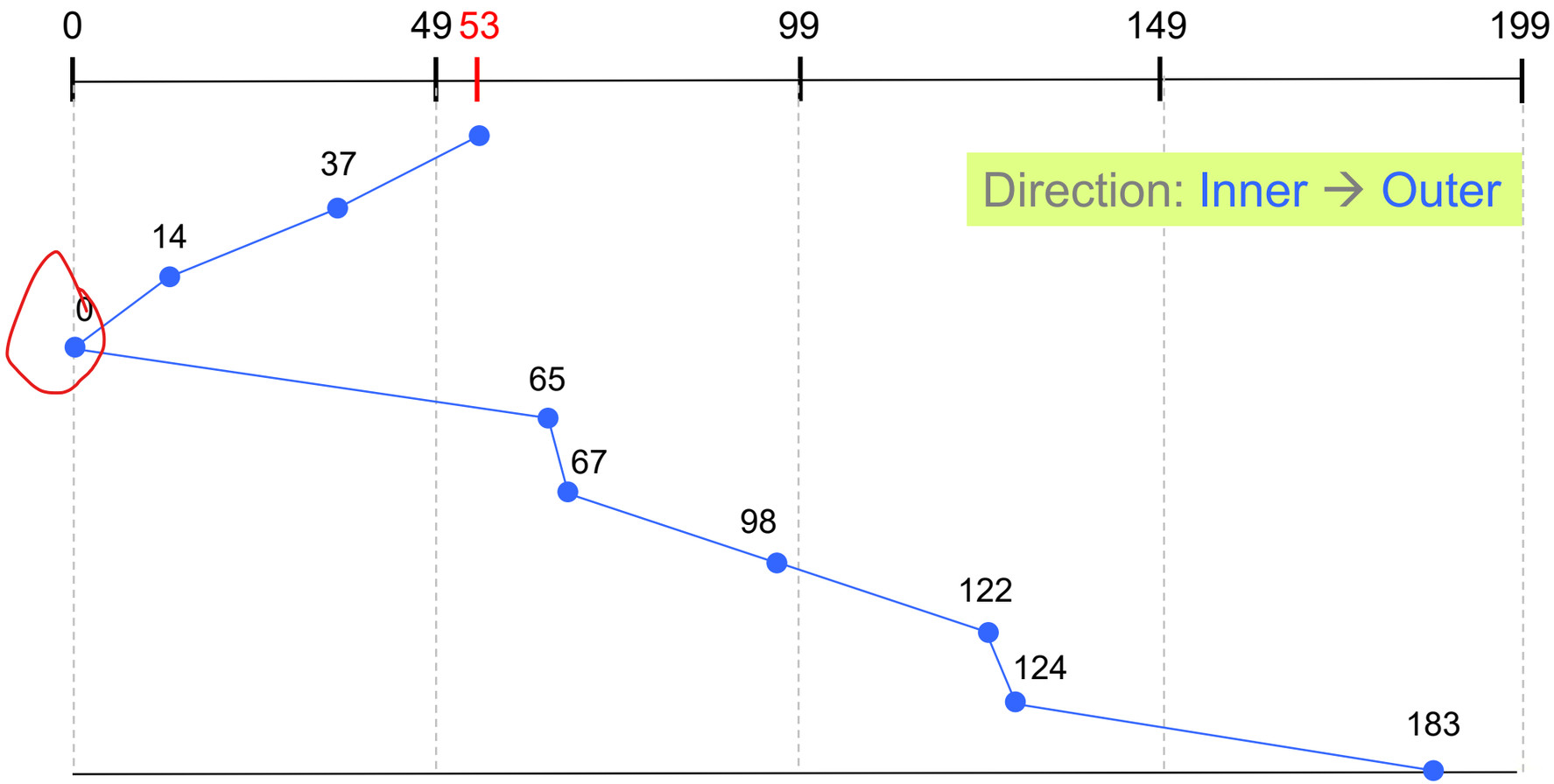
12.4

Disk Scheduling

Solution 5 :

- ✓ A request queue :

98, 183, 37, 122, 14, 124, 65, 67



Total distance =

$(53-37)+$

$(37-14)+$

$(14-0)+$

$(65-0)+$

$(67-65)+$

$(98-67)+$

$(122-98)+$

$(124-122)+$

$(183-124)$

$=25+23+14+65+2+31+24+2+59=245$ cylinders (head movement).

12.4

Disk Scheduling

Exercise 12.2 :

We illustrate the **SCAN** disk scheduling algorithms with the following information:

- ✓ A disk drive with 200 cylinders (0-199);
- ✓ A request queue :

  98, 183, 37, 122, 14, 124, 65, 67
- ✓ Head pointer 53.
- ✓ **Direction**: the disk arm is moving to innermost.

Calculate the total distance (in cylinders) that the disk arm moves to satisfy the requests.

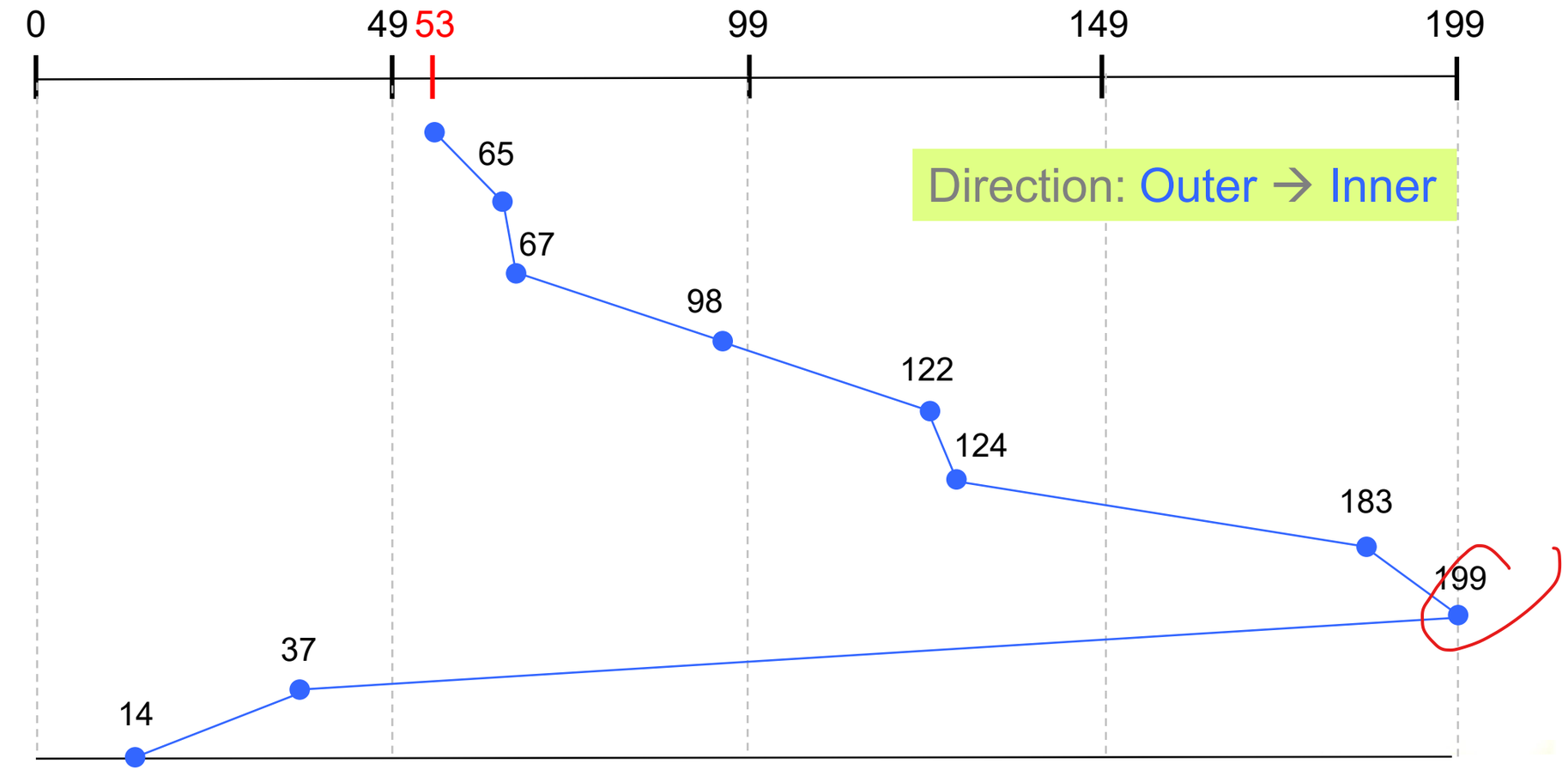
12.4

Disk Scheduling

Solution 12.2 :

- A request queue :

98, 183, 37, 122, 14, 124, 65, 67



Total distance =

$$(65-53)+$$
$$(67-65)+$$
$$(98-67)+$$
$$(122-98)+$$
$$(124-122)+$$
$$(183-124)+$$
$$(199-183)+$$
$$(199-37)+$$
$$(37-14)$$

$=12+2+30+23+84+24+2+59=331$ cylinders (head movement).

Disk Scheduling

(c-ii) C-SCAN

- Provides a more uniform wait time than **SCAN**.
- The head moves from one end (**inward sweep**) of the disk to the other, servicing requests as it goes:
  - When it reaches the other end, however, it immediately **returns** to the beginning of the disk, without servicing any requests on the return trip.
- Treats the cylinders as a circular list that wraps around from the last cylinder to the first one.

Disk Scheduling

Example 6 :

(Page: 548)

We illustrate the **C-SCAN** disk scheduling algorithms with the following information:

- ✓ A disk drive with 200 cylinders (0-199);
- ✓ A request queue :

  98, 183, 37, 122, 14, 124, 65, 67
- ✓ Head pointer 53.

Calculate the total distance (in cylinders) that the disk arm moves to satisfy the requests.

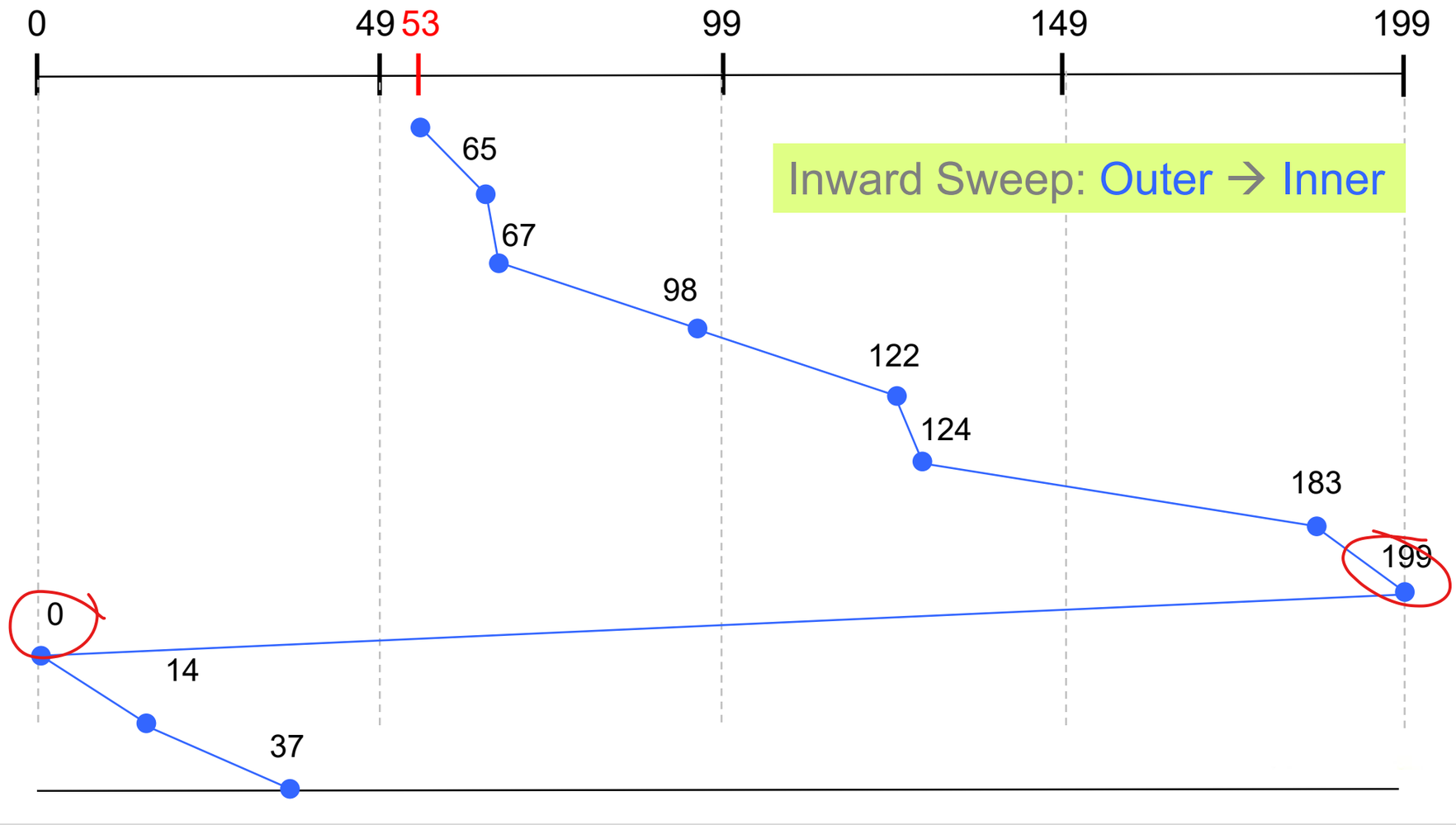
Disk Scheduling

Solution 6 :

(Page: 548)

- ✓ A request queue :

98, 183, 37, 122, 14, 124, 65, 67



Total distance =

$$(65-53)+$$
$$(67-65)+$$
$$(98-67)+$$
$$(122-98)+$$
$$(124-122)+$$
$$(183-124)+$$
$$(199-183)+$$
$$(199-0)+$$
$$(14-0)+$$
$$(37-14)$$

$$=12+2+31+24+2+59+16+199+14+23=382$$ cylinders (head movement).

Disk Scheduling

(d) LOOK and C-LOOK

- As described before, both SCAN and C-SCAN move the disk arm across the full width of the disk.
- The arms goes only as far as the final request in each direction, then reverses direction immediately, without going all the way to the end of the disk.
- Version of SCAN and C-SCAN that follow this pattern is called **LOOK** and **C-LOOK** disk scheduling.

Disk Scheduling

Example 7 :

We illustrate the **LOOK** disk scheduling algorithms with the following information:

- ✓ A disk drive with 200 cylinders (0-199);
- ✓ A request queue :

  98, 183, 37, 122, 14, 124, 65, 67
- ✓ Head pointer 53.
- ✓ **Direction**: the disk arm is moving toward 0.

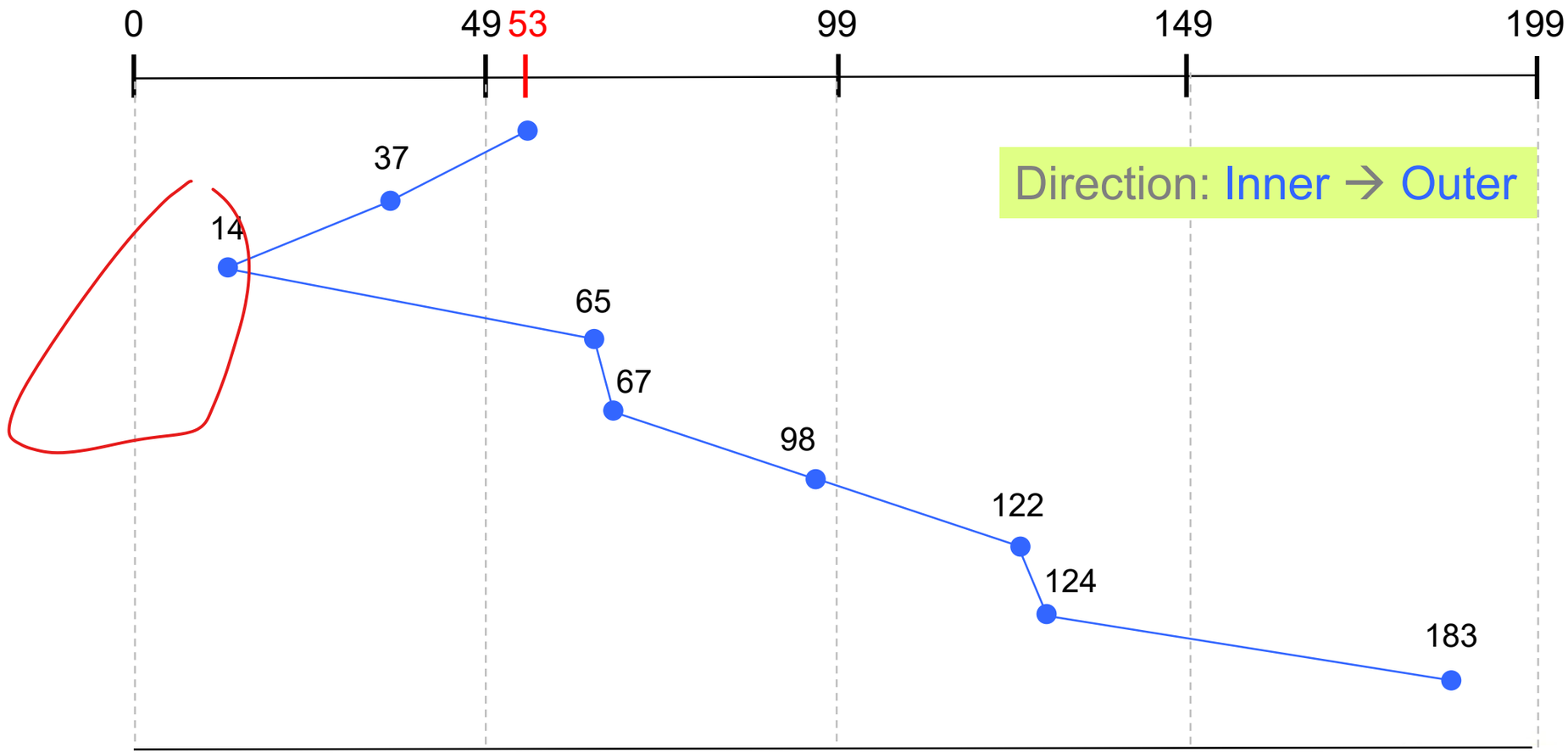
Calculate the total distance (in cylinders) that the disk arm moves to satisfy the requests.

Disk Scheduling

Solution 7 :

- A request queue :

98, 183, 37, 122, 14, 124, 65, 67



Total distance =

$$(53-37)+$$
$$(37-14)+$$
$$(65-14)+$$
$$(67-65)+$$
$$(98-67)+$$
$$(122-98)+$$
$$(124-122)+$$
$$(183-124)$$

$$=16+23+51+2+31+24+2+59=208$$ cylinders (head movement).

12.4

Disk Scheduling

Exercise 12.3 :

We illustrate the **LOOK** disk scheduling algorithms with the following information:

- ✓ A disk drive with 200 cylinders (0-199);
- ✓ A request queue :

  98, 183, 37, 122, 14, 124, 65, 67
- ✓ Head pointer 53.
- ✓ **Direction**: the disk arm is moving to innermost.

Calculate the total distance (in cylinders) that the disk arm moves to satisfy the requests.

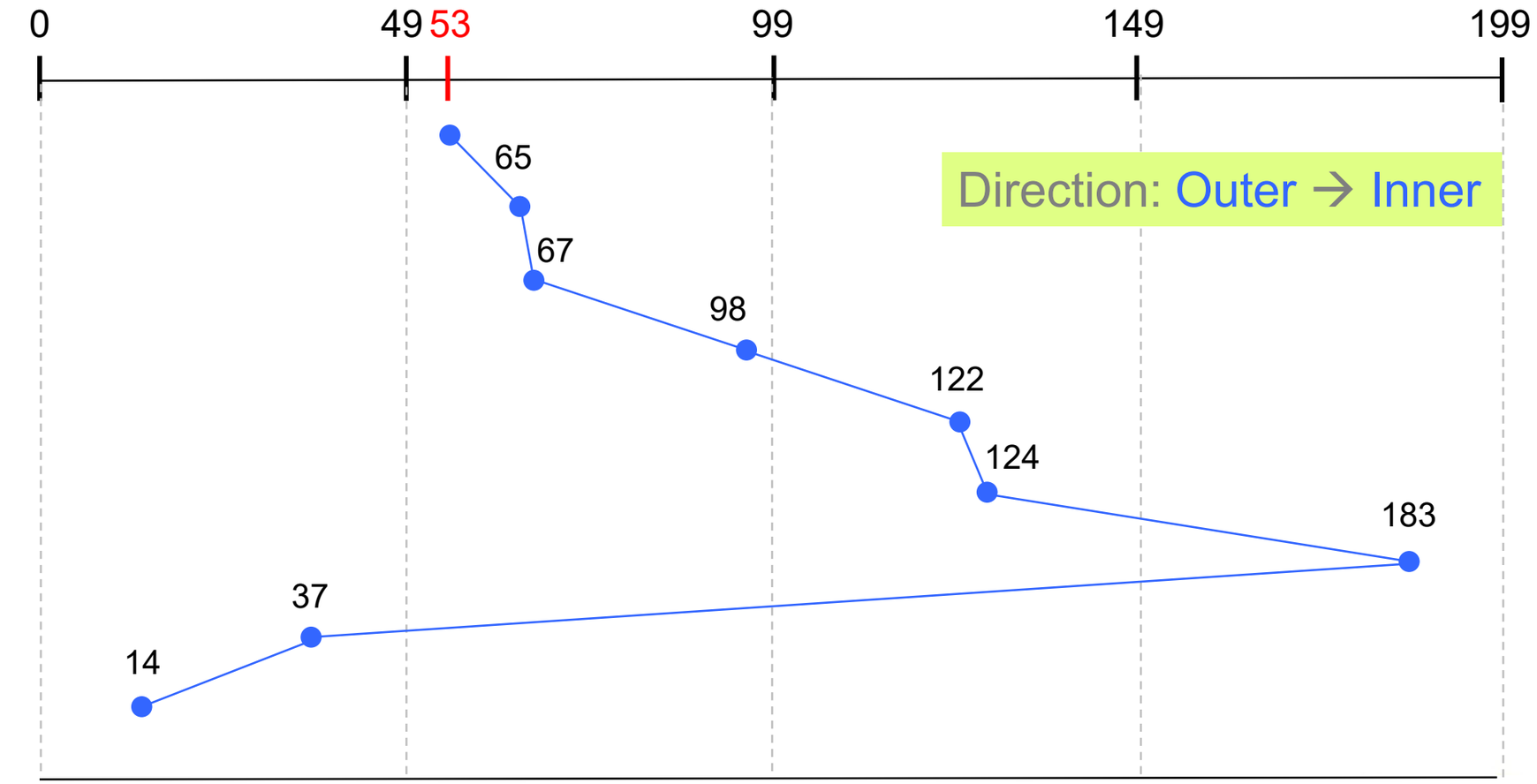
12.4

Disk Scheduling

Solution 12.3 :

- A request queue :

98, 183, 37, 122, 14, 124, 65, 67



Total distance =

$$(65-53)+$$ 11
$$(67-65)+$$ 2
$$(98-67)+$$
$$(122-98)+$$
$$(124-122)+$$
$$(183-124)+$$
$$(183-37)+$$
$$(37-14)$$

$$=12+2+30+23+84+24+2+59=331$$ cylinders (head movement).

12.4

Disk Scheduling

Example 8 :

(Page: 549)

We illustrate the **C-LOOK** disk scheduling algorithms with the following information:

- ✓ A disk drive with 200 cylinders (0-199);
- ✓ A request queue :

  98, 183, 37, 122, 14, 124, 65, 67
- ✓ Head pointer 53.

Calculate the total distance (in cylinders) that the disk arm moves to satisfy the requests.

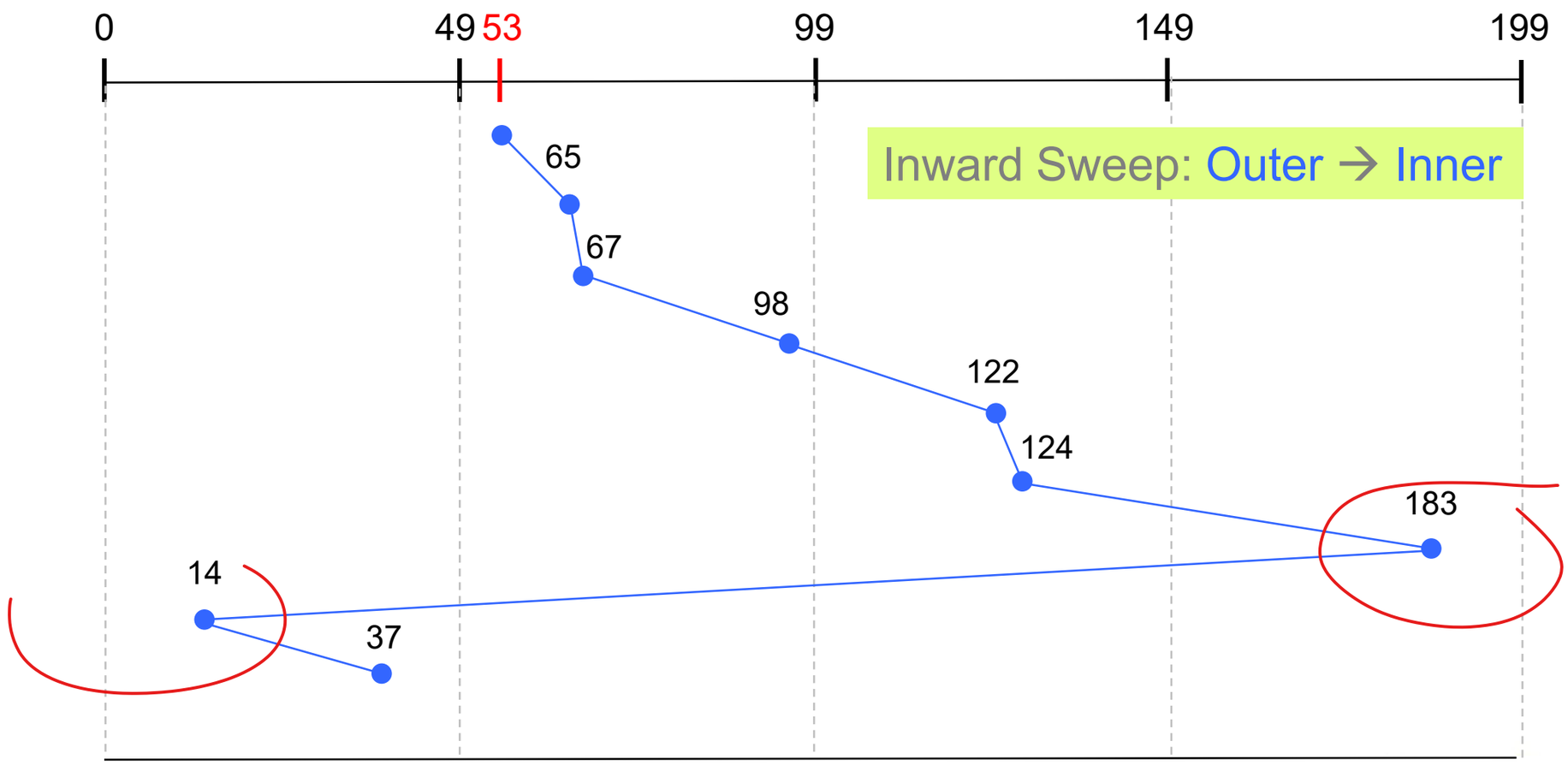
Disk Scheduling

Solution 8 :

(Page: 549)

- ✓ A request queue :

98, 183, 37, 122, 14, 124, 65, 67



Total distance =

$$(65-53)+$$
$$(67-65)+$$
$$(98-67)+$$
$$(122-98)+$$
$$(124-122)+$$
$$(183-124)+$$
$$(183-14)+$$
$$(37-14)$$

$$=12+2+31+24+2+59+169+23=322$$ cylinders (head movement).